

HỌC VIỆN NÔNG NGHIỆP VIỆT NAM
Khoa Công Nghệ Thông Tin

—***—



BÀI GIẢNG
MÔN HỌC MÁY VÀ KHAI PHÁ DỮ LIỆU
(Mã học phần: TH94005)

Năm học: 2025

NỘI DUNG



Giáo Trình Trí Tuệ Nhân Tạo, Học Máy Và Học Sâu

Vũ Văn Hiệu, Lê Khắc Định, Nguyễn Quỳnh Nga, Vũ Thị Anh Trâm, Phạm Quang Huy, NXB Thanh Niên, 2024

- § 1 - Nhập môn về Học máy và Khai phá dữ liệu
- § 2 - Học có giám sát (supervised learning)
- § 3 - Học không giám sát (unsupervised learning)
- § 4 - Biểu diễn và tiền xử lý dữ liệu
- § 5 - Đánh giá và cải thiện mô hình học



BÀI 1

NHẬP MÔN VỀ HỌC MÁY



▶ Học máy là gì?



- ▶ Học máy là gì?
- ▶ Phạm vi của "học máy"?



- ▶ Học máy là gì?
- ▶ Phạm vi của "học máy"?
- ▶ Python và thư viện khai thác ứng dụng học máy



- ▶ Học máy là gì?
- ▶ Phạm vi của "học máy"?
- ▶ Python và thư viện khai thác ứng dụng học máy
- ▶ Một số ví dụ về ứng dụng của học máy



Học máy là gì?



Học máy:

- ▶ Tiếng Anh: Machine learning



Học máy:

- ▶ Tiếng Anh: Machine learning
- ▶ Là việc trích xuất tri thức từ dữ liệu



Học máy:

- ▶ Tiếng Anh: Machine learning
- ▶ Là việc trích xuất tri thức từ dữ liệu
- ▶ Cải thiện việc thực hiện một công việc trên các kinh nghiệm đã có



- ▶ Nhận biết mã code từ các ký tự viết tay (MNIST)



- ▶ Nhận biết mã code từ các ký tự viết tay (MNIST)
- ▶ Xác định bất thường từ dữ liệu ảnh y tế



- ▶ Nhận biết mã code từ các ký tự viết tay (MNIST)
- ▶ Xác định bất thường từ dữ liệu ảnh y tế
- ▶ Xác định lỗi trong các phiên giao dịch của thẻ tín dụng



- ▶ Nhận biết mã code từ các ký tự viết tay (MNIST)
- ▶ Xác định bất thường từ dữ liệu ảnh y tế
- ▶ Xác định lỗi trong các phiên giao dịch của thẻ tín dụng
- ▶ Dự báo thư rác (spam emails)



- ▶ Nhận biết mã code từ các ký tự viết tay (MNIST)
- ▶ Xác định bất thường từ dữ liệu ảnh y tế
- ▶ Xác định lỗi trong các phiên giao dịch của thẻ tín dụng
- ▶ Dự báo thư rác (spam emails)
- ▶ Nhận biết mã thư tín từ người gửi viết trên thư



- ▶ Đọc biển số xe qua trạm kiểm soát

- ▶ Đọc biển số xe qua trạm kiểm soát
- ▶ Xác định chủ đề từ tập các bài trao đổi, bài đăng trên mạng xã hội



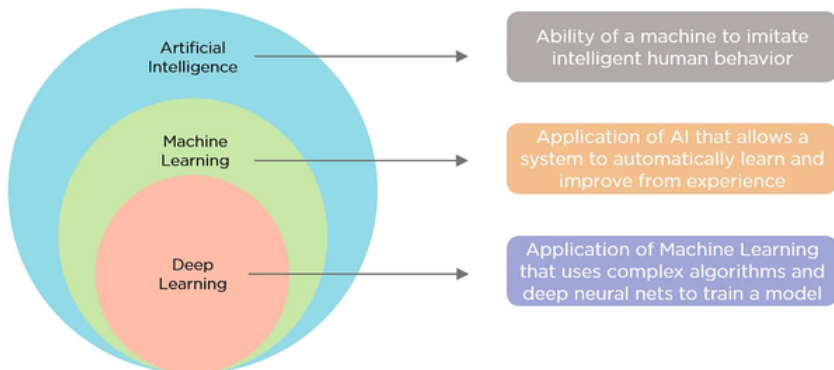
- ▶ Đọc biển số xe qua trạm kiểm soát
- ▶ Xác định chủ đề từ tập các bài trao đổi, bài đăng trên mạng xã hội
- ▶ Phân nhóm ưu tiên các khách hàng



- ▶ Đọc biển số xe qua trạm kiểm soát
- ▶ Xác định chủ đề từ tập các bài trao đổi, bài đăng trên mạng xã hội
- ▶ Phân nhóm ưu tiên các khách hàng
- ▶ Phát hiện các khác thường trong hành vi truy cập người dùng vào trang web

- ▶ Trí tuệ nhân tạo (Artificial Intelligence)
- ▶ Học máy (Machine learning)
- ▶ Học sâu (Deep learning)

Phạm vi¹



¹ Simplilearn



Nhận biết spam email?

²<https://security.virginia.edu/examples-phishing>

Nhận biết spam email?

Sent: Monday, May 09, 2016 10:07 AM
To:
Subject: Fwd: [UVa Library - Circulation] VIRGINIA WARNING: Closing & Deleting Your Account in Progress!

VIRGINIA WARNING: Closing & Deleting Your Account in Progress!

Hello User!

We received your instructions to delete your account **1**

We will process your request within 24 hours. **2**

All features associated with your account will be lost.

To retain your account, kindly Cancel Request to continue using our services

CANCEL REQUEST IMMEDIATELY **3**

Thank You,
Account Team

<http://bit.ly/1WTXQzB>

Please do not reply to this message. Mail sent to this address cannot be answered. **4**

²<https://security.virginia.edu/examples-phishing>



Ví dụ về học máy - Bài toán nhận biết cảm xúc khuôn mặt

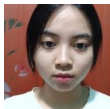
Nhóm 1: Khuôn mặt vui



Nhóm 1: Khuôn mặt vui



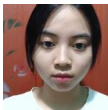
Nhóm 2: Khuôn mặt tâm trạng



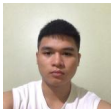
Nhóm 1: Khuôn mặt vui



Nhóm 2: Khuôn mặt tâm trạng



Khuôn mặt dưới đây thuộc nhóm nào?





- ▶ Học có giám sát là tổng quát hóa quá trình xử lý ra quyết định từ dữ liệu đã biết



- ▶ Học có giám sát là tổng quát hóa quá trình xử lý ra quyết định từ dữ liệu đã biết
- ▶ Thành công đa số của học máy là các xử lý ra quyết định tự động từ việc tổng quát hóa từ các dữ liệu đã biết



- ▶ Học có giám sát là tổng quát hóa quá trình xử lý ra quyết định từ dữ liệu đã biết
- ▶ Thành công đa số của học máy là các xử lý ra quyết định tự động từ việc tổng quát hóa từ các dữ liệu đã biết → **học có giám sát**



- ▶ Học có giám sát là tổng quát hóa quá trình xử lý ra quyết định từ dữ liệu đã biết
- ▶ Thành công đa số của học máy là các xử lý ra quyết định tự động từ việc tổng quát hóa từ các dữ liệu đã biết → **học có giám sát**
- ▶ Ví dụ về học có giám sát

- ▶ Học có giám sát là tổng quát hóa quá trình xử lý ra quyết định từ dữ liệu đã biết
- ▶ Thành công đa số của học máy là các xử lý ra quyết định tự động từ việc tổng quát hóa từ các dữ liệu đã biết → **học có giám sát**
- ▶ Ví dụ về học có giám sát
 - Nhận biết mã code từ các ký tự viết tay (e.g. dữ liệu MNIST)

- ▶ Học có giám sát là tổng quát hóa quá trình xử lý ra quyết định từ dữ liệu đã biết
- ▶ Thành công đa số của học máy là các xử lý ra quyết định tự động từ việc tổng quát hóa từ các dữ liệu đã biết → **học có giám sát**
- ▶ Ví dụ về học có giám sát
 - Nhận biết mã code từ các ký tự viết tay (e.g. dữ liệu MNIST)
 - Xác định bất thường từ dữ liệu ảnh y tế (e.g ảnh IVUS)

- ▶ Học có giám sát là tổng quát hóa quá trình xử lý ra quyết định từ dữ liệu đã biết
- ▶ Thành công đa số của học máy là các xử lý ra quyết định tự động từ việc tổng quát hóa từ các dữ liệu đã biết → **học có giám sát**
- ▶ Ví dụ về học có giám sát
 - Nhận biết mã code từ các ký tự viết tay (e.g. dữ liệu MNIST)
 - Xác định bất thường từ dữ liệu ảnh y tế (e.g ảnh IVUS)
 - Xác định lỗi trong các phiên giao dịch của thẻ tín dụng.

- **Bài toán phân lớp (Classification):** giá trị mục tiêu của dữ liệu là các nhãn lớp, là một lựa chọn từ một danh sách các khả năng được xác định trước. Ví dụ:
- Phân loại thư rác (spam: yes/no?)
 - Phân lớp các loại hoa Diên Vĩ...

- ▶ **Bài toán phân lớp (Classification):** giá trị mục tiêu của dữ liệu là các nhãn lớp, là một lựa chọn từ một danh sách các khả năng được xác định trước. Ví dụ:
 - Phân loại thư rác (spam: yes/no?)
 - Phân lớp các loại hoa Diên Vĩ...
- ▶ **Bài toán hồi quy (Regression):** giá trị mục tiêu của dữ liệu một số liên tục trong thuật ngữ toán học. Ví dụ:
 - Dự đoán thu nhập hàng năm của một người dựa trên trình độ học vấn, tuổi tác và nơi sinh sống
 - Dự đoán giá cả...



- ▶ Học không giám sát bao gồm tất cả các bài toán học máy mà dữ liệu học chưa biết nhãn, không có người hướng dẫn chỉ bảo thuật toán để học



- ▶ Học không giám sát bao gồm tất cả các bài toán học máy mà dữ liệu học chưa biết nhãn, không có người hướng dẫn chỉ bảo thuật toán để học
- ▶ Trong học không giám sát, thuật toán học chỉ được cung cấp dữ liệu đầu vào và được yêu cầu trích xuất kiến thức từ dữ liệu này.



- ▶ Học không giám sát bao gồm tất cả các bài toán học máy mà dữ liệu học chưa biết nhãn, không có người hướng dẫn chỉ bảo thuật toán để học
- ▶ Trong học không giám sát, thuật toán học chỉ được cung cấp dữ liệu đầu vào và được yêu cầu trích xuất kiến thức từ dữ liệu này.
- ▶ Ví dụ về học không giám sát

- ▶ Học không giám sát bao gồm tất cả các bài toán học máy mà dữ liệu học chưa biết nhãn, không có người hướng dẫn chỉ bảo thuật toán để học
- ▶ Trong học không giám sát, thuật toán học chỉ được cung cấp dữ liệu đầu vào và được yêu cầu trích xuất kiến thức từ dữ liệu này.
- ▶ Ví dụ về học không giám sát
 - Phân vùng ảnh (image segmentation)



- ▶ Học không giám sát bao gồm tất cả các bài toán học máy mà dữ liệu học chưa biết nhãn, không có người hướng dẫn chỉ bảo thuật toán để học
- ▶ Trong học không giám sát, thuật toán học chỉ được cung cấp dữ liệu đầu vào và được yêu cầu trích xuất kiến thức từ dữ liệu này.
- ▶ Ví dụ về học không giám sát
 - Phân vùng ảnh (image segmentation)
 - Phân nhóm khách hàng tiềm năng



► **Biến đổi tập dữ liệu (Data transform)**

- Giảm số chiều PCA, Projection
- Hàm nhân (kernel) biến đổi dữ liệu
- Xử lý dữ liệu thiếu
- Chuẩn hóa dữ liệu ...

► **Biến đổi tập dữ liệu (Data transform)**

- Giảm số chiều PCA, Projection
- Hàm nhân (kernel) biến đổi dữ liệu
- Xử lý dữ liệu thiếu
- Chuẩn hóa dữ liệu ...

► **Phân cụm dữ liệu (Clustering)**

- K-means
- DBSCAN
- HDBSCAN ...



- ▶ Phần quan trọng của học máy là hiểu rõ công việc cần xử lý, dữ liệu gì và mối liên hệ của nó với nhiệm vụ muốn giải quyết.

- ▶ Phần quan trọng của học máy là hiểu rõ công việc cần xử lý, dữ liệu gì và mối liên hệ của nó với nhiệm vụ muốn giải quyết.
- ▶ Sẽ không là cách hiệu quả nếu chọn ngẫu nhiên một thuật toán áp dụng vào dữ liệu mà mình có, mà cần phải hiểu rõ những gì đang diễn ra trong tập dữ liệu thu thập được trước khi bắt đầu xây dựng mô hình.

- ▶ Phần quan trọng của học máy là hiểu rõ công việc cần xử lý, dữ liệu gì và mối liên hệ của nó với nhiệm vụ muốn giải quyết.
- ▶ Sẽ không là cách hiệu quả nếu chọn ngẫu nhiên một thuật toán áp dụng vào dữ liệu mà mình có, mà cần phải hiểu rõ những gì đang diễn ra trong tập dữ liệu thu thập được trước khi bắt đầu xây dựng mô hình.
- ▶ Mỗi thuật toán đều khác nhau về loại dữ liệu và bối cảnh bài toán mà nó phù hợp nhất.



Những câu hỏi cần lưu ý trong bài toán học máy:



Những câu hỏi cần lưu ý trong bài toán học máy:

1. Tôi đang trả lời câu hỏi gì? Dữ liệu tôi thu thập có trả lời được câu hỏi đó không?



Những câu hỏi cần lưu ý trong bài toán học máy:

1. Tôi đang trả lời câu hỏi gì? Dữ liệu tôi thu thập có trả lời được câu hỏi đó không?
2. Cách nào tốt nhất để diễn đạt các câu hỏi cần giải quyết thành bài toán học máy?



Những câu hỏi cần lưu ý trong bài toán học máy:

1. Tôi đang trả lời câu hỏi gì? Dữ liệu tôi thu thập có trả lời được câu hỏi đó không?
2. Cách nào tốt nhất để diễn đạt các câu hỏi cần giải quyết thành bài toán học máy?
3. Tôi đã thu thập đủ dữ liệu thể hiện tính chất bài toán mà tôi cần giải quyết hay không?



Những câu hỏi cần lưu ý trong bài toán học máy:



Những câu hỏi cần lưu ý trong bài toán học máy:

4. Thuộc tính nào của dữ liệu mà tôi cần trích xuất làm cho các dự báo để có thể trả lời đúng?

Những câu hỏi cần lưu ý trong bài toán học máy:

4. Thuộc tính nào của dữ liệu mà tôi cần trích xuất làm cho các dự báo để có thể trả lời đúng?
5. Làm thế nào tôi có thể đo được sự thành công của ứng dụng học máy?

Những câu hỏi cần lưu ý trong bài toán học máy:

4. Thuộc tính nào của dữ liệu mà tôi cần trích xuất làm cho các dự báo để có thể trả lời đúng?
5. Làm thế nào tôi có thể đo được sự thành công của ứng dụng học máy?
6. Làm thế nào để giải pháp học máy có thể làm việc với các phần khác trong sản phẩm nghiên cứu hay kinh doanh?



Dữ liệu huấn luyện (training data) là tập dữ liệu đã biết nhãn nhằm phục vụ cho việc đào tạo mô hình học.

Dữ liệu kiểm tra (test data) là tập dữ liệu dùng để đánh giá khả năng học của mô hình.

Khái niệm

Trong học có giám sát, một mô hình trên dữ liệu được huấn luyện và sau đó có thể đưa ra dự đoán trên dữ liệu mới, chưa được nhìn thấy có cùng đặc điểm với tập huấn luyện mà chúng ta đã sử dụng. Nếu một mô hình có khả năng đưa ra dự đoán chính xác trên dữ liệu chưa được nhìn thấy, chúng ta nói rằng nó có khả năng khái quát hóa từ tập huấn luyện sang tập kiểm tra.

Khái niệm

Trong học có giám sát, một mô hình trên dữ liệu được huấn luyện và sau đó có thể đưa ra dự đoán trên dữ liệu mới, chưa được nhìn thấy có cùng đặc điểm với tập huấn luyện mà chúng ta đã sử dụng. Nếu một mô hình có khả năng đưa ra dự đoán chính xác trên dữ liệu chưa được nhìn thấy, chúng ta nói rằng nó có khả năng khái quát hóa từ tập huấn luyện sang tập kiểm tra.

- ▶ Học máy cần xây dựng một mô hình có khả năng khái quát hóa càng chính xác càng tốt.

Khái niệm

Thông thường, chúng ta xây dựng mô hình sao cho nó có thể đưa ra dự đoán chính xác trên tập dữ liệu huấn luyện. Nếu tập dữ liệu huấn luyện và tập dữ liệu kiểm tra có đủ điểm chung, chúng ta kỳ vọng mô hình cũng sẽ chính xác trên tập dữ liệu kiểm tra. Tuy nhiên, có một số trường hợp điều này có thể sai. Độ chính xác dự báo trên tập huấn luyện có thể cao nhưng trên tập kiểm tra lại thấp. Trường hợp như vậy gọi là quá khớp (overfitting).

Khái niệm

Nếu mô hình học quá đơn giản thì mô hình đó có thể không nắm bắt được hết các khía cạnh và sự biến đổi trong dữ liệu, và mô hình sẽ hoạt động kém hiệu quả ngay cả trên tập huấn luyện. Việc lựa chọn mô hình quá đơn giản được gọi là hiện tượng chưa khớp (underfitting).



1. Datasets on Scikitlearn
2. Datasets on Tensorflow
3. Datasets on Kaggle
4. Datasets on Github



- ▶ Download miễn phí trên <https://www.python.org/>
- ▶ Các packages cho xử lý dữ liệu
 - scikit-learn
 - pandas
 - numpy
 - matplotlib
 - seaborn
 - opencv

Setosa



Versicolor



Virginica³



³nguồn: Wikipedia



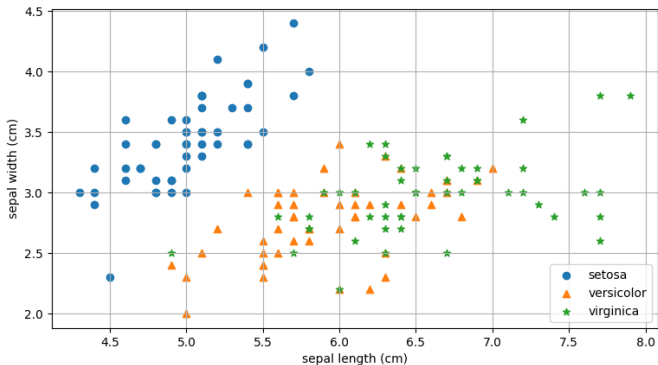
Đọc thông tin về dữ liệu hoa Diên Vĩ:

```
from sklearn import datasets
iris = datasets.load_iris()
# Hiển thị tên các thuộc tính
print(iris.feature_names)
# Hiển thị tên các loài hoa Diên Vĩ
print(iris.target_names)
# Hiển thị giá trị các trường dữ liệu
print(iris.data)
# Các lớp {0, 1, 2} ứng với dữ liệu
print(iris.target)
```

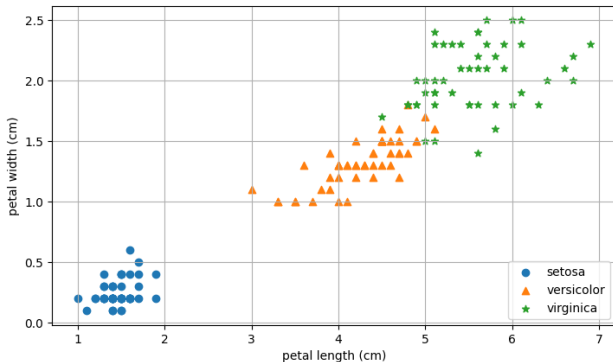
Biểu diễn tương quan giữa các thuộc tính:

```
1. from matplotlib import pyplot as plt
2. from sklearn import datasets
3. iris = datasets.load_iris()
4. x,y = iris.data,iris.target
5. types = iris.target_names
6. features = iris.feature_names
7. _, ax = plt.subplots()
8. m = ["o","^","*"]
9. f1, f2 = 1 ,2 # Biểu diễn dài và rộng của đài hoa
10. for i in range(3):
11.     xi = x[y==i]
12.     ax.scatter(xi[:,f1],xi[:,f2], marker=m[i],label=types[i])
13. ax.set(xlabel=features[f1], ylabel=features[f2])
14. ax.legend(loc="lower right")
15. plt.show()
```

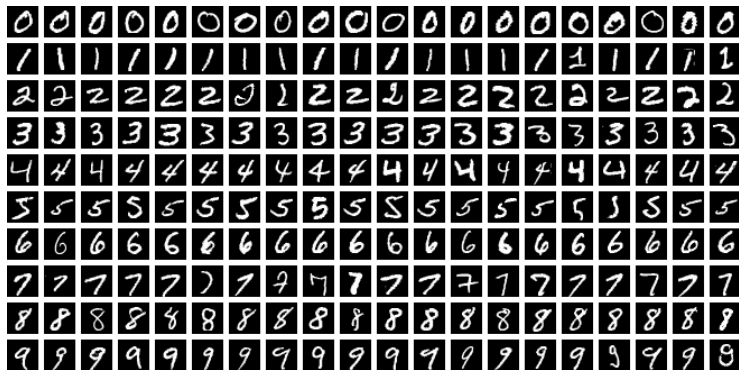
Độ dài và Độ rộng của Đài hoa (Sepal):



Độ dài và Độ rộng của Cánh hoa (Petal):



MNIST datasets gồm 70,000 ảnh xám (28×28) các ký tự số viết tay



(Nguồn Wikipedia)



MNIST datasets trong sklearn gồm 1797 ảnh xám (8×8):

```
from sklearn import datasets
digits = datasets.load_digits()
# Hiển thị dữ liệu các điểm ảnh
print(digits.data)
# Hiển thị nhãn
print(digits.target)
```




```
from matplotlib import pyplot as plt
from sklearn import datasets
import numpy as np

# Đọc dữ liệu các điểm ảnh
x = datasets.load_digits()["data"]
y = datasets.load_digits()["target"]

# Biểu diễn dữ liệu đầu tiên thành ảnh
fig, ax = plt.subplots(1)
ax.imshow(np.reshape(x[0], (8,8)), cmap="gray")
plt.show()
```



Agriculture and Farming Dataset on Kaggle:

- Train_agriculture.csv
- Test_agriculture.xlsx

```
import pandas as pd

train_data = pd.read_csv("../train_agriculture.csv")

test_data =
pd.read_excel("../test_agriculture.xlsx")

# Hiển thị thông tin dữ liệu train
print(train_data.info())

# Hiển thị thông tin dữ liệu test
print(test_data.info())
```

```
import numpy as np  
from matplotlib import pyplot as plt  
import cv2  
data = cv2.imread("../tomato.jpg")  
cv2.imshow("Tomato",data)  
plt.show()
```





BÀI 2

HỌC CÓ GIÁM SÁT

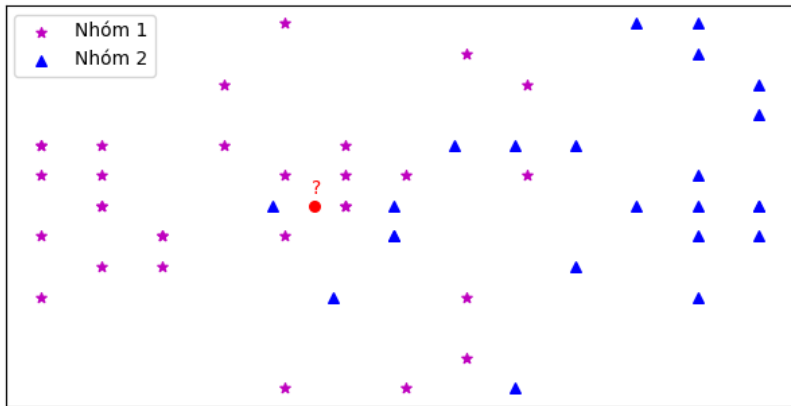
Phương pháp k-Láng giềng gần nhất (kNN) và Tiền xử lý dữ liệu

- ▶ Phương pháp k-láng giềng gần nhất
(kNN=**k** **N**earest **N**eighbors)

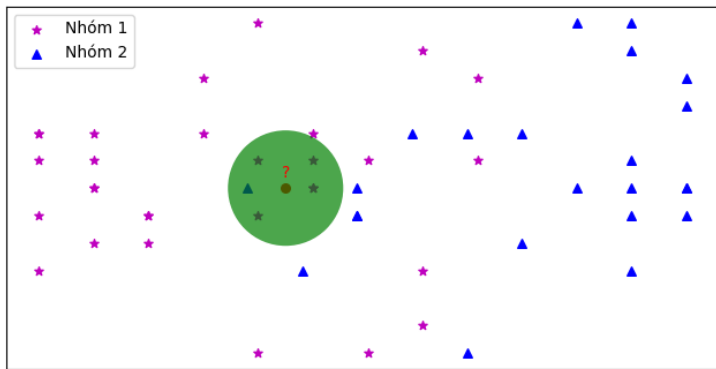
- ▶ Phương pháp k-láng giềng gần nhất
(kNN=**k** Nearest **N**eighbors)
- ▶ Biến đổi dữ liệu (Data transformation)

- ▶ Phương pháp k-láng giềng gần nhất
(kNN=**k** Nearest **N**eighbors)
- ▶ Biến đổi dữ liệu (Data transformation)
- ▶ Thực hành trên Python

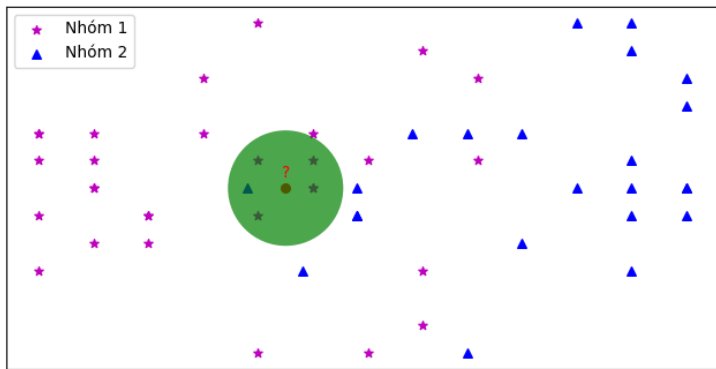
Bài toán: Xác định phần tử ● thuộc nhóm 1 hay nhóm 2?



Tục ngữ: "Gần mực thì đen gần đèn thì rạng"

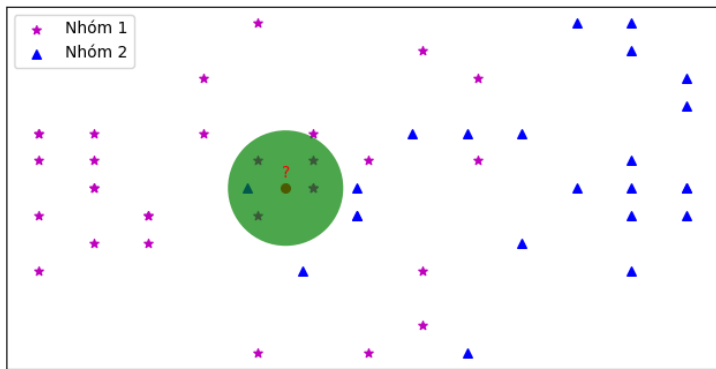


Tục ngữ: "Gần mực thì đen gần đèn thì rạng"



- Phần tử ● thuộc nhóm 1

Tục ngữ: "Gần mực thì đen gần đèn thì rạng"



- Phần tử ● thuộc nhóm 1
- Số láng giềng của ● xem xét là $k=5$

Algorithm 1 Thuật toán phân lớp kNN

Input: Dữ liệu, nhãn (x, y) , số $k \in \mathbb{N}$
và x^* chưa biết nhãn

Output: Hiển thị nhãn của x^* trong nhóm y

for $x_i \in x$ **do**

 Tính khoảng cách x^* tới x_i

end for

$S \leftarrow k$ phần tử có khoảng cách ngắn nhất đến x^*

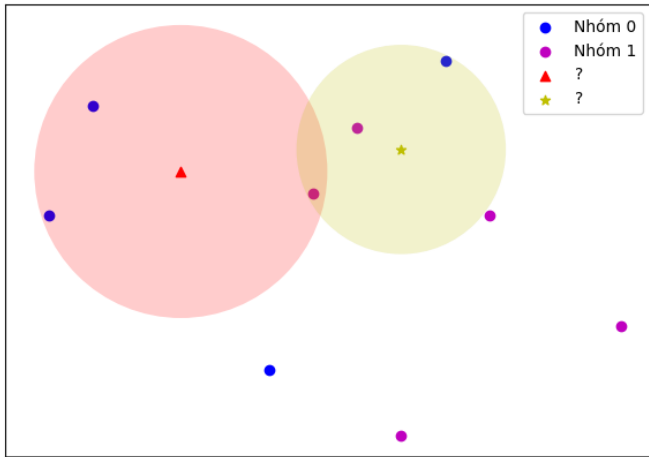
Nhãn $x^* \leftarrow$ nhãn trội trong các phần tử của S .

Cho dữ liệu có 2 thuộc tính X_1 , X_2 và nhãn Y ở bảng:

X_1	X_2	Y
2	3	0
2.7	3.4	1
2.5	2.3	0
3	3	1
2.9	3.7	0
2.8	2	1
2.6	3.1	1
2.1	3.5	0
3.3	2.5	1
2.3	3.2	?
2.8	3.3	?

Hãy xác định nhãn của hai điểm dữ liệu cuối cùng theo dữ liệu đã biết ở trên.

k-NN với $k = 3$



Cho dữ liệu thông tin về xe ô tô bị mất cắp ở bảng:

TT	Màu sắc	Kiểu xe	Nguồn gốc	Bị trộm
1.	Đỏ	Thể thao	Nội địa	Có
2.	Cam	Thể thao	Nội địa	Không
3.	Cam	Thể thao	Nhập khẩu	Có
4.	Xanh	SUV	Nhập khẩu	Không
5.	Đỏ	SUV	Nhập khẩu	Không
6.	Đỏ	Thể thao	Nhập khẩu	Có
7.	Cam	Thể thao	Nội địa	Không
8.	Cam	SUV	Nội địa	Có
9.	Xanh	Thể thao	Nhập khẩu	Có
10.	Đỏ	SUV	Nội địa	?

Có thể dùng kNN xác định nhãn của dữ liệu cuối cùng theo thông tin đã biết ở trên?

Giả sử $x^1 = (x_i^1)$ và $x^2 = (x_i^2)$ là 2 đối tượng thông tin thu thập được với các trường thuộc tính giá trị thực

- Khoảng cách Euclide (L2)

$$d(x^1, x^2) = \sqrt{\sum_{i=1}^n (x_i^1 - x_i^2)^2}$$

- Khoảng cách Manhattan (L1)

$$d(x^1, x^2) = \sum_{i=1}^n |x_i^1 - x_i^2|$$

Giả sử $x^1 = (x_i^1)$ và $x^2 = (x_i^2)$ là 2 đối tượng thông tin thu thập được với các trường thuộc tính giá trị thực

- Khoảng cách Hamming

$$d(x^1, x^2) = \sum_{i=1}^n \text{diff}(x_i^1, x_i^2), \text{ với } \text{diff}(x_i^1, x_i^2) = \begin{cases} 1 & \text{nếu } x_i^1 \neq x_i^2 \\ 0 & \text{nếu } x_i^1 = x_i^2 \end{cases}$$

- Khoảng cách Cosine

$$d(x^1, x^2) = 1 - \frac{x^1 \cdot x^2}{\|x^1\| \cdot \|x^2\|} = 1 - \frac{\sum_{i=1}^n x_i^1 \cdot x_i^2}{\sqrt{\sum_{i=1}^n (x_i^1)^2 \cdot \sum_{i=1}^n (x_i^2)^2}}$$



Ví dụ 1: Hãy sử dụng khoảng cách Euclide và phương pháp kNN với $k=3$ xác định nhãn của 2 phần tử còn lại trong dữ liệu dưới đây:

X_1	X_2	Y
2	3	0
2.7	3.4	1
2.5	2.3	0
3	3	1
2.9	3.7	0
2.8	2	1
2.6	3.1	1
2.1	3.5	0
3.3	2.5	1
2.3	3.2	?
2.8	3.3	?

Ví dụ 2: Sử dụng độ đo thích hợp và phương pháp kNN với $k = 3$ cho biết kết quả nhãn của bản ghi chưa biết:

TT	Màu sắc	Kiểu xe	Nguồn gốc	Bị trộm
1.	Đỏ	Thể thao	Nội địa	Có
2.	Cam	Thể thao	Nội địa	Không
3.	Cam	Thể thao	Nhập khẩu	Có
4.	Xanh	SUV	Nhập khẩu	Không
5.	Đỏ	SUV	Nhập khẩu	Không
6.	Đỏ	Thể thao	Nhập khẩu	Có
7.	Cam	Thể thao	Nội địa	Không
8.	Cam	SUV	Nội địa	Có
9.	Xanh	Thể thao	Nhập khẩu	Có
10.	Đỏ	SUV	Nội địa	?

- Khi dữ liệu cho ở dạng phạm trù, thể loại (categories), dạng text (có/không, ...) cần đưa về dạng số để tính toán khoảng cách thì cần chuyển đổi dữ liệu từ dạng text/categories về dạng số

Ví dụ:

"Yes"	→	1	"Đỏ"	→	0
"No"	→	0	"Xanh"	→	1
			"Cam"	→	2

- Khi dữ liệu cho ở dạng phạm trù, thể loại (categories), dạng text (có/không, ...) cần đưa về dạng số để tính toán khoảng cách thì cần chuyển đổi dữ liệu từ dạng text/categories về dạng số

Ví dụ:

"Yes"	→	1	"Đỏ"	→	0
"No"	→	0	"Xanh"	→	1
			"Cam"	→	2

- Sử dụng packages `sklearn.compose`, `sklearn.preprocessing` cho việc chuyển đổi dữ liệu

Hãy số hóa dữ liệu trong bảng sau:

TT	Màu sắc	Kiểu xe	Nguồn gốc	Bị trộm
1.	Đỏ	Thể thao	Nội địa	Có
2.	Cam	Thể thao	Nội địa	Không
3.	Cam	Thể thao	Nhập khẩu	Có
4.	Xanh	SUV	Nhập khẩu	Không
5.	Đỏ	SUV	Nhập khẩu	Không
6.	Đỏ	Thể thao	Nhập khẩu	Có
7.	Cam	Thể thao	Nội địa	Không
8.	Cam	SUV	Nội địa	Có
9.	Xanh	Thể thao	Nhập khẩu	Có
10.	Đỏ	SUV	Nội địa	?



Biến đổi dữ liệu về dạng số thực - Data Transformation

```
from sklearn.preprocessing import OrdinalEncoder
import numpy as np
import pandas as pd
data = pd.DataFrame({'Màu sắc': ['Đỏ', 'Cam', 'Cam', "Xanh", 'Đỏ', 'Đỏ', 'Cam', \
                                'Cam', 'Xanh'],
                    'Kiểu xe': ['Thể thao', 'Thể thao', 'Thể thao', 'SUV', \
                                'SUV', 'Thể thao', 'Thể thao', 'SUV', 'Thể thao'],
                    'Nguồn gốc': ['Nội địa', 'Nội địa', 'Nhập khẩu', 'Nhập khẩu', \
                                'Nhập khẩu', 'Nhập khẩu', 'Nội địa', 'Nội địa', \
                                'Nhập khẩu'],
                    'Bị trộm': ['Có', 'Không', 'Có', 'Không', 'Không', 'Có', 'Không', \
                                'Có', 'Có']})
x = data.drop('Bị trộm', axis=1).to_numpy()
y = data['Bị trộm'].to_numpy()

transformer = OrdinalEncoder()
x1 = transformer.fit_transform(x)
```

Bài tập 1:

Sử dụng dữ liệu **Iris** trong **sklearn.datasets**, hãy cho biết trường hợp kích thước hoa (dài/rộng đài, dài/rộng cánh) là (3cm/2.2cm, 4cm/3.5cm) thì nằm trong nhóm Setosa, Versicolor hay Virginica bằng phương pháp kNN với $k=3$, khoảng cách Euclidean?

Bài tập 2:

Cho dữ liệu thông tin về một loài sinh vật¹ ở bảng dưới đây. Hãy cho biết dự báo cho trường hợp thứ 9.

No.	Color	Legs	Height	Smelly	Species
1.	White	3	Short	Yes	M
2.	Green	2	Tall	No	M
3.	Green	3	Short	Yes	M
4.	White	3	Short	Yes	M
5.	Green	2	Short	No	H
6.	White	2	Tall	No	H
7.	White	2	Tall	No	H
8.	White	2	Short	Yes	H
9.	Green	2	Tall	No	?

¹<https://www.youtube.com/watch?v=z8K-598fqSo>



Cú pháp sử dụng sklearn cho phương pháp kNN:

```
from sklearn import neighbors  
  
# n_neighbors là giá trị tham số k  
  
knn = neighbors.KNeighborsClassifier(n_neighbors=3)  
  
# Đào tạo knn với dữ liệu đã biết (x,y)  
knn.fit(x,y)  
  
# Dự báo cho dữ liệu mới x*  
knn.predict(x*)
```

Bài tập 3:

Cho dữ liệu thông tin về một loài sinh vật² ở bảng dưới đây.

No.	Color	Legs	Height	Smelly	Species
1.	White	3	Short	Yes	M
2.	Green	2	Tall	No	M
3.	Green	3	Short	Yes	M
4.	White	3	Short	Yes	M
5.	Green	2	Short	No	H
6.	White	2	Tall	No	H
7.	White	2	Tall	No	H
8.	White	2	Short	Yes	H

Hãy chuyển dữ liệu về dạng số.

²<https://www.youtube.com/watch?v=z8K-598fqSo>

Bài tập 4:

Cho dữ liệu thông tin về triệu trứng bệnh cúm³ ở bảng dưới đây. Hãy số hóa dữ liệu dưới đây:

Symptom	Occupation	Ailment
sneezing	nurse	flu
sneezing	farmer	hayfever
headache	builder	concussion
headache	builder	flu
sneezing	teacher	flu
headache	teacher	concussion

³users.sussex.ac.uk/~christ/crs/ml/lec02b.html

- Dữ liệu dạng số cần chuẩn hóa theo phân phối chuẩn (**StandardScaler**)

- Dữ liệu dạng số cần chuẩn hóa theo phân phối chuẩn (**StandardScaler**)
- Dữ liệu dạng số cần căn chỉnh về khoảng đơn vị $[0, 1]$ (**MinMaxScaler**, **MaxAbsScaler**)

- Dữ liệu dạng số cần chuẩn hóa theo phân phối chuẩn (**StandardScaler**)
- Dữ liệu dạng số cần căn chỉnh về khoảng đơn vị $[0, 1]$ (**MinMaxScaler**, **MaxAbsScaler**)
- Công cụ **StandardScaler**, **MinMaxScaler**, **MaxAbsScaler** trong package **sklearn.preprocessing**

Cho dữ liệu dưới đây:

X_1	X_2	Y
2	3	0
2.7	3.4	1
2.5	2.3	0
3	3	1
2.9	3.7	0
2.8	2	1
2.6	3.1	1
2.1	3.5	0
3.3	2.5	1

Hãy chuẩn hóa các trường thuộc tính X_1 và X_2 về dải đơn vị $[0, 1]$ hoặc theo phân phối chuẩn.

Bài tập 5:

Sử dụng dữ liệu `Iris` trong `sklearn.datasets`, hãy chuẩn hóa các trường thông tin về độ dài/rộng đài hoa, độ dài/rộng cánh hoa về phân phối chuẩn.

- Dữ liệu thu thập có thể bị thiếu cần phải bổ sung hoặc loại bỏ cho quá trình xử lý tiếp theo

- Dữ liệu thu thập có thể bị thiếu cần phải bổ sung hoặc loại bỏ cho quá trình xử lý tiếp theo
- Bổ sung dữ liệu vào bản ghi có trường dữ liệu thiếu `SimpleImputer`, `IterativeImputer` trong package `sklearn.impute`

- Dữ liệu thu thập có thể bị thiếu cần phải bổ sung hoặc loại bỏ cho quá trình xử lý tiếp theo
- Bổ sung dữ liệu vào bản ghi có trường dữ liệu thiếu `SimpleImputer`, `IterativeImputer` trong package `sklearn.impute`
- Loại bỏ bản ghi có trường dữ liệu mất mát bằng công cụ `pandas.DataFrame.dropna()`

Cho dữ liệu dưới đây:

X_1	X_2	X_3
2	3	4
2	4	1
5	2.3	0
3	3	1
2	7	5
8		1
	3	1
2	3	0
3	5	7

Hãy bổ sung dữ liệu bị thiếu để được bộ hoàn chỉnh cho các bước xử lý sau.

Bài tập 6:

Cho dữ liệu phát hiện gian lận ngân hàng⁴. Hãy thực hiện các công việc sau:

1. Hãy đọc dữ liệu từ file excel.
2. Số hóa các cột dữ liệu text.
3. Điền thêm các bản ghi có dữ liệu bị thiếu sử dụng `sklearn.impute`.
4. Chuẩn hóa các các cột dữ liệu thuộc tính sau về khoảng $[0, 1]$: `Transaction_Amount`, `Time_of_Transaction`, `Account_Age`, `Number_of_Transactions_Last_24H`

⁴<https://www.kaggle.com/datasets/ranjitmandal/fraud-detection-dataset-csv>



BÀI 3

HỌC CÓ GIÁM SÁT

Cây quyết định (Decision Tree) và Đánh giá hiệu quả phân lớp



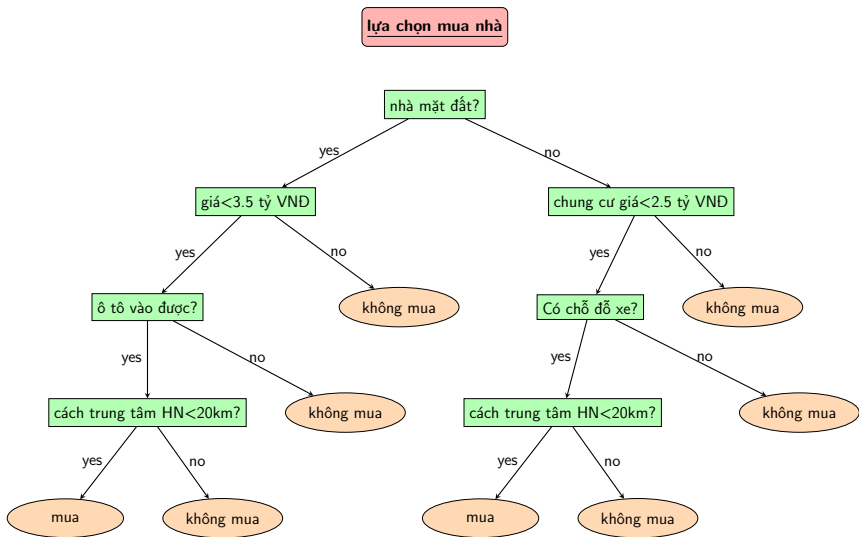
- ▶ Phương pháp Cây quyết định (Decision Tree)



- ▶ Phương pháp Cây quyết định (Decision Tree)
- ▶ Độ đo đánh giá hiệu quả phân lớp



- ▶ Phương pháp Cây quyết định (Decision Tree)
- ▶ Độ đo đánh giá hiệu quả phân lớp
- ▶ Thực hành trên Python



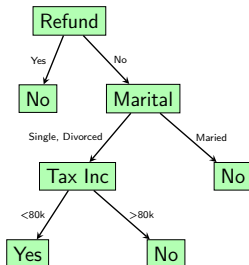
Cho bảng dữ liệu sau¹

ID	Refund	Marital Status	Taxable Income	Cheat
1	yes	single	125k	no
2	no	married	100k	no
3	no	single	70k	no
4	yes	married	120k	no
5	no	divorced	95k	yes
6	no	married	60k	no
7	yes	divorced	220k	no
8	no	single	85k	yes
9	no	married	75k	no
10	no	single	90k	yes

¹ Introduction to Data Mining, Pang-Ning Tan & et al.

Cho bảng dữ liệu sau¹

ID	Refund	Marital Status	Taxable Income	Cheat
1	yes	single	125k	no
2	no	married	100k	no
3	no	single	70k	no
4	yes	married	120k	no
5	no	divorced	95k	yes
6	no	married	60k	no
7	yes	divorced	220k	no
8	no	single	85k	yes
9	no	married	75k	no
10	no	single	90k	yes



¹ Introduction to Data Mining, Pang-Ning Tan & et al.



1. Mỗi cây gồm nút, nhánh và nút lá.



1. Mỗi cây gồm nút, nhánh và nút lá.
2. Nút thể hiện thuộc tính, nhánh thể hiện giá trị hoặc khoảng giá trị của nút nó xuất phát. Nút lá thể hiện nhãn của dữ liệu hay giá trị của dữ liệu và là nút cuối cùng không có nhánh.

1. Mỗi cây gồm **nút**, **nhánh** và **nút lá**.
2. Nút thể hiện thuộc tính, nhánh thể hiện giá trị hoặc khoảng giá trị của nút nó xuất phát. Nút lá thể hiện nhãn của dữ liệu hay giá trị của dữ liệu và là nút cuối cùng không có nhánh.
3. Với cây quyết định được xây dựng, mỗi mẫu dữ liệu sẽ được phân lớp bằng cách duyệt cây từ nút gốc đến một nút lá, nút lá đó là nhãn của mẫu dữ liệu đã cho.

Hãy xây dựng cây quyết định từ bảng dữ liệu sau bắt đầu từ nút **Marital Status**

ID	Refund	Marital Status	Taxable Income	Cheat
1	yes	single	125k	no
2	no	married	100k	no
3	no	single	70k	no
4	yes	married	120k	no
5	no	divorced	95k	yes
6	no	married	60k	no
7	yes	divorced	220k	no
8	no	single	85k	yes
9	no	married	75k	no
10	no	single	90k	yes



- Bước 1:** Lựa chọn nút xuất phát là thuộc tính có thể phân chia nhánh tốt nhất dựa trên các độ đo như **Information Gain (Entropy)**, **GINI index** được xác định từ tập dữ liệu tương ứng.
- Bước 2:** Nhánh tương ứng với giá trị/khoảng giá trị của nút lựa chọn. Mỗi nhánh sẽ xác định tập con dữ liệu dựa theo giá trị của nút.
- Bước 3:** Thực hiện quá trình lặp ở **Bước 1** cho lựa chọn nút tiếp theo với tập con dữ liệu mới dựa trên giá trị của nhánh đó cho đến khi gặp điều kiện dừng.
- Bước 4:** Điều kiện dừng bao gồm: khi các tập con có cùng nhãn, hoặc khi không còn thuộc tính nào để thực hiện phân nhánh tiếp, hoặc điều kiện xác định trước được thêm vào.

Định nghĩa

Giả sử dữ liệu $D = \{(x_i, y_i)\}_{i=1\dots n}$ có tập các nhãn

$Label(D) = \{1, \dots, k\}$. D_i là tập con của D có nhãn là i . Khi đó Entropy của tập D được xác định bởi:

$$Entropy(D) = - \sum_{i=1}^k \frac{|D_i|}{|D|} \log_2 \frac{|D_i|}{|D|}$$

Định nghĩa

Giả sử dữ liệu $D = \{(x_i, y_i)\}_{i=1\dots n}$ có tập các nhãn

$\text{Label}(D) = \{1, \dots, k\}$. D_i là tập con của D có nhãn là i . Khi đó Entropy của tập D được xác định bởi:

$$\text{Entropy}(D) = - \sum_{i=1}^k \frac{|D_i|}{|D|} \log_2 \frac{|D_i|}{|D|}$$

- $0 \leq \text{Entropy}(D) \leq 1$

Định nghĩa

Giả sử dữ liệu $D = \{(x_i, y_i)\}_{i=1\dots n}$ có tập các nhãn $\text{Label}(D) = \{1, \dots, k\}$. D_i là tập con của D có nhãn là i . Khi đó Entropy của tập D được xác định bởi:

$$\text{Entropy}(D) = - \sum_{i=1}^k \frac{|D_i|}{|D|} \log_2 \frac{|D_i|}{|D|}$$

- $0 \leq \text{Entropy}(D) \leq 1$
- Nếu $\text{Entropy}(D) = 0$ thì tập D là thuần nhất nghĩa là các phần tử gần như cùng nhãn.

Định nghĩa

Giả sử dữ liệu $D = \{(x_i, y_i)\}_{i=1\dots n}$ có tập các nhãn $\text{Label}(D) = \{1, \dots, k\}$. D_i là tập con của D có nhãn là i . Khi đó Entropy của tập D được xác định bởi:

$$\text{Entropy}(D) = - \sum_{i=1}^k \frac{|D_i|}{|D|} \log_2 \frac{|D_i|}{|D|}$$

- $0 \leq \text{Entropy}(D) \leq 1$
- Nếu $\text{Entropy}(D) = 0$ thì tập D là thuần nhất nghĩa là các phần tử gần như cùng nhãn.
- Nếu $\text{Entropy}(D) = 1$ thì tập D có độ nhiễu cao nhất nghĩa là nhãn có khả năng xuất hiện như nhau.

Cho dữ liệu thông tin về xe ô tô bị mất cắp ở bảng:

TT	Màu sắc	Kiểu xe	Nguồn gốc	Bị trộm
1.	Đỏ	Thể thao	Nội địa	Có
2.	Cam	Thể thao	Nội địa	Không
3.	Cam	Thể thao	Nhập khẩu	Có
4.	Xanh	SUV	Nhập khẩu	Không
5.	Đỏ	SUV	Nhập khẩu	Không
6.	Đỏ	Thể thao	Nhập khẩu	Có
7.	Cam	Thể thao	Nội địa	Không
8.	Cam	SUV	Nội địa	Có
9.	Xanh	Thể thao	Nhập khẩu	Có

Tính Entropy của tập dữ liệu đã cho?

Định nghĩa

Giả sử A là một thuộc tính trong dữ liệu D cần xem xét, $Values(A)$ là tập các giá trị của A . Với mỗi $\nu \in Values(A)$, ký hiệu D_ν là tập con của D mà thuộc tính A nhận giá trị ν . Khi đó **Information Gain (IG)** của thuộc tính A trong dữ liệu D được xác định bởi:

$$IG(D, A) = Entropy(D) - \sum_{\nu \in Values(A)} \frac{|D_\nu|}{|D|} Entropy(D_\nu)$$

Định nghĩa

Giả sử A là một thuộc tính trong dữ liệu D cần xem xét, $Values(A)$ là tập các giá trị của A . Với mỗi $\nu \in Values(A)$, ký hiệu D_ν là tập con của D mà thuộc tính A nhận giá trị ν . Khi đó **Information Gain (IG)** của thuộc tính A trong dữ liệu D được xác định bởi:

$$IG(D, A) = Entropy(D) - \sum_{\nu \in Values(A)} \frac{|D_\nu|}{|D|} Entropy(D_\nu)$$

- Khi $IG(D, A)$ lớn thì mức độ thông tin tốt hơn khi rẽ nhánh theo thuộc tính A trong dữ liệu D .

Bài tập: Xây dựng cây quyết định theo thuật toán CLS dựa vào IG của các thuộc tính theo dữ liệu dưới đây:

TT	Màu sắc	Kiểu xe	Nguồn gốc	Bị trộm
1.	Đỏ	Thể thao	Nội địa	Có
2.	Cam	Thể thao	Nội địa	Không
3.	Cam	Thể thao	Nhập khẩu	Có
4.	Xanh	SUV	Nhập khẩu	Không
5.	Đỏ	SUV	Nhập khẩu	Không
6.	Đỏ	Thể thao	Nhập khẩu	Có
7.	Cam	Thể thao	Nội địa	Không
8.	Cam	SUV	Nội địa	Có
9.	Xanh	Thể thao	Nhập khẩu	Có

Định nghĩa

Giả sử dữ liệu D có tập các nhãn $\text{Label}(D) = \{1, \dots, k\}$. D_i là tập con của D có nhãn là i . Khi đó GINI của tập D được xác định bởi:

$$\text{GINI}(D) = 1 - \sum_{i=1}^k \left(\frac{|D_i|}{|D|} \right)^2$$

Định nghĩa

Giả sử dữ liệu D có tập các nhãn $\text{Label}(D) = \{1, \dots, k\}$. D_i là tập con của D có nhãn là i . Khi đó GINI của tập D được xác định bởi:

$$\text{GINI}(D) = 1 - \sum_{i=1}^k \left(\frac{|D_i|}{|D|} \right)^2$$

- Khi $\text{GINI}(D)$ phản ánh độ mất cân xứng giữa các nhãn trong dữ liệu D .

Định nghĩa

Giả sử A là một thuộc tính trong dữ liệu D cần xem xét, $Values(A)$ là tập các giá trị của A . Với mỗi $\nu \in Values(A)$, ký hiệu D_ν là tập con của D mà thuộc tính A nhận giá trị ν . Khi đó **GINI** của thuộc tính A trong dữ liệu D được xác định bởi:

$$GINI(D, A) = \sum_{\nu \in Values(A)} \frac{|D_\nu|}{|D|} GINI(D_\nu)$$

Định nghĩa

Giả sử A là một thuộc tính trong dữ liệu D cần xem xét, $Values(A)$ là tập các giá trị của A . Với mỗi $\nu \in Values(A)$, ký hiệu D_ν là tập con của D mà thuộc tính A nhận giá trị ν . Khi đó **GINI** của thuộc tính A trong dữ liệu D được xác định bởi:

$$GINI(D, A) = \sum_{\nu \in Values(A)} \frac{|D_\nu|}{|D|} GINI(D_\nu)$$

- Khi $GINI(D, A)$ lớn thì mức độ thông tin tốt hơn khi rẽ nhánh theo thuộc tính A trong dữ liệu D .

Xây dựng cây quyết định theo thuật toán CLS dựa vào GINI của các thuộc tính theo dữ liệu dưới đây:

TT	Màu sắc	Kiểu xe	Nguồn gốc	Bị trộm
1.	Đỏ	Thể thao	Nội địa	Có
2.	Cam	Thể thao	Nội địa	Không
3.	Cam	Thể thao	Nhập khẩu	Có
4.	Xanh	SUV	Nhập khẩu	Không
5.	Đỏ	SUV	Nhập khẩu	Không
6.	Đỏ	Thể thao	Nhập khẩu	Có
7.	Cam	Thể thao	Nội địa	Không
8.	Cam	SUV	Nội địa	Có
9.	Xanh	Thể thao	Nhập khẩu	Có

Bài tập 1: Cho dữ liệu về cháy nắng ở bảng ²

Name	Hair	Height	Weight	Lotion	Result
Alex	Brown	Short	Average	Yes	None
Annie	Blonde	Short	Average	No	Burn
Danna	Blonde	Tall	Average	Yes	None
Emily	Red	Average	Heavy	No	Burn
John	Brown	Average	Heavy	No	None
Katie	Blonde	Short	Light	Yes	None
Pete	Brown	Tall	Heavy	No	None
Sarah	Blonde	Average	Light	No	Burn
Ann	Red	Average	Heavy	Yes	?

1. Hãy xây dựng cây quyết định và cho biết kết quả nhãn của bản ghi chưa biết.
2. Lập chương trình xử lý trong Python dùng `sklearn.tree`

²Data Mining: Practical Machine Learning Tools and Techniques, Ian H. Witten & Eibe Frank

Bài tập 2: Tìm nhãn của bản ghi cuối chưa biết dưới đây bằng cây quyết định.

TT	Màu sắc	Kiểu xe	Nguồn gốc	Bị trộm
1.	Đỏ	Thể thao	Nội địa	Có
2.	Cam	Thể thao	Nội địa	Không
3.	Cam	Thể thao	Nhập khẩu	Có
4.	Xanh	SUV	Nhập khẩu	Không
5.	Đỏ	SUV	Nhập khẩu	Không
6.	Đỏ	Thể thao	Nhập khẩu	Có
7.	Cam	Thể thao	Nội địa	Không
8.	Cam	SUV	Nội địa	Có
9.	Xanh	Thể thao	Nhập khẩu	Có
10.	Đỏ	SUV	Nội địa	?

Bài tập 3: Cho dữ liệu thông tin về một loài sinh vật³ ở bảng dưới đây.
Tìm nhãn của bản ghi cuối chưa biết bằng cây quyết định theo dữ liệu đã biết trên đó.

No.	Color	Legs	Height	Smelly	Species
1.	White	3	Short	Yes	M
2.	Green	2	Tall	No	M
3.	Green	3	Short	Yes	M
4.	White	3	Short	Yes	M
5.	Green	2	Short	No	H
6.	White	2	Tall	No	H
7.	White	2	Tall	No	H
8.	White	2	Short	Yes	H
9.	Green	2	Tall	No	?

³<https://www.youtube.com/watch?v=z8K-598fqSo>

Bài tập 4: Cho dữ liệu thông tin về triệu chứng bệnh cúm⁴ ở bảng dưới đây. Tìm nhãn xây dựng cây quyết định dựa trên Information Gain và GINI.

Symptom	Occupation	Ailment
sneezing	nurse	flu
sneezing	farmer	hayfever
headache	builder	concussion
headache	builder	flu
sneezing	teacher	flu
headache	teacher	concussion

⁴users.sussex.ac.uk/~christ/crs/ml/lec02b.html

HỌC CÂY QUYẾT ĐỊNH

DECISION TREE LEARNING

Đã học

- Phương pháp học có giám sát, $D = (X, Y)$,,
 - K-láng giềng gần nhất (k-NN)
 - Học cây quyết định (Decision tree learning, DT)

NỘI DUNG

- Giới thiệu Decision Tree
- Giải thuật học cây quyết định
- Độ lợi thông tin và lựa chọn thuộc tính
- Các phiên bản cải tiến C4.5 và CART
- Đánh giá hiệu năng

Giới thiệu

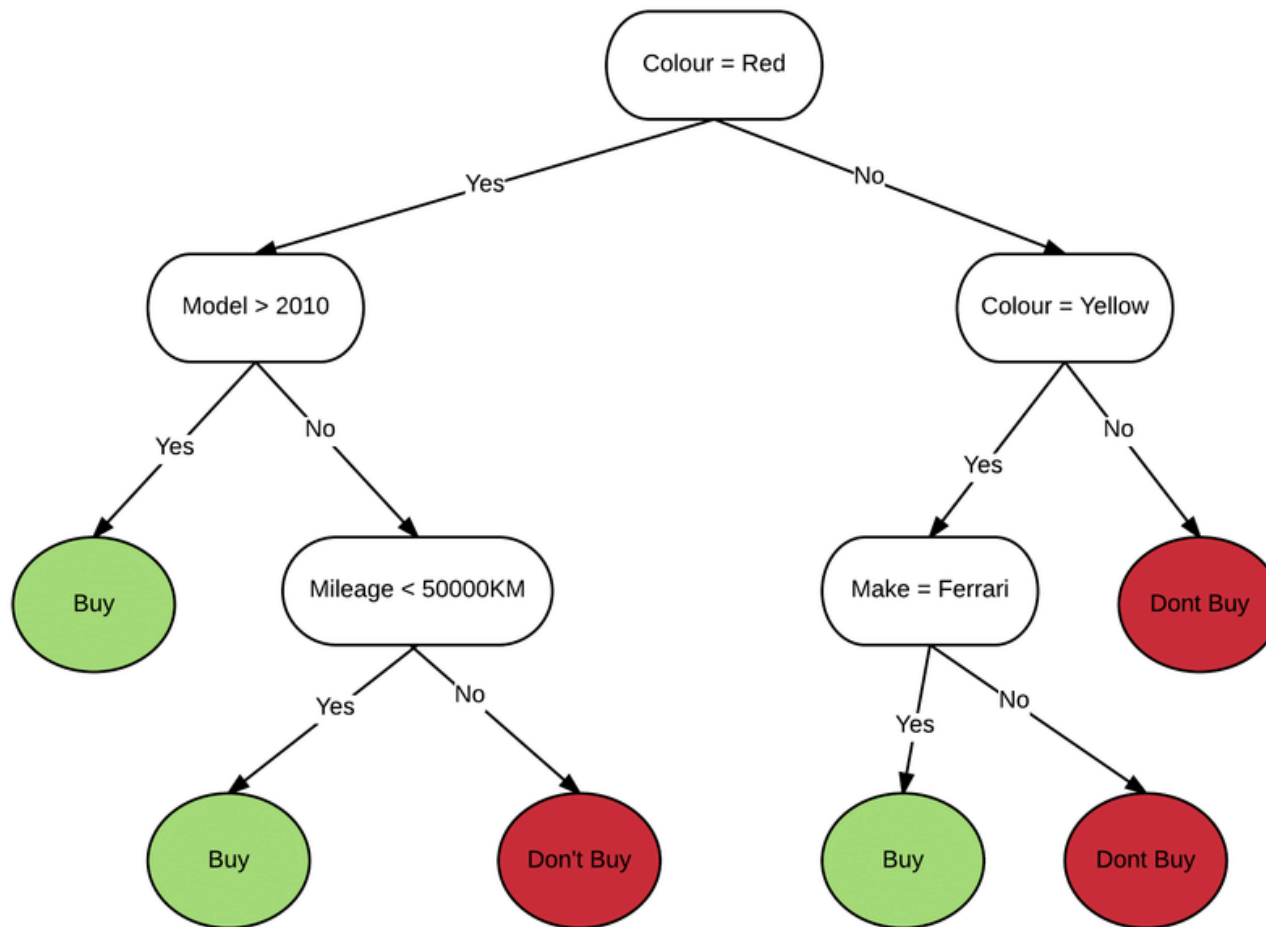
Cây trong tự nhiên



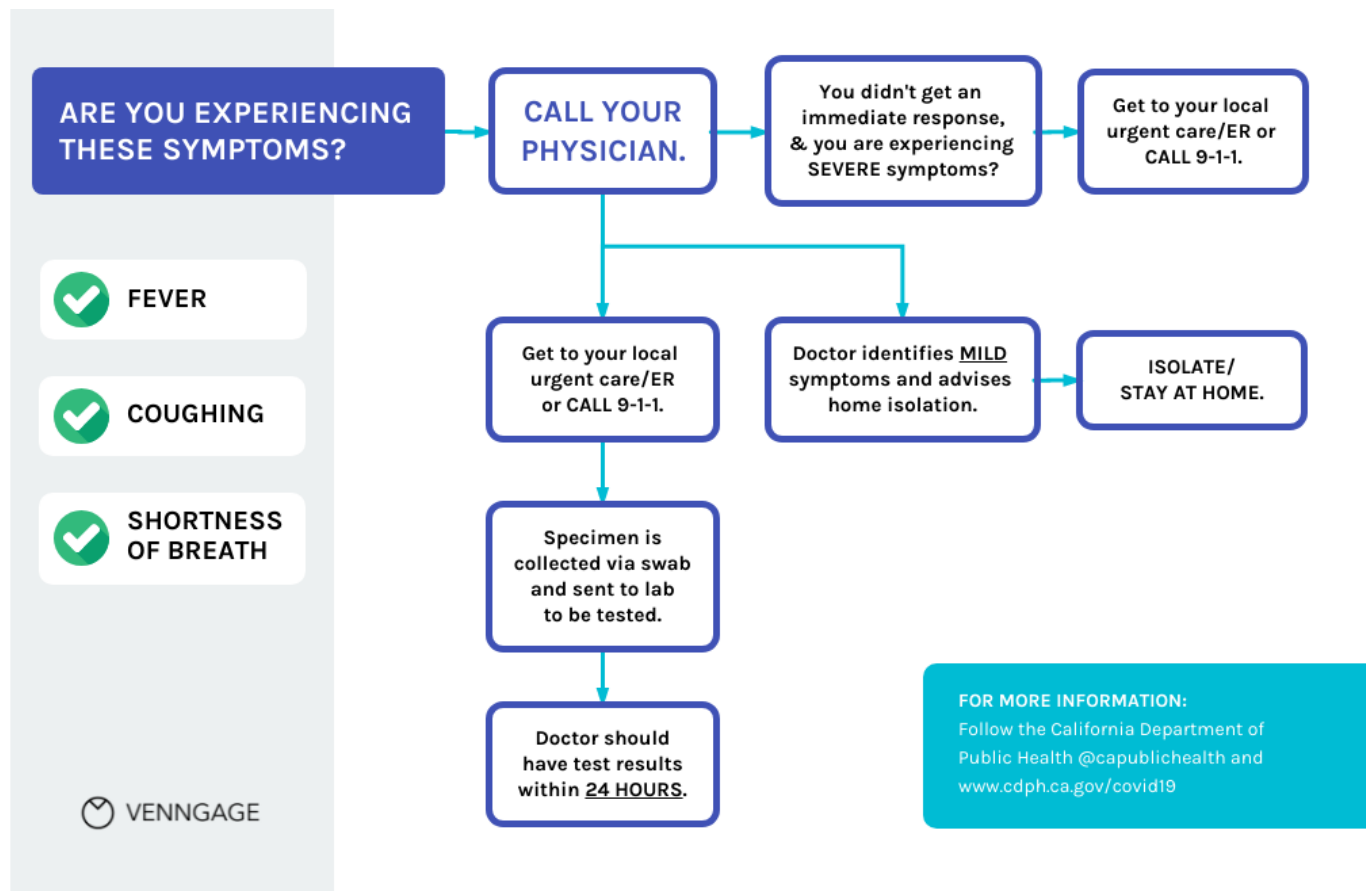
Hình: nguồn internet



Cây quyết định: mua xe?



Cây quyết định: test covid?



Trước đây, cây được tạo ra bằng cách vẽ tay



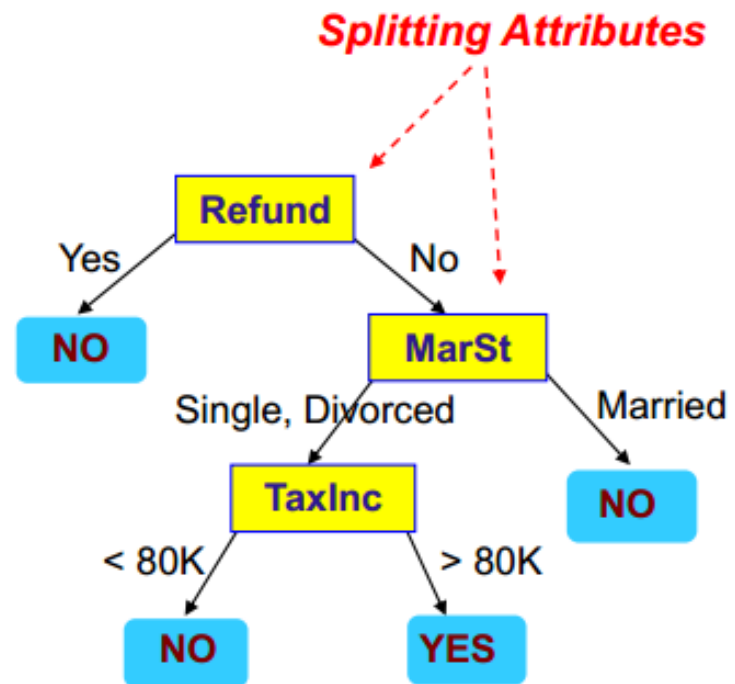
By User:Gokul Jadhav - Own work, Public Domain,
<https://commons.wikimedia.org/w/index.php?curid=4404547>

Cây trong học máy:

Dựng mô hình cây từ dữ liệu

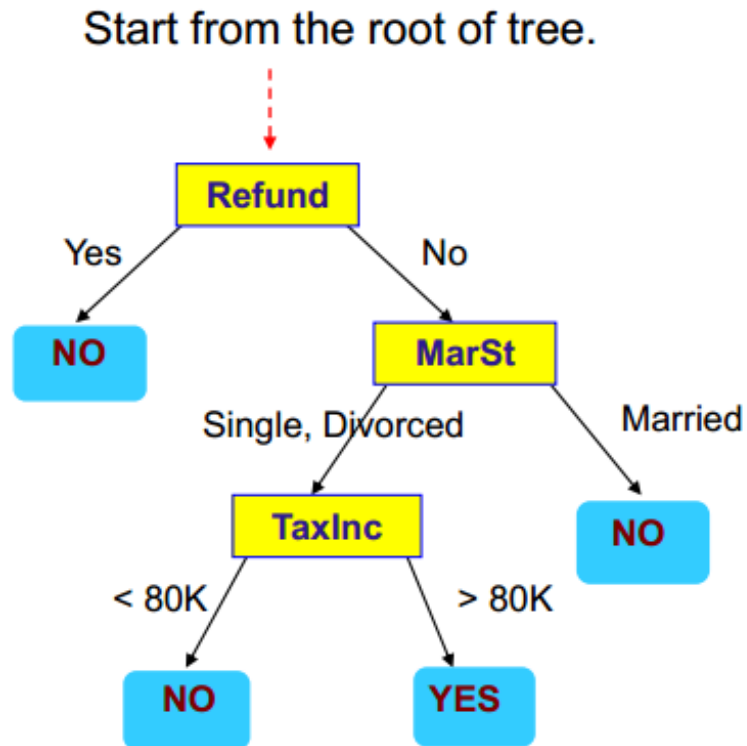
<i>Tid</i>	<i>Refund</i>	<i>Marital Status</i>	<i>Taxable Income</i>	<i>Cheat</i>
1	Yes	Single	125K	No
2	No	Married	100K	No
3	No	Single	70K	No
4	Yes	Married	120K	No
5	No	Divorced	95K	Yes
6	No	Married	60K	No
7	Yes	Divorced	220K	No
8	No	Single	85K	Yes
9	No	Married	75K	No
10	No	Single	90K	Yes

Training Data



Model: Decision Tree

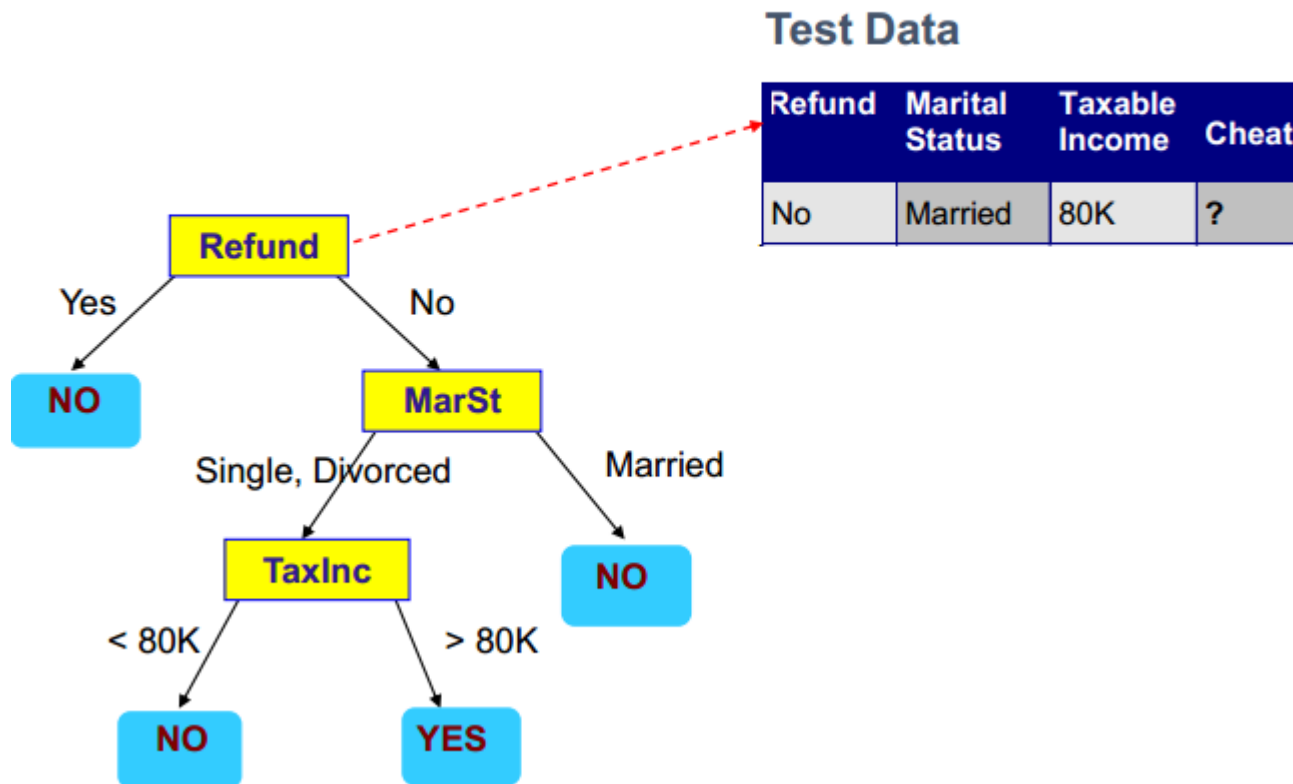
- Ví dụ: hỗ trợ ra quyết định
ứng dụng mô hình vào dự đoán nhãn của dữ liệu mới



Test Data

Refund	Marital Status	Taxable Income	Cheat
No	Married	80K	?

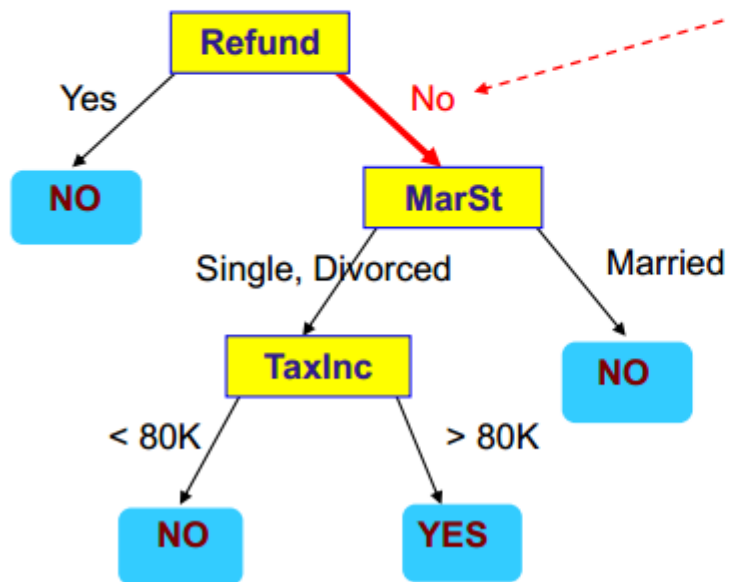
Apply Model to Test Data



Apply Model to Test Data

Test Data

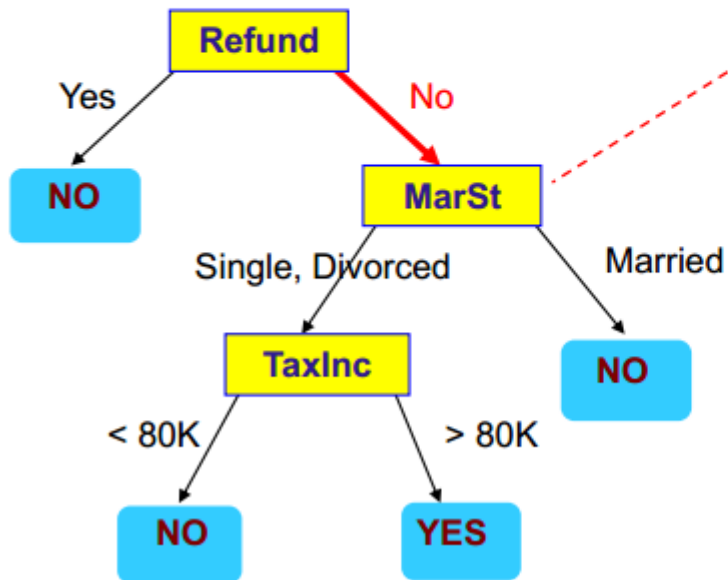
Refund	Marital Status	Taxable Income	Cheat
No	Married	80K	?



Apply Model to Test Data

Test Data

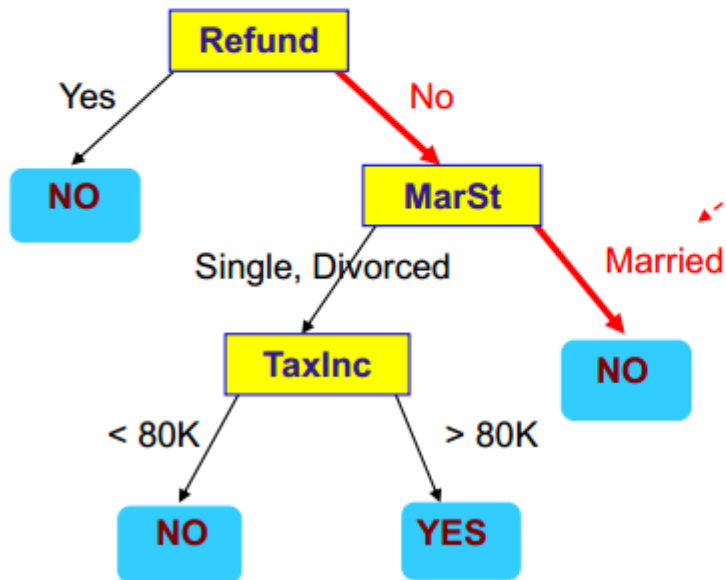
Refund	Marital Status	Taxable Income	Cheat
No	Married	80K	?



Apply Model to Test Data

Test Data

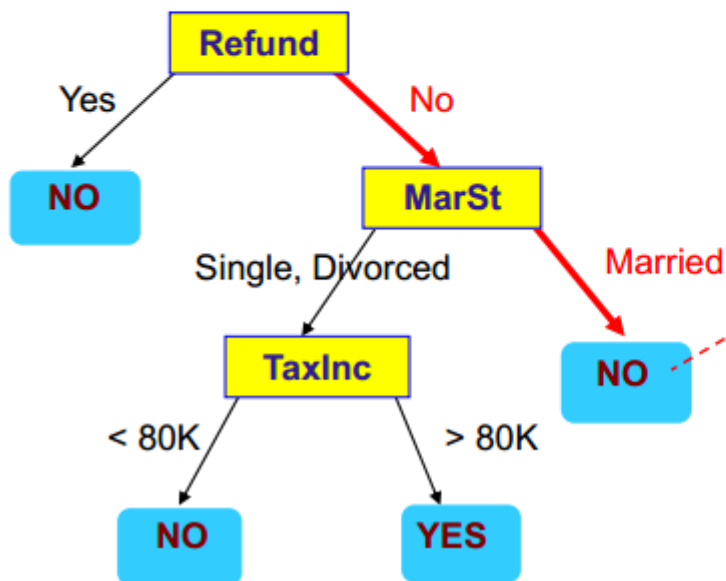
Refund	Marital Status	Taxable Income	Cheat
No	Married	80K	?



Apply Model to Test Data

Test Data

Refund	Marital Status	Taxable Income	Cheat
No	Married	80K	?



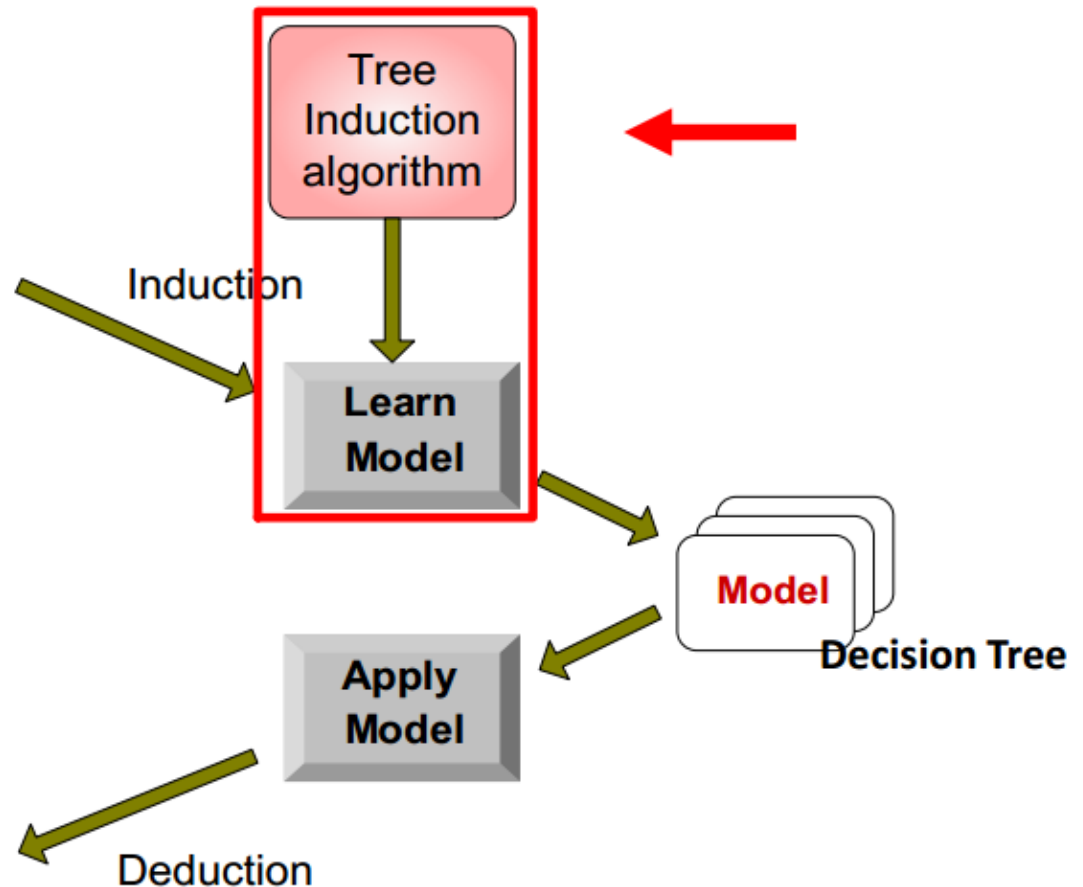
Assign Cheat to "No"

Cây quyết định cho phân lớp dữ liệu

Tid	Attrib1	Attrib2	Attrib3	Class
1	Yes	Large	125K	No
2	No	Medium	100K	No
3	No	Small	70K	No
4	Yes	Medium	120K	No
5	No	Large	95K	Yes
6	No	Medium	60K	No
7	Yes	Large	220K	No
8	No	Small	85K	Yes
9	No	Medium	75K	No
10	No	Small	90K	Yes

Training Set

Tid	Attrib1	Attrib2	Attrib3	Class
11	No	Small	55K	?
12	Yes	Medium	80K	?
13	Yes	Large	110K	?
14	No	Small	95K	?
15	No	Large	67K	?



Cây quyết định

- Cây quyết định (Decision tree): Là một công cụ phổ biến sử dụng mô hình dạng cây để phân lớp và dự báo
- Tốc độ học tương đối nhanh so với các phương pháp khác
- Cây có thể được chuyển thành tập luật một cách dễ dàng
- Độ chính xác khá tốt
- Được áp dụng thành công trong nhiều bài toán ứng dụng thực tế.

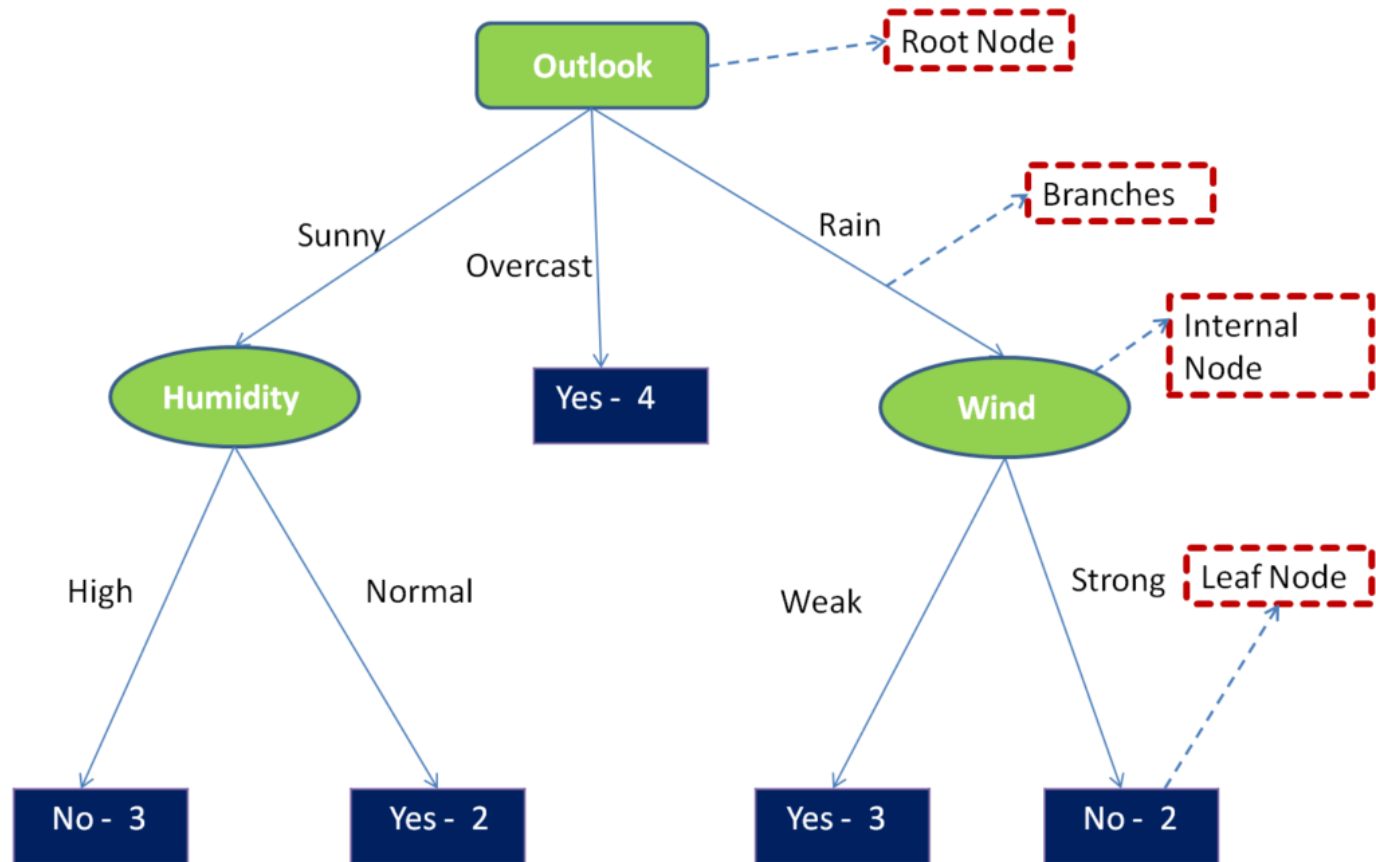
Học cây quyết định

- Học cây quyết định (Decision tree learning)
 - Để học (xấp xỉ) một hàm mục tiêu (target function). Nếu hàm mục tiêu có giá trị:
 - rời rạc (discrete valued) $\rightarrow$ gọi là hàm phân lớp
 - liên tục $\rightarrow$ hàm hồi quy
 - Hàm phân lớp được biểu diễn bởi một cây quyết định
- Cây quyết định có thể học trên tập dữ liệu có chứa nhiễu/lỗi (noisy data).

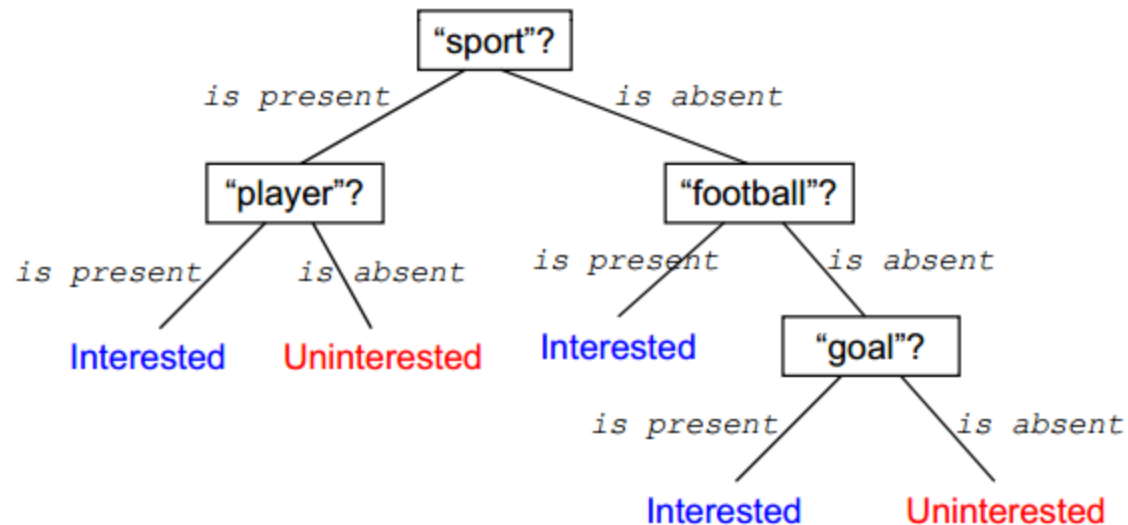
Biểu diễn DT

- Mỗi nút trong (internal node) biểu diễn *một thuộc tính* của dữ liệu cần kiểm tra (test) các giá trị
- Mỗi nhánh (branch) từ một nút tương ứng với một giá trị có thể của thuộc tính gắn với nút đó
- Mỗi nút lá (leaf node) biểu diễn 1 nhãn lớp (a class label)
- Một cây quyết định học được sẽ phân lớp đối với một mẫu, bằng cách duyệt cây từ nút gốc đến một nút lá → nhãn lớp gắn với nút lá đó sẽ được gán cho mẫu dữ liệu cần được phân lớp.

- Ví dụ DT



- Vd: Những tin tức được quan tâm?

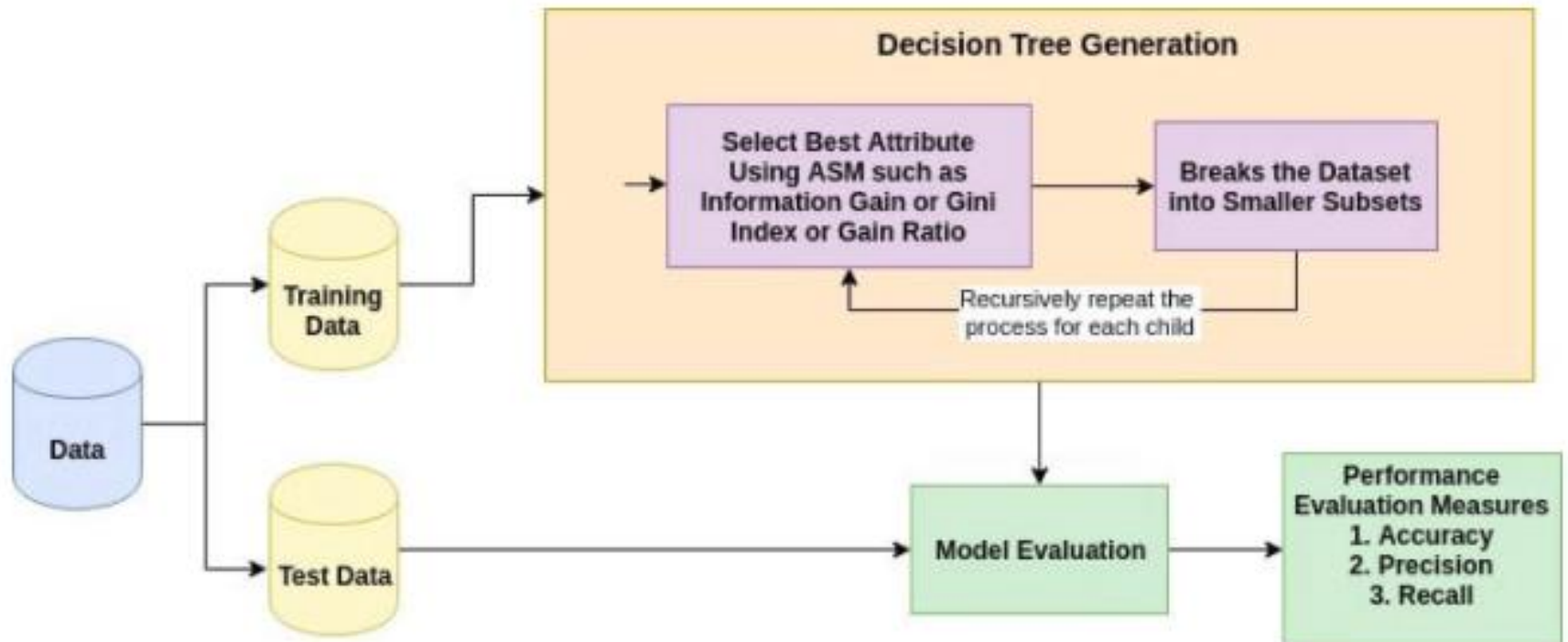


$[("sport" \text{ is present}) \wedge ("player" \text{ is present})] \vee$

$[("sport" \text{ is absent}) \wedge ("football" \text{ is present})] \vee$

$[("sport" \text{ is absent}) \wedge ("football" \text{ is absent}) \wedge ("goal" \text{ is present})]$

Dựng cây quyết định



Source: DataCamp

Dựng cây quyết định

- Xây dựng cây: thực hiện đệ quy chia tập mẫu dữ liệu huấn luyện cho đến khi các mẫu ở mỗi nút lá thuộc cùng một lớp.
 - Các mẫu huấn luyện xuất phát nằm ở nút gốc
 - Chọn một thuộc tính để phân chia tập mẫu huấn luyện thành các nhánh
 - Tiếp tục lặp việc xây dựng cây quyết định cho các nhánh, quá trình dừng khi:
 - Tất cả các mẫu đều được phân lớp
 - Không còn thuộc tính nào có thể dùng để chia mẫu

Các giải thuật xây dựng DT

- Phân loại:
 - **CLS** (Concept Learning System)
 - **ID3** (Iterative Dichotomiser 3)
 - C4.5
- Hồi quy: CART

Giải thuật CLS

- B1. Tạo một nút gốc T gồm tất cả các mẫu huấn luyện
- B2. Nếu tất cả các mẫu trong T có nhãn “Yes” thì gán nhãn nút T là “Yes” và dừng.
- B3. Nếu tất cả các mẫu trong T có nhãn “No” thì gán nhãn nút T là “No” và dừng.
- B4. Nếu các mẫu trong T có cả “Yes” và “No” thì
 - Chọn một thuộc tính A có các giá trị $a_1, \dots, a_n$
 - Chia tập mẫu theo giá trị của A thành các tập con $T_1, \dots, T_n$.
 - Tạo n nút con T_i ($i=1..n$) với nút cha là nút T
- B5. Thực hiện lặp lại cho các nút con T_i ($i=1..n$) và quay lại B2.

Giải thuật CLS

- Nhận xét:
 - Tại bước 4 có thể chọn 1 thuộc tính tùy ý
 - Thứ tự các thuộc tính khác nhau sẽ cho ra cây có hình dạng khác nhau
 - Việc lựa chọn thuộc tính sẽ ảnh hưởng đến độ sâu, độ rộng, và độ phức tạp của cây.

Một số khái niệm

- Gọi S là tập mẫu dữ liệu cần xử lý, S sẽ được phân thành m lớp $\{C_1, C_2, \dots, C_m\}$
- $|C_i|$: lực lượng của C_i , đặt $s_i = |C_i|$, $s = |S|$
- A tập các thuộc tính $\{a_1, \dots, a_n\}$
- Phân hoạch S theo thuộc tính A thành n tập $\{S_1, S_2, \dots, S_n\}$.
- s_{ij} là số phần tử của lớp C_i trong tập S_j

Entropy – độ đo thông tin

- Entropy của tập thông tin S , ký hiệu $E(S)$ hay $E(s_1, \dots, s_n)$

$$E(S) = E(s_1, s_2, \dots, s_m) = - \sum_{i=1}^m \frac{s_i}{s} \log_2 \frac{s_i}{s}$$

Entropy(S) = 0 : tập thông tin S là thuần nhất

Entropy(S) = 1 : tập thông tin S là độ nhiễu cao nhất

$0 < \text{Entropy}(S) < 1$: tập thông tin S có số lượng mẫu thuộc các loại khác nhau

- Entropy của thuộc tính A ứng với tập S

$$E(S, A) = \sum_{j=1}^n \frac{s_{1j} + s_{2j} + \dots + s_{mj}}{s} E(S_j);$$

$$E(S_j) = \sum_{i=1}^m - \frac{s_{ij}}{|S_j|} \log_2 \frac{s_{ij}}{|S_j|}$$

Information Gain

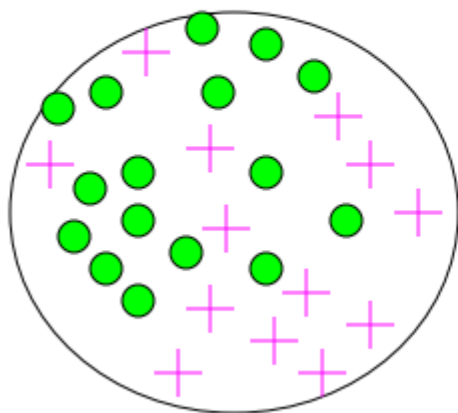
Impurity/Entropy (informal)

- Measures the level of **impurity** in a group of examples

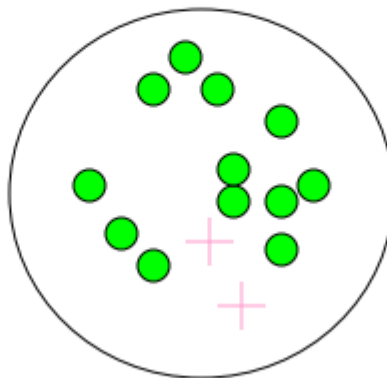


- Impurity

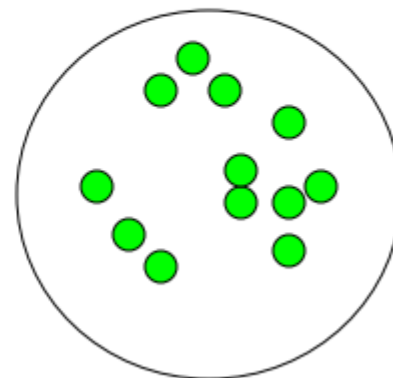
Very impure group



Less impure



**Minimum
impurity**



Information Gain – độ lợi thông tin

- Information Gain hay Gain: là thông tin thu được (độ giảm entropy) khi thực hiện (biết) phân nhánh
là đại lượng xác định hiệu xuất phân lớp các mẫu dữ liệu ứng với mỗi thuộc tính

$$\text{Gain}(S,A) = E(S) - E(S,A)$$

Giải thuật ID3 – ý tưởng

- Thực hiện giải thuật tìm kiếm tham lam (greedy search) đối với không gian các cây quyết định
- Xây dựng (học) một cây quyết định theo chiến lược top-down, bắt đầu từ nút gốc
- Ở mỗi nút, thuộc tính kiểm tra là thuộc tính có *khả năng phân loại tốt nhất* đối với các ví dụ học gắn với nút đó.
- Tạo mới một cây con (sub-tree) của nút hiện tại cho mỗi giá trị có thể của thuộc tính kiểm tra, và *tập học sẽ được tách ra* (thành các tập con) tương ứng với cây con vừa tạo
- Mỗi thuộc tính chỉ được phép xuất hiện tối đa 1 lần đối với bất kỳ một đường đi nào trong cây
- Quá trình phát triển (học) cây quyết định sẽ tiếp tục cho đến khi thỏa mãn điều kiện dừng (các mẫu đã được phân loại, hoặc không còn thuộc tính nào).

Giải thuật ID3

ID3 cho phân lớp nhị phân

- B1. Tạo một nút T gồm tất cả các mẫu huấn luyện
- B2. Nếu tất cả các mẫu trong T có nhãn “Yes” thì gán nhãn nút T là “Yes” và dừng.
- B3. Nếu tất cả các mẫu trong T có nhãn “No” thì gán nhãn nút T là “No” và dừng.
- B4. Nếu mẫu trong T có cả “Yes” và “No” thì
 - Chọn thuộc tính A có $\text{Gain}(S,A)$ lớn nhất để phân nhánh
- B5. Thực hiện lặp cho các nút con T_i ($i=1..n$) và quay lại B2.

Ví dụ: S

	Quang cảnh	Nhiệt độ	Độ ẩm	Gió	Chơi tennis
D1	Nắng	Nóng	Cao	Nhẹ	Không
D2	Nắng	Nóng	Cao	Mạnh	Không
D3	Âm u	Nóng	Cao	Mạnh	Có
D4	Mưa	Ấm áp	Cao	Nhẹ	Có
D5	Mưa	Mát	TB	Nhẹ	Có
D6	Mưa	Mát	TB	Mạnh	Không
D7	Âm u	Mát	TB	Mạnh	Có
D8	Nắng	Ấm áp	Cao	Nhẹ	Không
D9	Nắng	Mát	TB	Nhẹ	Có
D10	Mưa	Ấm áp	TB	Nhẹ	Có
D11	Nắng	Ấm áp	TB	Mạnh	Có
D12	Âm u	Ấm áp	Cao	Mạnh	Có
D13	Âm u	Nóng	TB	Nhẹ	Có
D14	Mưa	Ấm áp	Cao	Mạnh	Không

$$\text{Entropy}(S) = -\left(\frac{9}{14}\right)\log_2\left(\frac{9}{14}\right) - \left(\frac{5}{14}\right)\log_2\left(\frac{5}{14}\right) = 0.94$$

$$\text{Gain}(S, \text{Quang cảnh}) =$$

$$E(S) - \left(\frac{5}{14}\right)E(S_{\text{Nắng}}) - \left(\frac{4}{14}\right)E(S_{\text{Âm_u}}) - \left(\frac{5}{14}\right)E(S_{\text{Mưa}}) = 0.94 - \left(\frac{5}{14}\right)0.971 - \left(\frac{4}{14}\right)0.0 - \left(\frac{5}{14}\right)0.0971 = 0.247$$

$$\text{Gain}(S, \text{Nhiệt độ}) = 0.029$$

$$\text{Gain}(S, \text{Độ ẩm}) = 0.151$$

$$\text{Gain}(S, \text{Gió}) = 0.048$$

$$\text{Entropy}(S_{\text{Nắng}}) = -\left(\frac{2}{5}\right)\log_2\left(\frac{2}{5}\right) - \left(\frac{3}{5}\right)\log_2\left(\frac{3}{5}\right) = 0.97$$

$$\text{Gain}(S_{\text{Nắng}}, \text{Nhiệt độ}) = 0.57$$

$$\text{Gain}(S_{\text{Nắng}}, \text{Độ ẩm}) = 0.97$$

$$\text{Gain}(S_{\text{Nắng}}, \text{Gió}) = 0.019$$

Giải thuật ID3

Chú ý đối với **thuộc tính liên tục** ta sử dụng chỉ số GINI

$$GINI(t) = 1 - \sum_j p^2(j/t)$$

$p(j/t)$ là tần suất của lớp j trong nút t

max = $1 - 1/n$ khi các mẫu phân bố đều trên các lớp

min = 0 khi các mẫu thuộc về một lớp

Khi chia nút p thành k nhánh, chất lượng của phép chia

$$GINI_{chia} = \sum_{i=1}^k \frac{n_i}{n} GINI(i)$$

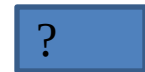
Trong đó n_i là số mẫu trong nút i ; n là số mẫu trong nút p .

Ví dụ: tập dữ liệu Sunburn

Name	Hair	Height	Weight	Lotion	Result
Sarah	Blonde	Average	Light	No	Burn
Dana	Blonde	Tall	Average	Yes	None
Alex	Brown	Short	Average	Yes	None
Annie	Blonde	Short	Average	No	Burn
Emily	Red	Average	Heavy	No	Burn
Pete	Brown	Tall	Heavy	No	None
John	Brown	Average	Heavy	No	None
Katie	Blonde	Short	Light	Yes	None

Name	Hair	Height	Weight	Lotion	Result
Sarah	Blonde	Average	Light	No	Burn
Dana	Blonde	Tall	Average	Yes	None
Alex	Brown	Short	Average	Yes	None
Annie	Blonde	Short	Average	No	Burn
Emily	Red	Average	Heavy	No	Burn
Pete	Brown	Tall	Heavy	No	None
John	Brown	Average	Heavy	No	None
Katie	Blonde	Short	Light	Yes	None

Entropy:
 $H(S) = -\sum_i p(c_i) \log p(c_i)$



$$\begin{aligned}
 \text{InfoGain}(S, A) &= H(S) - H(S \mid A) \\
 &= H(S) - \sum_a p(A = a) H(S \mid A = a) \\
 &= H(S) - \sum_{a \in \text{Values}(A)} \frac{|S_a|}{|S|} H(S_a)
 \end{aligned}$$

- Bài tập: sử dụng độ đo Information Gain để chọn thuộc tính tốt nhất để
 1. Dựng cây quyết định cho Dữ liệu ở Slide 4
 2. Dựng cây quyết định cho Dữ liệu Sunburn

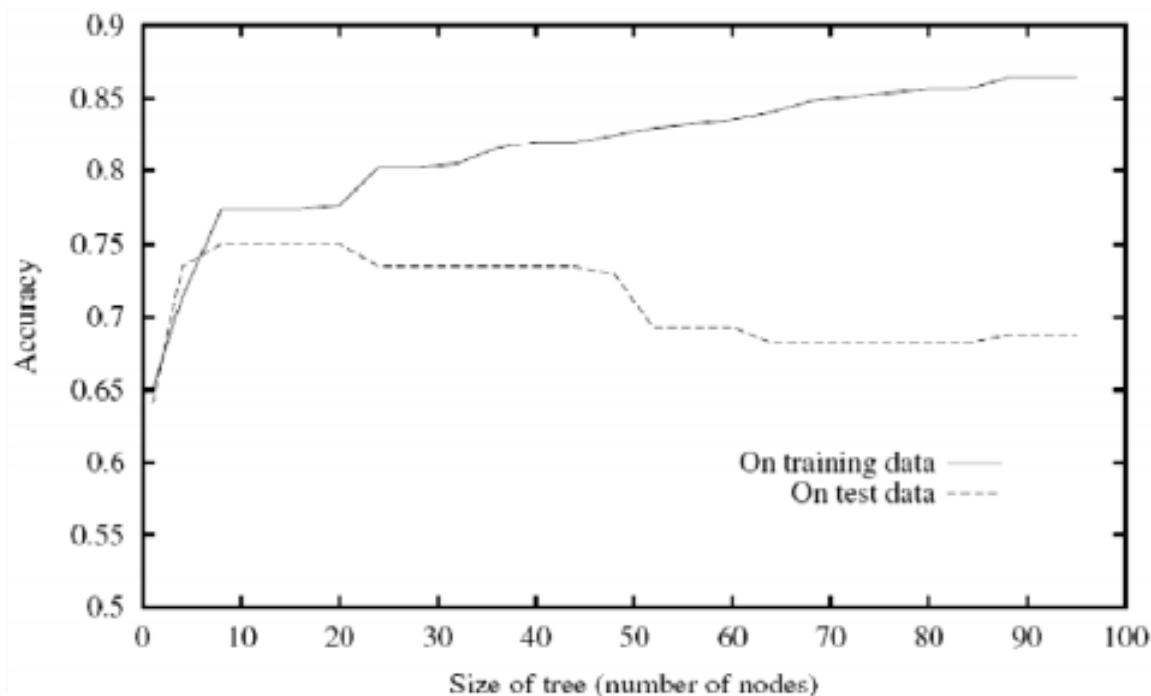
Nộp lên folder của mình trên Google drive.

Một số vấn đề

- Cây quyết định học được quá phù hợp (over-fit) với các mẫu dữ liệu huấn luyện
 - Xử lý các thuộc tính có kiểu giá trị liên tục (kiểu số thực)
 - Các đánh giá phù hợp hơn (tốt hơn Information Gain) đối với việc xác định thuộc tính kiểm tra cho một nút
 - Xử lý các ví dụ học thiếu giá trị thuộc tính (missing-value attributes)
 - Xử lý các thuộc tính có chi phí (cost) khác nhau
- Cải tiến của giải thuật ID3 với tất cả các vấn đề nêu trên được giải quyết: giải thuật C4.5

Học quá (over-fitting)

Tiếp tục quá trình học cây quyết định sẽ làm giảm độ chính xác đối với tập thử nghiệm mặc dù tăng độ chính xác đối với tập học



[Mitchell, 1997]

Tránh over-fitting

- 2 chiến lược
 - Ngừng việc học (phát triển) cây quyết định sớm hơn, trước khi nó đạt tới cấu trúc cây cho phép phân loại (khớp) hoàn hảo tập huấn luyện
 - Học (phát triển) cây đầy đủ (tương ứng với cấu trúc cây hoàn toàn phù hợp đối với tập huấn luyện), và sau đó thực hiện quá trình tỉa (to post-prune) cây ,,
- Chiến lược tỉa cây đầy đủ (Post-pruning over-fit trees) thường cho hiệu quả tốt hơn trong thực tế

Lý do: Chiến lược “ngừng sớm” việc học cây cần phải đánh giá chính xác được khi nào nên ngừng việc học (phát triển) cây – Khó xác định!

Các thuộc tính có giá trị liên tục

- Cần xác định (chuyển đổi thành) các thuộc tính có giá trị rời rạc, bằng cách chia khoảng giá trị liên tục thành một tập các khoảng (intervals) không giao nhau.
- Đối với thuộc tính (có giá trị liên tục) A , tạo một thuộc tính mới kiểu nhị phân A_v sao cho: A_v là đúng nếu $A > v$, và là sai nếu ngược lại
- Xác định giá trị ngưỡng v “tốt nhất”?
→ Chọn giá trị ngưỡng v nhằm sinh ra giá trị *Information Gain* cao nhất.

- Vấn đề với Information gain:
 - Có thiên hướng lựa chọn các thuộc tính có nhiều giá trị (attributes with many values).

Một đánh giá khác: Gain Ratio

→ Giảm ảnh hưởng của các thuộc tính có (rất) nhiều giá trị

$$GainRatio(S, A) = \frac{Gain(S, A)}{SplitInformation(S, A)}$$

$$SplitInformation(S, A) = - \sum_{v \in Values(A)} \frac{|S_v|}{|S|} \log_2 \frac{|S_v|}{|S|}$$

(trong đó $Values(A)$ là tập các giá trị có thể của thuộc tính A , và $S_v = \{x \mid x \in S, x_A = v\}$)

Xử lý các thuộc tính thiếu giá trị

- Giả sử thuộc tính A là một thuộc tính kiểm tra ở nút n
- Nếu mẫu x không có (thiếu) giá trị đối với thuộc tính A (x_A không xác định)?
- Gọi S_n là tập các mẫu học gắn với nút n có giá trị đối với thuộc tính A
 - Giải pháp 1: x_A là giá trị phổ biến nhất đối với thuộc tính A trong số các ví dụ thuộc tập S_n
 - Giải pháp 2: x_A là giá trị phổ biến nhất đối với thuộc tính A trong số các ví dụ thuộc tập S_n có cùng phân lớp với x.

Cây C4.5

- Phiên bản cải tiến của **ID3**:
 - Độ đo được sử dụng: Gain $\rightarrow$ Gain ratio, Gain*Gain/cost
 - Thông tin không đầy đủ (thuộc tính thiếu dữ liệu) $\rightarrow$ điền vào
 - Các thuộc tính có giá trị liên tục $\rightarrow$ rời rạc hóa
 - Thực hiện cắt tỉa cây, tránh hiện tượng “học quá”
 - Rút ra các luật suy diễn.

Đánh giá hiệu năng của thuật toán (tiếp bài sau)

- Độ chính xác (Accuracy):
 - on validation data
 - K-fold cross validation
- Chi phí do phân lớp sai (Misclassification cost): Sometimes more accuracy is desired for some classes than others.
- Minimum description length (MDL):
 - Favor good accuracy on compact model
 - $MDL = \text{model_size}(\text{tree}) + \text{errors}(\text{tree})$

Regression Tree (cây hồi quy, sau)

- A variant of decision trees for dealing with **continuous target attribute**
- Estimation problem: approximate real-valued functions: e.g., weight, income, **scoring**
- A leaf node is marked with a real value or a linear function: e.g., the mean of the target values of the examples at the node.
- Measure of impurity: e.g., variance, standard deviation,...

Dùng cây quyết định khi nào?

- Các mẫu học được biểu diễn bằng các cặp (thuộc tính, giá trị)
 - Phù hợp với các thuộc tính có giá trị rời rạc
 - Đối với các thuộc tính có giá trị liên tục, phải rời rạc hóa ,,
- Hàm mục tiêu có giá trị đầu ra là các giá trị rời rạc
 - Ví dụ: Phân loại các ví dụ vào lớp phù hợp ,,
- Phù hợp khi hàm mục tiêu được biểu diễn ở dạng tách rời (disjunctive)
- Tập huấn luyện có thể chứa nhiều/lỗi
 - Lỗi trong phân loại (nhãn lớp) của các mẫu dữ liệu
 - Lỗi trong giá trị thuộc tính biểu diễn của mẫu dữ liệu ,,
- Tập huấn luyện có thể chứa các thuộc tính thiếu giá trị (giá trị của một số thuộc tính là không xác định đối với một số phần tử dữ liệu học).

Bài tập và thực hành

RID	age	income	student	credit-rating	class:buy_computer
1	youth	high	no	fair	no
2	youth	high	no	excellent	no
3	middle-aged	high	no	fair	yes
4	senior	medium	no	fair	yes
5	senior	low	yes	fair	yes
6	senior	low	yes	excellent	no
7	middle-aged	low	yes	excellent	yes
8	youth	medium	no	fair	no
9	youth	low	yes	fair	yes
10	senior	medium	yes	fair	yes
11	youth	medium	yes	excellent	yes
12	middle-aged	medium	no	excellent	yes
13	middle-aged	high	yes	fair	yes
14	senior	medium	no	excellent	no

- Huấn luyện (dựng cây quyết định) từ D
- Đánh giá hiệu năng của cây trên D và trên tập test D1 như sau:

RID	Age	Income	Student	C-Rate	Buy
1	Youth	Low	Yes	Fair	?
2	Senior	Medium	No	Excellent	?

Thảo luận

Đánh giá hiệu năng của giải thuật học máy



NỘI DUNG

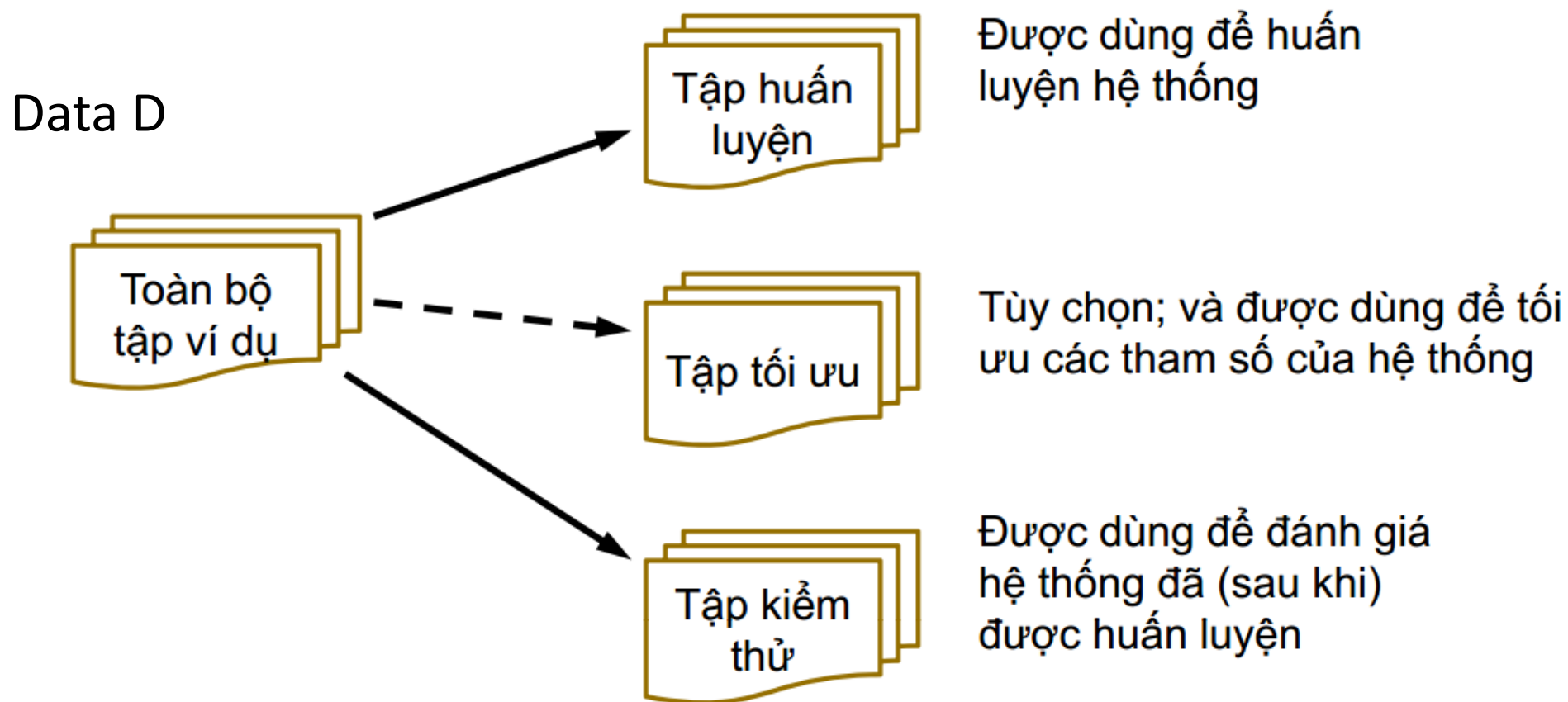
Đánh giá hiệu năng của các giải thuật phân loại

- Các phương pháp đánh giá
- Các tiêu chí đánh giá

Đánh giá hiệu năng của các kỹ thuật phân loại

- Việc đánh giá hiệu năng hệ thống học máy thường được thực hiện dựa trên thực nghiệm (experiments).
- Đã học các kỹ thuật phân lớp:
 - K-NN
 - Decision Trees
- Đánh giá độ chính xác
- So sánh độ chính xác

Đánh giá hiệu năng của các kỹ thuật phân loại



- Để có thể có một đánh giá đáng tin cậy về hiệu năng của hệ thống:
 - Tập huấn luyện càng lớn thì hiệu năng của hệ thống học càng tốt
 - Tập kiểm thử càng lớn thì việc đánh giá càng chính xác
 - Vấn đề: Có thể rất khó để có được các tập dữ liệu (rất) lớn
- Hiệu năng của hệ thống không chỉ phụ thuộc vào giải thuật học máy được sử dụng, mà còn phụ thuộc vào:
 - Phân bố lớp (Class distribution)
 - Chi phí của việc phân lớp sai (Cost of misclassification)
 - Kích thước của tập huấn luyện (Size of the training set)
 - Kích thước của tập kiểm thử (Size of the test set)

CÁC PHƯƠNG PHÁP ĐÁNH GIÁ

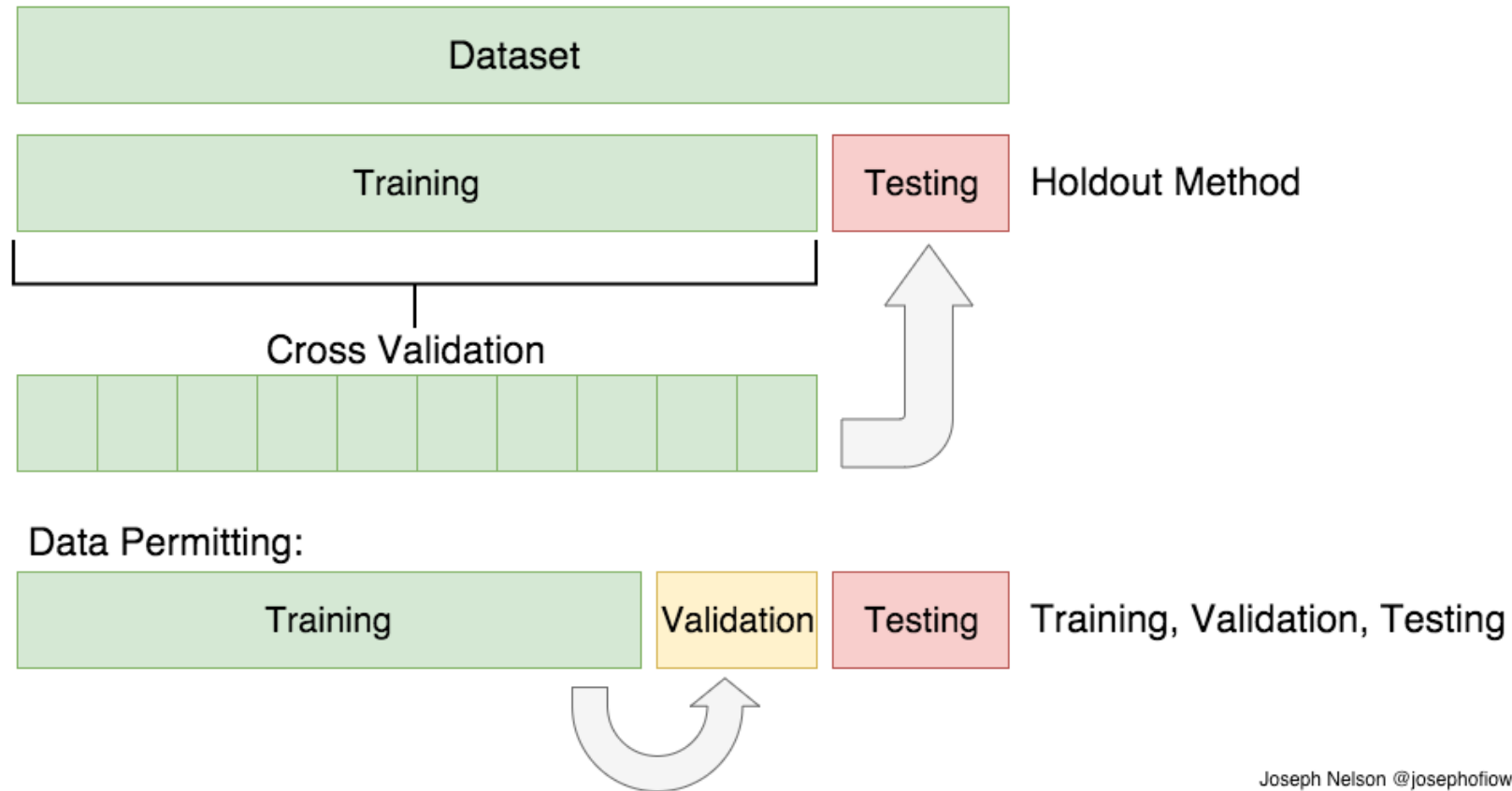
Các cách phân chia tập dữ liệu:

- Hold-out
- Cross-validation
 - k-fold
 - Leave-one-out cross-validation
- Bootstrap sampling

Hold-out (splitting)

- Toàn bộ tập ví dụ D được chia thành 2 tập con **không giao nhau**
 - Tập huấn luyện D_{train} – để huấn luyện hệ thống
 - Tập kiểm thử D_{test} – để đánh giá hiệu năng của hệ thống đã học

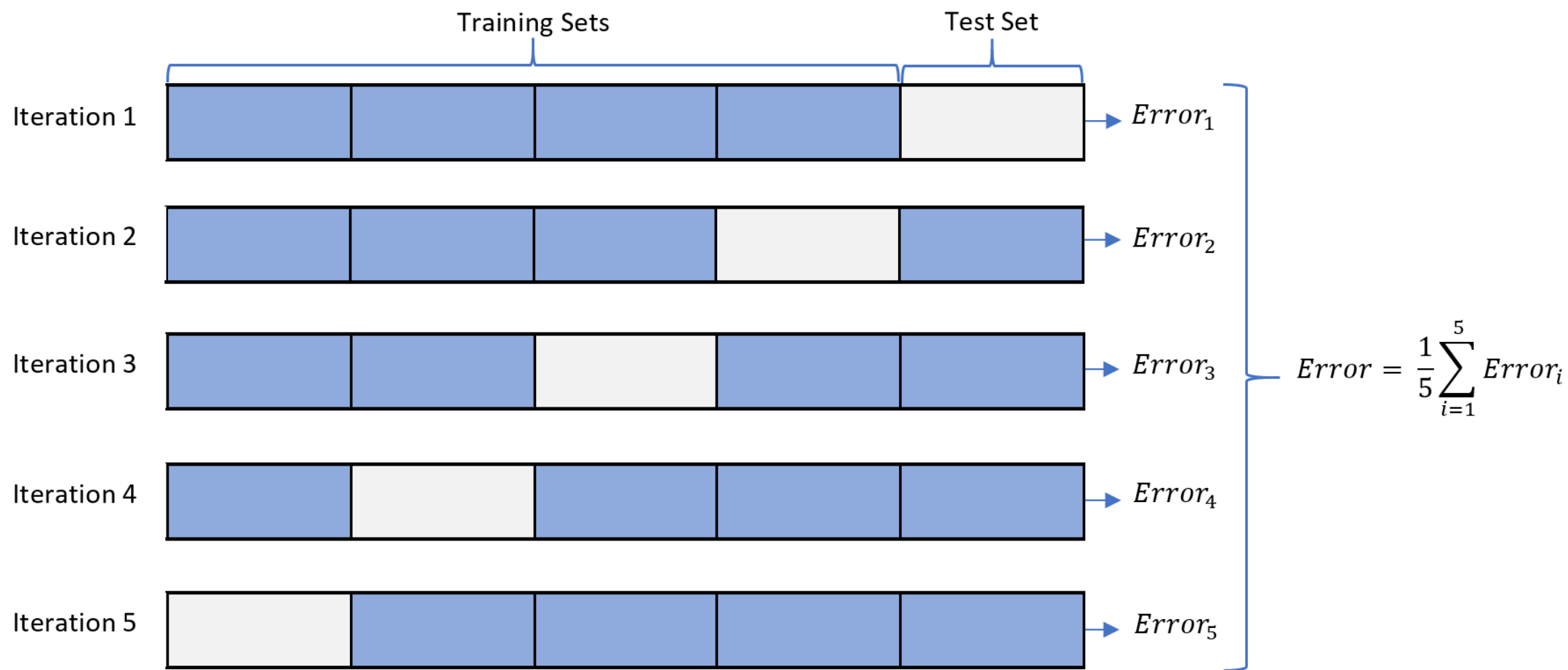
→ $D = D_{train} \cup D_{test}$, và thường là $|D_{train}| \gg |D_{test}|$
- Các yêu cầu:
 - Bất kỳ ví dụ nào thuộc vào tập kiểm thử D_{test} đều không được sử dụng trong quá trình huấn luyện hệ thống
 - Bất kỳ ví dụ nào được sử dụng trong giai đoạn huấn luyện hệ thống (i.e., thuộc vào D_{train}) đều không được sử dụng trong giai đoạn đánh giá hệ thống
 - Các ví dụ kiểm thử trong D_{test} cho phép một đánh giá không thiên vị đối với hiệu năng của hệ thống
- Các lựa chọn thường gặp: $|D_{train}| = (2/3) \cdot |D|$, $|D_{test}| = (1/3) \cdot |D|$
- Phù hợp khi ta có tập ví dụ D có kích thước lớn



Joseph Nelson @josephofiowa

Cross-validation

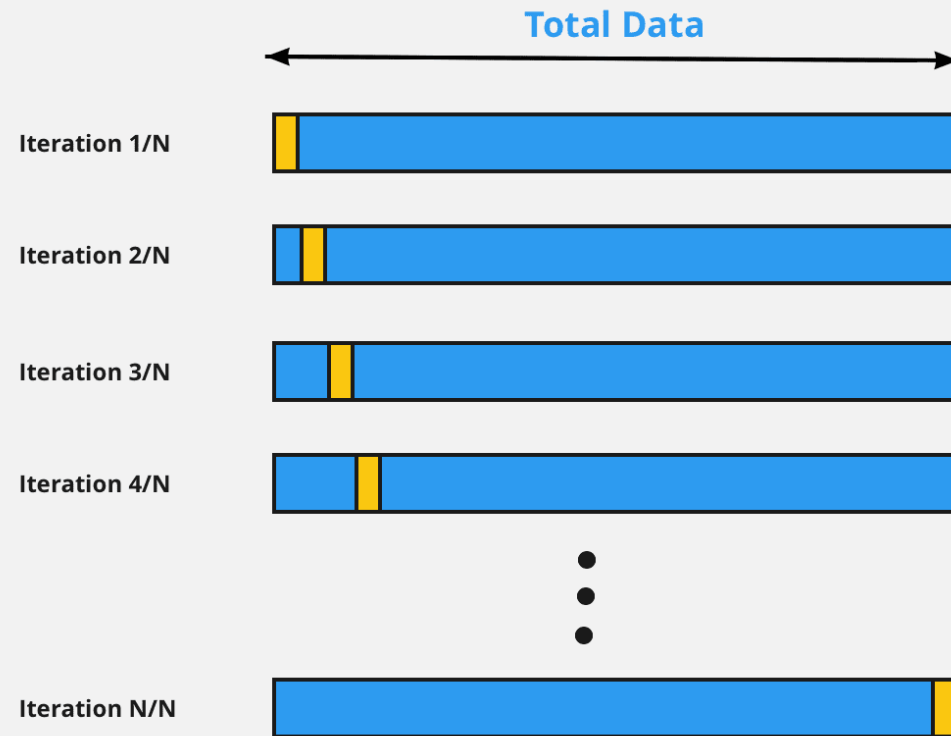
- Để tránh việc trùng lặp giữa các tập kiểm thử (một số ví dụ cùng xuất hiện trong các tập kiểm thử khác nhau)
- k -fold cross-validation
 - Tập toàn bộ các ví dụ D được chia thành k tập con **không giao nhau** (gọi là “*fold*”) có kích thước xấp xỉ nhau
 - Mỗi lần (trong số k lần) lặp, một tập con được sử dụng làm tập kiểm thử, và $(k-1)$ tập con còn lại được dùng làm tập huấn luyện
 - k giá trị lỗi (mỗi giá trị tương ứng với một *fold*) được tính trung bình cộng để thu được giá trị lỗi tổng thể
- Các lựa chọn thông thường của k : 10, hoặc 5
- Thông thường, mỗi tập con (fold) được lấy mẫu phân tầng (xấp xỉ phân bố lớp) trước khi áp dụng quá trình đánh giá Cross-validation
- Phù hợp khi ta có tập ví dụ D vừa và nhỏ



Leave-one-out cross-validation

- Một trường hợp (kiểu) của phương pháp Cross-validation
 - Số lượng các nhóm (folds) bằng kích thước của tập dữ liệu ($k=|D|$)
 - Mỗi nhóm (fold) chỉ bao gồm một ví dụ
- Khai thác tối đa (triệt để) tập ví dụ ban đầu
- Không hề có bước lấy mẫu ngẫu nhiên (no random sub-sampling)
- Áp dụng lấy mẫu phân tầng (stratification) không phù hợp
 - Vì ở mỗi bước lặp, tập thử nghiệm chỉ gồm có một ví dụ
- Chi phí tính toán (rất) cao
- Phù hợp khi ta có một tập ví dụ D (rất) nhỏ

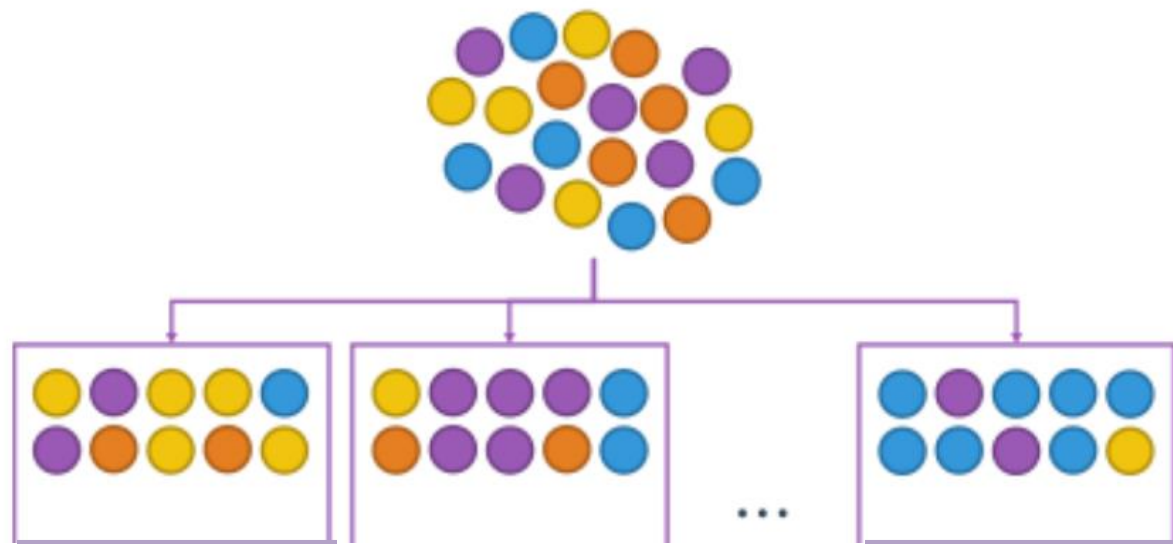
LOOCV: Leave One Out Cross Validation



dataaspirant.com

Bootstrap sampling (1)

- Phương pháp Cross-validation sử dụng việc lấy mẫu không lặp lại (sampling without replacement)
 - Đối với mỗi ví dụ, *một khi đã được chọn (được sử dụng), thì không thể được chọn (sử dụng) lại* cho tập huấn luyện
- Phương pháp Bootstrap sampling sử dụng việc ***lấy mẫu có lặp lại (sampling with replacement)*** để tạo nên tập huấn luyện
 - Giả sử tập toàn bộ D bao gồm n ví dụ
 - Lấy mẫu có lặp lại n lần đối với tập D , để tạo nên tập huấn luyện D_{train} gồm n ví dụ
 - Từ tập D , lấy ra ngẫu nhiên một ví dụ x (nhưng **không loại bỏ** x khỏi tập D)
 - Đưa ví dụ x vào trong tập huấn luyện: $D_{train} = D_{train} \cup x$
 - Lặp lại 2 bước trên n lần
 - Sử dụng tập D_{train} để huấn luyện hệ thống
 - Sử dụng tất cả các ví dụ thuộc D **nhưng không thuộc** D_{train} để tạo nên tập thử nghiệm: $D_{test} = \{z \in D; z \notin D_{train}\}$



Original Data

D

Bootstrapping

D_{train}

$D_{\text{test?}}$

Bootstrap sampling (2)

- Trong mỗi bước lặp, một ví dụ có xác suất $= \left(1 - \frac{1}{n}\right)$ để không được lựa chọn đưa vào tập huấn luyện
- Vì vậy, xác suất để một ví dụ (sau quá trình lấy mẫu lặp lại – bootstrap sampling) được đưa vào tập kiểm thử là:

$$\left(1 - \frac{1}{n}\right)^n \approx e^{-1} \approx 0.368$$

- Có nghĩa rằng:
 - Tập huấn luyện (có kích thước $=n$) bao gồm xấp xỉ 63.2% các ví dụ trong D (Lưu ý: Một ví dụ thuộc tập D có thể **xuất hiện nhiều lần** trong tập D_{train})
 - Tập kiểm thử (có kích thước $<n$) bao gồm xấp xỉ 36.8% các ví dụ trong D (Lưu ý: Một ví dụ thuộc tập D chỉ có thể **xuất hiện tối đa 1 lần** trong tập D_{test})
- Phù hợp khi ta có một tập dữ liệu D có kích thước (rất) nhỏ

CÁC TIÊU CHÍ ĐÁNH GIÁ

- Tính chính xác (Accuracy)
 - Mức độ dự đoán (phân lớp) chính xác của hệ thống (đã được huấn luyện) đối với các ví dụ kiểm chứng (test instances)
- Tính hiệu quả (Efficiency)
 - Chi phí về thời gian và tài nguyên (bộ nhớ) cần thiết cho việc huấn luyện và kiểm thử hệ thống
- Khả năng xử lý nhiễu (Robustness)
 - Khả năng xử lý (chịu được) của hệ thống đối với các ví dụ nhiễu (lỗi) hoặc thiếu giá trị

Các tiêu chí đánh giá

- Khả năng mở rộng (Scalability)
 - Hiệu năng của hệ thống (vd: tốc độ học/ phân loại) thay đổi như thế nào đối với kích thước của tập dữ liệu
- Khả năng diễn giải (Interpretability)
 - Mức độ dễ hiểu (đối với người sử dụng) của các kết quả và hoạt động của hệ thống
- Mức độ phức tạp (Complexity)
 - Mức độ phức tạp của mô hình hệ thống (hàm mục tiêu) học được.

Độ chính xác (Accuracy)

- Đối với bài toán phân loại : Là tỷ lệ phân loại đúng.

→ Giá trị (kết quả) đầu ra của hệ thống là một giá trị định danh

$$Accuracy = \frac{1}{|D_{test}|} \sum_{x \in D_{test}} Identical(o(x), c(x)); \quad Identical(a, b) = \begin{cases} 1, & \text{if } (a = b) \\ 0, & \text{if otherwise} \end{cases}$$

- x : Một ví dụ trong tập thử nghiệm D_{test}
- $o(x)$: Giá trị đầu ra (phân lớp) bởi hệ thống đối với ví dụ x
- $c(x)$: Phân lớp thực sự (đúng) đối với ví dụ x

- Đối với bài toán hồi quy (dự đoán)

→ Giá trị (kết quả) đầu ra của hệ thống là một giá trị số

$$Error = \frac{1}{|D_{test}|} \sum_{x \in D_{test}} Error(x); \quad Error(x) = |d(x) - o(x)|$$

- $o(x)$: Giá trị đầu ra (dự đoán) bởi hệ thống đối với ví dụ x
- $d(x)$: Giá trị đầu ra thực sự (đúng) đối với ví dụ x
- *Accuracy* là một hàm đảo (inverse function) đối với *Error*

- Độ chính xác (Accuracy) có phải là một độ đo tốt cho hiệu năng của giải thuật học máy?
- Gợi ý: Trường hợp dữ liệu không cân bằng.

Ma trận nhầm lẫn (Confusion matrix)

- Còn được gọi là Contingency Table
- **Chỉ được sử dụng đối với bài toán phân loại**
 - Không thể áp dụng cho bài toán hồi quy (dự đoán)

- **TP_i** : Số lượng các ví dụ thuộc lớp c_i được phân loại chính xác vào lớp c_i
- **FP_i** : Số lượng các ví dụ không thuộc lớp c_i bị phân loại nhầm vào lớp c_i
- **TN_i** : Số lượng các ví dụ không thuộc lớp c_i được phân loại (chính xác)
- **FN_i** : Số lượng các ví dụ thuộc lớp c_i bị phân loại nhầm (vào các lớp khác c_i)

Lớp c_i		Được phân lớp bởi hệ thống	
		Thuộc	Ko thuộc
Phân lớp thực sự (đúng)	Thuộc	TP_i	FN_i
	Ko thuộc	FP_i	TN_i

Precision and Recall

■ Precision đối với lớp c_i

→ Tổng số các ví dụ thuộc lớp c_i được phân loại chính xác chia cho tổng số các ví dụ được phân loại vào lớp c_i

$$Precision(c_i) = \frac{TP_i}{TP_i + FP_i}$$

■ Recall đối với lớp c_i

→ Tổng số các ví dụ thuộc lớp c_i được phân loại chính xác chia cho tổng số các ví dụ thuộc lớp c_i

$$Recall(c_i) = \frac{TP_i}{TP_i + FN_i}$$

Precision and Recall

- Làm thế nào để tính toán được giá trị Precision và Recall (một cách tổng thể) cho toàn bộ các lớp $C=\{c_i\}$?

- Trung bình vi mô (Micro-averaging)

$$\text{Precision} = \frac{\sum_{i=1}^{|C|} TP_i}{\sum_{i=1}^{|C|} (TP_i + FP_i)} \quad \text{Recall} = \frac{\sum_{i=1}^{|C|} TP_i}{\sum_{i=1}^{|C|} (TP_i + FN_i)}$$

- Trung bình vĩ mô (Macro-averaging)

$$\text{Precision} = \frac{\sum_{i=1}^{|C|} \text{Precision}(c_i)}{|C|} \quad \text{Recall} = \frac{\sum_{i=1}^{|C|} \text{Recall}(c_i)}{|C|}$$

F1 (F-score)

- Tiêu chí đánh giá F_1 là sự kết hợp của 2 tiêu chí đánh giá *Precision* và *Recall*

$$F_1 = \frac{2 \cdot \text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}} = \frac{2}{\frac{1}{\text{Precision}} + \frac{1}{\text{Recall}}}$$

- F_1 là một **trung bình điều hòa (harmonic mean)** của các tiêu chí *Precision* và *Recall*
 - F_1 có xu hướng lấy giá trị gần với giá trị nào nhỏ hơn giữa 2 giá trị *Precision* và *Recall*
 - F_1 có giá trị lớn nếu cả 2 giá trị *Precision* và *Recall* đều lớn

Ví dụ

- Tính độ chính xác và độ truy hồi (Precision, Recall)

<i>Classes</i>	<i>yes</i>	<i>no</i>	<i>Total</i>	<i>Recognition (%)</i>
<i>yes</i>	90	210	300	30.00
<i>no</i>	140	9560	9700	98.56
Total	230	9770	10,000	96.40

Figure 8.16 Confusion matrix for the classes *cancer = yes* and *cancer = no*.

- Độ chính xác và độ truy hồi còn gọi là độ nhạy và độ đặc hiệu (dữ liệu y tế).

Ứng dụng

- Câu hỏi
- Bài tập

Tài liệu tham khảo

- Sách cung cấp
- ML course, Stanford
- N. N Quang, Bài giảng Học máy, ĐH BK Hà Nội



BÀI 5

HỒI QUI TUYẾN TÍNH-LINEAR REGRESSION



- ▶ Mô hình hồi qui?



- ▶ Mô hình hồi qui?
- ▶ Overfitting



- ▶ Mô hình hồi qui?
- ▶ Overfitting
- ▶ Thực hành



Dữ liệu:

x	x_1	x_2	$\dots$	x_n
y	y_1	y_2	$\dots$	y_n



Dữ liệu:

x	x_1	x_2	$\dots$	x_n
y	y_1	y_2	$\dots$	y_n

Hàm hồi qui:

$$y = \alpha + \beta.x$$



Dữ liệu:

x	x_1	x_2	$\dots$	x_n
y	y_1	y_2	$\dots$	y_n

Hàm hồi qui:

$$y = \alpha + \beta.x$$

Phương trình xác định hệ số:

$$\begin{cases} n.\alpha + (\sum_{i=1}^n x_i).\beta = \sum_{i=1}^n y_i \\ (\sum_{i=1}^n x_i).\alpha + (\sum_{i=1}^n x_i^2).\beta = \sum_{i=1}^n x_i y_i \end{cases}$$



Công thức tìm hàm hồi qui cho 2 biến

Dữ liệu:

x	x_1	x_2	$\dots$	x_n
y	y_1	y_2	$\dots$	y_n
z	z_1	z_2	$\dots$	z_n



Công thức tìm hàm hồi qui cho 2 biến

Dữ liệu:

x	x_1	x_2	$\dots$	x_n
y	y_1	y_2	$\dots$	y_n
z	z_1	z_2	$\dots$	z_n

Hàm hồi qui:

$$z = \alpha + \beta.x + \gamma.y$$

Dữ liệu:

x	x_1	x_2	$\dots$	x_n
y	y_1	y_2	$\dots$	y_n
z	z_1	z_2	$\dots$	z_n

Hàm hồi qui:

$$z = \alpha + \beta \cdot x + \gamma \cdot y$$

Phương trình xác định hệ số:

$$\begin{cases} n \cdot \alpha + \left(\sum_{i=1}^n x_i\right) \cdot \beta + \left(\sum_{i=1}^n y_i\right) \cdot \gamma = \sum_{i=1}^n z_i \\ \left(\sum_{i=1}^n x_i\right) \cdot \alpha + \left(\sum_{i=1}^n x_i^2\right) \cdot \beta + \left(\sum_{i=1}^n x_i y_i\right) \cdot \gamma = \sum_{i=1}^n x_i z_i \\ \left(\sum_{i=1}^n y_i\right) \cdot \alpha + \left(\sum_{i=1}^n x_i \cdot y_i\right) \cdot \beta + \left(\sum_{i=1}^n y_i^2\right) \cdot \gamma = \sum_{i=1}^n y_i z_i \end{cases}$$

Hàm hồi qui:

$$y = \alpha_0 + \sum_{j=1}^m \alpha_j \cdot x^{(j)}$$

$$\left\{ \begin{array}{l} n \cdot \alpha_0 + \sum_{i=1}^n x_i^{(1)} \alpha_1 + \sum_{i=1}^n x_i^{(2)} \alpha_2 + \dots + \sum_{i=1}^n x_i^{(m)} \alpha_m = \sum_{i=1}^n y_i \\ \sum_{i=1}^n x_i^{(1)} \alpha_0 + \sum_{i=1}^n [x_i^{(1)}]^2 \alpha_1 + \dots + \sum_{i=1}^n x_i^{(1)} x_i^{(m)} \alpha_m = \sum_{i=1}^n x_i^{(1)} y_i \\ \vdots \\ \sum_{i=1}^n x_i^{(m)} \alpha_0 + \sum_{i=1}^n x_i^{(1)} x_i^{(m)} \alpha_1 + \dots + \sum_{i=1}^n [x_i^{(m)}]^2 \alpha_m = \sum_{i=1}^n x_i^{(m)} y_i \end{array} \right.$$

Học dựa trên xác suất

Phân lớp Bayes

NỘI DUNG

- Giới thiệu về xác suất
- Định lý Bayes
- Đánh giá khả năng có thể nhất
- Phương pháp phân loại Naïve Bayes
- Ứng dụng.

Khái niệm cơ bản về xác suất

- Giả sử có một thí nghiệm mà kết quả của nó mang tính ngẫu nhiên (phụ thuộc vào khả năng có thể xảy ra). Ví dụ: gieo 1 quân xúc xắc
- *Không gian các khả năng S*. Tập hợp tất cả các kết quả có thể xảy ra
Ví dụ: $S = \{1, 2, 3, 4, 5, 6\}$ đối với thí nghiệm gieo xúc xắc
- *Sự kiện E*. Một tập con của không gian các khả năng
Ví dụ: $E = \{1\}$: kết quả quân xúc xắc đổ ra là 1
Ví dụ: $E = \{1, 3, 5\}$: kết quả quân xúc xắc đổ ra là một số lẻ
- *Không gian các sự kiện W*. Không gian (thế giới) mà các kết quả của sự kiện có thể xảy ra
Ví dụ: W bao gồm tất cả các lần đổ xúc xắc
- *Biến ngẫu nhiên A*. Một biến ngẫu nhiên biểu diễn (diễn đạt) một sự kiện, và có một mức độ về khả năng xảy ra sự kiện này.

Biểu diễn xác suất

$P(A)$: “Phần của không gian (thể giới) mà trong đó A là đúng”

Không gian sự kiện
của (không gian của
tất cả các giá trị có
thể xảy ra của A)



[<http://www.cs.cmu.edu/~awm/tutorials>]

Các biến ngẫu nhiên 2 giá trị

Một biến ngẫu nhiên 2 giá trị (nhị phân) có thể nhận một trong 2 giá trị đúng (`true`) hoặc sai (`false`)

Các tiên đề

- $0 \leq P(A) \leq 1$
- $P(\text{true}) = 1$
- $P(\text{false}) = 0$
- $P(A \vee B) = P(A) + P(B) - P(A \wedge B)$

Các hệ quả

- $P(\text{not } A) \equiv P(\sim A) = 1 - P(A)$
- $P(A) = P(A \wedge B) + P(A \wedge \sim B)$

Các biến ngẫu nhiên đa trị

Một biến ngẫu nhiên nhiều giá trị có thể nhận một trong số k (>2) giá trị $\{v_1, v_2, \dots, v_k\}$

$$P(A = v_i \wedge A = v_j) = 0 \text{ if } i \neq j$$

$$P(A=v_1 \vee A=v_2 \vee \dots \vee A=v_k) = 1$$

$$P(A = v_1 \vee A = v_2 \vee \dots \vee A = v_i) = \sum_{j=1}^i P(A = v_j)$$

$$\sum_{j=1}^k P(A = v_j) = 1$$

$$P(B \wedge [A = v_1 \vee A = v_2 \vee \dots \vee A = v_i]) = \sum_{j=1}^i P(B \wedge A = v_j)$$

[<http://www.cs.cmu.edu/~awm/tutorials>]

Xác suất có điều kiện (1)

- $P(A|B)$ là phần của không gian (thế giới) mà trong đó A là đúng, với điều kiện (đã biết) là B đúng

Mí dụ:

- A: Tôi sẽ đi đá bóng vào ngày mai
- B: Trời sẽ không mưa vào ngày mai
- $P(A|B)$: Xác suất của việc tôi sẽ đi đá bóng vào ngày mai nếu
(đã biết rằng) trời sẽ không mưa (vào ngày mai)

Xác suất có điều kiện (2)

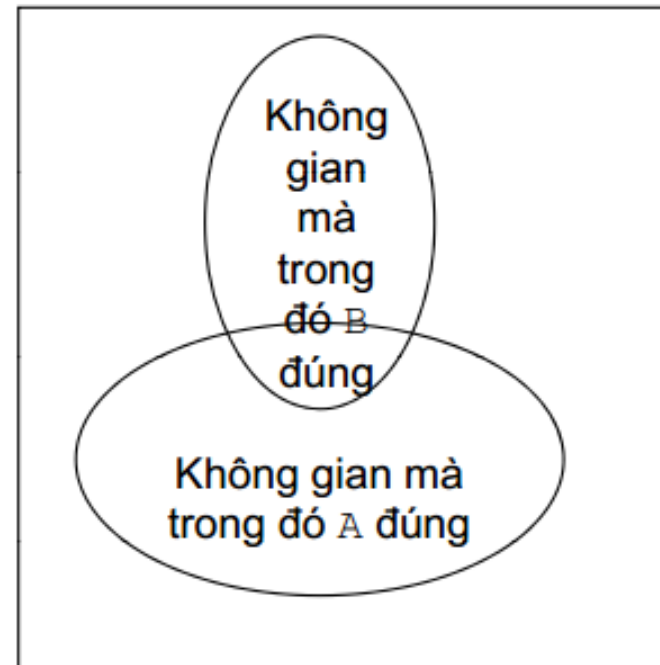
Định nghĩa: $P(A|B) = \frac{P(A, B)}{P(B)}$

Các hệ quả:

$$P(A, B) = P(A|B) \cdot P(B)$$

$$P(A|B) + P(\sim A|B) = 1$$

$$\sum_{i=1}^k P(A = v_i | B) = 1$$



Các biến độc lập về xác suất (1)

- Hai sự kiện A và B được gọi là *độc lập về xác suất* nếu xác suất của sự kiện A là như nhau đối với các trường hợp:
 - Khi sự kiện B xảy ra, hoặc
 - Khi sự kiện B không xảy ra, hoặc
 - Không có thông tin (không biết gì) về việc xảy ra của sự kiện B
- Ví dụ
 - A: Tôi sẽ thi tiếng Anh
 - B: Tuấn sẽ mua máy tính
 - $P(A|B) = P(A)$
 - “Dù Tuấn có mua máy tính hay không cũng không ảnh hưởng tới quyết định của tôi về việc thi tiếng Anh.”

Các biến độc lập về xác suất (2)

Từ định nghĩa của các biến độc lập về xác suất

$P(A|B) = P(A)$, chúng ta thu được các luật như sau

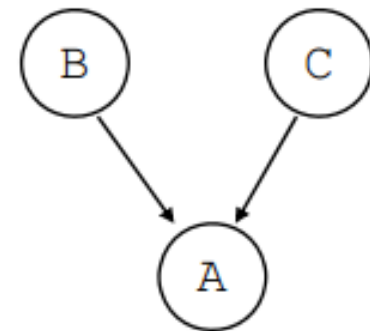
- $P(\sim A|B) = P(\sim A)$
- $P(B|A) = P(B)$
- $P(A, B) = P(A) \cdot P(B)$
- $P(\sim A, B) = P(\sim A) \cdot P(B)$
- $P(A, \sim B) = P(A) \cdot P(\sim B)$
- $P(\sim A, \sim B) = P(\sim A) \cdot P(\sim B)$

Xác suất có điều kiện với >2 biến

$P(A|B, C)$ là xác suất của A đối với (đã biết) B và C

Ví dụ

- A: Tôi sẽ đi dạo bờ sông vào sáng mai
- B: Thời tiết sáng mai rất đẹp
- C: Tôi sẽ dậy sớm vào sáng mai
- $P(A|B, C)$: Xác suất của việc tôi sẽ đi dạo dọc bờ sông vào sáng mai, nếu (đã biết rằng) thời tiết sáng mai rất đẹp và tôi sẽ dậy sớm vào sáng mai



$P(A|B, C)$

Độc lập có điều kiện

Hai biến A và C được gọi là ***độc lập có điều kiện*** đối với biến B , nếu xác suất của A đối với B bằng xác suất của A đối với B và C

Công thức định nghĩa: $P(A|B, C) = P(A|B)$

Ví dụ

- A : Tôi sẽ đi đá bóng vào ngày mai
- B : Trận đá bóng ngày mai sẽ diễn ra trong nhà
- C : Ngày mai trời sẽ không mưa
- $P(A|B, C) = P(A|B)$

→ Nếu biết rằng trận đấu ngày mai sẽ diễn ra trong nhà, thì xác suất của việc tôi sẽ đi đá bóng ngày mai không phụ thuộc vào thời tiết

Các quy tắc quan trọng của xác suất

Quy tắc chuỗi (chain rule)

- $P(A, B) = P(A|B) \cdot P(B) = P(B|A) \cdot P(A)$
- $P(A|B) = P(A, B) / P(B) = P(B|A) \cdot P(A) / P(B)$
- $P(A, B|C) = P(A, B, C) / P(C) = P(A|B, C) \cdot P(B, C) / P(C)$
 $= P(A|B, C) \cdot P(B|C)$

Độc lập về xác suất và độc lập có điều kiện

- $P(A|B) = P(A)$; nếu A và B là độc lập về xác suất
- $P(A, B|C) = P(A|C) \cdot P(B|C)$; nếu A và B là độc lập có điều kiện đối với C
- $P(A_1, \dots, A_n|C) = P(A_1|C) \dots P(A_n|C)$; nếu $A_1, \dots, A_n$ là độc lập có điều kiện đối với C

ĐỊNH LÝ BAYES

$$P(h | D) = \frac{P(D | h).P(h)}{P(D)}$$

- $P(h)$: Xác suất trước (tiên nghiệm) của giả thiết (phân loại) h
- $P(D)$: Xác suất trước (tiên nghiệm) của việc quan sát được dữ liệu D
- $P(D | h)$: Xác suất (có điều kiện) của việc quan sát được dữ liệu D , nếu biết giả thiết (phân loại) h là đúng
- $P(h | D)$: Xác suất (có điều kiện) của giả thiết (phân loại) h là đúng, nếu quan sát được dữ liệu D
 - **Các phương pháp phân loại dựa trên xác suất sẽ sử dụng xác suất có điều kiện (*posterior probability*) này!**

Suy luận Bayes

$$P(h | D) = \frac{P(D | h).P(h)}{P(D)}$$

- Công thức nói rằng xác suất đúng của giả thuyết h khi quan sát được bằng chứng (dữ liệu) D , bằng với xác suất cho rằng chúng ta sẽ quan sát được bằng chứng D nếu giả thuyết h là đúng, nhân với xác suất tiên nghiệm của h , tất cả chia cho xác suất tiên nghiệm của việc quan sát được bằng chứng D .

Ví dụ: $P(\text{cúm}) = 0.001$ $P(\text{sốt}) = 0.003$
 $P(\text{cúm And sốt}) = 0.000003$
nhưng cúm và sốt là cá sự kiện không độc lập
các chuyên gia cho biết: $P(\text{sốt} | \text{cúm}) = 0.9$

Bằng chứng (triệu chứng): bệnh nhân bị sốt

Giả thuyết (bệnh): bệnh nhân bị cảm cúm

$$P(\text{cúm}|\text{sốt}) = \frac{P(\text{cúm}) * P(\text{sốt}|\text{cúm})}{P(\text{sốt})} = \frac{0.001 * 0.9}{0.003} = 0.3$$

Các con số ở vế phải thì dễ đạt được hơn con số ở vế trái

Định lý Bayes – Ví dụ (1)

Giả sử chúng ta có tập dữ liệu sau (dự đoán 1 người có chơi tennis)?

Ngày	Ngoài trời	Nhiệt độ	Độ ẩm	Gió	Chơi tennis
N1	Nắng	Nóng	Cao	Yếu	Không
N2	Nắng	Nóng	Cao	Mạnh	Không
N3	Âm u	Nóng	Cao	Yếu	Có
N4	Mưa	Bình thường	Cao	Yếu	Có
N5	Mưa	Mát mẻ	Bình thường	Yếu	Có
N6	Mưa	Mát mẻ	Bình thường	Mạnh	Không
N7	Âm u	Mát mẻ	Bình thường	Mạnh	Có
N8	Nắng	Bình thường	Cao	Yếu	Không
N9	Nắng	Mát mẻ	Bình thường	Yếu	Có
N10	Mưa	Bình thường	Bình thường	Yếu	Có
N11	Nắng	Bình thường	Bình thường	Mạnh	Có
N12	Âm u	Bình thường	Cao	Mạnh	Có

Định lý Bayes – Ví dụ (2)

Dữ liệu D . *Ngoài trời là nắng và Gió là mạnh*

Giả thiết (phân loại) h . Anh ta chơi tennis

Xác suất trước $P(h)$. Xác suất rằng anh ta chơi tennis (bất kể *Ngoài trời* như thế nào và *Gió* ra sao)

Xác suất trước $P(D)$. Xác suất rằng *Ngoài trời là nắng và Gió là mạnh*

$P(D|h)$. Xác suất *Ngoài trời là nắng và Gió là mạnh*, nếu biết rằng anh ta chơi tennis

$P(h|D)$. Xác suất anh ta chơi tennis, nếu biết rằng *Ngoài trời là nắng và Gió là mạnh*

→ Chúng ta quan tâm đến giá trị xác suất sau (*posterior probability*) này!

Xác suất hậu nghiệm cực đại (MAP)

Với một tập các giả thiết (các phân lớp) có thể H , hệ thống học sẽ tìm **giả thiết có thể xảy ra nhất (the most probable hypothesis)** $h \in H$ đối với các dữ liệu quan sát được D

Giả thiết h này được gọi là giả thiết có xác suất hậu nghiệm cực đại (**Maximum a posteriori – MAP**)

$$h_{MAP} = \arg \max_{h \in H} P(h | D)$$

$$h_{MAP} = \arg \max_{h \in H} \frac{P(D | h) \cdot P(h)}{P(D)} \quad (\text{bởi định lý Bayes})$$

$$h_{MAP} = \arg \max_{h \in H} P(D | h) \cdot P(h) \quad (P(D) \text{ là như nhau đối với các giả thiết } h)$$

MAP – Ví dụ

Tập H bao gồm 2 giả thiết (có thể)

- h_1 : Anh ta chơi tennis
- h_2 : Anh ta không chơi tennis

Tính giá trị của 2 xác suất có điều kiện: $P(h_1 | D)$, $P(h_2 | D)$

Giả thiết có thể nhất $h_{MAP} = h_1$ nếu $P(h_1 | D) \geq P(h_2 | D)$; ngược lại thì $h_{MAP} = h_2$

Bởi vì $P(D) = P(D, h_1) + P(D, h_2)$ là như nhau đối với cả 2 giả thiết h_1 và h_2 , nên có thể bỏ qua đại lượng $P(D)$

Vì vậy, cần tính 2 biểu thức: $P(D | h_1) \cdot P(h_1)$ và $P(D | h_2) \cdot P(h_2)$, và đưa ra quyết định tương ứng

- Nếu $P(D | h_1) \cdot P(h_1) \geq P(D | h_2) \cdot P(h_2)$, thì kết luận là anh ta chơi tennis
- Ngược lại, thì kết luận là anh ta không chơi tennis

Đánh giá khả năng có thể nhất (MLE)

Phương pháp MAP: Với một tập các giả thiết có thể H , cần tìm một giả thiết cực đại hóa giá trị: $P(D|h) \cdot P(h)$

Giả sử (assumption) trong phương pháp **đánh giá khả năng có thể nhất (Maximum likelihood estimation – MLE)**: Tất cả các giả thiết đều có giá trị xác suất trước như nhau: $P(h_i) = P(h_j)$, $\forall h_i, h_j \in H$

Phương pháp MLE tìm giả thiết cực đại hóa giá trị $P(D|h)$; trong đó $P(D|h)$ được gọi là *khả năng có thể (likelihood)* của dữ liệu D đối với h

Giả thiết có khả năng nhất (maximum likelihood hypothesis)

$$h_{ML} = \arg \max_{h \in H} P(D|h)$$

MLE – Ví dụ

Tập H bao gồm 2 giả thiết có thể

- h_1 : Anh ta chơi tennis
- h_2 : Anh ta không chơi tennis

D: Tập dữ liệu (các ngày) mà trong đó thuộc tính *Outlook* có giá trị *Sunny* và thuộc tính *Wind* có giá trị *Strong*

Tính 2 giá trị khả năng xảy ra (likelihood values) của dữ liệu D đối với 2 giả thiết: $P(D|h_1)$ và $P(D|h_2)$

- $P(\text{Outlook}=\text{Sunny}, \text{Wind}=\text{Strong}|h_1) = 1/8$
- $P(\text{Outlook}=\text{Sunny}, \text{Wind}=\text{Strong}|h_2) = 1/4$

Giả thiết MLE $h_{MLE}=h_1$ nếu $P(D|h_1) \geq P(D|h_2)$; và ngược lại thì $h_{MLE}=h_2$

→ Bởi vì $P(\text{Outlook}=\text{Sunny}, \text{Wind}=\text{Strong}|h_1) < P(\text{Outlook}=\text{Sunny}, \text{Wind}=\text{Strong}|h_2)$, hệ thống kết luận rằng:
Anh ta sẽ không chơi tennis!

Phân lớp Naïve Bayes (1)

Biểu diễn bài toán phân loại (classification problem)

- Một tập học D_{train} , trong đó mỗi ví dụ học x được biểu diễn là một vector n chiều: $(x_1, x_2, \dots, x_n)$
- Một tập xác định các nhãn lớp: $C = \{c_1, c_2, \dots, c_m\}$
- Với một ví dụ (mới) z , thì z sẽ được phân vào lớp nào?

Mục tiêu: Xác định phân lớp có thể (phù hợp) nhất đối với z

$$c_{MAP} = \arg \max_{c_i \in C} P(c_i | z)$$

$$c_{MAP} = \arg \max_{c_i \in C} P(c_i | z_1, z_2, \dots, z_n)$$

$$c_{MAP} = \arg \max_{c_i \in C} \frac{P(z_1, z_2, \dots, z_n | c_i) \cdot P(c_i)}{P(z_1, z_2, \dots, z_n)} \quad (\text{bởi định lý Bayes})$$

Phân lớp Naïve Bayes (2)

Để tìm được phân lớp có thể nhất đối với $z \dots$

$$c_{MAP} = \arg \max_{c_i \in C} P(z_1, z_2, \dots, z_n | c_i).P(c_i) \quad (P(z_1, z_2, \dots, z_n) \text{ là như nhau với các lớp})$$

Giả sử (assumption) trong phương pháp phân loại Naïve Bayes. Các thuộc tính là *độc lập có điều kiện (conditionally independent)* đối với các lớp

$$P(z_1, z_2, \dots, z_n | c_i) = \prod_{j=1}^n P(z_j | c_i)$$

Phân loại Naïve Bayes tìm phân lớp có thể nhất đối với z

$$c_{NB} = \arg \max_{c_i \in C} P(c_i). \prod_{j=1}^n P(z_j | c_i)$$

Naïve Bayes classifier – Ví dụ (1)

- Một sinh viên trẻ với thu nhập trung bình và mức đánh giá tín dụng bình thường liệu sẽ mua một cái máy tính?

Rec. ID	Age	Income	Student	Credit_Rating	Buy_Computer
1	Young	High	No	Fair	No
2	Young	High	No	Excellent	No
3	Medium	High	No	Fair	Yes
4	Old	Medium	No	Fair	Yes
5	Old	Low	Yes	Fair	Yes
6	Old	Low	Yes	Excellent	No
7	Medium	Low	Yes	Excellent	Yes
8	Young	Medium	No	Fair	No
9	Young	Low	Yes	Fair	Yes
10	Old	Medium	Yes	Fair	Yes
11	Young	Medium	Yes	Excellent	Yes
12	Medium	Medium	No	Excellent	Yes
13	Medium	High	Yes	Fair	Yes
14	Old	Medium	No	Excellent	No

Naïve Bayes classifier – Ví dụ (2)

Biểu diễn bài toán phân loại

- $z = (\text{Age}=\text{Young}, \text{Income}=\text{Medium}, \text{Student}=\text{Yes}, \text{Credit_Rating}=\text{Fair})$
- Có 2 phân lớp có thể: c_1 (“Mua máy tính”) và c_2 (“Không mua máy tính”)

Tính giá trị xác suất trước cho mỗi phân lớp

- $P(c_1) = 9/14$
- $P(c_2) = 5/14$

Tính giá trị xác suất của mỗi giá trị thuộc tính đối với mỗi phân lớp

- | | |
|---|--|
| • $P(\text{Age}=\text{Young} c_1) = 2/9;$ | $P(\text{Age}=\text{Young} c_2) = 3/5$ |
| • $P(\text{Income}=\text{Medium} c_1) = 4/9;$ | $P(\text{Income}=\text{Medium} c_2) = 2/5$ |
| • $P(\text{Student}=\text{Yes} c_1) = 6/9;$ | $P(\text{Student}=\text{Yes} c_2) = 1/5$ |
| • $P(\text{Credit_Rating}=\text{Fair} c_1) = 6/9;$ | $P(\text{Credit_Rating}=\text{Fair} c_2) = 2/5$ |

Naïve Bayes classifier – Ví dụ (2)

Tính toán xác suất có thể xảy ra (likelihood) của ví dụ z đối với mỗi phân lớp

- Đối với phân lớp c_1

$$P(z|c_1) = P(\text{Age}=\text{Young}|c_1).P(\text{Income}=\text{Medium}|c_1).P(\text{Student}=\text{Yes}|c_1). \\ P(\text{Credit_Rating}=\text{Fair}|c_1) = (2/9).(4/9).(6/9).(6/9) = 0.044$$

- Đối với phân lớp c_2

$$P(z|c_2) = P(\text{Age}=\text{Young}|c_2).P(\text{Income}=\text{Medium}|c_2).P(\text{Student}=\text{Yes}|c_2). \\ P(\text{Credit_Rating}=\text{Fair}|c_2) = (3/5).(2/5).(1/5).(2/5) = 0.019$$

Xác định phân lớp có thể nhất (the most probable class)

- Đối với phân lớp c_1

$$P(c_1).P(z|c_1) = (9/14).(0.044) = 0.028$$

- Đối với phân lớp c_2

$$P(c_2).P(z|c_2) = (5/14).(0.019) = 0.007$$

→ Kết luận: Anh ta (z) sẽ mua một máy tính!

Naïve Bayes classifier – lưu ý (1)

Nếu không có ví dụ nào gắn với phân lớp c_i có giá trị thuộc tính x_j ...

$$P(x_j | c_i) = 0, \text{ và vì vậy: } P(c_i) \cdot \prod_{j=1}^n P(x_j | c_i) = 0$$

Giải pháp: Sử dụng phương pháp Bayes để ước lượng $P(x_j | c_i)$

$$P(x_j | c_i) = \frac{n(c_i, x_j) + mp}{n(c_i) + m}$$

- $n(c_i)$: số lượng các ví dụ học gắn với phân lớp c_i
- $n(c_i, x_j)$: số lượng các ví dụ học gắn với phân lớp c_i có giá trị thuộc tính x_j
- p : ước lượng đối với giá trị xác suất $P(x_j | c_i)$
 - Các ước lượng đồng mức: $p = 1/k$, với thuộc tính f_j có k giá trị có thể
- m : một hệ số (trọng số)
 - Để bổ sung cho $n(c_i)$ các ví dụ thực sự được quan sát với thêm m mẫu ví dụ với ước lượng p

Naïve Bayes classifier – lưu ý (2)

Giới hạn về độ chính xác trong tính toán của máy tính

- $P(x_j | c_i) < 1$, đối với mọi giá trị thuộc tính x_j và phân lớp c_i
- Vì vậy, khi số lượng các giá trị thuộc tính là rất lớn, thì:

$$\lim_{n \rightarrow \infty} \left(\prod_{j=1}^n P(x_j | c_i) \right) = 0$$

Giải pháp: Sử dụng hàm lôgarit cho các giá trị xác suất

$$c_{NB} = \arg \max_{c_i \in C} \left(\log \left[P(c_i) \cdot \prod_{j=1}^n P(x_j | c_i) \right] \right)$$
$$c_{NB} = \arg \max_{c_i \in C} \left(\log P(c_i) + \sum_{j=1}^n \log P(x_j | c_i) \right)$$

Ứng dụng NB cho phân loại văn bản (1)

Biểu diễn bài toán phân loại văn bản

- Tập học D_{train} , trong đó mỗi ví dụ học là một biểu diễn văn bản gắn với một nhãn lớp: $D = \{(d_k, c_i)\}$
- Một tập các nhãn lớp xác định: $C = \{c_i\}$

Giai đoạn học

- Từ tập các văn bản trong D_{train} , trích ra tập các từ khóa (keywords/terms): $T = \{t_j\}$
- Gọi $D_{c_i} (\subseteq D_{\text{train}})$ là tập các văn bản trong D_{train} có nhãn lớp c_i
- Đối với mỗi phân lớp c_i
 - Tính giá trị xác suất trước của phân lớp c_i : $P(c_i) = \frac{|D_{c_i}|}{|D|}$
 - Đối với mỗi từ khóa t_j , tính xác suất từ khóa t_j xuất hiện đối với lớp c_i

$$P(t_j | c_i) = \frac{(\sum_{d_k \in D_{c_i}} n(d_k, t_j)) + 1}{(\sum_{d_k \in D_{c_i}} \sum_{t_m \in T} n(d_k, t_m)) + |T|}$$

$(n(d_k, t_j))$: số lần xuất hiện của từ khóa t_j trong văn bản d_k

Ứng dụng NB cho phân loại văn bản (2)

Giai đoạn phân lớp đối với một văn bản mới d

- Từ văn bản d , trích ra tập T_d gồm các từ khóa (keywords) t_j đã được định nghĩa trong tập T ($T_d \subseteq T$)
- **Giả sử (assumption).** Xác suất từ khóa t_j xuất hiện đối với lớp c_i là độc lập đối với vị trí của từ khóa đó trong văn bản

$$P(t_j \text{ ở vị trí } k | c_i) = P(t_j \text{ ở vị trí } m | c_i), \quad \forall k, m$$

- Đối với mỗi phân lớp c_i , tính giá trị likelihood của văn bản d đối với c_i

$$P(c_i) \cdot \prod_{t_j \in T_d} P(t_j | c_i)$$

- Phân lớp văn bản d thuộc vào lớp c^*

$$c^* = \arg \max_{c_i \in C} P(c_i) \cdot \prod_{t_j \in T_d} P(t_j | c_i)$$

Tổng kết NB

- Một trong các phương pháp học máy được áp dụng phổ biến nhất trong thực tế
- Dựa trên định lý Bayes
- Việc phân loại dựa trên các giá trị xác suất của các khả năng xảy ra của các giả thiết (phân loại)

Mặc dù đặt giả sử về sự độc lập có điều kiện của các thuộc tính đối với các phân lớp, nhưng phương pháp phân loại Naïve Bayes vẫn thu được các kết quả phân loại tốt trong nhiều lĩnh vực ứng dụng thực tế

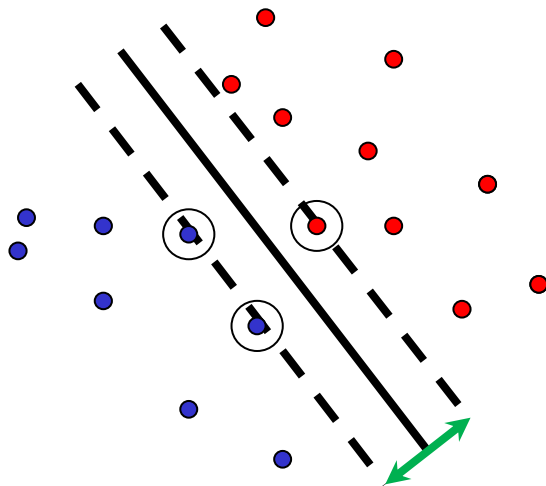
- Khi nào nên sử dụng?
 - Có một tập huấn luyện có kích thước lớn hoặc vừa
 - Các ví dụ được biểu diễn bởi một số lượng lớn các thuộc tính
 - **Các thuộc tính độc lập có điều kiện đối với các phân lớp**

Tài liệu tham khảo

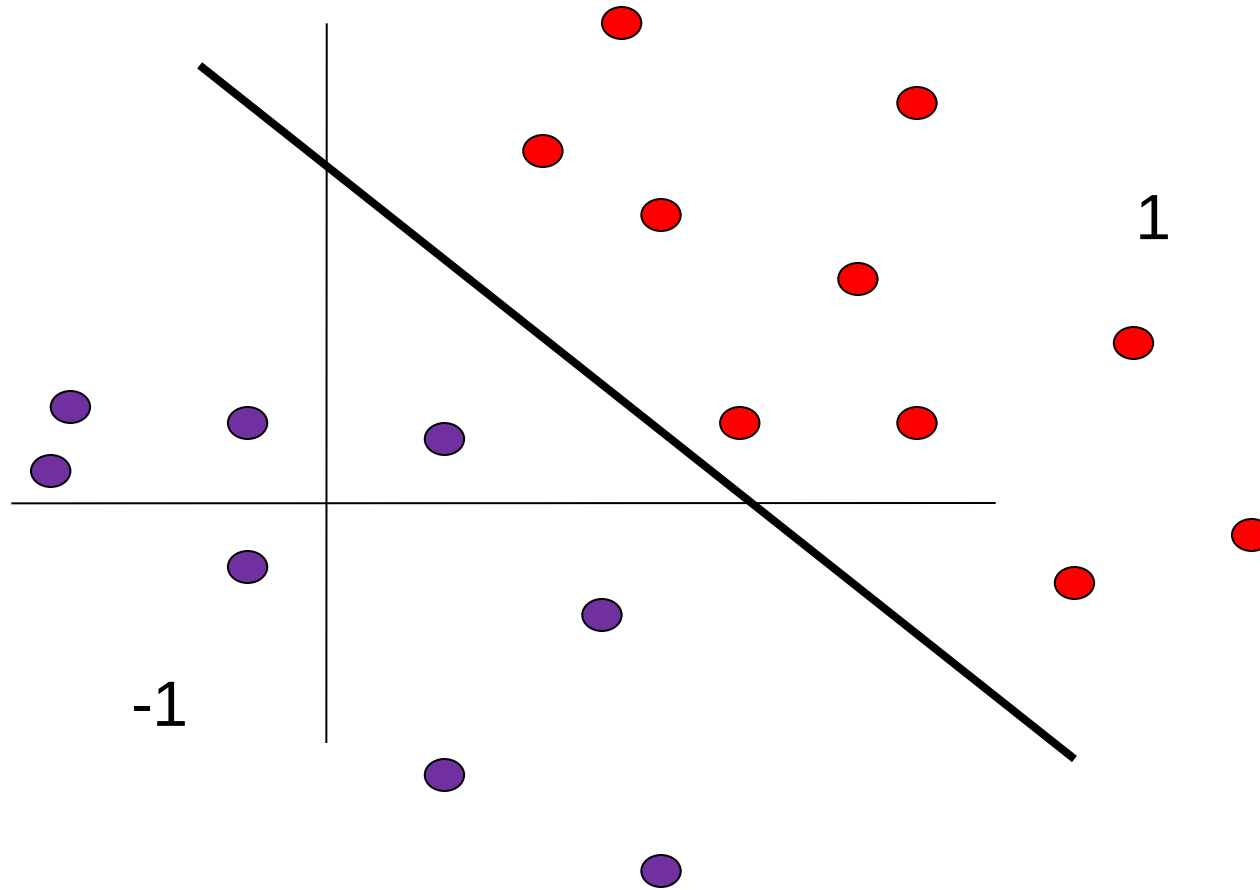
- R. Duda, P. Hart & D. Stork, ***Pattern Classification*** (2nd ed.), Wiley
- S. Shalev-Shwartz and S. Ben-David, *Understanding Machine Learning: From Theory to Algorithms*, Cambridge University Press, 2014
- Many slides are adapted from Quang N. N., ĐH Bách Khoa HN.

Q&A

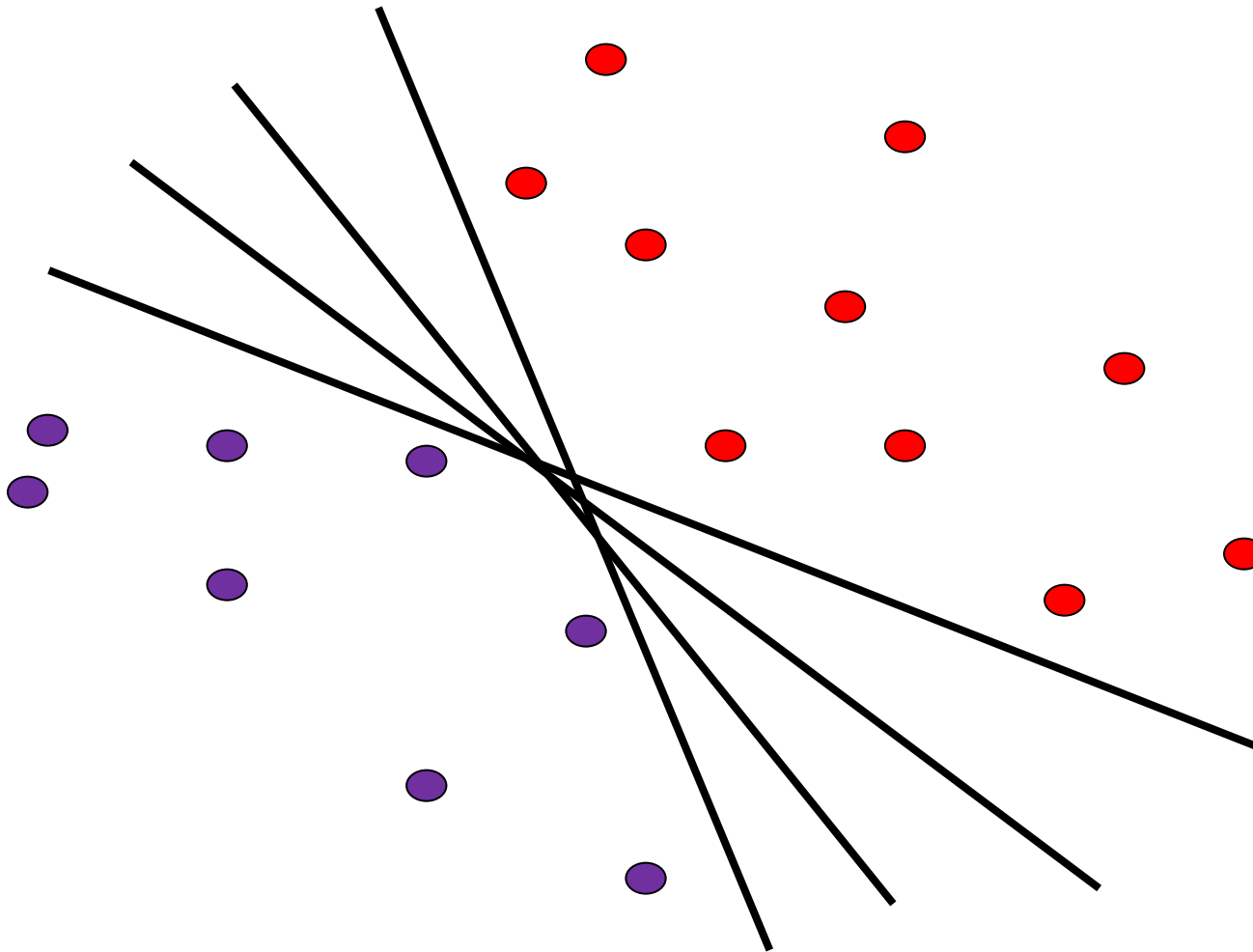
Support Vector Machines (SVM)



Phân loại tuyến tính



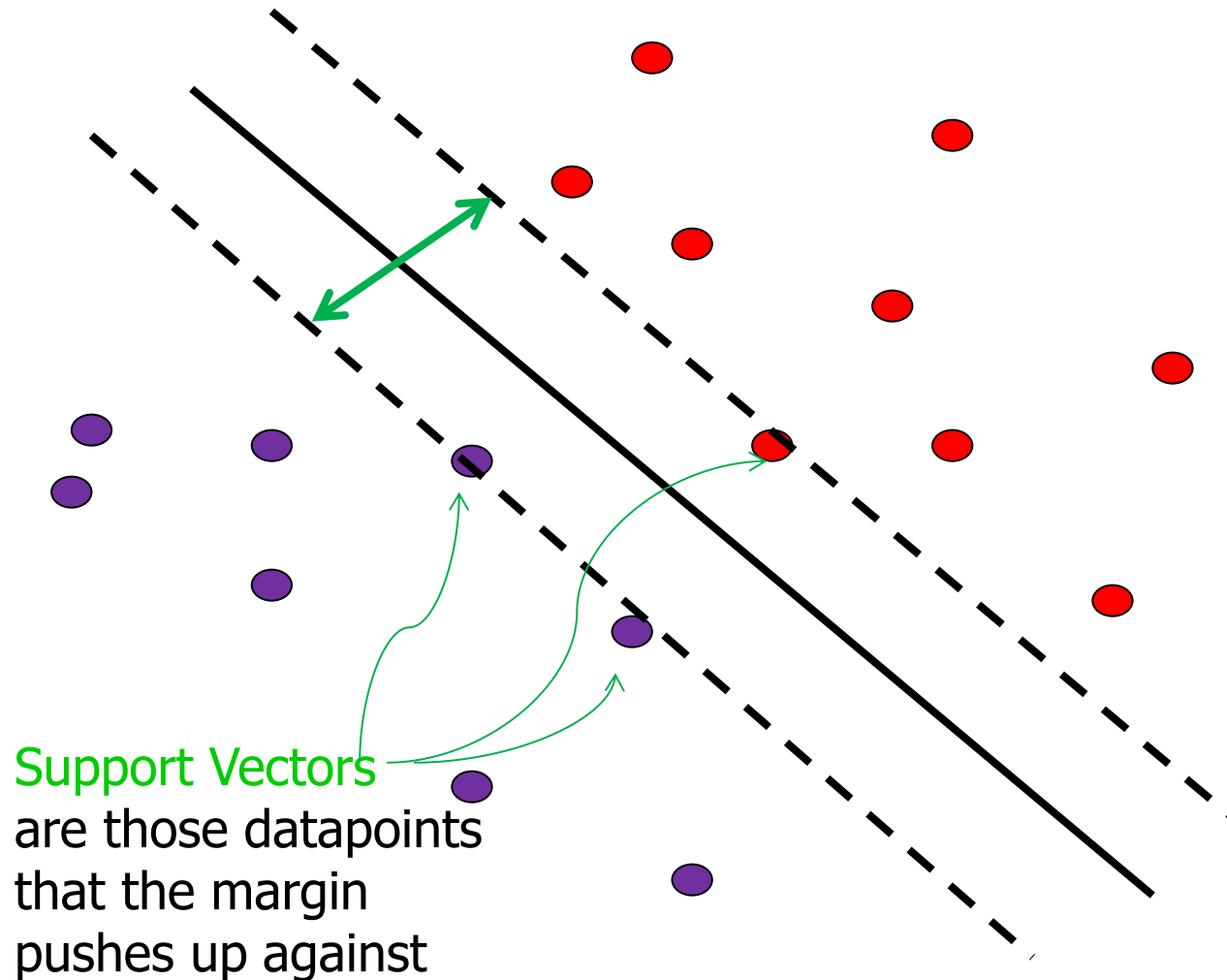
Phân loại tuyến tính



- Tìm hàm tuyến tính phân chia mẫu dương (positive samples) và mẫu âm (negative samples)

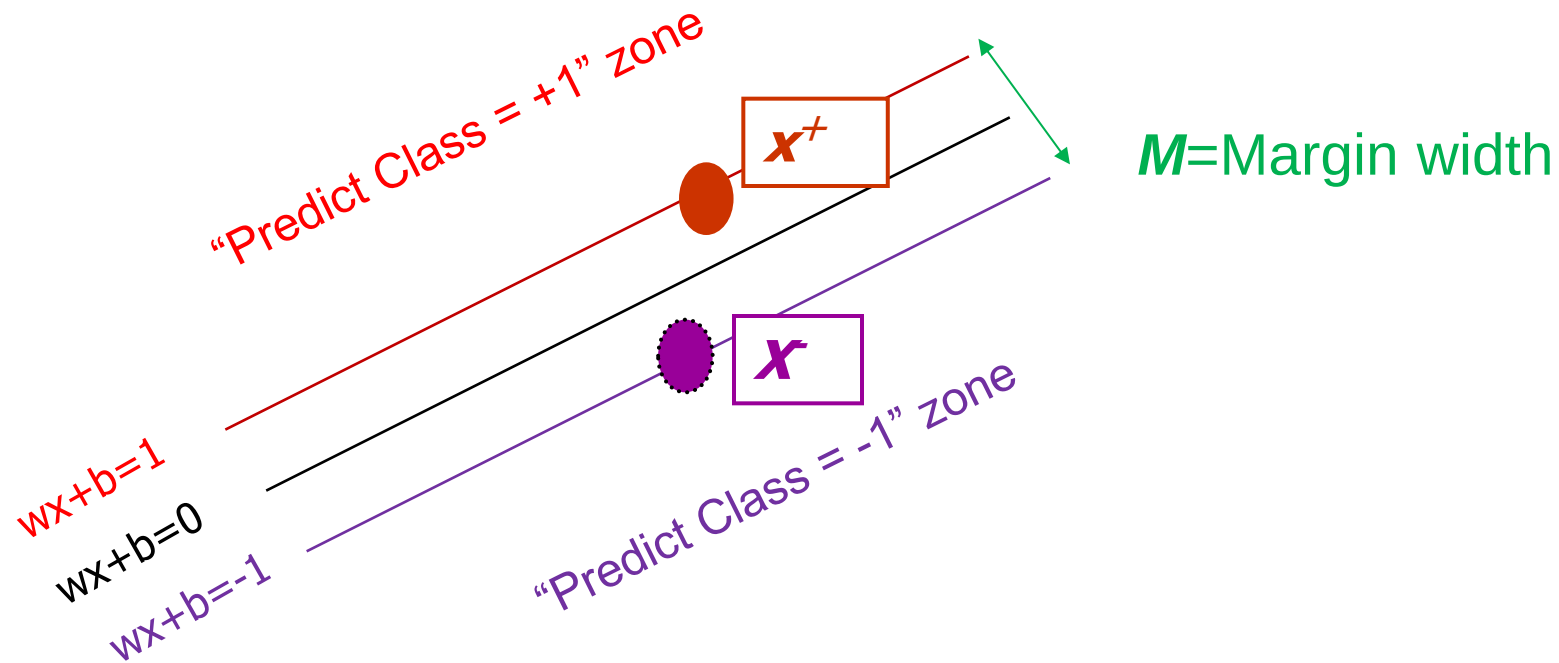
Hàm nào tốt nhất?

Support Vector Machines (SVMs)



- Là bộ phân loại dựa trên xây dựng **siêu phẳng** phân cách tối ưu (*đường thẳng trong trường hợp 2d*)
- Cực đại **lề (margin)** giữa các mẫu positives và các mẫu negatives

Mô hình của SVM tuyến tính



What we know:

- $w \cdot x^+ + b = +1$
- $w \cdot x^- + b = -1$

$$\Rightarrow w \cdot (x^+ - x^-) = 2$$

$$M = \frac{(x^+ - x^-) \cdot w}{|w|} = \frac{2}{|w|}$$

SVM tuyến tính

Tìm siêu phẳng cực đại hóa lề phân cách

- ♦ Mục tiêu: 1) Phân lớp đúng tất cả các mẫu dữ liệu

$$\begin{array}{ll} \mathbf{w} \mathbf{x}_i + \mathbf{b} \geq 1 & \text{if } y_i = +1 \\ \mathbf{w} \mathbf{x}_i + \mathbf{b} \leq 1 & \text{if } y_i = -1 \\ y_i (\mathbf{w} \mathbf{x}_i + \mathbf{b}) \geq 1 & \text{for all } i \end{array} \quad \left. \vphantom{\begin{array}{l} \mathbf{w} \mathbf{x}_i + \mathbf{b} \geq 1 \\ \mathbf{w} \mathbf{x}_i + \mathbf{b} \leq 1 \end{array}} \right\} \quad \curvearrowright$$

2) Cực đại hóa $M = \frac{2}{|\mathbf{w}|}$

hay cực tiểu hóa $\frac{1}{2} \mathbf{w}^t \mathbf{w}$

- ♦ Chuyển thành bài toán tối ưu toàn phương, giải để tìm $\mathbf{w}$ và $\mathbf{b}$:

Minimize $\Phi(\mathbf{w}) = \frac{1}{2} \mathbf{w}^t \mathbf{w}$

subject to $y_i (\mathbf{w} \mathbf{x}_i + \mathbf{b}) \geq 1 \quad \forall i$

Tìm siêu phẳng cực đại hóa lề phân cách

1. Cực đại hóa lề $2/\|\mathbf{w}\|$
2. Phân loại chính xác các mẫu huấn luyện:

$$\mathbf{x}_i \text{ positive } (y_i = 1) : \quad \mathbf{x}_i \cdot \mathbf{w} + b \geq 1$$

$$\mathbf{x}_i \text{ negative } (y_i = -1) : \quad \mathbf{x}_i \cdot \mathbf{w} + b \leq -1$$

- Bài toán *tối ưu* toàn phương:

$$\text{Cực tiểu hóa} \quad \frac{1}{2} \mathbf{w}^T \mathbf{w}$$

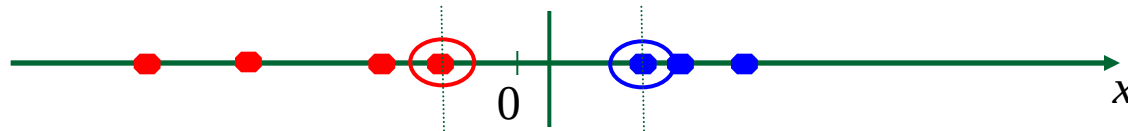
$$\text{s.t. } y_i(\mathbf{w} \cdot \mathbf{x}_i + b) \geq 1$$

Câu hỏi

- Làm sao nếu dữ liệu không thể phân tách tuyến tính?

SVMs phi tuyến

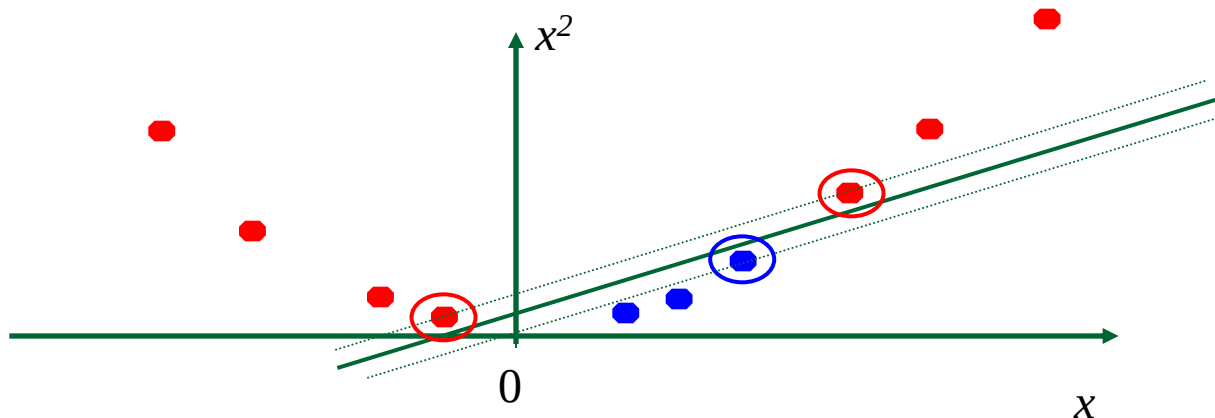
- Nếu dữ liệu có thể phân tách tuyến tính với một số nhiễu thì SVM thực hiện tốt.



- Nhưng nếu dữ liệu quá khó thì làm sao?

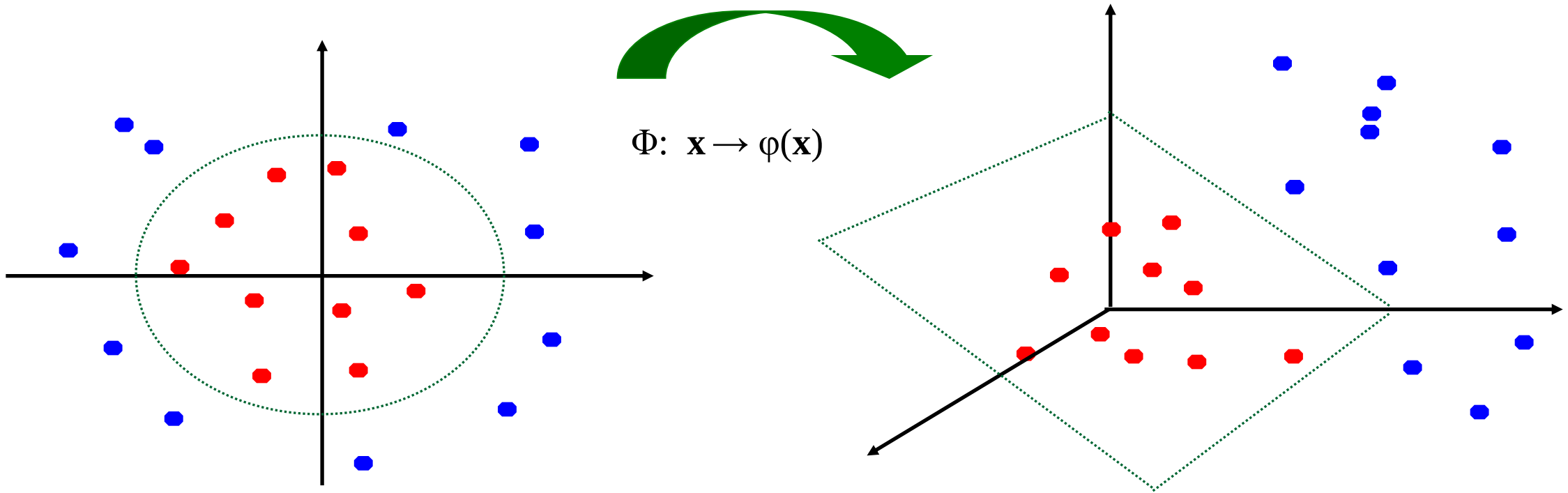


- Ánh xạ dữ liệu vào không gian cao chiều:



SVMs phi tuyến: không gian đặc trưng

- Ý tưởng chung: không gian dữ liệu ban đầu được ánh xạ vào không gian đặc trưng cao chiều hơn nơi mà tập mẫu huấn luyện có thể phân tách tuyến tính được.



SVMs phi tuyến

- *Kỹ thuật hàm nhân (kernel trick)*: Thay vì tính toán trực tiếp hàm chuyển đổi $\varphi(\mathbf{x})$, ta định nghĩa **hàm nhân** K thỏa mãn

$$K(\mathbf{x}_i, \mathbf{x}_j) = \varphi(\mathbf{x}_i) \cdot \varphi(\mathbf{x}_j)$$

- Kỹ thuật này đưa ra lời giải phi tuyến trong không gian dữ liệu ban đầu

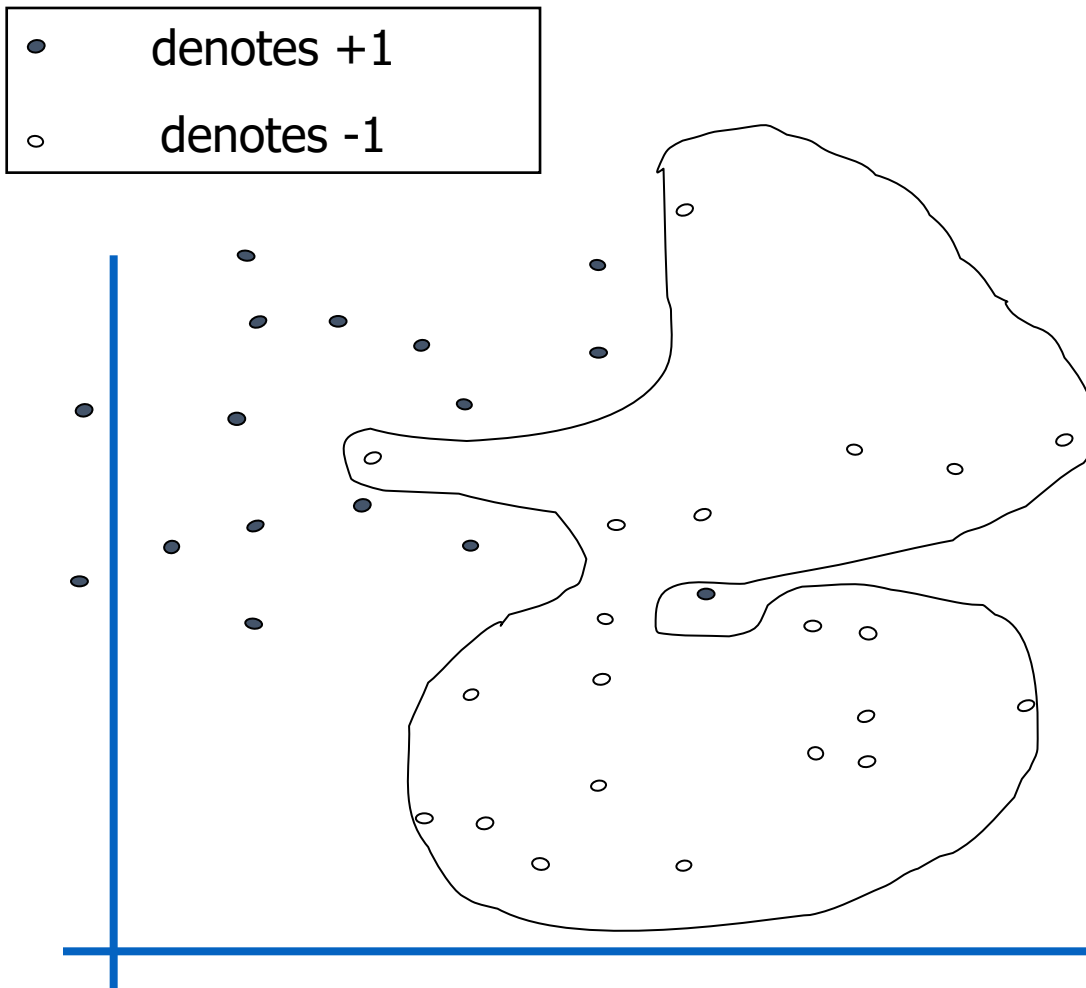
$$\sum_i \alpha_i y_i K(\mathbf{x}_i, \mathbf{x}) + b$$

Ví dụ một số hàm nhân

- Tuyến tính: $K(\mathbf{x}_i, \mathbf{x}_j) = \mathbf{x}_i^T \mathbf{x}_j$
- Gaussian RBF: $K(\mathbf{x}_i, \mathbf{x}_j) = \exp\left(-\frac{\|\mathbf{x}_i - \mathbf{x}_j\|^2}{2\sigma^2}\right)$
- Giao histogram: $K(\mathbf{x}_i, \mathbf{x}_j) = \sum_k \min(\mathbf{x}_i(k), \mathbf{x}_j(k))$
- Đa thức bậc p : $K(\mathbf{x}_i, \mathbf{x}_j) = (1 + \mathbf{x}_i^T \mathbf{x}_j)^p$
- Sigmoid: $K(\mathbf{x}_i, \mathbf{x}_j) = \tanh(\beta_0 \mathbf{x}_i^T \mathbf{x}_j + \beta_1)$

Khi huấn luyện SVM, cần xác định các tham số tương ứng với mỗi loại kernel được sử dụng.

Khi dữ liệu có nhiễu

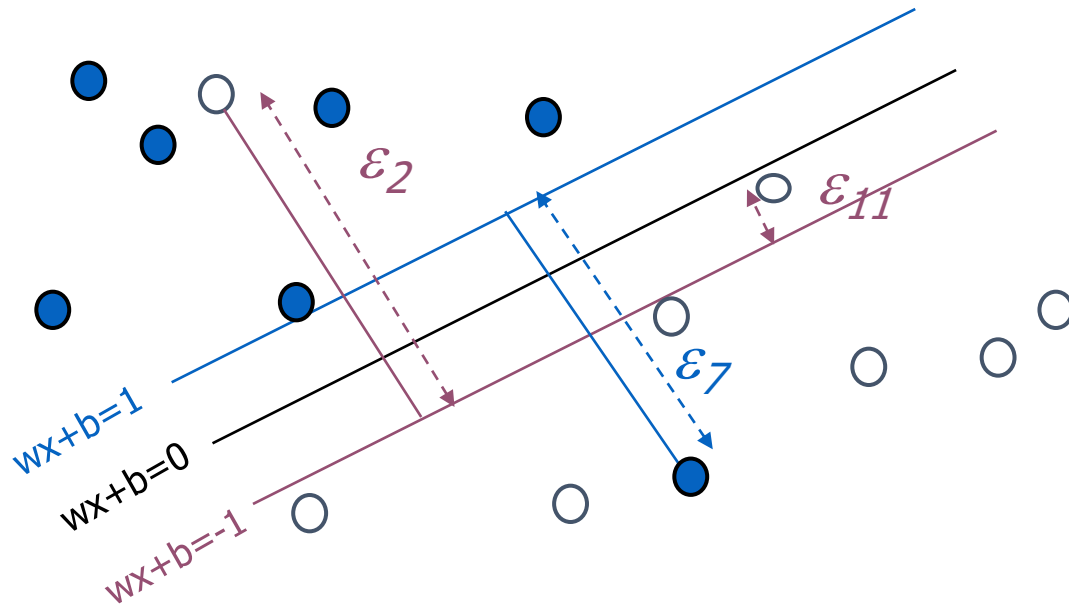


- ◆ Biên cứng (Hard Margin): Các điểm dữ liệu phải được phân lớp chính xác, không có sai số khi huấn luyện
- ◆ Điều gì xảy ra khi dữ liệu có nhiễu?
 - Giải pháp 1: use very powerful kernels

OVERFITTING!

Phân lớp với biên mềm

Thêm biến giả ξ_i cho phép có sai số cho những điểm khó phân lớp hoặc những điểm là nhiễu.



Điều kiện tối ưu trở thành:

Minimize

$$\frac{1}{2} \mathbf{w} \cdot \mathbf{w} + C \sum_{k=1}^R \varepsilon_k$$

C là tham số phải xác định khi huấn luyện SVM.

SVMs cho bài toán phân loại

1. Trích xuất véc tơ đặc trưng cho các mẫu huấn luyện.

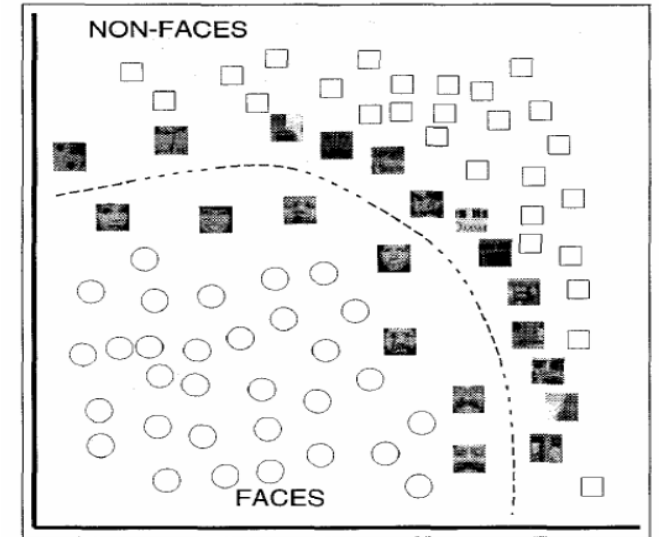
$D = (X, Y)$, X vector đặc trưng, Y - nhãn

1. Lựa chọn một hàm nhân (kernel)

2. Tính ma trận khoảng cách hàm nhân *kernel matrix* giữa các mẫu huấn luyện

3. Sử dụng “kernel matrix” huấn luyện SVM: xác định véc tơ hỗ trợ và các tham số w của mô hình

4. Phân loại mẫu dữ liệu mới: tính khoảng cách hàm nhân giữa mẫu mới và các véc tơ hỗ trợ, lắp w vào tính toán và kiểm tra dấu của kết quả biểu thức đầu ra.



Câu hỏi

- Làm sao nếu dữ liệu không thể phân tách tuyến tính?
- **Làm thế nào nếu có nhiều hơn hai lớp đối tượng?**

SVMs nhiều lớp

- Kết hợp nhiều SVM nhị phân
- **One vs. all**
 - Training: huấn luyện SVM cho từng lớp vs. phần còn lại
 - Testing: áp dụng từng SVM với mẫu test và gán nhãn cho lớp của SVM trả lại kết quả score cao nhất.
- **One vs. one**
 - Training: huấn luyện SVM cho từng cặp hai lớp
 - Testing: mỗi SVM “vote” một nhãn để gán cho mẫu test

SVMs: Ưu và nhược điểm

- Pros

- Sử dụng SVM hàm nhân rất linh hoạt và mạnh mẽ
- Thường số lượng véc tơ hỗ trợ khá thưa – hiệu quả đối với thời gian test
- Kết quả rất tốt trong thực tế, thậm chí đối với trường hợp tập huấn luyện bé

- Cons

- Không có SVM “trực tiếp” cho bài toán đa lớp, phải kết hợp các SVMs nhị phân
- Việc lựa chọn hàm nhân tốt nhất không đơn giản
- Độ phức tạp tính toán, bộ nhớ
 - Lúc huấn luyện, phải tính ma trận hàm nhân cho các cặp mẫu dữ liệu
 - Thời gian huấn luyện có thể rất lâu đối với các bài toán kích thước lớn

Khuyến nghị các bước sử dụng SVM cho người mới bắt đầu

- Chuyển dữ liệu về format sử dụng cho các gói công cụ SVM
 - Chuẩn hóa dữ liệu
 - [Thử với linear kernel nếu là bài toán đơn giản]
 - Thử với RBF kernel $K(\mathbf{x}, \mathbf{y}) = e^{-\gamma \|\mathbf{x} - \mathbf{y}\|^2}$
 - Dùng phương pháp kiểm tra chéo (cross-validation) để tìm các tham số C và γ tốt nhất
 - Dùng các giá trị C and γ tìm được để huấn luyện trên toàn bộ dữ liệu
 - Kiểm thử mô hình thu được (test).
-
- TK: <https://www.csie.ntu.edu.tw/~cjlin/papers/guide/guide.pdf>

Bài toán ứng dụng

Phân loại phương tiện giao thông

Car



Bus



Truck



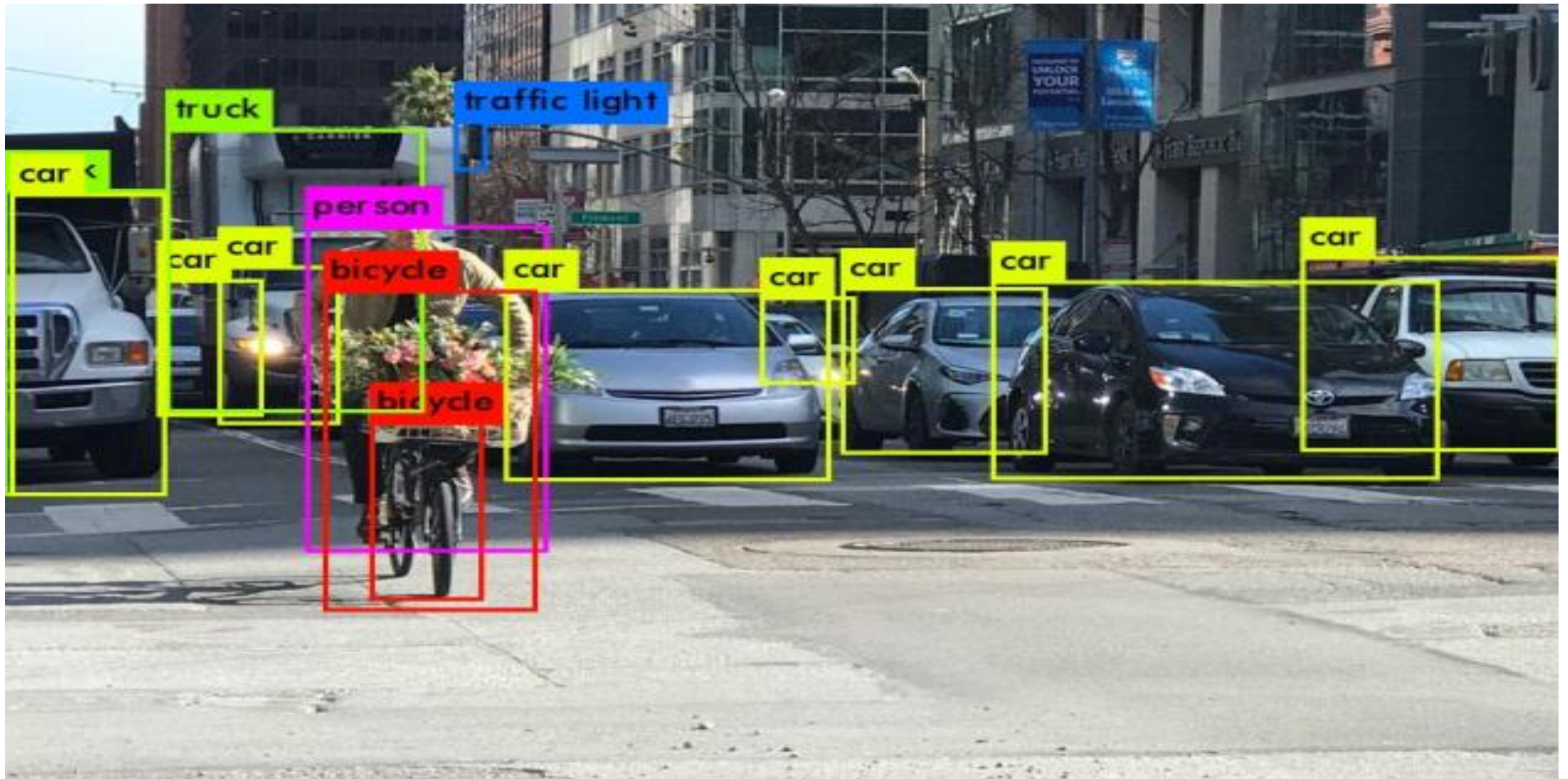
Motorbike



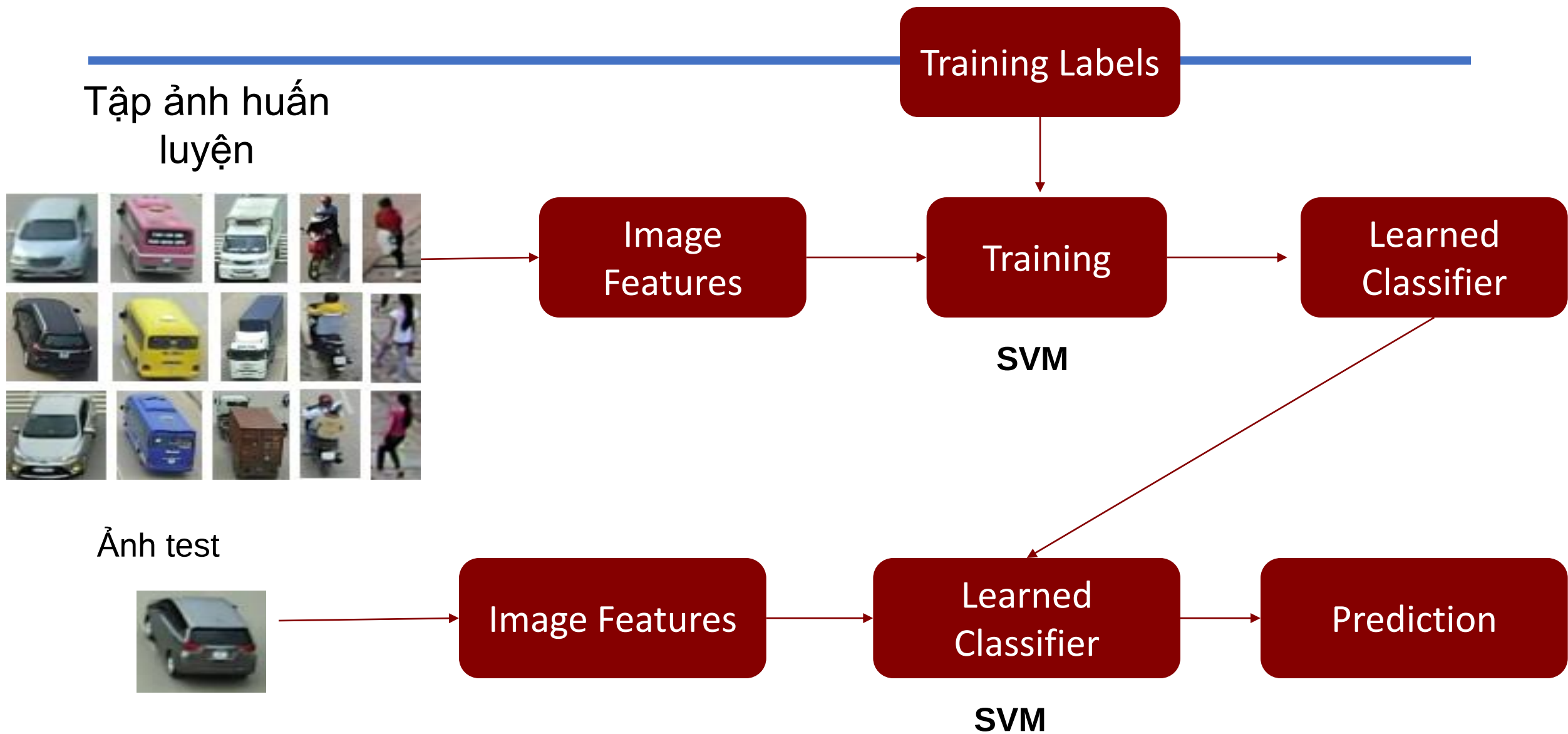
Pedestrian



Phân loại phương tiện giao thông



Sơ đồ quá trình huấn luyện và kiểm tra



Thực hành Matlab

Thực hành với `svmtrain` và `svmclassify` trong Matlab (cũ)

- Svmtrain: Hàm huấn luyện bộ phân lớp SVM
- Cú pháp:
 `SVMStruct = svmtrain(Training,Group)`
 `SVMStruct = svmtrain(Training,Group,Name,Value)`
- Mô tả:
 - `SVMStruct = svmtrain(Training,Group)` returns a structure, `SVMStruct`, containing information about the trained support vector machine (SVM) classifier.
 - `SVMStruct = svmtrain(Training,Group,Name,Value)` returns a structure with additional options specified by one or more `Name,Value` pair arguments.

Kiểm tra mô hình học được trên mẫu dữ liệu mới:

- `species = svmclassify(svmStruct,[5 2],'showplot',true)`
- `hold on;plot(5,2,'ro','MarkerSize',12);hold off`

Matlab help:

- <https://www.mathworks.com/help/stats/support-vector-machines-for-binary-classification.html>
- <https://www.mathworks.com/help/stats/referencelist.html?type=function&category=support-vector-machine-classification>
- <https://www.mathworks.com/help/stats/support-vector-machines-for-binary-classification.html#bsr5b6n>

- SVM validation (important for parameter tuning)

- `CVSVMModel = crossval(SVMModel);`

`% 10-fold by default`

- `kloss = kfoldLoss(CVSVMModel);`

`% SVM testing`

`pred = predict(SVMModel, testX);`

Thực hành

- Thực hành với thư viện SVM:
 - Thư viện: <https://www.csie.ntu.edu.tw/~cjlin/libsvm/index.html>
 - Công cụ: <https://www.csie.ntu.edu.tw/~cjlin/libsvmtools/>
 - Dữ liệu: <https://www.csie.ntu.edu.tw/~cjlin/libsvmtools/datasets/>
- File hướng dẫn sử dụng LibSVM trong link.

Thực hành

Python functions for SVM, see documentation at:

- scikit-learn: <http://scikit-learn.org/stable/modules/svm.html>
- LibSVM: <http://www.csie.ntu.edu.tw/~cjlin/libsvm/>

Lưu ý:

- scikit-learn sử dụng LibSVM để tính toán.
- LibSVM có chứa phần tìm kiếm tham số tối ưu dựa trên cài đặt Matlab cho SVM, và gồm SVM cho bài toán phân loại nhiều lớp.
- Tham khảo:
https://www.di.ens.fr/willow/teaching/recvis10/final_project

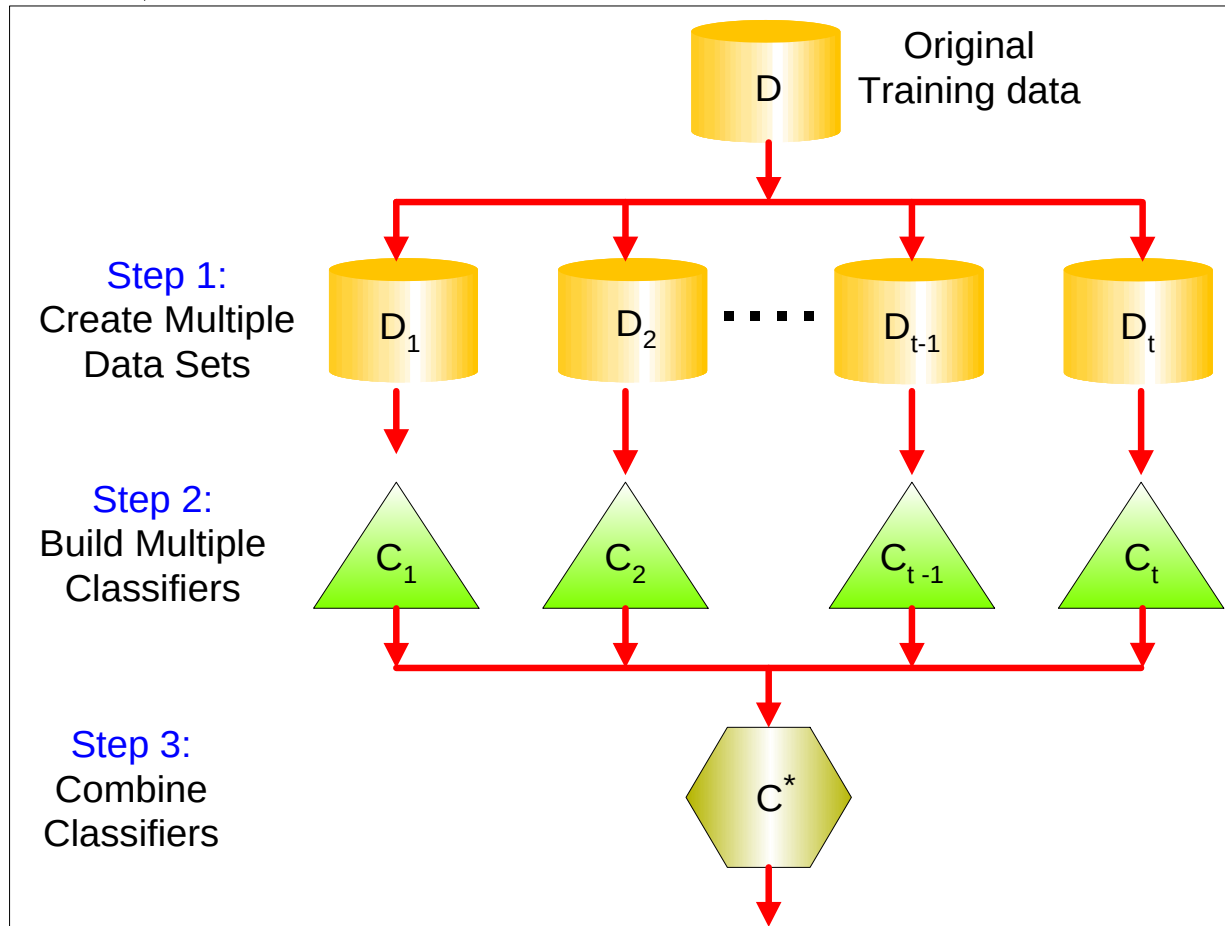
CÁC PHƯƠNG PHÁP HỌC KẾT HỢP BOOSTING VÀ RANDOM FOREST

NỘI DUNG

- Giới thiệu
- Thuật toán Bagging
- Thuật toán Boosting
- Thuật toán Random forest

Ý tưởng

Kết hợp các bộ phân lớp đơn giản để tạo thành bộ phân lớp mạnh (Ensemble methods – kết hợp, đồng thuận)



Thuật toán Bagging

- Also known as bootstrap aggregation

Original Data	1	2	3	4	5	6	7	8	9	10	D
Bagging (Round 1)	7	8	10	8	2	5	10	10	5	9	D1
Bagging (Round 2)	1	4	9	1	2	3	2	7	3	2	D2
Bagging (Round 3)	1	8	5	10	5	5	9	6	3	7	D3

- Sampling uniformly with replacement
- Build classifier on each bootstrap sample
- 0.632 bootstrap
- Each bootstrap sample D_i contains approx. 63.2% of the original training data
- Remaining (36.8%) are used as test set

Bagging

- Độ chính xác:

$$Acc(M) = \sum_{i=1}^k (0.632 * Acc(M_i)_{test_set} + 0.368 * Acc(M_i)_{train_set})$$

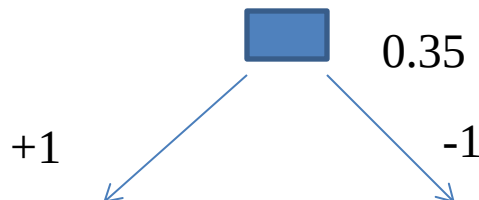
- Làm việc tốt với các bộ dữ liệu nhỏ
- Ví dụ:

x	0.1	0.2	0.3	0.4	0.5	0.6	0.7	0.8	0.9	1.0
y	1	1	1	-1	-1	-1	-1	1	1	1

- Dựng cây quyết định dạng **Decision stump**: chỉ có 2 nhánh

Error = 3/10

Accuracy = 70%



Bagging

- Decision Stump
- Single level decision binary tree
- Entropy – $x \leq 0.35$ or $x \leq 0.75$
- Accuracy at most 70%

Bagging Round 1:

x	0.1	0.2	0.2	0.3	0.4	0.4	0.5	0.6	0.9	0.9
y	1	1	1	1	-1	-1	-1	-1	1	1

$x \leq 0.35 \implies y = 1$

$x > 0.35 \implies y = -1$

Bagging Round 2:

x	0.1	0.2	0.3	0.4	0.5	0.8	0.9	1	1	1
y	1	1	1	-1	-1	1	1	1	1	1

$x \leq 0.65 \implies y = 1$

$x > 0.65 \implies y = 1$

Bagging Round 3:

x	0.1	0.2	0.3	0.4	0.4	0.5	0.7	0.7	0.8	0.9
y	1	1	1	-1	-1	-1	-1	-1	1	1

$x \leq 0.35 \implies y = 1$

$x > 0.35 \implies y = -1$

Bagging Round 4:

x	0.1	0.1	0.2	0.4	0.4	0.5	0.5	0.7	0.8	0.9
y	1	1	1	-1	-1	-1	-1	-1	1	1

$x \leq 0.3 \implies y = 1$

$x > 0.3 \implies y = -1$

Bagging Round 5:

x	0.1	0.1	0.2	0.5	0.6	0.6	0.6	1	1	1
y	1	1	1	-1	-1	-1	-1	1	1	1

$x \leq 0.35 \implies y = 1$

$x > 0.35 \implies y = -1$

Bagging Round 6:

x	0.2	0.4	0.5	0.6	0.7	0.7	0.7	0.8	0.9	1
y	1	-1	-1	-1	-1	-1	-1	1	1	1

$x \leq 0.75 \implies y = -1$

$x > 0.75 \implies y = 1$

Bagging Round 7:

x	0.1	0.4	0.4	0.6	0.7	0.8	0.9	0.9	0.9	1
y	1	-1	-1	-1	-1	1	1	1	1	1

$x \leq 0.75 \implies y = -1$

$x > 0.75 \implies y = 1$

Bagging Round 8:

x	0.1	0.2	0.5	0.5	0.5	0.7	0.7	0.8	0.9	1
y	1	1	-1	-1	-1	-1	-1	1	1	1

$x \leq 0.75 \implies y = -1$

$x > 0.75 \implies y = 1$

Bagging Round 9:

x	0.1	0.3	0.4	0.4	0.6	0.7	0.7	0.8	1	1
y	1	1	-1	-1	-1	-1	-1	1	1	1

$x \leq 0.75 \implies y = -1$

$x > 0.75 \implies y = 1$

Bagging Round 10:

x	0.1	0.1	0.1	0.1	0.3	0.3	0.8	0.8	0.9	0.9
y	1	1	1	1	1	1	1	1	1	1

$x \leq 0.05 \implies y = -1$

$x > 0.05 \implies y = 1$

Figure 5.35. Example of bagging.

Round	x=0.1	x=0.2	x=0.3	x=0.4	x=0.5	x=0.6	x=0.7	x=0.8	x=0.9	x=1.0
1	1	1	1	-1	-1	-1	-1	-1	-1	-1
2	1	1	1	1	1	1	1	1	1	1
3	1	1	1	-1	-1	-1	-1	-1	-1	-1
4	1	1	1	-1	-1	-1	-1	-1	-1	-1
5	1	1	1	-1	-1	-1	-1	-1	-1	-1
6	-1	-1	-1	-1	-1	-1	-1	1	1	1
7	-1	-1	-1	-1	-1	-1	-1	1	1	1
8	-1	-1	-1	-1	-1	-1	-1	1	1	1
9	-1	-1	-1	-1	-1	-1	-1	1	1	1
10	1	1	1	1	1	1	1	1	1	1
Sum	2	2	2	-6	-6	-6	-6	2	2	2
Sign	1	1	1	-1	-1	-1	-1	1	1	1
True Class	1	1	1	-1	-1	-1	-1	1	1	1

Figure 5.36. Example of combining classifiers constructed using the bagging approach.

Accuracy of ensemble classifier: 100% 😊

Thuật toán Boosting

- Thay đổi phân bố của dữ liệu huấn luyện bằng cách tập trung vào các mẫu bị phân lớp sai
 - Ban đầu trọng số của các mẫu được khởi tạo bằng nhau
- Sau mỗi bước:
 - **Tăng trọng số** cho những phần tử bị phân lớp **sai**
 - **Giảm** trọng số cho các phần tử được phân lớp **đúng**.

Adaboost

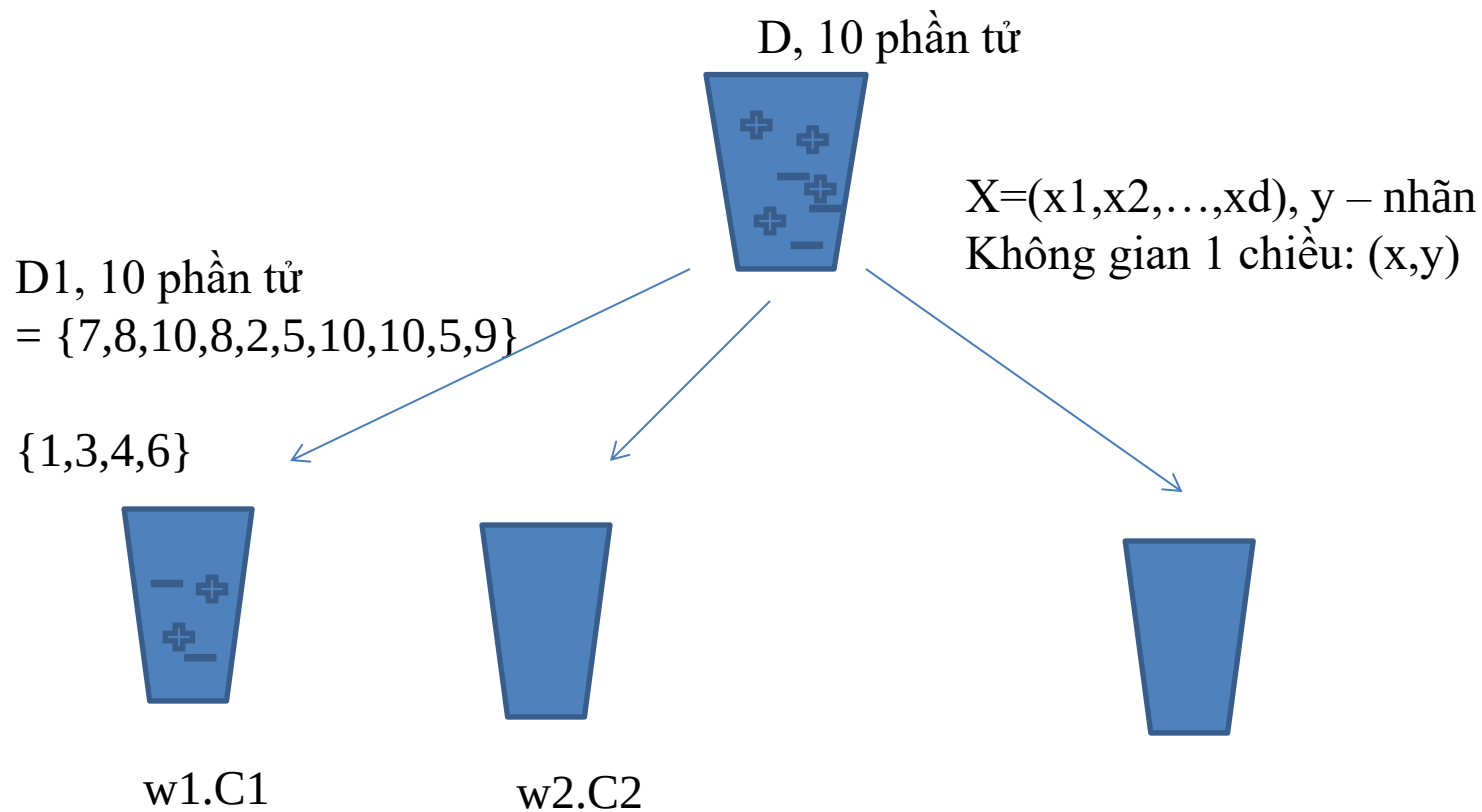
- Input:
 - Training set D containing d tuples
 - k rounds
 - A classification learning scheme
- Output:
 - A composite model

Adaboost

- Data set D containing d class-labeled tuples $(X_1, y_1), (X_2, y_2), (X_3, y_3), \dots, (X_d, y_d)$
- Initially assign equal weight $w = 1/d$ to each tuple
- To generate k base classifiers, we need k rounds (or iterations)
- Round i , tuples from D are sampled with replacement, to form D_i (size d)
- Each tuple's chance of being selected depends on its weight.

Adaboost

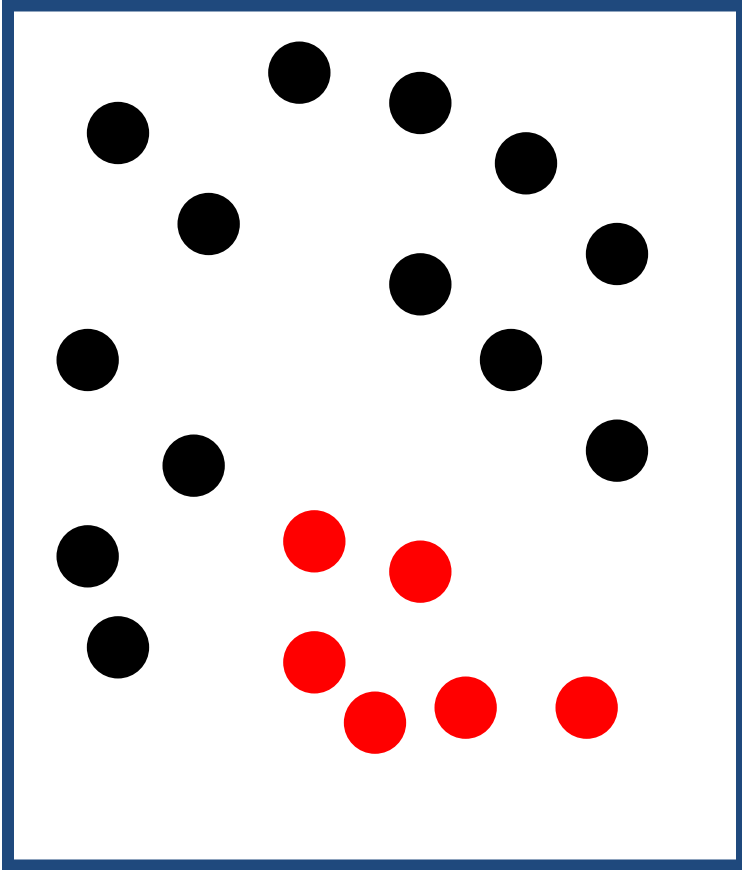
- Base classifier M_i , is derived from training tuples of D_i
- Error of M_i is tested using D_i
- Weights of training tuples are adjusted depending on how they were classified
 - Correctly classified: Decrease weight
 - Incorrectly classified: Increase weight
- Weight of a tuple indicates how hard it is to classify it (directly proportional).



Xây dựng 1 mô hình phân lớp
Dạng cây Decision tree

$C_strong = \text{kết hợp các } C_weak$
 $\text{kết hợp } w_i.C_i$

Boosting



Given:

- set of labeled training samples
- weight distribution over them

Algorithm:

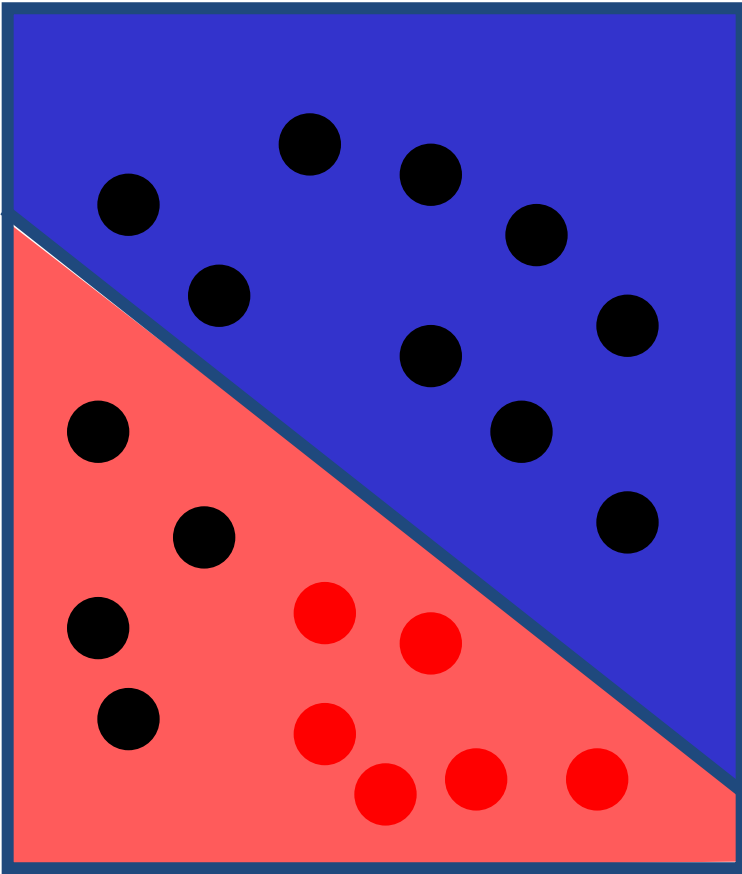
for $n = 1$ to N

- train a weak classifier (c_i) using samples and weight dist.
- calculate error
- calculate weight for c_i
- update weight dist.

next

Y. Freund and R. Schapire. **A decision-theoretic generalization of on-line learning and an application to boosting.** Journal of Computer and System Sciences, 1997.

Boosting



Given:

- set of labeled training samples
- weight distribution over them

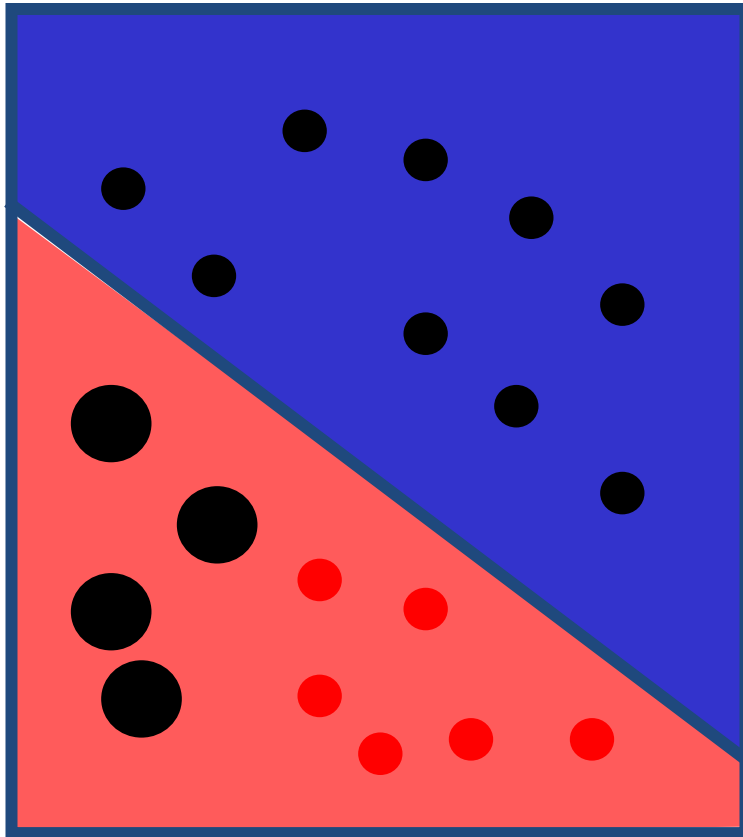
Algorithm:

for $n = 1$ to N

- train a weak classifier using samples and weight dist.
- calculate error
- calculate weight
- update weight dist.

next

Boosting



Given:

- set of labeled training samples
- weight distribution over them

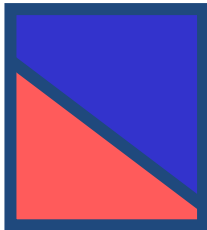
Algorithm:

for $n = 1$ to N

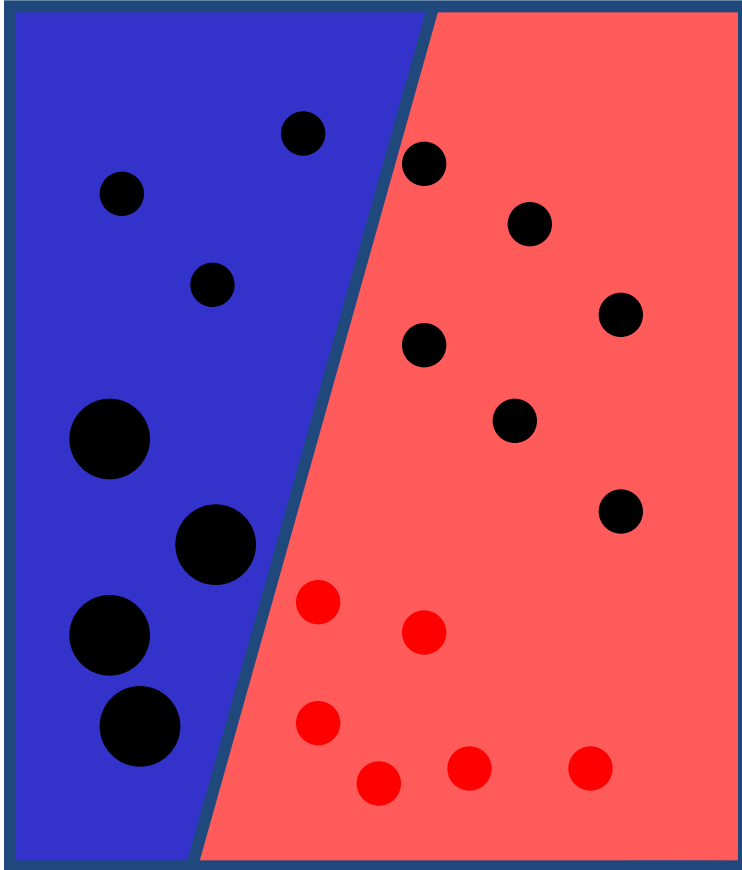
- train a weak classifier using samples and weight dist.
- calculate error
- calculate weight
- update weight dist.

next

α_1



Boosting



Given:

- set of labeled training samples
- weight distribution over them

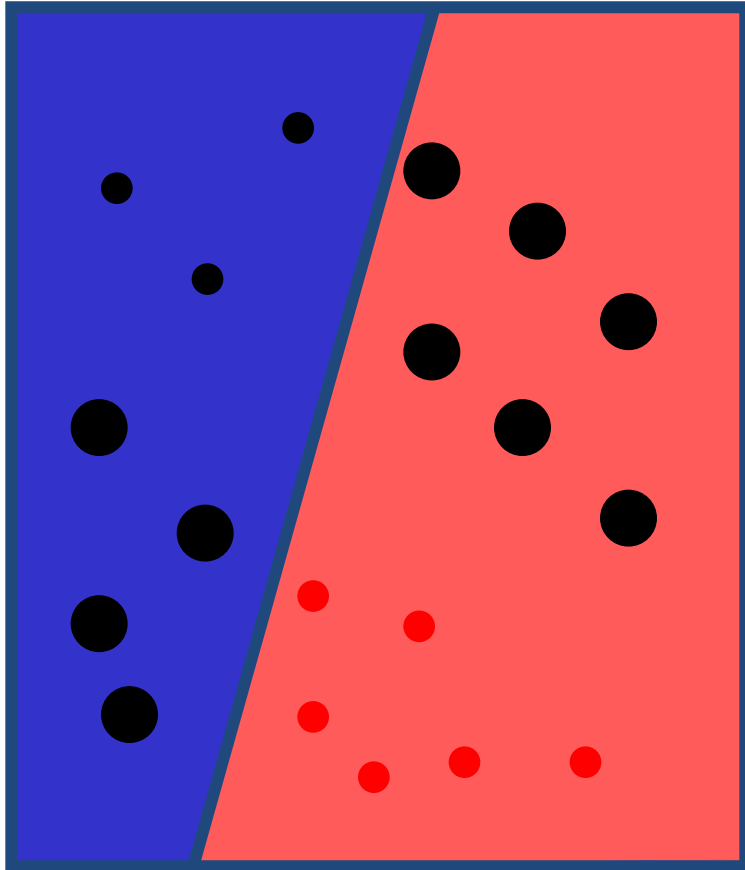
Algorithm:

for $n = 1$ to N

- train a weak classifier using samples and weight dist.
- calculate error
- calculate weight
- update weight dist.

next

Boosting



Given:

- set of labeled training samples
- weight distribution over them

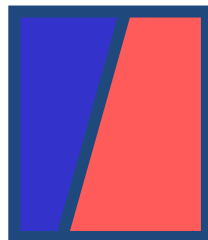
Algorithm:

for n = 1 to N

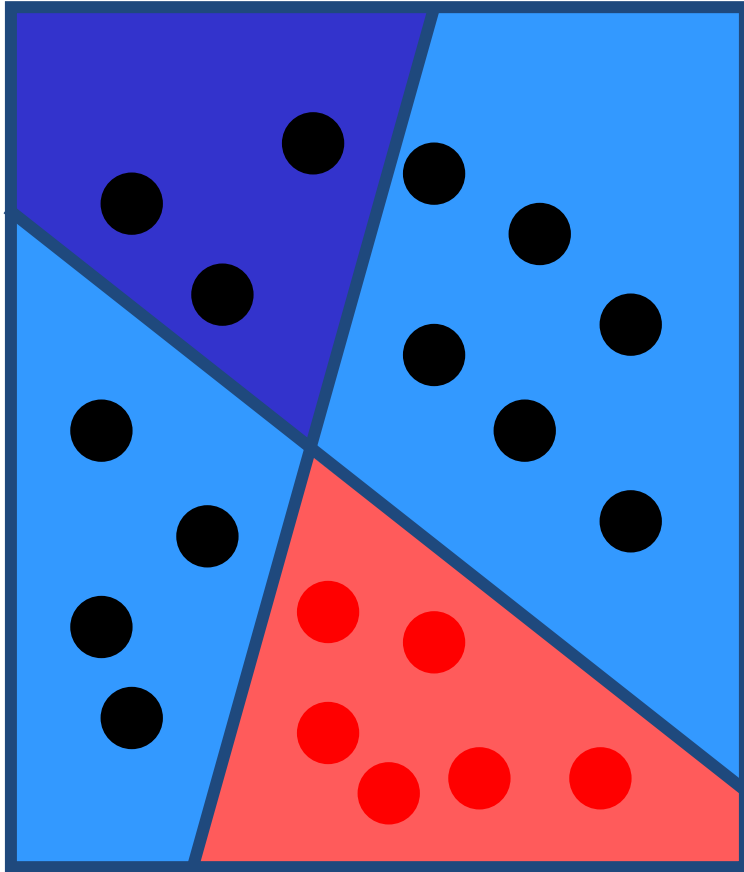
- train a weak classifier using samples and weight dist.
- calculate error
- calculate weight
- update weight dist.

next

α_2



Boosting



Given:

- set of labeled training samples
- weight distribution over them

Algorithm:

for $n = 1$ to N

- train a weak classifier using samples and weight dist.
- calculate error
- calculate weight
- update weight dist.

next

Result:

$$h^{strong}(x) = \text{sign}\left(\sum_{n=1}^N \alpha_n \cdot h_n^{weak}(x)\right)$$

$$= \alpha_1 \cdot \begin{array}{|c|} \hline \text{Dark Blue Triangle} \\ \hline \end{array} + \alpha_2 \cdot \begin{array}{|c|} \hline \text{Dark Blue Triangle} \\ \hline \end{array}$$

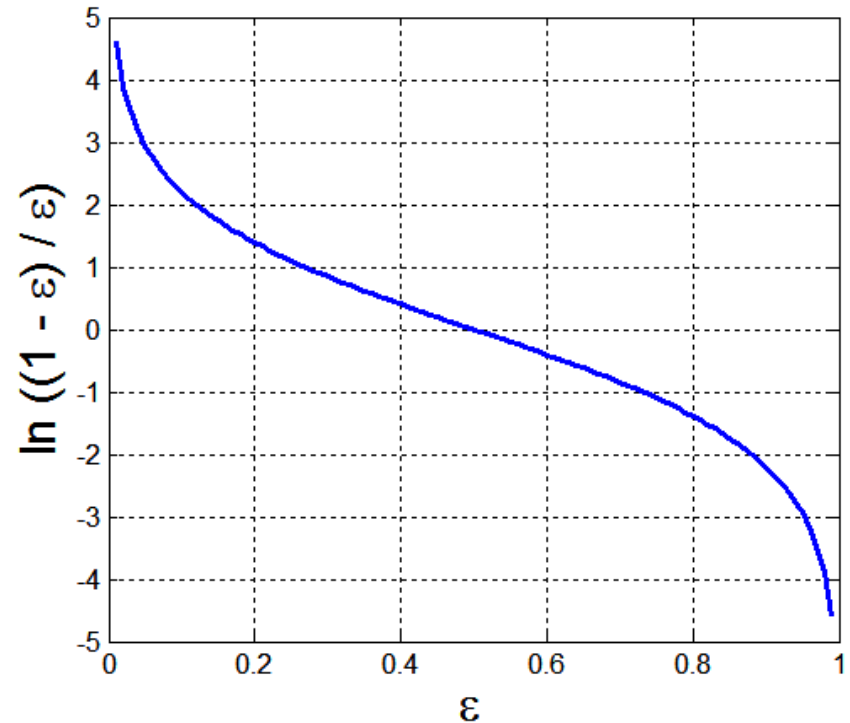
AdaBoost

- Base classifiers: $C_1, C_2, \dots, C_T$
(weak classifiers)
- Error rate:

$$\varepsilon_i = \frac{1}{N} \sum_{j=1}^N w_j \delta(C_i(x_j) \neq y_j)$$

- Importance of a classifier:

$$\alpha_i = \frac{1}{2} \ln \left(\frac{1 - \varepsilon_i}{\varepsilon_i} \right)$$



AdaBoost

- Weight update:

$$w_i^{(j+1)} = \frac{w_i^{(j)}}{Z_j} \begin{cases} \exp^{-\alpha_j} & \text{if } C_j(x_i) = y_i \\ \exp^{\alpha_j} & \text{if } C_j(x_i) \neq y_i \end{cases}$$

where Z_j is the normalization factor

- If any intermediate rounds produce error rate higher than 50%, the weights are reverted back to $1/n$ and the re-sampling procedure is repeated
- Classification:

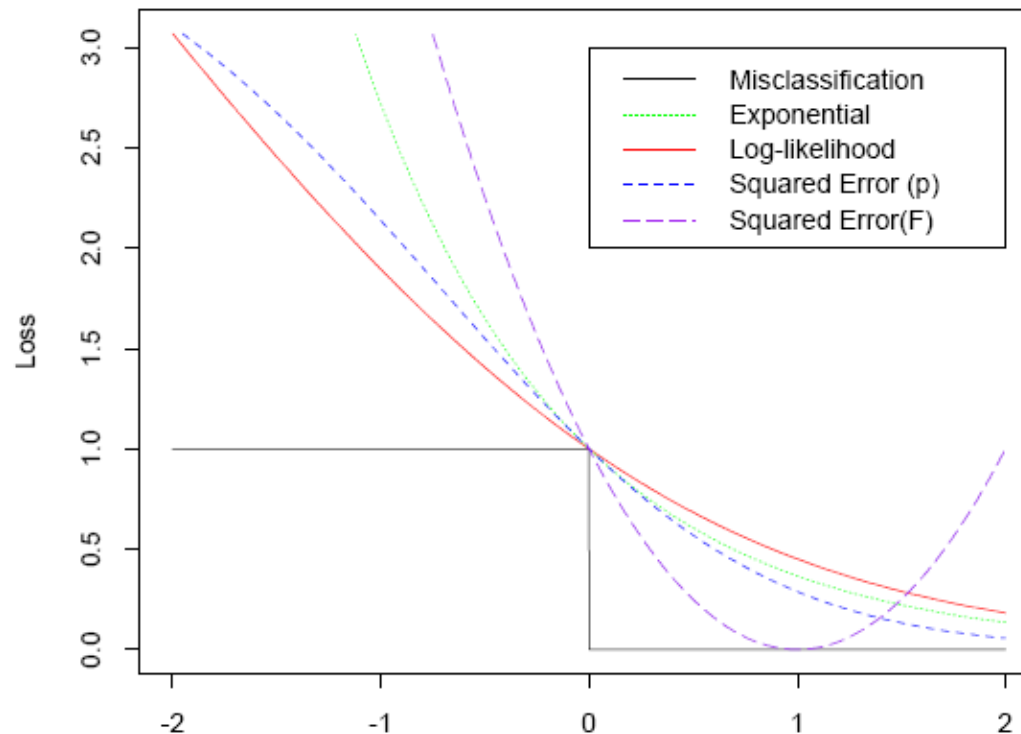
$$C^*(x) = \arg \max_y \sum_{j=1}^T \alpha_j \delta(C_j(x) = y)$$

Lựa chọn hàm lỗi

$$J(F) = E[e^{-yF(x)}]?$$

$$- E[\log(1 + e^{-2yF(x)})]$$

Losses as Approximations to Misclassification Error

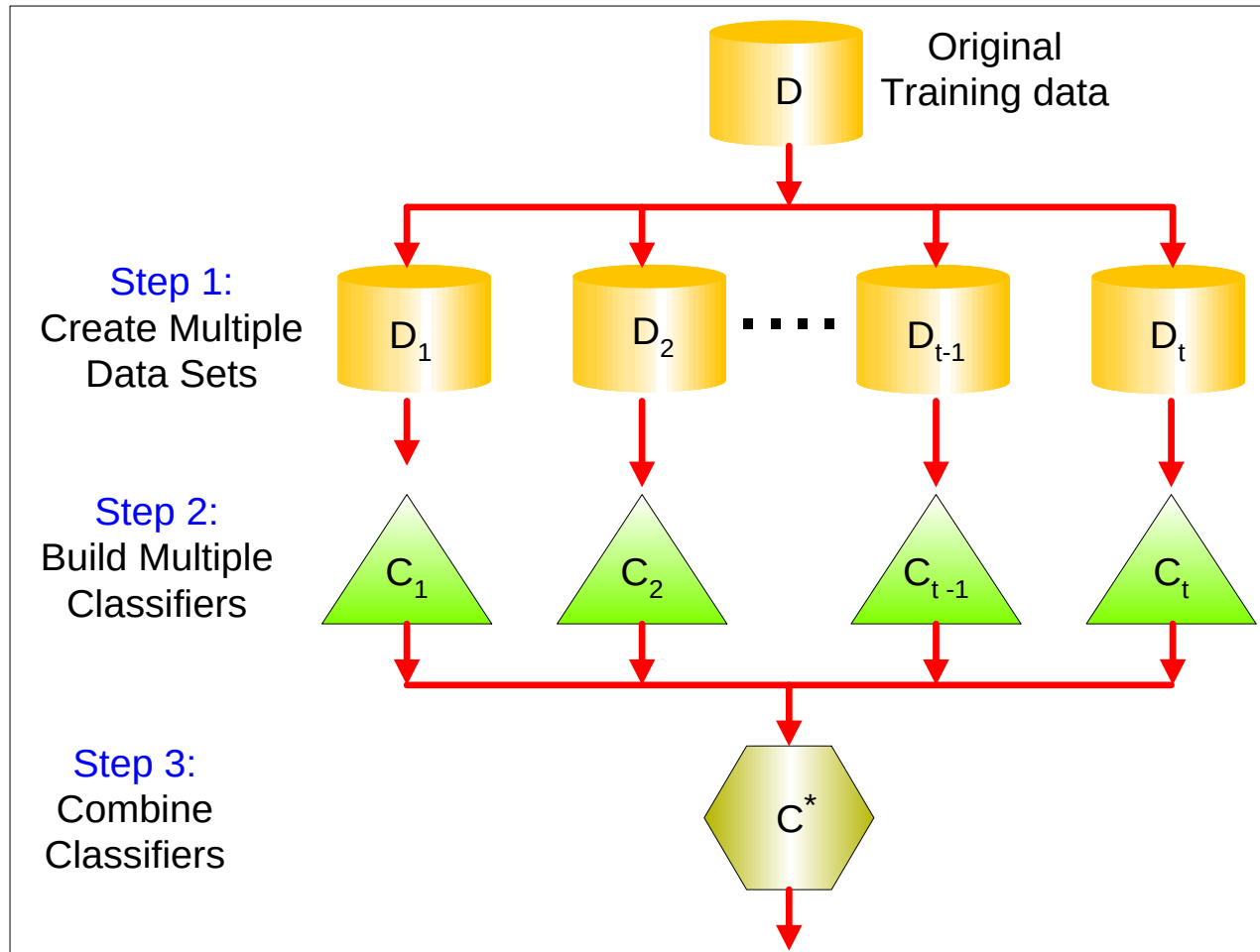


Bagging (Breiman 1994,...)

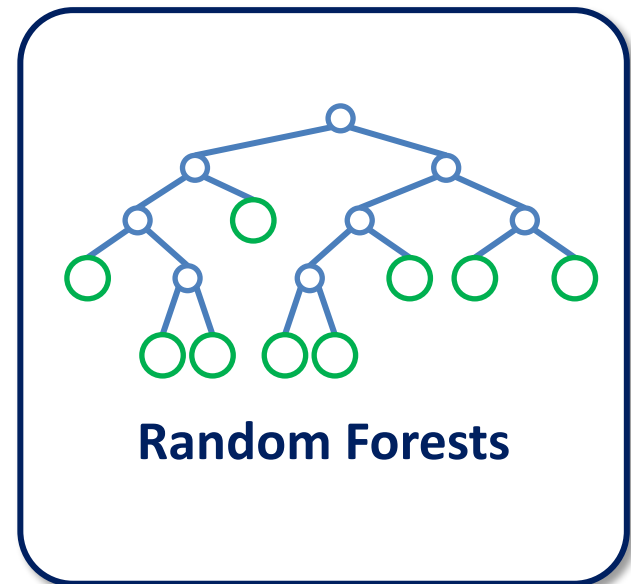
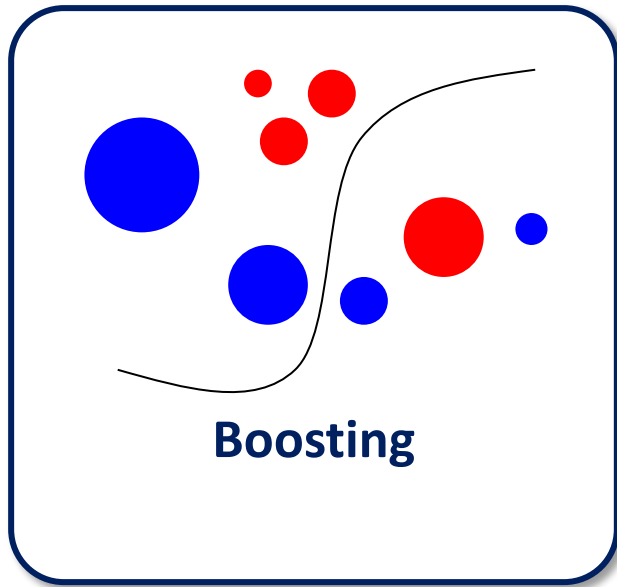
Boosting (Freund and Schapire 1995, Friedman et al. 1998)

Random forests (Breiman 2001,...)

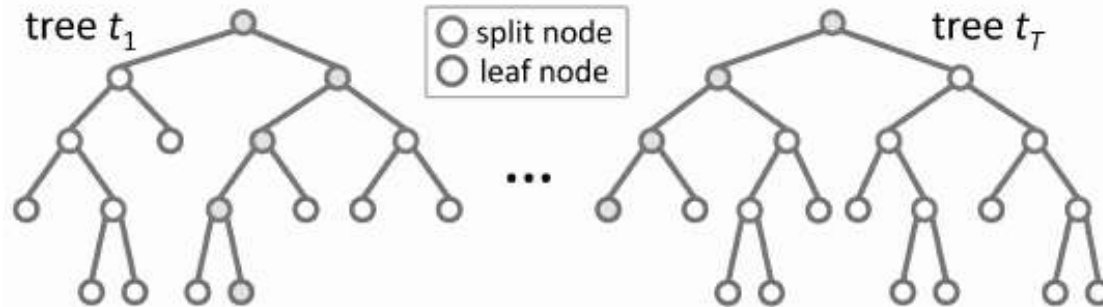
Thuật toán Rừng ngẫu nhiên – Ý tưởng



Boosting vs Random forest

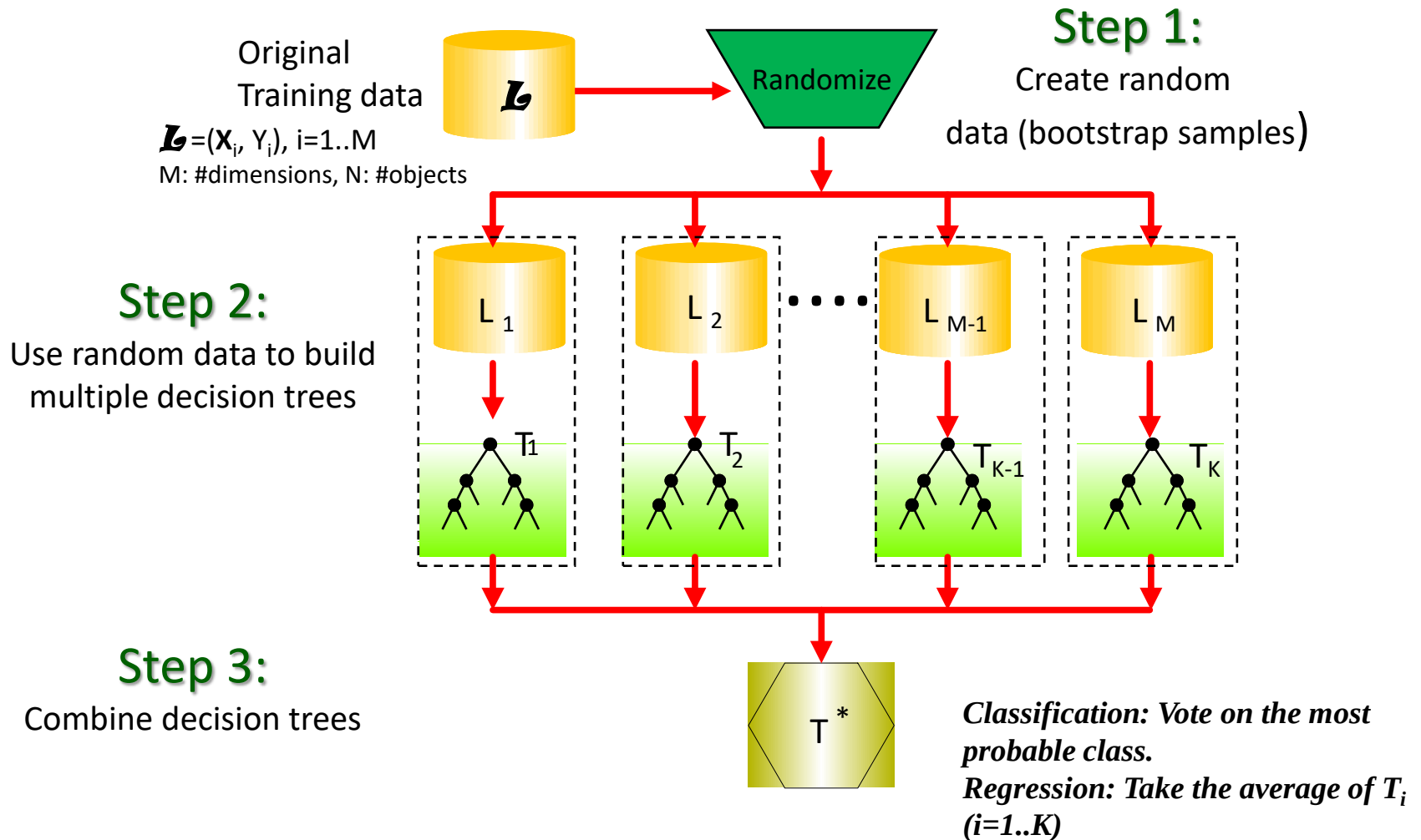


Random Forest



- Random forests (RF) are a combination of tree predictors
- Each tree depends on the values of a random vector sampled independently
- The generalization error depends on the strength of the individual trees and the correlation between them
- Using a random selection of features yields results favorable to AdaBoost, and are more robust w.r.t. noise.

Random Forest



The Random Forests algorithm

Given a training set D

For $i = 1$ to k do:

Build **subset D_i** by sampling with replacement from D

Learn **tree T_i** from D_i

At each node:

Choose best split from **random subset of F features**

Each tree grows to the largest extent, and no pruning

Make predictions according to **majority vote** of the set of k trees.

Random Forest

In classification/regression:

- High-dimensional data (thousands of features).
- Multiple data types.
- Missing value.
- Big data.

	f1	f2	f3	f4	f5	...	Thousands of features
1	X	Y	Z	X	3	7	...
2	C	V	Z	Y	5	4	...
	...	...	...	...	...	...	...
	...	...	...	...	...	...	...
	...	...	...	...	...	...	...
	Z	C	E	□	0	9	...
n-4	X	V	S	X	□	8	...
n-3	X	C	□	X	□	0	...
n-2	□	V	E	Y	6	8	...
n-1	X	S	V	Y	5	7	...
n	Z	S	V	X	0	8	...

Một số tính chất của RF

- Có đầy đủ các điểm mạnh của Cây quyết định, nhưng có hiệu năng tốt hơn
- Hoạt động hiệu quả với các bộ dữ liệu lớn
- Có thể xử lý hàng ngàn biến đầu vào
- Có thể ước lượng các biến quan trọng cho phân lớp
- Có thể xử lý hiệu quả (duy trì độ chính xác) với các tập dữ liệu có thuộc tính bị thiếu/mất một số giá trị.
- Có thể dùng để xác định mối quan hệ giữa các biến.

Thực hành ứng dụng

Thuật toán phân cụm K-mean và Mean-shift

Nội dung buổi học

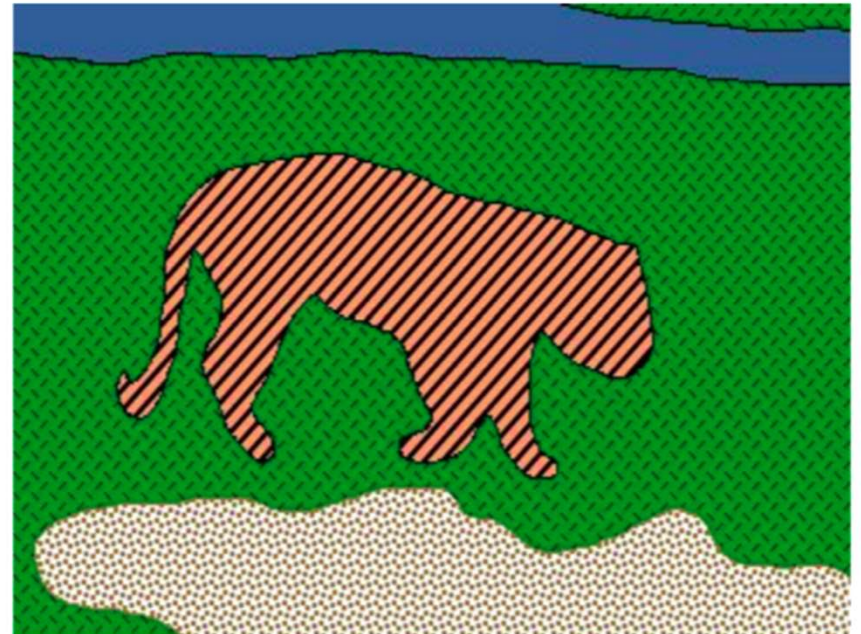
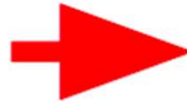
1. Giới thiệu bài toán phân cụm
2. Phân cụm K-mean
3. Phân cụm Mean-shift
4. Ứng dụng

Nội dung buổi học

1. **Giới thiệu bài toán phân cụm**
2. Phân cụm K-mean
3. Phân cụm Mean-shift
4. Ứng dụng trong phân khúc khách hàng

1. Giới thiệu bài toán phân cụm

- Ứng dụng: Phân đoạn ảnh (image segmentation)
- Mục tiêu: Xác định tập các điểm ảnh tương đồng biểu diễn các đối tượng trong ảnh

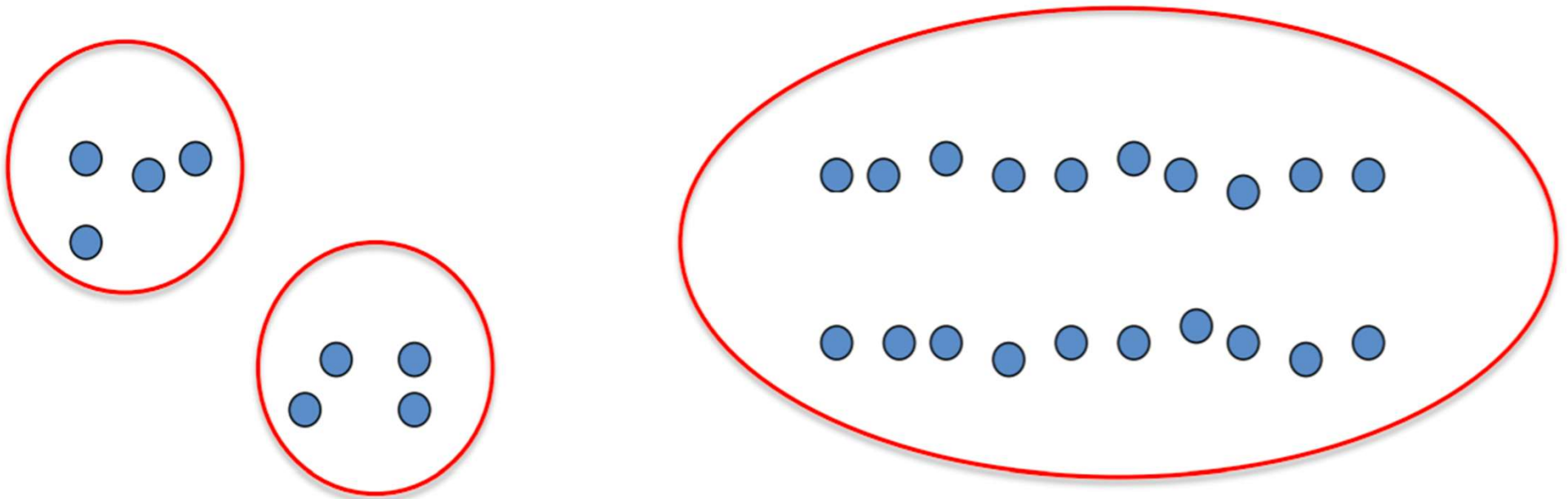


Bài toán phân cụm (Clustering)

- Là phương pháp học *không giám sát* để phân cụm (gom nhóm) tập mẫu dữ liệu thành các cụm (nhóm) dựa trên các thuộc tính của dữ liệu.
- Yêu cầu dữ liệu, nhưng không cần gán nhãn: $D(X, -)$ không có Y
- Ứng dụng: nhận dạng mẫu. Ví dụ:
 - Nhóm các email
 - Đặc trưng các nhóm khách hàng
 - Các vùng của ảnh.
- Sử dụng khi chúng ta không biết nhãn của dữ liệu → Bài toán đi tìm mối quan hệ nội tại của dữ liệu.

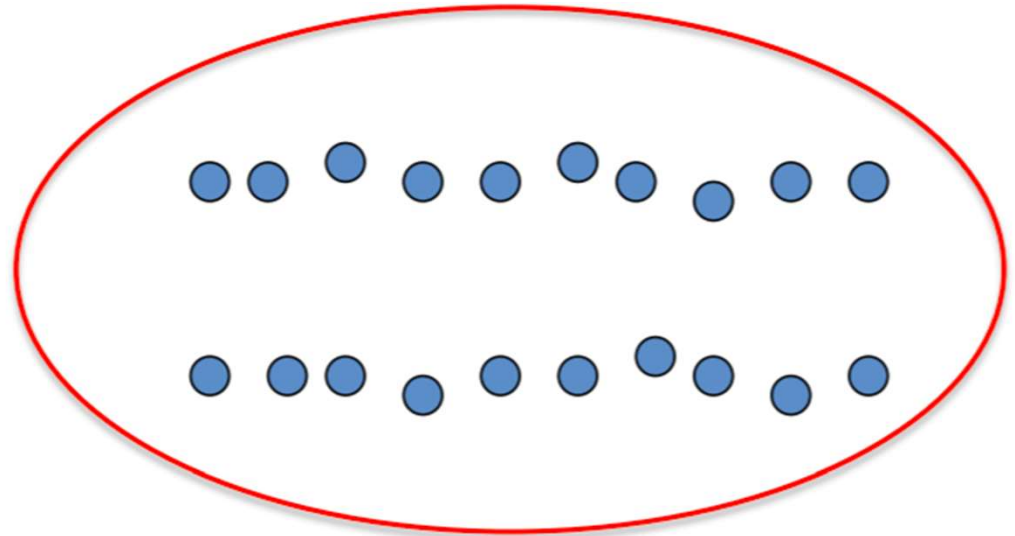
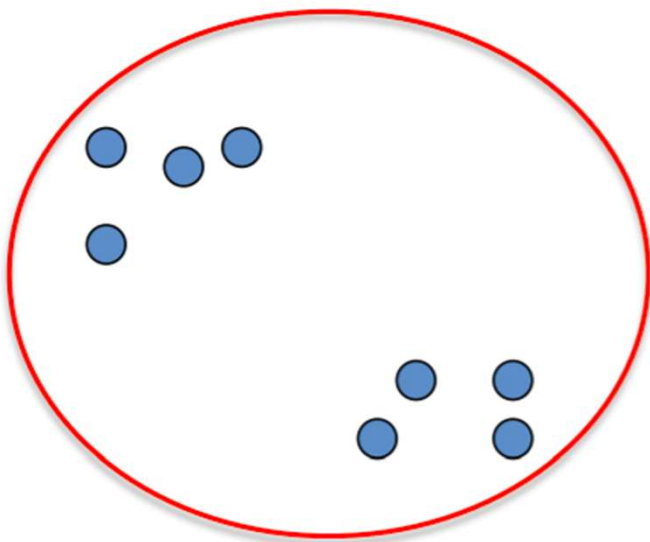
Kỹ thuật phân cụm

- Ý tưởng: nhóm các mẫu dữ liệu có đặc trưng giống nhau
- Ví dụ: các điểm trong không gian 2D



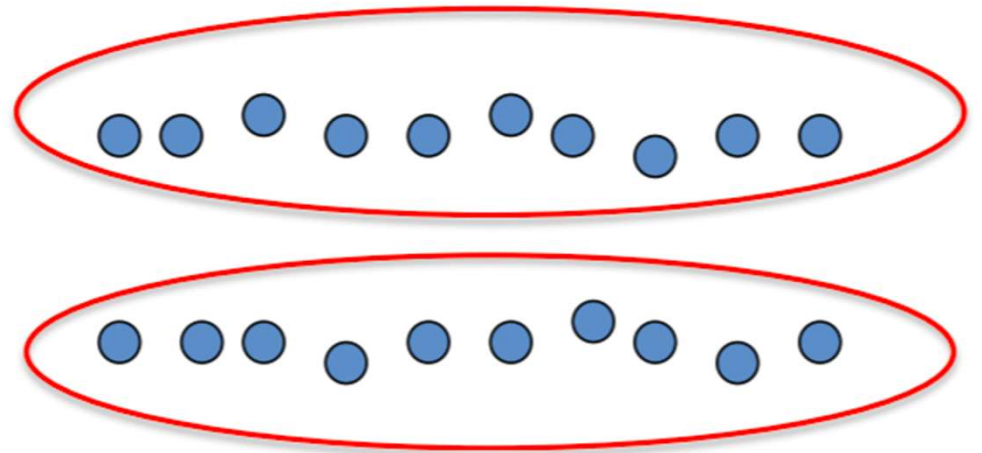
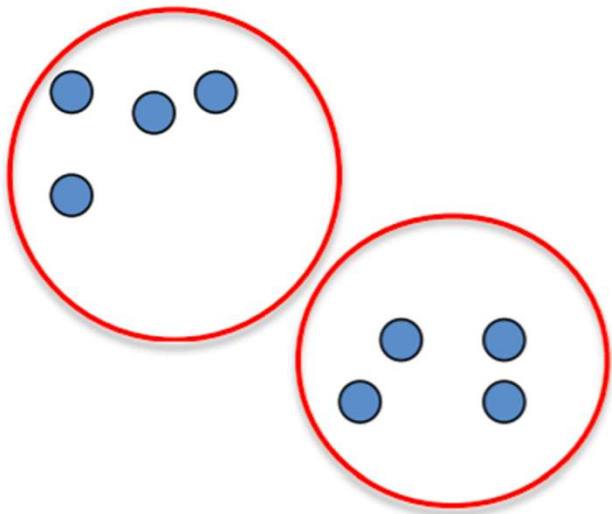
Kỹ thuật phân cụm

- Ý tưởng: nhóm các mẫu dữ liệu có đặc trưng giống nhau
- Ví dụ: các điểm trong không gian 2D



Kỹ thuật phân cụm

- Ý tưởng: nhóm các mẫu dữ liệu có đặc trưng giống nhau
- Ví dụ: các điểm trong không gian 2D



- Làm thế nào để gom nhóm các dữ liệu “gần nhau”?

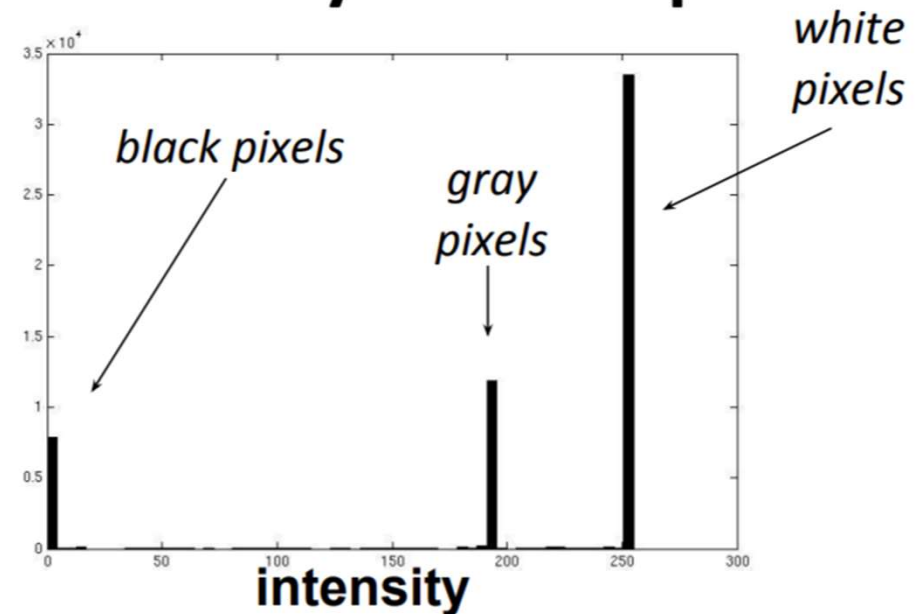
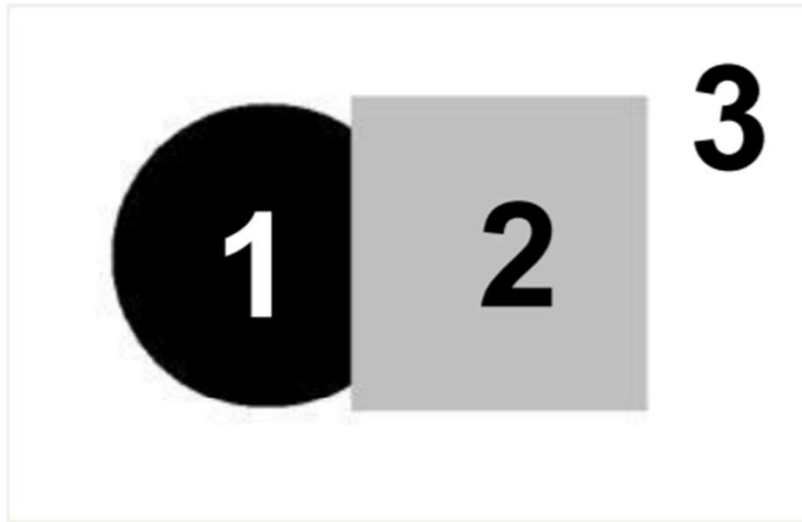
Độ đo tương tự (similarity metric)

- Độ đo đơn giản: khoảng cách Euclidean

$$\text{dist}(\vec{x}, \vec{y}) = \|\vec{x} - \vec{y}\|_2^2$$

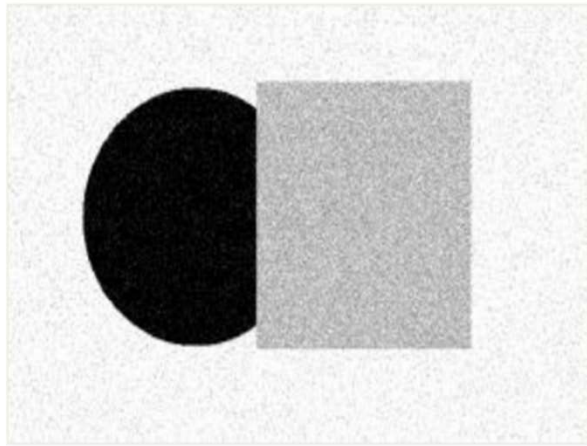
- Kết quả phân cụm dữ liệu phụ thuộc nhiều vào loại độ đo tương tự mà chúng ta chọn để đo sự tương đồng giữa các điểm dữ liệu.

Ví dụ về Phân đoạn ảnh (Example)

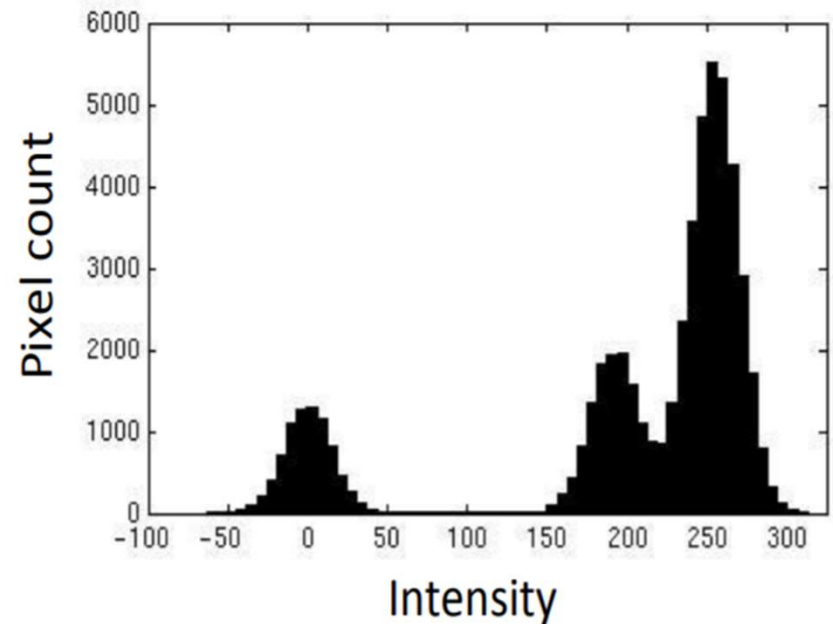


- Để phân đoạn ảnh này ta có thể dùng cường độ sáng để phân biệt 3 vùng ảnh: đen, xám, trắng
- Tức là phân đoạn ảnh dựa trên đặc trưng là cường độ sáng.
- Điều gì sẽ xảy ra nếu ảnh có màu sắc không đơn giản như thế này?

Ví dụ về Phân đoạn ảnh (Example)

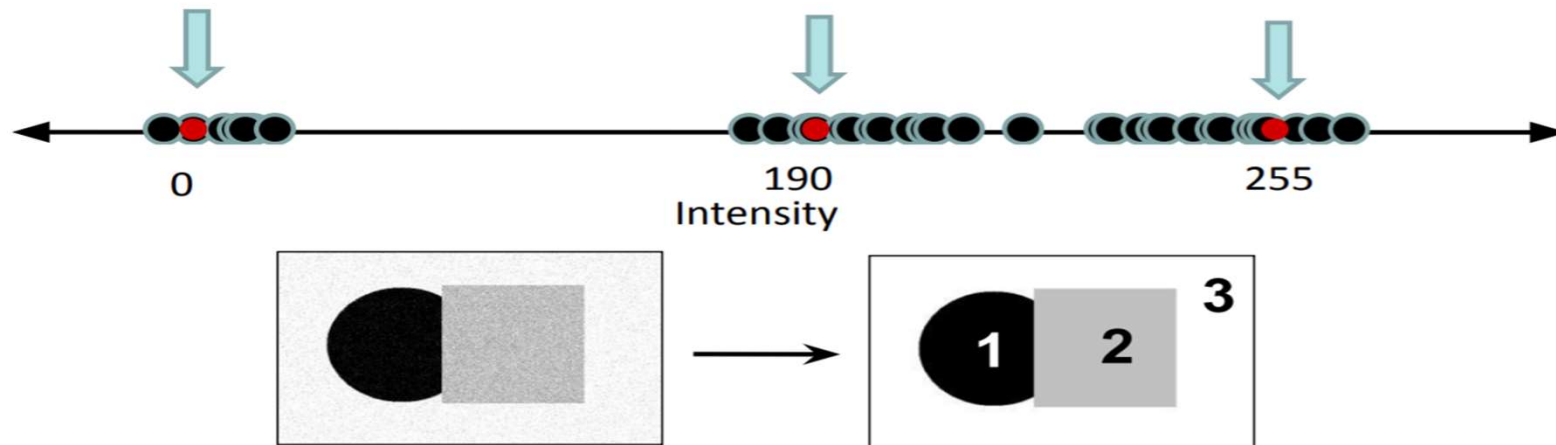


Input image



- Làm cách nào để xác định 3 nhóm cường độ sáng để phân đoạn ảnh?
- Cần kỹ thuật phân cụm!

Ví dụ về Phân đoạn ảnh (Example)



- Mục tiêu: chọn 3 “tâm” biểu diễn 3 loại cường độ sáng. Sau đó gán nhãn mỗi pixel dựa vào tâm nào nó gần nhất.
- Tâm cụm tốt nhất là những điểm mà làm giá trị tổng bình phương khoảng cách (Sum of Square Distance, SSD) nhỏ nhất.

$$SSD = \sum_i^k \sum_{x \in c_i} (x - c_i)^2$$

Hàm mục tiêu (Objective function)

- Giảm sai số việc gán nhầm các cụm

$$c^*, \delta^* = \arg \min_{c, \delta} \frac{1}{N} \sum_j^N \sum_i^k \delta_{ij} (c_i - x_j)^2$$

Cluster center Data

Whether x_j is assigned to c_i

Nội dung (Content)

1. Giới thiệu bài toán phân cụm
2. **Phân cụm K-mean**
3. Phân cụm Mean-shift
4. Ứng dụng trong phân khúc khách hàng

Phân cụm K-means (K-mean clustering)

➤ Khởi tạo

- Chọn k tâm cụm

➤ Lặp

- Bước gán cụm: Với mỗi điểm dữ liệu, gán nó với tâm cụm gần nhất
- Bước cập nhật cụm: Cập nhật lại tâm cụm là điểm trung bình của các điểm trong cụm

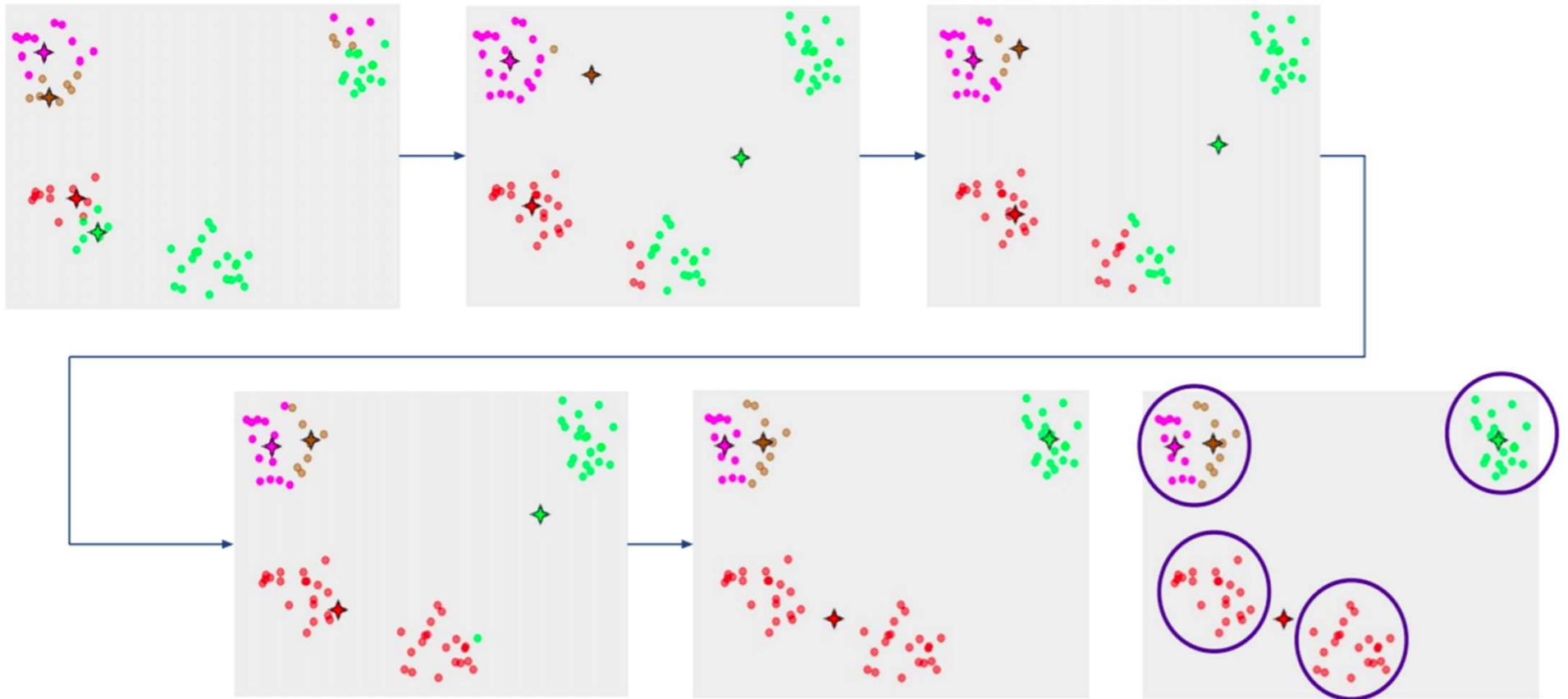
➤ Cho đến khi

- Đạt số vòng lặp tối đa
- Bước gán cụm không thay đổi cụm của các điểm
- Giá trị hàm mục tiêu giảm không đáng kể.

Phân cụm K-means (K-mean clustering)

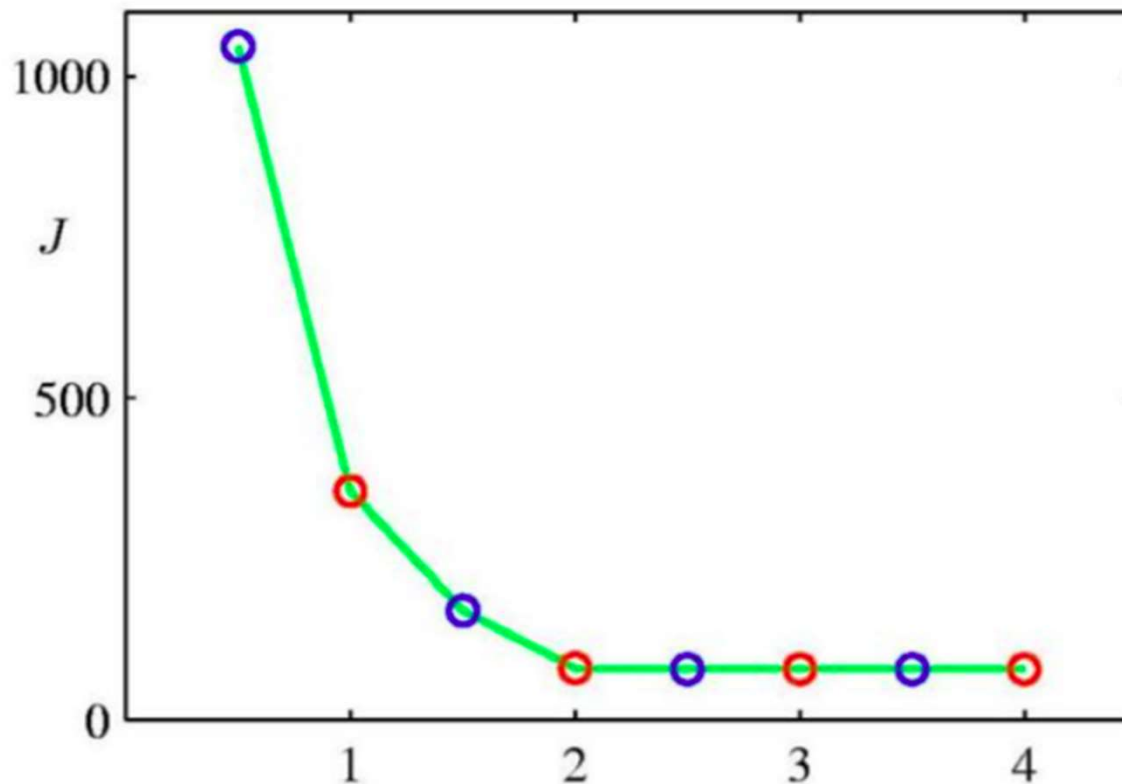
- Kỹ thuật này rất ‘nhạy cảm’ với việc khởi tạo
- Khởi tạo kém có thể dẫn tới:
 - Tốc độ hội tụ chậm
 - Kết quả phân cụm kém
- Làm cách nào khởi tạo:
 - Chọn ngẫu nhiên từ dữ liệu
 - Tìm điểm K “spread-out” (k-means++)
 - Nên cố gắng khởi tạo nhiều lần và chọn điểm tốt nhất

K-means: Khởi tạo (Initialize)



K-means: chọn K thế nào?

Plot các giá trị K khác nhau với giá trị hàm mục tiêu.



K-means: chọn K

- Sử dụng tập kiểm tra (validation)
- Thử một số lượng các cụm (cluster) khác nhau và xem kết quả đánh giá.

K-means: Độ đo tương tự và Điều kiện dừng

- Chọn 1 số độ đo tương đồng:
 - Euclidean (sử dụng nhiều nhất)
 - Cosine
 - non-linear (kernel k-means)
- Điều kiện dừng:
 - Đạt số vòng lặp tối đa
 - Không thay đổi bước gán các điểm vào cụm (hội tụ)
 - Giá trị hàm mục tiêu không giảm nhiều.

Đánh giá (Evaluation)

- Phương pháp sinh:

- Giá trị hàm lỗi

- Phương pháp phân biệt:

- Các điểm đã được gán vào các cụm đúng hay không dựa trên nhãn

- Chú ý: Phân cụm không giám sát không có mục đích phân biệt.

Ví dụ phân cụm ảnh với K-means

Image



Intensity-based clusters



Color-based clusters



Image source: Forsyth & Ponce

K-means: Ưu nhược điểm

- Ưu điểm:
 - Tìm tâm cụm bằng cách tối thiểu hóa hàm mục tiêu
 - Đơn giản, nhanh, dễ cài đặt
- Nhược điểm:
 - Cần chọn K
 - Nhạy cảm với nhiễu
 - Có thể rơi vào trạng thái cực trị cục bộ
 - Có thể rất chậm với dữ liệu nhiều chiều

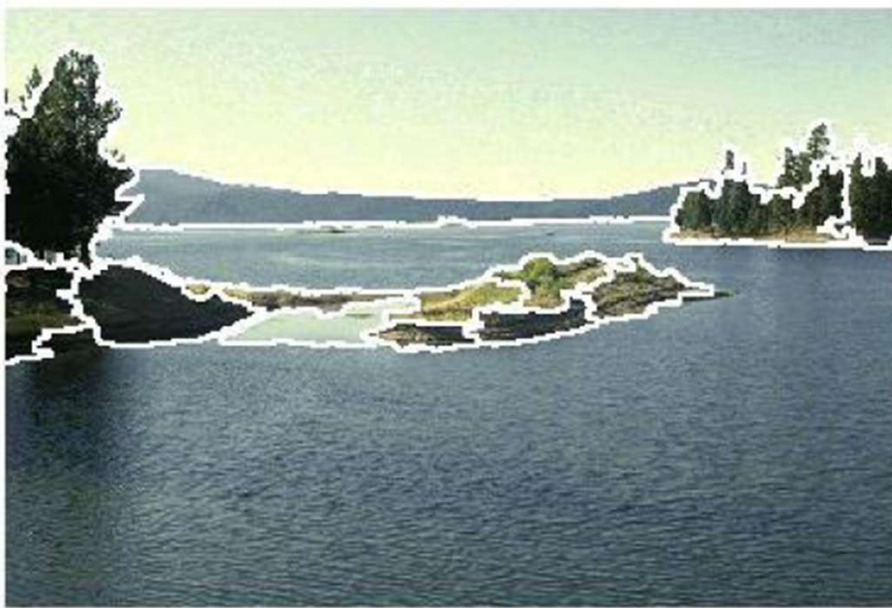
Nội dung

1. Giới thiệu bài toán phân cụm
2. Phân cụm K-mean
- 3. Phân cụm Mean-shift**
4. Ứng dụng trong phân khúc khách hàng

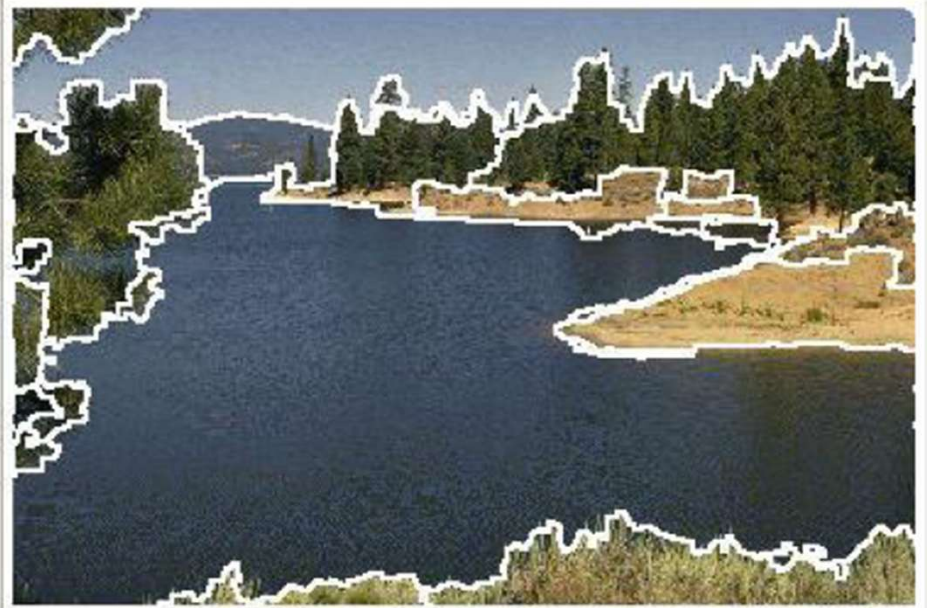
Mean-shift

Một kỹ thuật tiên tiến và linh hoạt cho phân đoạn dựa trên phân cụm.

Segmented "landscape 1"



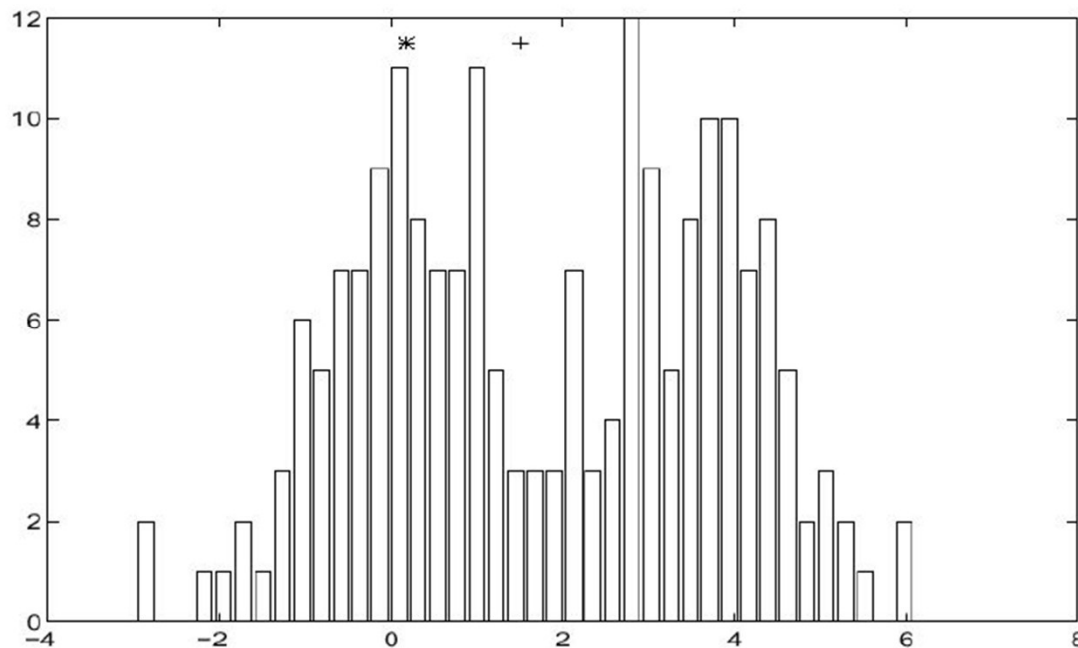
Segmented "landscape 2"



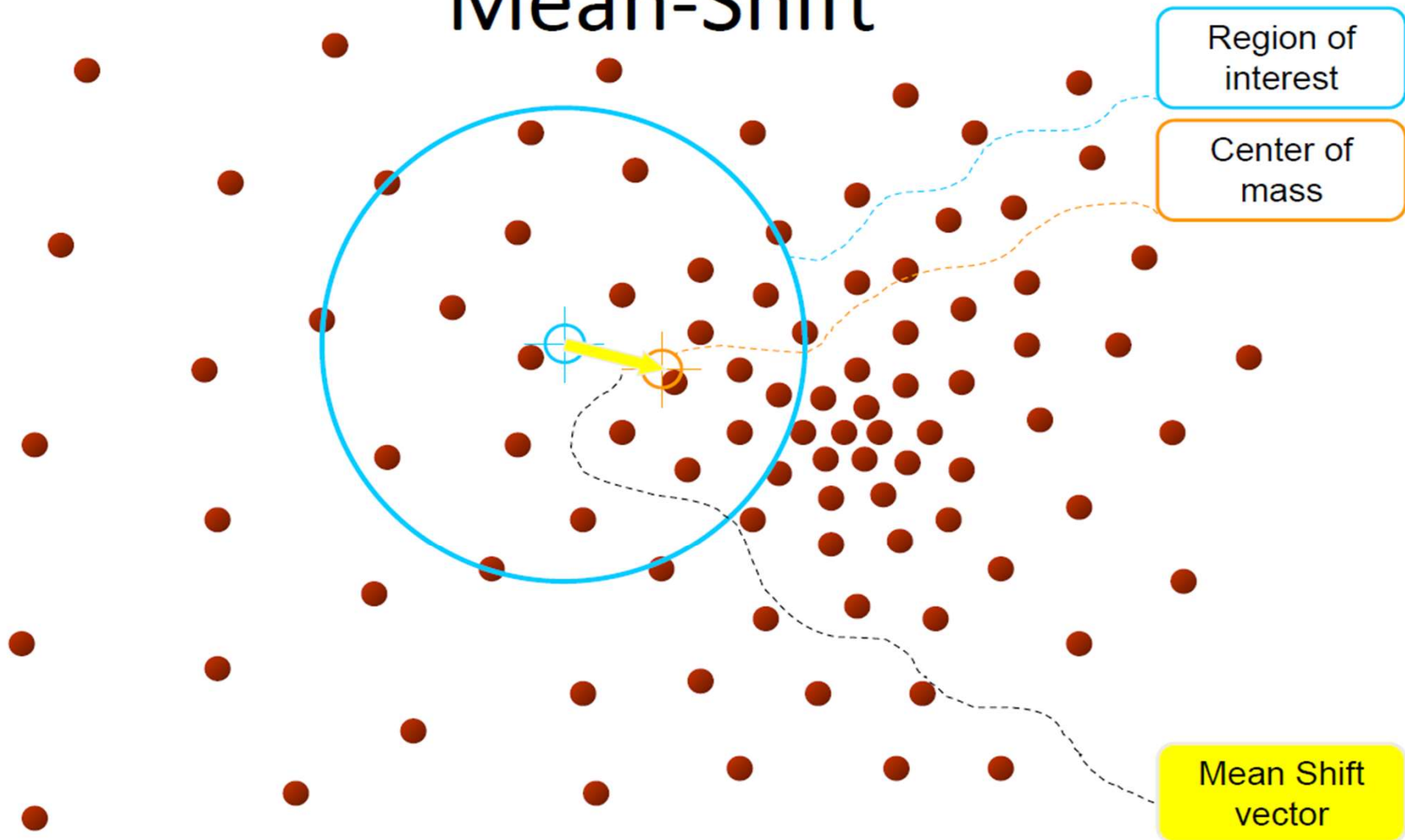
Thuật toán Mean-Shift

➤ Vòng lặp tìm kiếm

1. Khởi tạo dãy số ngẫu nhiên (random seed), cửa sổ W
2. Tính tâm (mean) W
3. Dịch (shift) cửa sổ tìm kiếm tới mean
4. Lặp lại bước 2 cho tới khi hội tụ



Mean-Shift

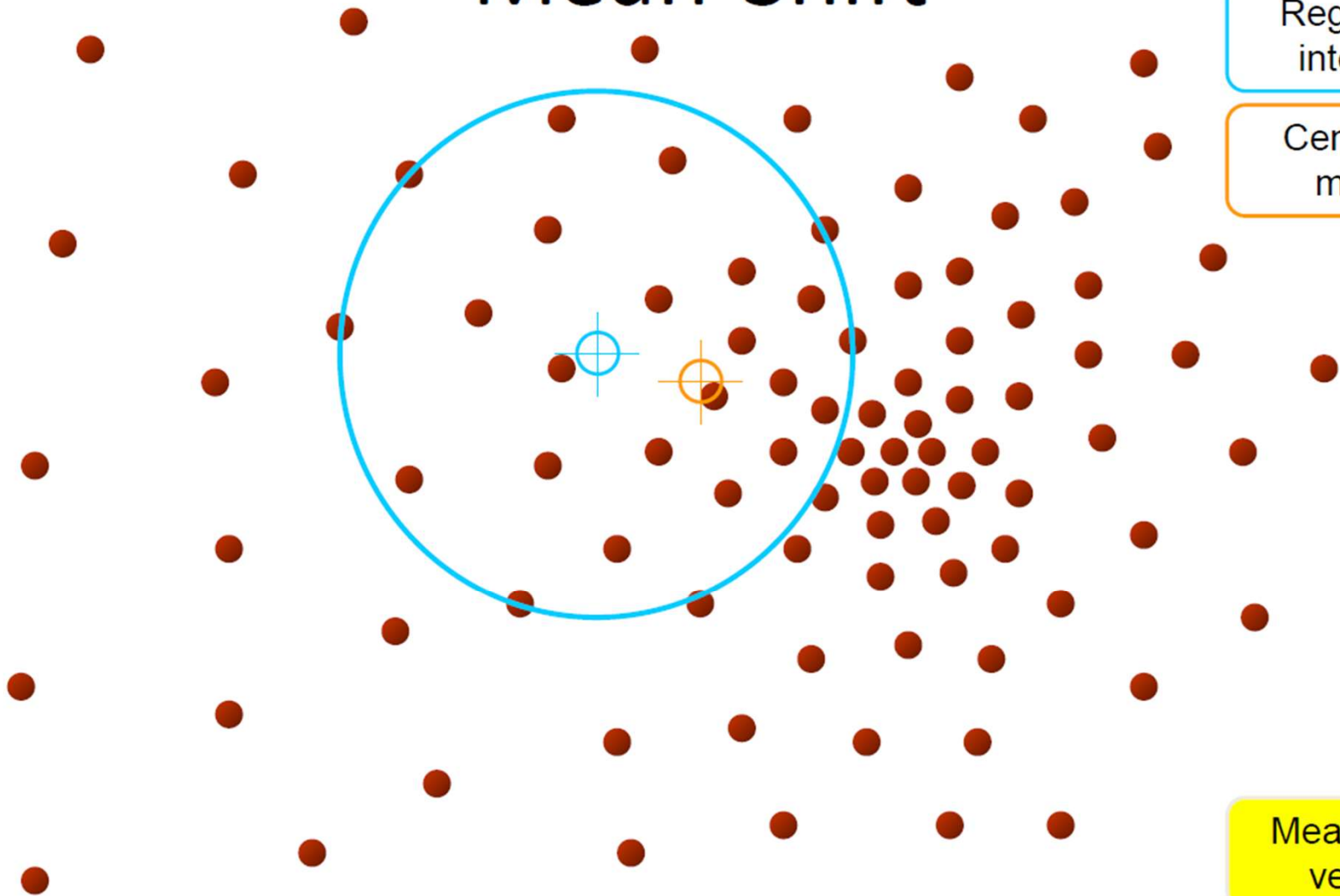


Mean-Shift

Region of
interest

Center of
mass

Mean Shift
vector

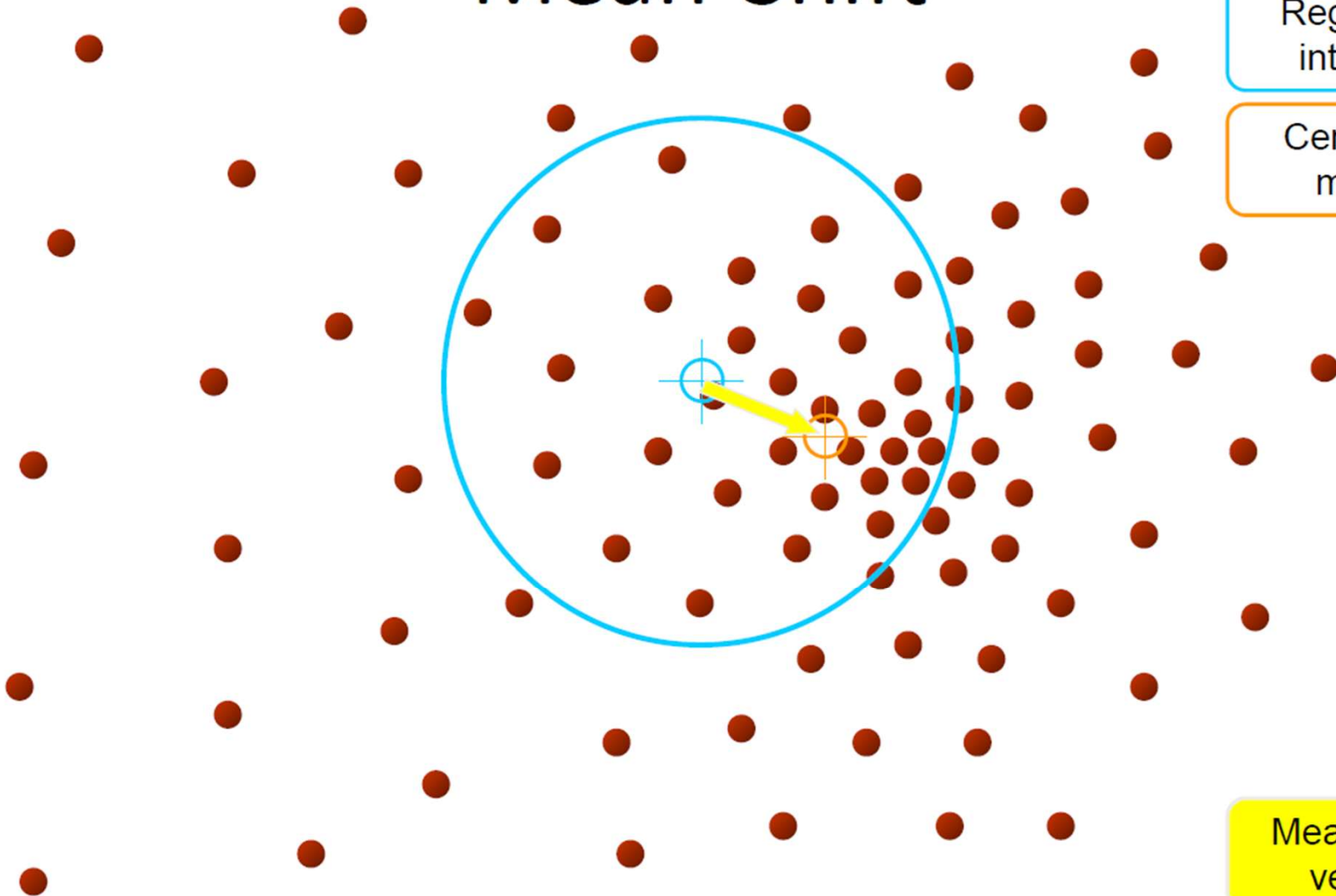


Mean-Shift

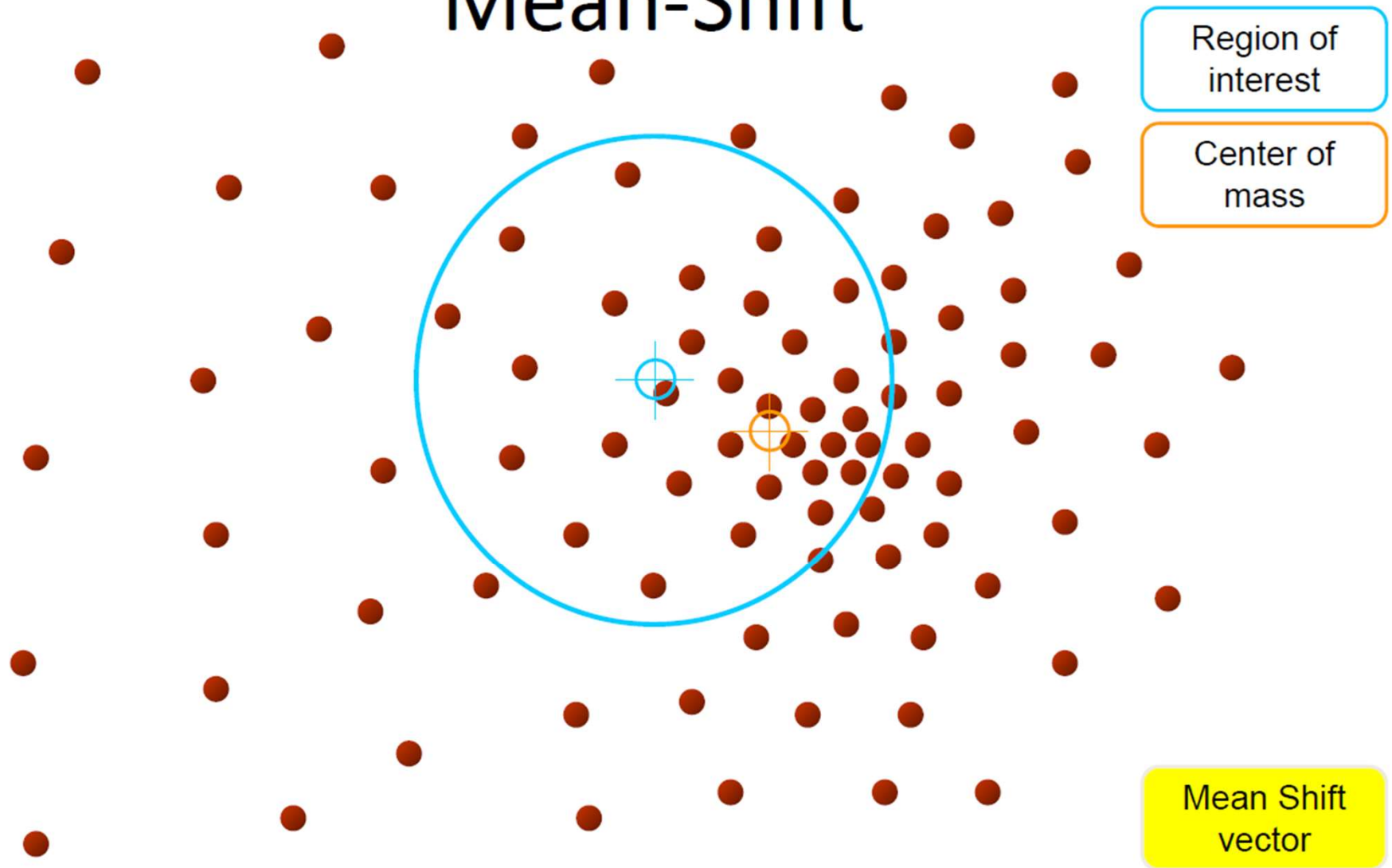
Region of
interest

Center of
mass

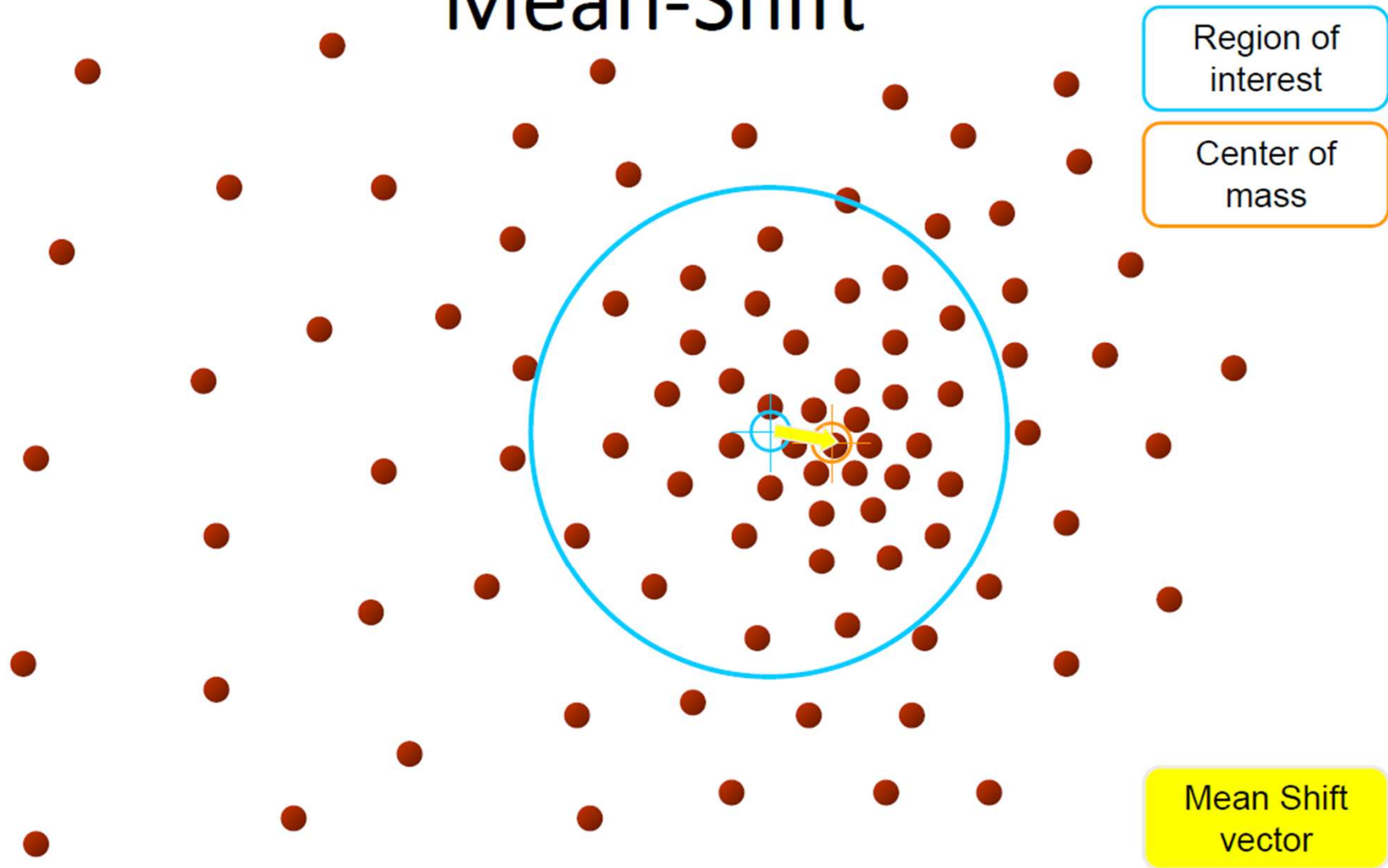
Mean Shift
vector



Mean-Shift



Mean-Shift

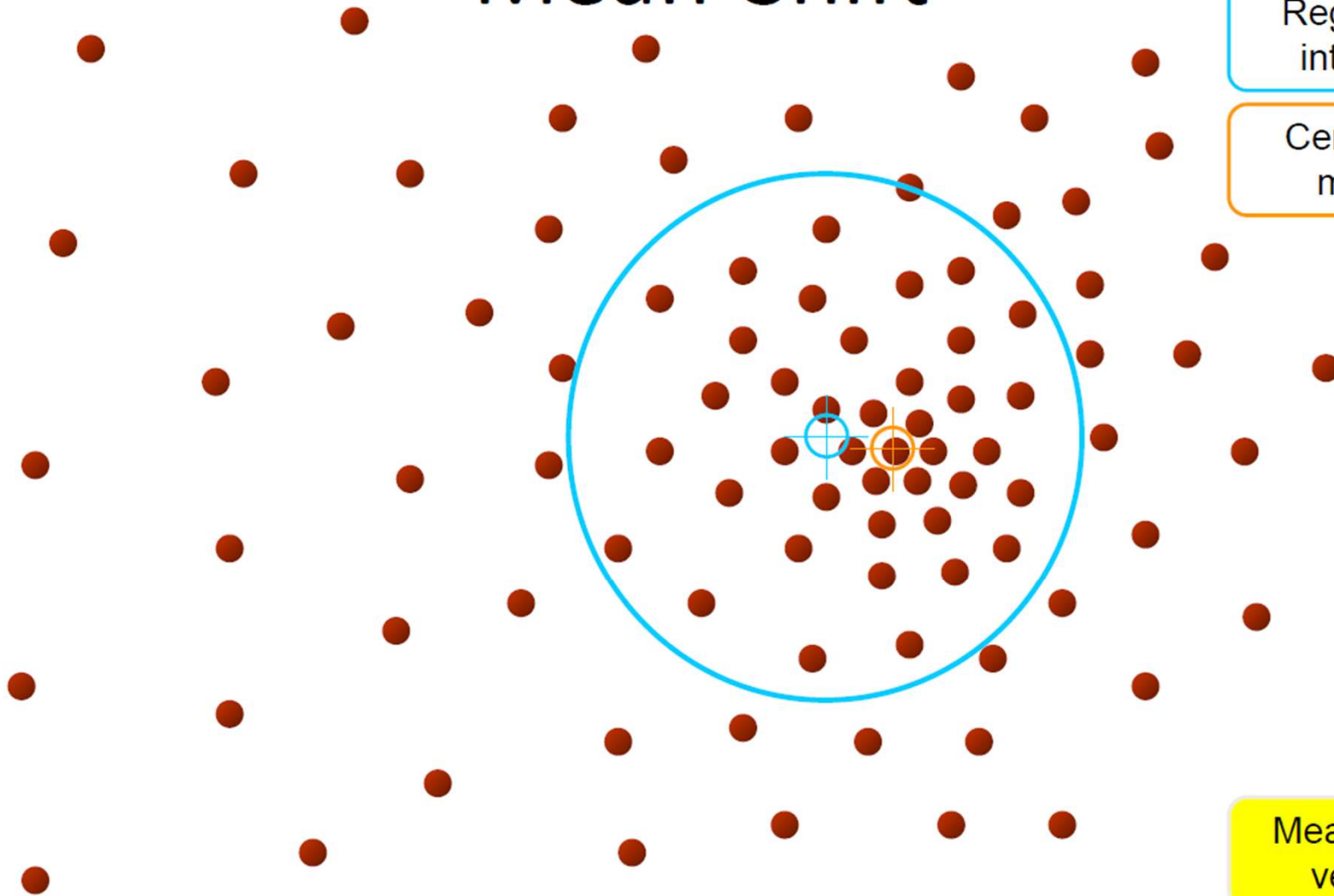


Mean-Shift

Region of
interest

Center of
mass

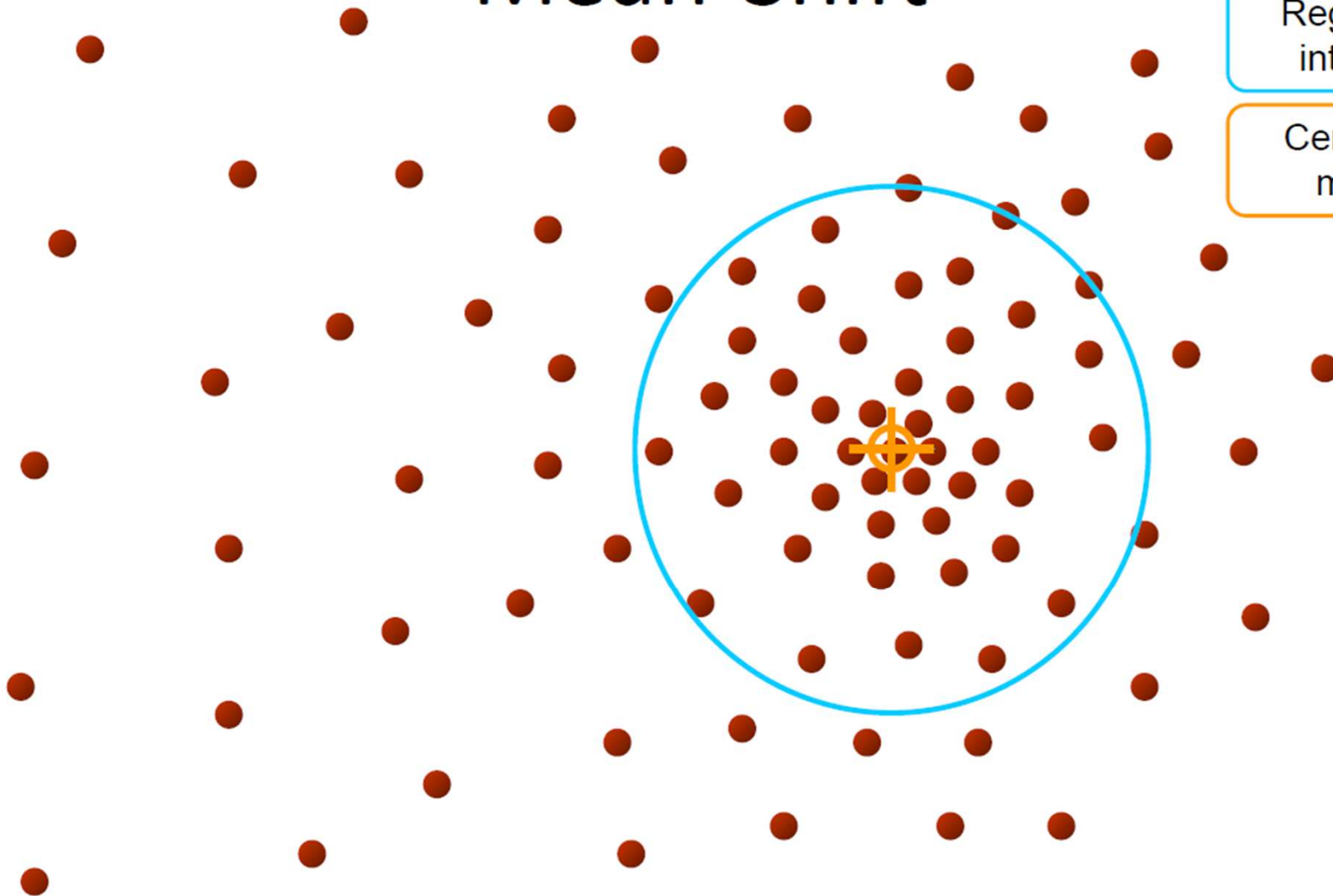
Mean Shift
vector



Mean-Shift

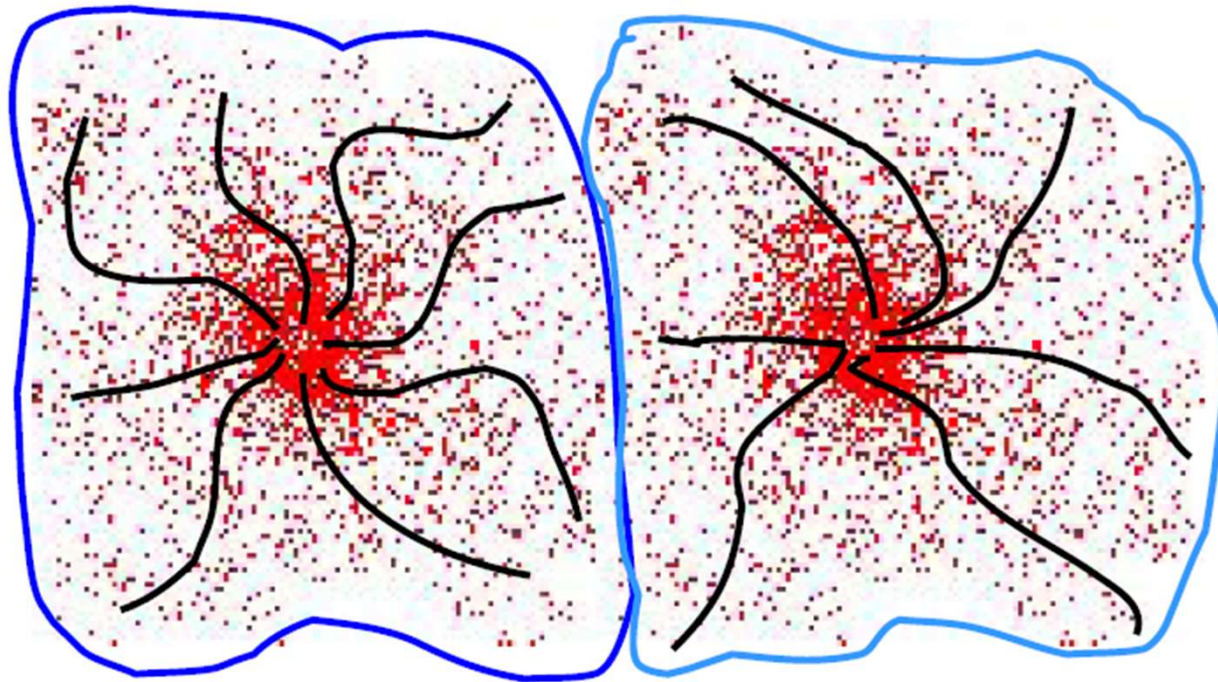
Region of
interest

Center of
mass



Phân cụm Mean-Shift

- Cụm: tất cả các điểm dữ liệu trong lưu vực thu hút của một mode
- Lưu vực thu hút: khu vực mà tất cả các quỹ đạo dẫn đến cùng một mode.



Ví dụ



Đánh giá Mean-shift

➤ Ưu điểm

- Không cần giả định nào trước cho các cụm dữ liệu
- Chỉ có một tham số kích thước cửa sổ (window size)
- Mạnh với các điểm ngoại lai (outlier)

➤ Nhược điểm

- Kết quả phụ thuộc vào kích thước cửa sổ (window size)
- Chi phí tính toán lớn
- Khó thích ứng với dữ liệu số chiều cao

Nội dung

1. Giới thiệu bài toán phân cụm
2. Phân cụm K-mean
3. Phân cụm Mean-shift
- 4. Ứng dụng**

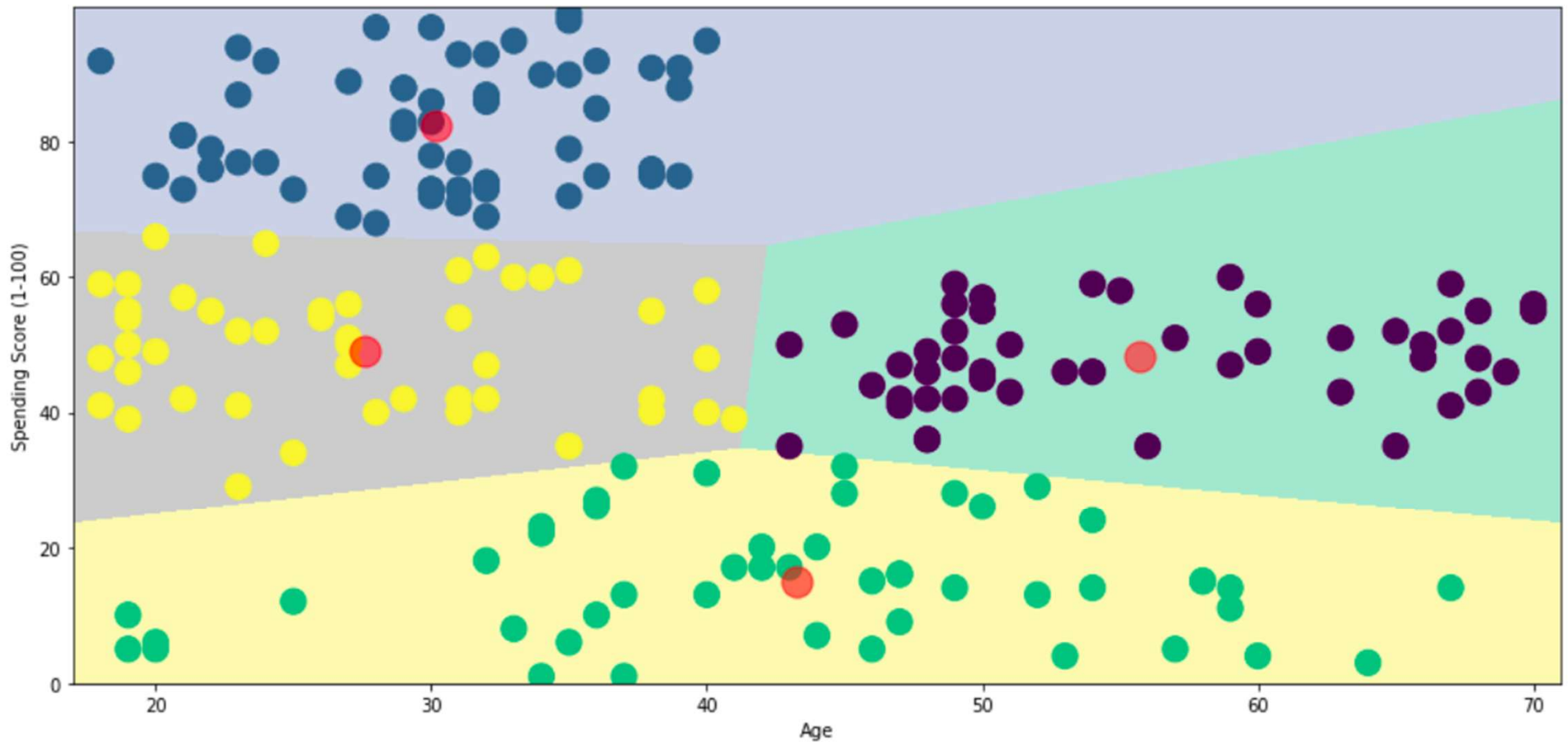
Ví dụ về phân cụm khách hàng

- Dữ liệu cơ bản về khách hàng như:
 - ID khách hàng
 - Tuổi
 - Giới tính
 - Thu nhập hàng năm
- Điểm chỉ tiêu: là điểm khách hàng nhận được dựa trên các thông số đã xác định như hành vi của khách hàng và dữ liệu mua hàng.
- Bài toán: Chủ trung tâm thương mại muốn hiểu khách hàng như [Khách hàng mục tiêu] để đội ngũ tiếp thị có thể hiểu và hoạch định chiến lược phù hợp.

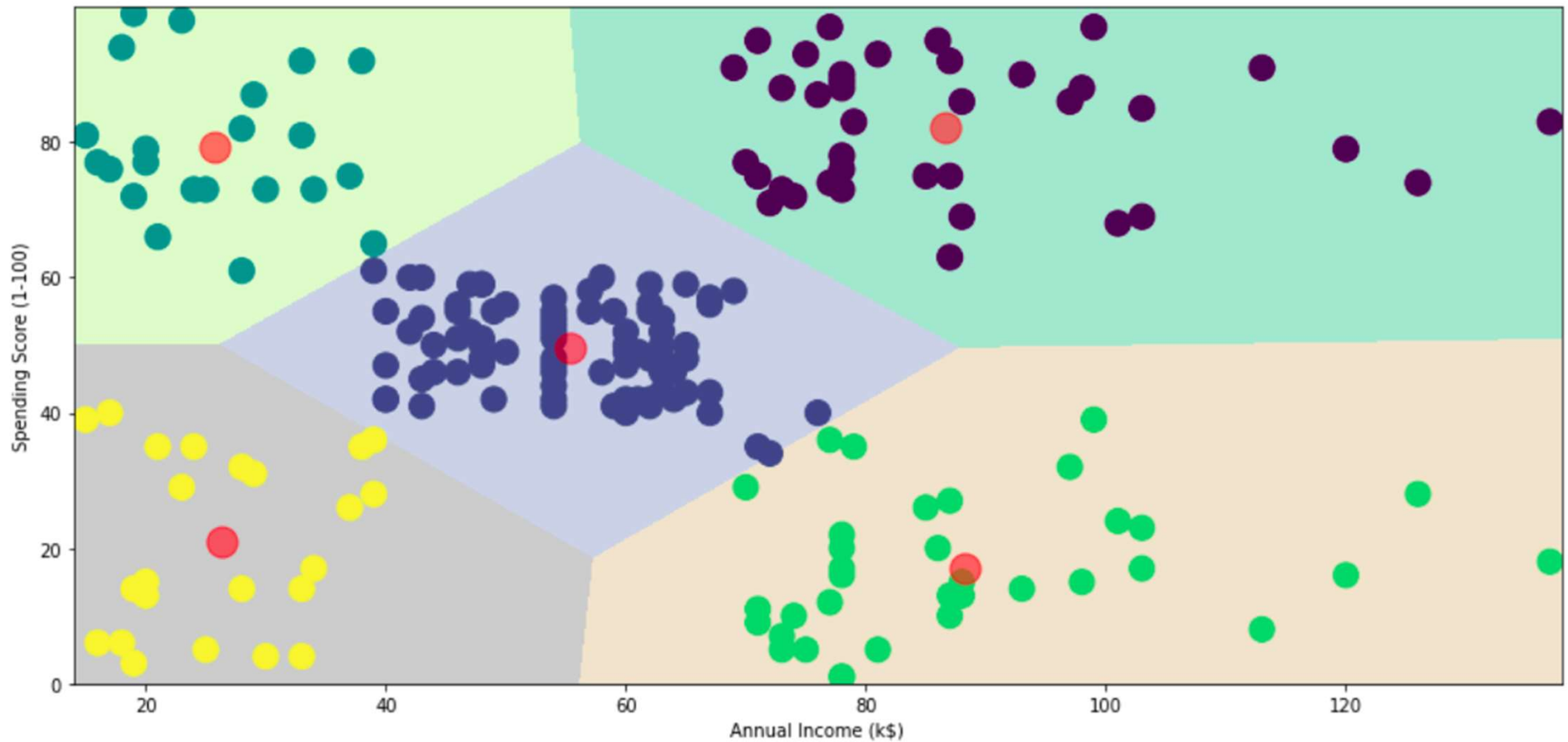
Dữ liệu phân đoạn khách hàng

	CustomerID	Gender	Age	Annual Income (k\$)	Spending Score (1-100)
0	1	Male	19	15	39
1	2	Male	21	15	81
2	3	Female	20	16	6
3	4	Female	23	16	77
4	5	Female	31	17	40

Kết quả phân cụm theo độ tuổi



Kết quả phân đoạn theo thu nhập hàng năm



Bài tập

Q&A

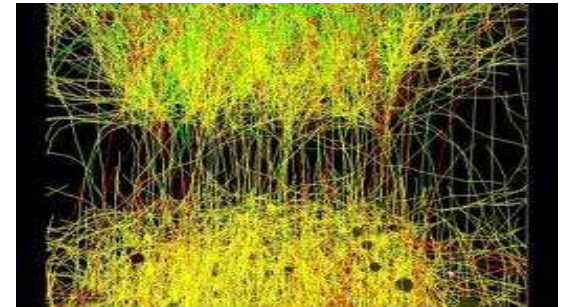
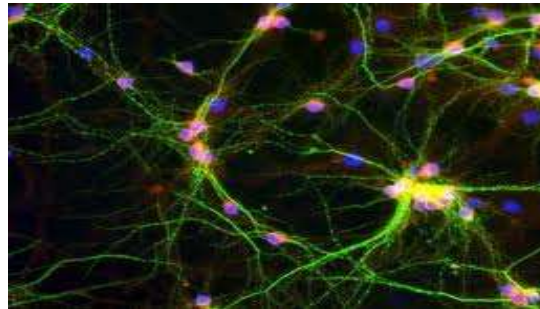
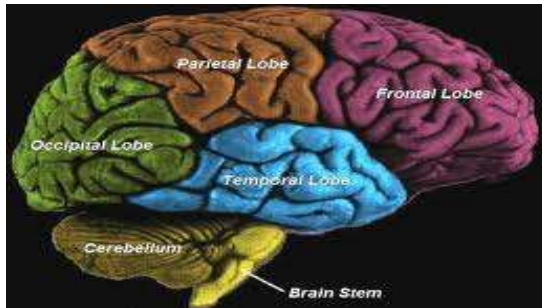
Mạng nơ-ron nhân tạo

Nội dung

1. Giới thiệu về mạng nơ-ron nhân tạo (ANN)
2. Các mô hình mạng nơ-ron
3. Giải thuật lan truyền ngược (Backpropagation)
4. Mạng tích chập CNN
5. Ứng dụng

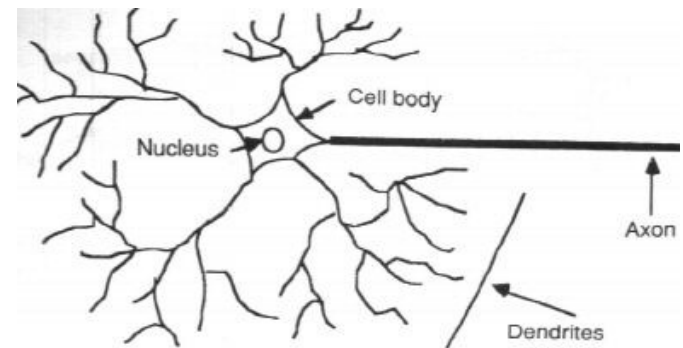
Giới thiệu về mạng nơ-ron nhân tạo Artificial Neural Network (ANN)

Mạng nơ-ron sinh học



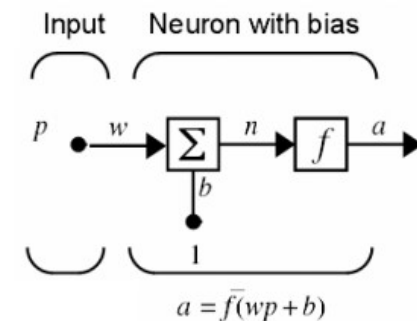
- Hàng trăm tỉ nơ-ron
- Mỗi nơ-ron có hàng ngàn kết nối
- Tổ chức thành các lớp (layers)

Q: Quá trình học của con người?



Artificial Neural Networks (ANN)

- ANN là một mô hình mạng chứa nhiều đơn vị xử lý, mô phỏng quá trình học tập và tính toán của bộ não sinh học.
- Mỗi đơn vị xử lý gọi là một nơ-ron nhân tạo. Mỗi nơ-ron nhân tạo giả lập một nơ-ron sinh học, gồm:
 - 1 đơn vị tính toán (tổng)
 - 1 ngưỡng kích hoạt (bias b) và
 - 1 hàm kích hoạt (hay hàm truyền, transfer function) đặc trưng cho tính chất của nơ-ron.



- Các nơ-ron nhân tạo được liên kết với nhau bằng các kết nối. Mỗi kết nối có một trọng số kết nối (weights), đặc trưng cho khả năng nhớ của ANN.

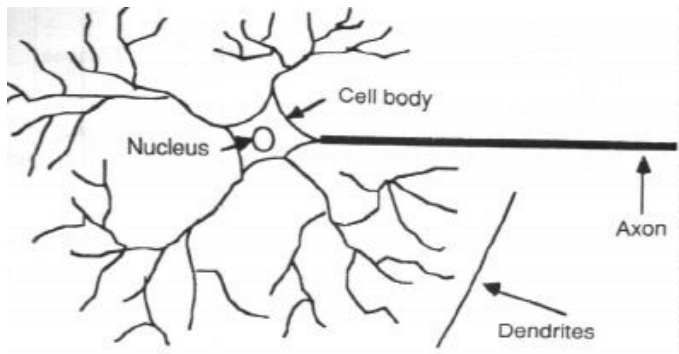
Artificial Neural Networks (ANN)

- ANN học từ dữ liệu thông qua hiệu chỉnh các trọng số kết nối giữa các nơ-ron
- Quá trình huấn luyện ANN là 1 quá trình điều chỉnh các ngưỡng kích hoạt và các trọng số kết nối, dựa trên dữ liệu học.

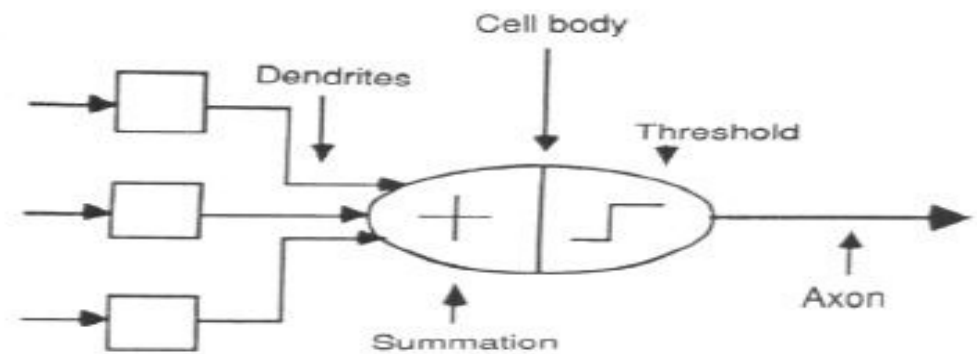
Bài toán đặt ra: Xây dựng một cấu trúc ANN và huấn luyện nó dựa trên dữ liệu thu thập được (gọi là tập dữ liệu học).

Neuron Model

- Mô hình của 1 nơ-ron:

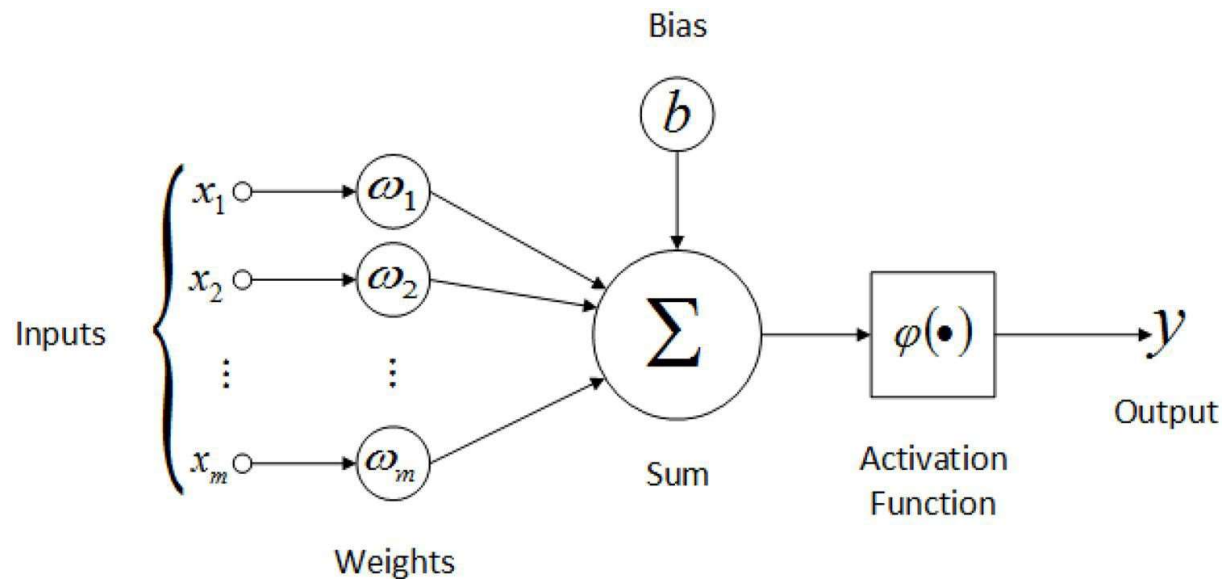


1 nơ-ron sinh học



1 nơ-ron nhân tạo

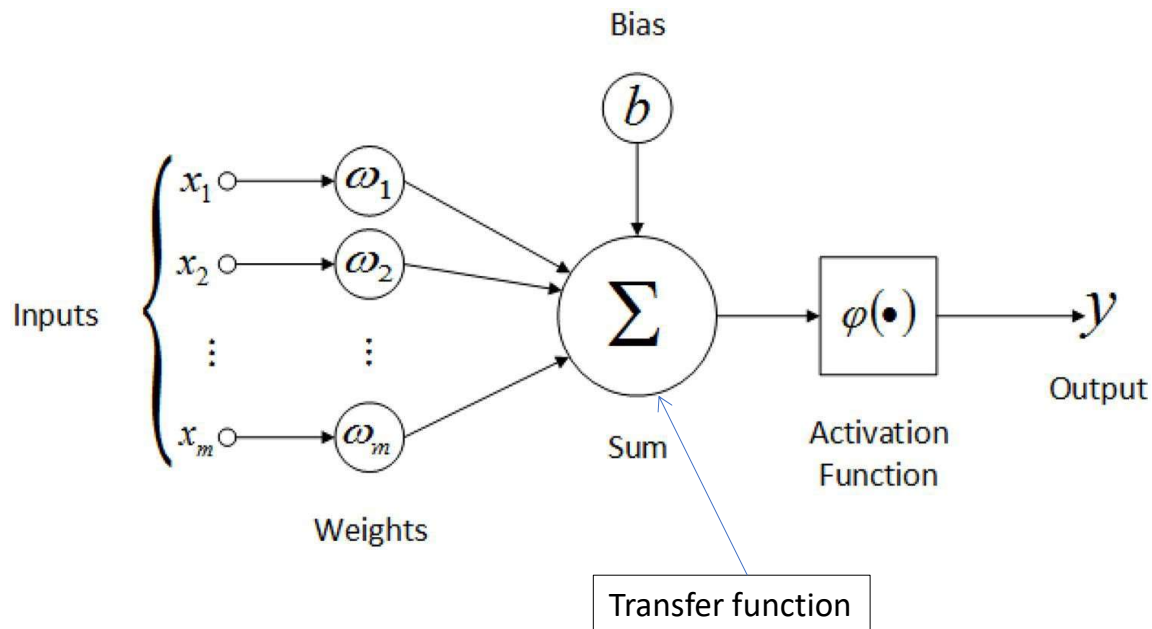
Biểu diễn toán học của 1 nơ-ron



- Tính tổng có trọng số của các tín hiệu vào: $\text{Sum}(W \cdot x)$
- Chuyển kết quả tính tổng qua một hàm kích hoạt (activation function)

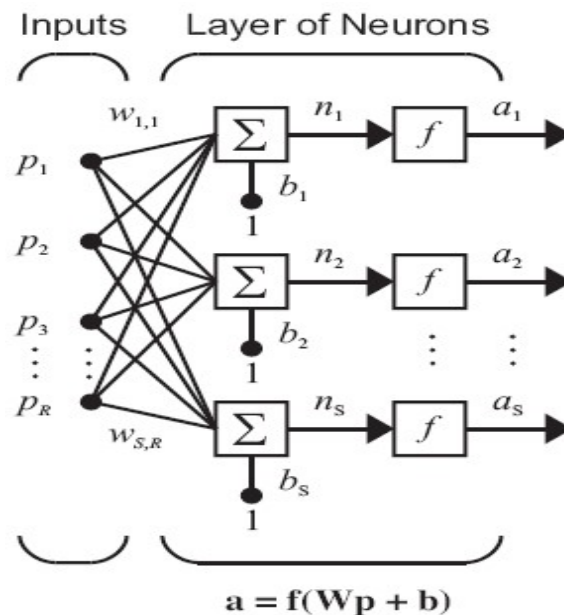
Hàm kích hoạt và hàm truyền

- Activation function and transfer function



Activation Function	Equation	Example	1D Graph
Linear	$\phi(z) = z$	Adaline, linear regression	
Unit Step (Heaviside Function)	$\phi(z) = \begin{cases} 0 & z < 0 \\ 0.5 & z = 0 \\ 1 & z > 0 \end{cases}$	Perceptron variant	
Sign (signum)	$\phi(z) = \begin{cases} -1 & z < 0 \\ 0 & z = 0 \\ 1 & z > 0 \end{cases}$	Perceptron variant	
Piece-wise Linear	$\phi(z) = \begin{cases} 0 & z \leq -1/2 \\ z + 1/2 & -1/2 \leq z \leq 1/2 \\ 1 & z \geq 1/2 \end{cases}$	Support vector machine	
Logistic (sigmoid)	$\phi(z) = \frac{1}{1 + e^{-z}}$	Logistic regression, Multilayer NN	
Hyperbolic Tangent (tanh)	$\phi(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}$	Multilayer NN, RNNs	
ReLU	$\phi(z) = \begin{cases} 0 & z < 0 \\ z & z > 0 \end{cases}$	Multilayer NN, CNNs	

Kiến trúc mạng (Network architecture)



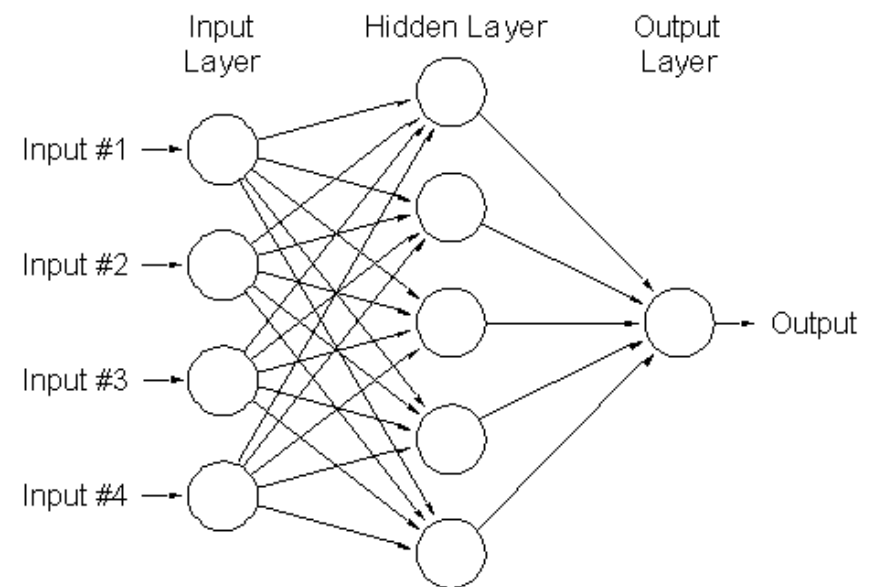
Where

R = number of elements in input vector

S = number of neurons in layer

- Mạng nơ-ron là một hệ thống gồm nhiều nơ-ron đơn lẻ kết nối với nhau, để thực hiện một chức năng tính toán nào đó.
- 2 hoặc nhiều neurons có thể kết hợp tạo thành 1 lớp (layer)
- 1 ANN có thể có 1 hoặc nhiều hơn 1 lớp (VD: mạng ANN 1 lớp, [Perceptron](#))

-
- Tính năng của mạng phức thuộc vào:
 - **Cấu trúc mạng**
 - **các trọng số kết nối** và quá trình tính toán tại các nơ-ron đơn lẻ (hàm truyền).
 - Mạng nơ-ron có thể học từ dữ liệu mẫu và tổng quát hóa dựa trên các mẫu đã học (mạng có khả năng nhớ)



Ví dụ hoạt động của ANN

- Xét bài toán: Cho 2 vector x và y . Xác định $y=f(x)=?$

x	1	2	3	4	5
y	2	4	6	8	10

$$y = f(x) = 2x$$

Cho $x = 7 \rightarrow y = 14$

- Thay đổi dữ liệu:

x	1	2	3	4	5
y	2.5	4.1	6	8.33	10.1

$$y = f(x) = ?$$

Cho $x = 7 \rightarrow y = ? \rightarrow$ khó xác định

- Ứng dụng ANN xác định $y=f(x)$:

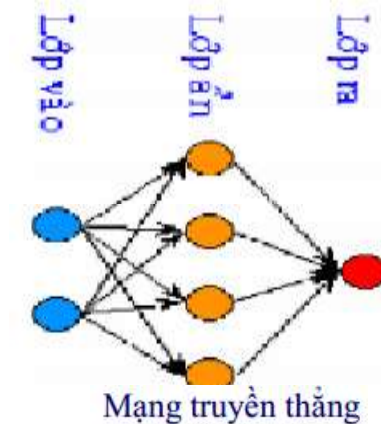
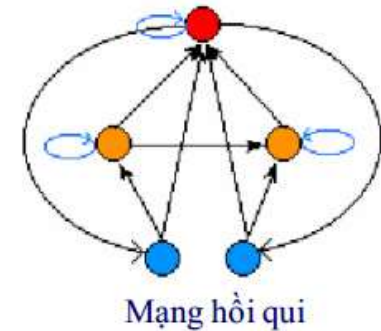
- Thu thập dữ liệu 2 vectors x và y (dữ liệu nhiều, độ chính xác lớn)
- Xây dựng mạng nơ-ron và huấn luyện mạng dựa trên dữ liệu thu thập
- Khi đó ANN hoạt động như 1 hàm: $y=f_{ANN}(x)$
- Với 1 giá trị x_n ta có $y_n = f_{ANN}(x_n) \sim f(x_n)$

✂ ANN có thể được huấn luyện để xấp xỉ một quan hệ phi tuyến bất kỳ

Các mô hình mạng nơ-ron

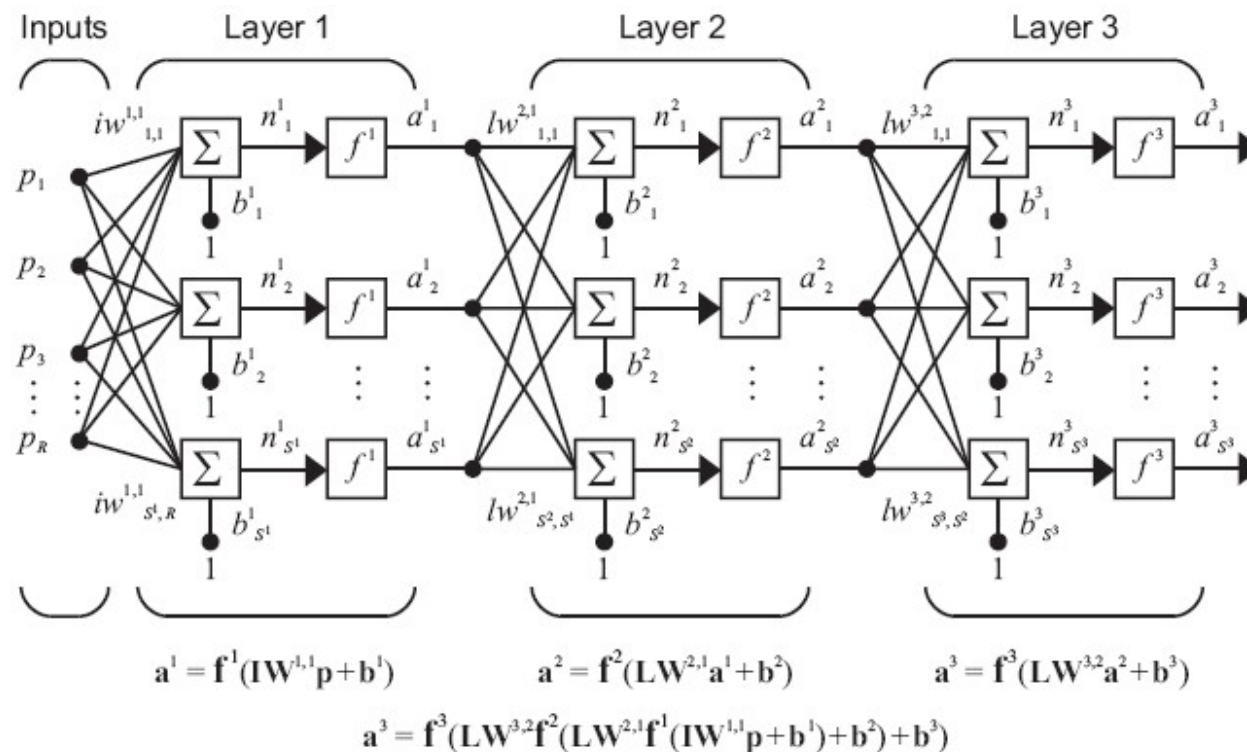
Phân loại mạng ANN

- Dựa theo kiểu kết nối các nơ-ron: có 2 loại
 - Mạng truyền thẳng (feedforward NN, **FNN**): Kết nối không tạo thành chu trình
 - Mạng hồi qui (recurrent NN, **RNN**): Kết nối tạo thành các chu trình
- Dựa vào số lớp: có 2 loại
 - Mạng 1 lớp
 - Mạng nhiều lớp: Gồm lớp vào, các lớp ẩn và lớp ra.
 - Lớp vào không tính số lớp.
 - Không kết nối trên cùng lớp; không nhảy lớp; không kết nối từ lớp sau ngược lại lớp trước.



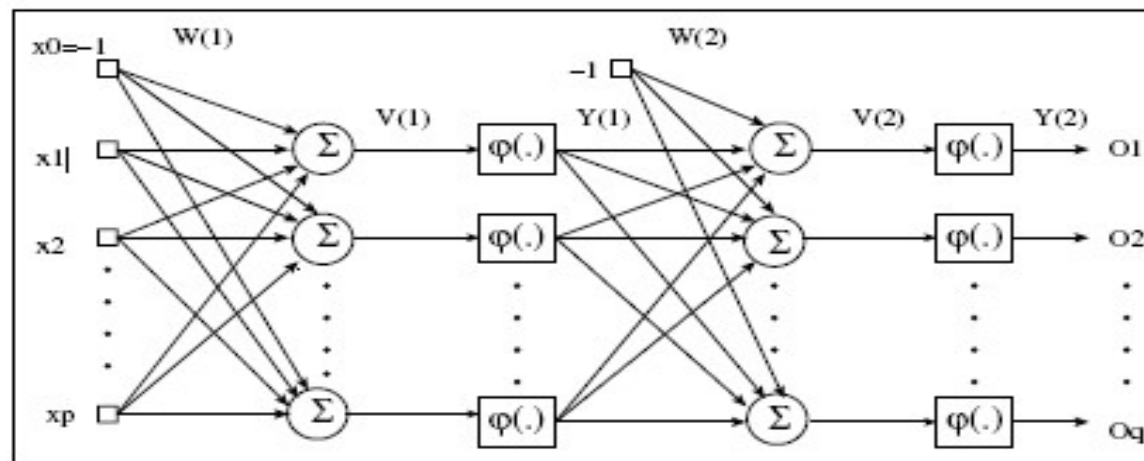
Mạng nhiều lớp truyền thẳng FNN

- Multilayer Perceptron (MLP)



- Dạng thức mô hình tính toán chung là như nhau cho các layers

ANN lan truyền thuận



- Mạng nơ-ron có thể được tổ chức thành nhiều lớp, với ngõ ra của lớp trước là ngõ vào của lớp sau.
- Quá trình tính toán trên mạng được thực hiện lần lượt trên từng lớp.

ANN lan truyền thuận

- Bias Nodes
 - 1 node được thêm vào mỗi lớp với đầu ra là 1 giá trị hằng
- Lan truyền thuận:
 - Tính toán các giá trị cho mỗi nơ-ron trên mỗi lớp từ đầu vào đến đầu ra
 - Với mỗi nơ-ron:
 - Tính trung bình trọng số các đầu vào
 - Tính toán hàm kích hoạt (activation function)

Ví dụ : Cho mạng 3 lớp như hình vẽ, tính giá trị ngõ ra, với:

- Lớp vào: $\mathbf{p} = [2, 1.5, 3]^T$

- Lớp ẩn 1:

$$\mathbf{b}^1 = [0.5, 0.6]^T$$

$$\mathbf{W}^1 = [0.1, 0.2, 0.3; 0.5, 0.4, 0.6]$$

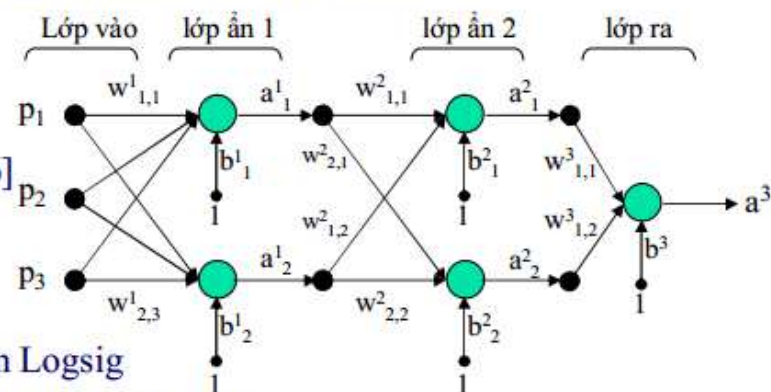
f^1 : hàm purelin

- Lớp ẩn 2:

$$\mathbf{b}^2 = [0.5, 0.6]^T$$

$$\mathbf{W}^2 = [0.5, 0.2; 0.1, 0.6]; f^2: \text{hàm Logsig}$$

- Lớp ra: $b^3 = 0.2; \mathbf{W}^3 = [0.4, 0.2]; f^3: \text{hàm Tangsig}$



- Ngõ ra lớp ẩn 1: Giống Ví dụ ở phần mạng nơ-ron 1 lớp. Ta có $a^1_1=1.9$ và $a^1_2=4$
- Ngõ ra lớp ẩn 2: ngõ vào lớp 2 chính là ngõ ra lớp 1

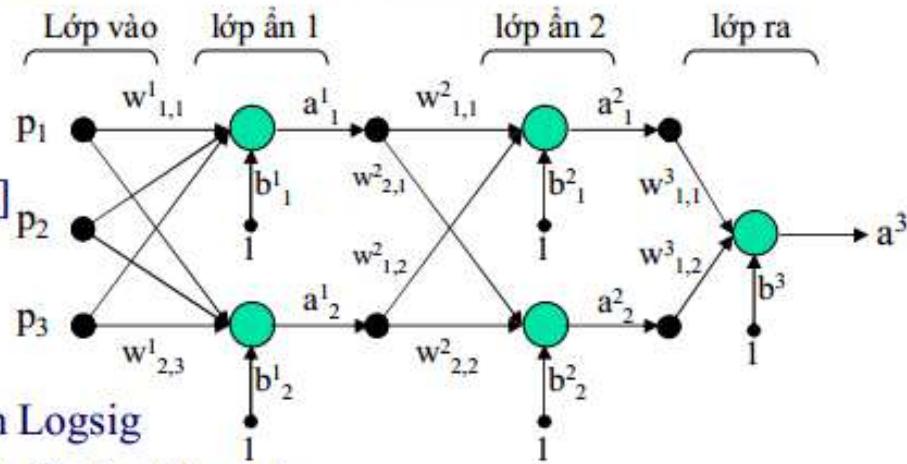
$$\mathbf{W}^2 \mathbf{p}^2 + \mathbf{b}^2 = [0.5, 0.2; 0.1, 0.6] * [1.9, 4]^T + [0.5, 0.6]^T = [2.25, 3.19]^T$$

$$\mathbf{a}^2 = f^2(\mathbf{W}^2 \mathbf{p}^2 + \mathbf{b}^2) = [f^2(2.25), f^2(3.19)]^T = \left[\frac{1}{1 + e^{-2.25}} \quad \frac{1}{1 + e^{-3.19}} \right]^T = [.9047, .9605]^T$$

$$\text{hay: } a^2_1 = 0.9047 \text{ và } a^2_2 = 0.9605$$

Ví dụ : Cho mạng 3 lớp như hình vẽ, tính giá trị ngõ ra, với:

- Lớp vào: $\mathbf{p} = [2, 1.5, 3]^T$
- Lớp ẩn 1:
 $\mathbf{b}^1 = [0.5, 0.6]^T$
 $\mathbf{W}^1 = [0.1, 0.2, 0.3; 0.5, 0.4, 0.6]$
 f^1 : hàm purelin
- Lớp ẩn 2:
 $\mathbf{b}^2 = [0.5, 0.6]^T$
 $\mathbf{W}^2 = [0.5, 0.2; 0.1, 0.6]$; f^2 : hàm Logsig
- Lớp ra: $b^3 = 0.2$; $\mathbf{W}^3 = [0.4, 0.2]$; f^3 : hàm Tangsig



$$\mathbf{W}^1 = \begin{bmatrix} 0.1 & 0.2 & 0.3 \\ 0.5 & 0.4 & 0.6 \end{bmatrix}$$

$$\mathbf{W}^2 = \begin{bmatrix} 0.5 & 0.2 \\ 0.1 & 0.6 \end{bmatrix}$$

$$\mathbf{a} = f(\mathbf{W}\mathbf{p} + \mathbf{b}) = ?$$

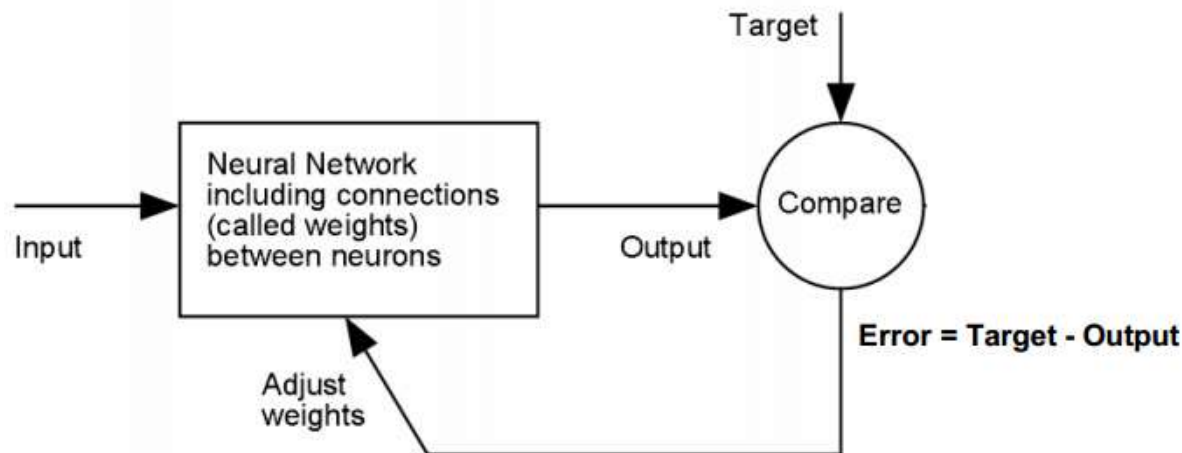
Huấn luyện mạng nơ-ron

Giải thuật lan truyền ngược BP (Back propagation)

Quá trình học của ANN

- Quá trình huấn luyện ANN có thể mô tả như sau:

Dựa trên sự sai biệt (*Error*) giữa ngõ ra mong muốn (*Target*) và ngõ ra thực tế của ANN (*Output*), giải thuật huấn luyện sẽ điều chỉnh các trọng số kết nối của ANN, sao cho: $\min\{Error\}$



Giải thuật huấn luyện ANN

- Giải thuật truyền ngược cập nhật các trọng số theo nguyên tắc:

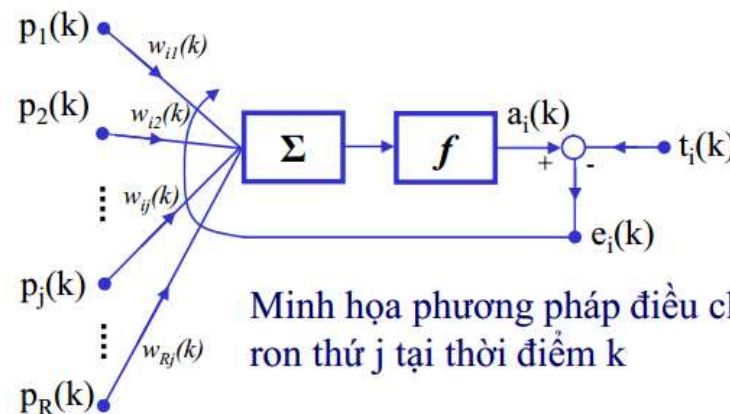
$$w_{ij}(k+1) = w_{ij}(k) + \eta g(k)$$

trong đó:

- $w_{ij}(k)$ là trọng số của kết nối từ nơ-ron j đến nơ-ron i , ở thời điểm hiện tại (vòng lặp k)
 - η là tốc độ học (learning rate, $0 < \eta \leq 1$)
 - $g(k)$ là gradient hiện tại
- Có nhiều phương pháp xác định gradient $g(k)$, dẫn tới có nhiều giải thuật truyền ngược cải tiến.

Giải thuật huấn luyện ANN

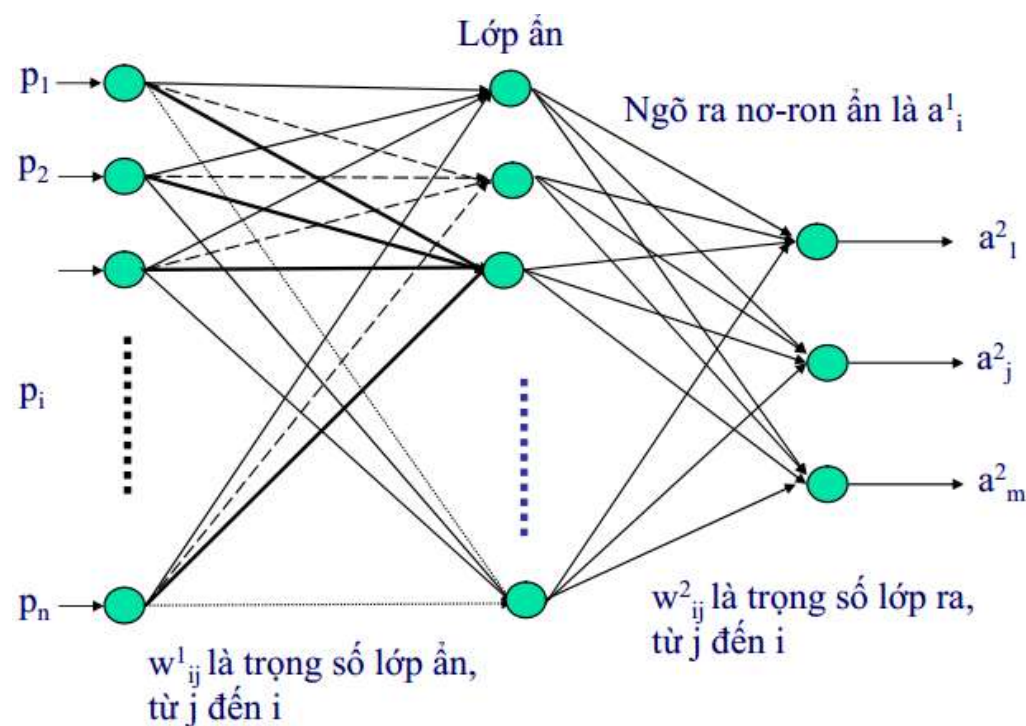
- Để cập nhật các trọng số cho mỗi chu kỳ huấn luyện, giải thuật truyền ngược cần 2 thao tác:
 - **Thao tác truyền thuận (forward pass phase):** Áp vector dữ liệu vào trong tập dữ liệu học cho ANN và tính toán các ngõ ra của nó.
 - **Thao tác truyền ngược (backward pass phase):** Xác định sai khác (lỗi) giữa ngõ ra thực tế của ANN và giá trị ngõ ra mong muốn trong tập dữ liệu học. Sau đó, truyền ngược lỗi này từ ngõ ra về ngõ vào của ANN và tính toán các giá trị mới của các trọng số, dựa trên giá trị lỗi này.



Minh họa phương pháp điều chỉnh trọng số nơ-ron thứ j tại thời điểm k

Giải thuật BP – Gradient descent

- Xét một MLP 2 lớp:



Giải thuật BP – Gradient descent

- Pha truyền thuận

Tính ngõ ra lớp ẩn (hidden layer):

$$n^1_i(k) = \sum_j w^1_{ij}(k) p_j(k) \quad \text{tại thời điểm } k$$

$$a^1_i(k) = f^1(n^1_i(k))$$

với f^1 là hàm kích truyền của các nơ-ron trên lớp ẩn.

Ngõ ra của lớp ẩn là ngõ vào của các nơ-ron trên lớp ra.

Tính ngõ ra ANN (output layer):

$$n^2_i(k) = \sum_j w^2_{ij}(k) a^1_j(k) \quad \text{tại thời điểm } k$$

$$a^2_i(k) = f^2(n^2_i(k))$$

với f^2 là hàm truyền của các nơ-ron trên lớp ra

Giải thuật BP – Gradient descent

- Pha truyền ngược

Tính tổng bình phương của lỗi: $E(k) = \frac{1}{2} \sum_i [t_i(k) - a_i(k)]^2$
với $t(k)$ là ngõ ra mong muốn tại k

Tính sai số các nơ-ron ngõ ra:

$$\Delta_i(k) = -\frac{\partial E(k)}{\partial n_i^2(k)} = [t_i(k) - a_i(k)] f'^2[n_i^2(k)]$$

Tính sai số các nơ-ron ẩn:

$$\delta_i(k) = -\frac{\partial E(k)}{\partial n_i^1(k)} = f'^1[n_i^1(k)] \sum_j \Delta_j(k) w_{ji}$$

Cập nhật trọng số

lớp ẩn: $w_{ij}^1(k+1) = w_{ij}^1(k) + \eta \delta_i(k) p_j(k)$

lớp ra: $w_{ij}^2(k+1) = w_{ij}^2(k) + \eta \Delta_i(k) a_j^1(k)$

Huấn luyện đến khi nào?

- Đủ số thời kỳ (epochs) ấn định trước
- Hàm mục tiêu đạt giá trị mong muốn
- Hàm mục tiêu phân kỳ.

Hàm mục tiêu MSE

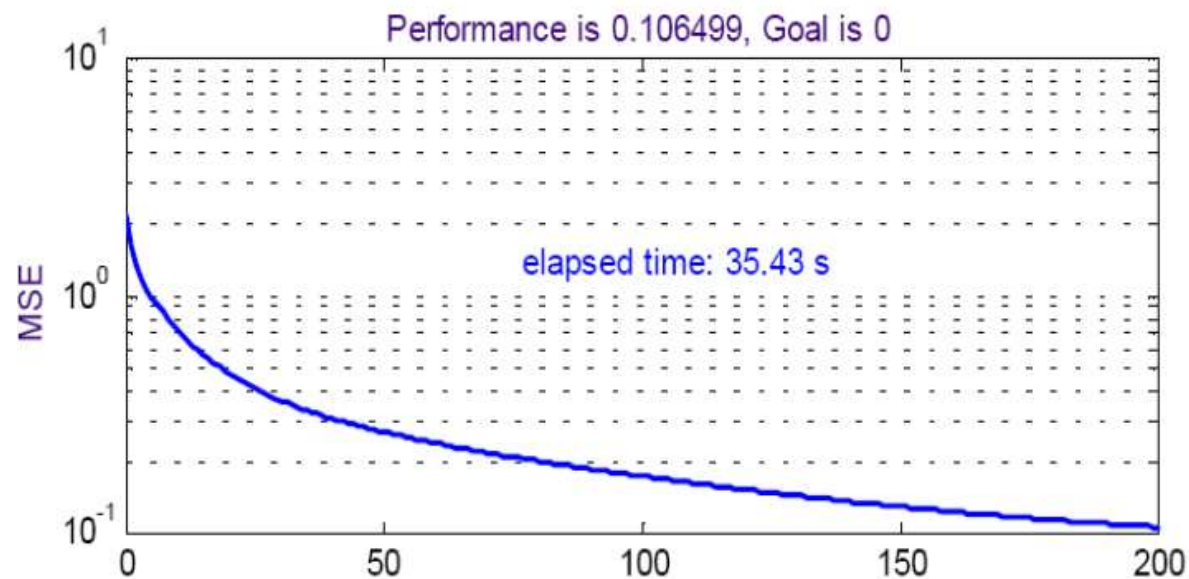
- Định nghĩa MSE: Giả sử ta có tập mẫu học: $\{p_1, t_1\}, \{p_2, t_2\}, \dots, \{p_N, t_N\}$, với $p = [p_1, p_2, \dots, p_N]$ là vector dữ liệu ngõ vào, $t = [t_1, t_2, \dots, t_N]$ là vector dữ liệu ngõ ra mong muốn. Gọi $a = [a_1, a_2, \dots, a_N]$ là vector dữ liệu ra thực tế thu được khi đưa vector dữ liệu vào p qua mạng. MSE:

- $$\mathbf{a} = f(\mathbf{w}\mathbf{p} + \mathbf{b})$$

$$MSE = \frac{1}{N} \sum_{i=1}^N (t_i - a_i)^2 \quad \text{được gọi là hàm mục tiêu}$$

- Mean Square Error (MSE) là lỗi bình phương trung bình, được xác định trong quá trình huấn luyện mạng. MSE được xem như là một trong những tiêu chuẩn đánh giá sự thành công của quá trình huấn luyện.
- MSE càng nhỏ, độ chính xác của ANN càng cao.
- Có thể sử dụng một số hàm sai số khác.

- MSE được xác định sau mỗi chu kỳ huấn luyện mạng (epoch) và được xem như 1 mục tiêu cần đạt đến. Quá trình huấn luyện kết thúc (đạt kết quả tốt) khi MSE đủ nhỏ.



-
- Cải tiến tốc độ hội tụ của giải thuật gradient descent: 1 nguyên tắc cập nhật trọng số của ANN:

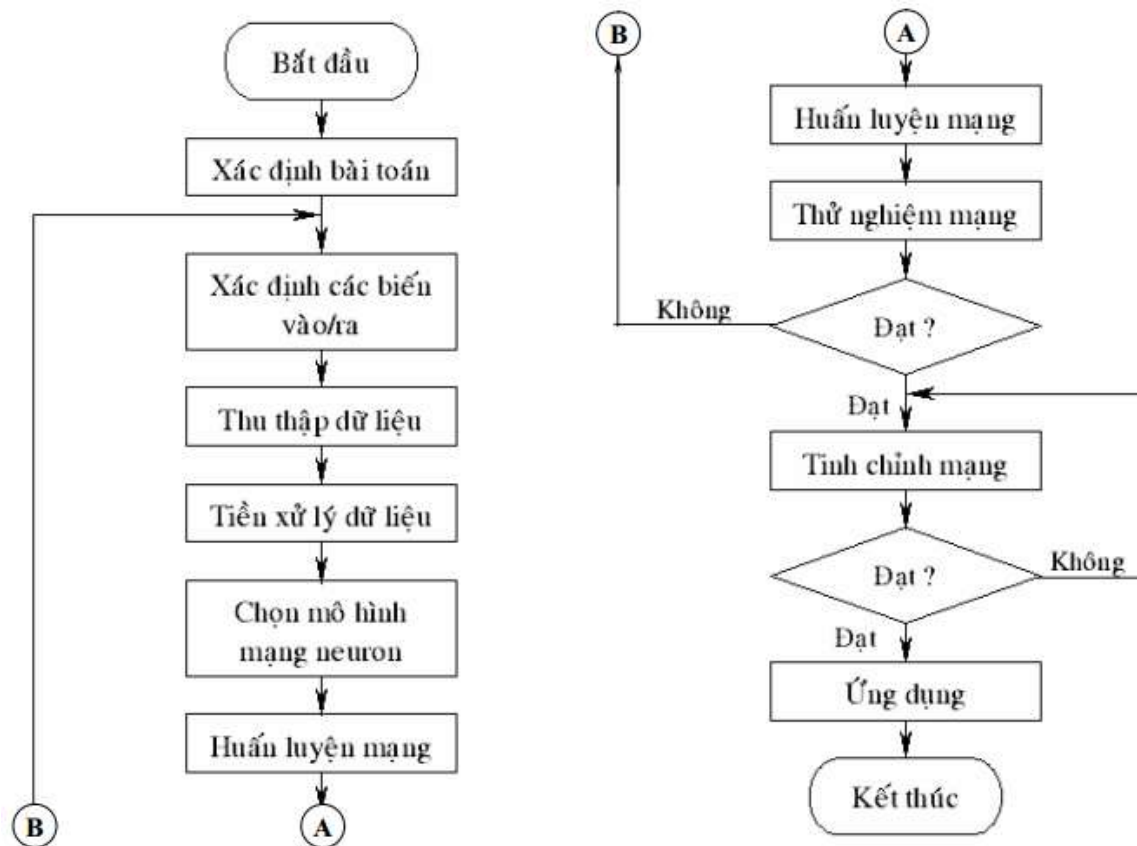
$$w_{ij}(k+1) = w_{ij}(k) + \nabla w_{ij}(k)$$

$$\nabla w_{ij}(k) = \eta g(k) + \mu \nabla w_{ij}(k-1)$$

với $g(k)$: gradient; η : tốc độ học
 $\nabla_{ij}(k-1)$ là giá trị trước đó của $\nabla_{ij}(k)$
 μ : momentum

- Thông thường, tổng giá trị của moment và tốc độ học nên gần bằng 1:
 $\mu \in [0.8 \ 1]$; $\eta \in [0 \ 0.2]$

Quy trình thiết kế ANN



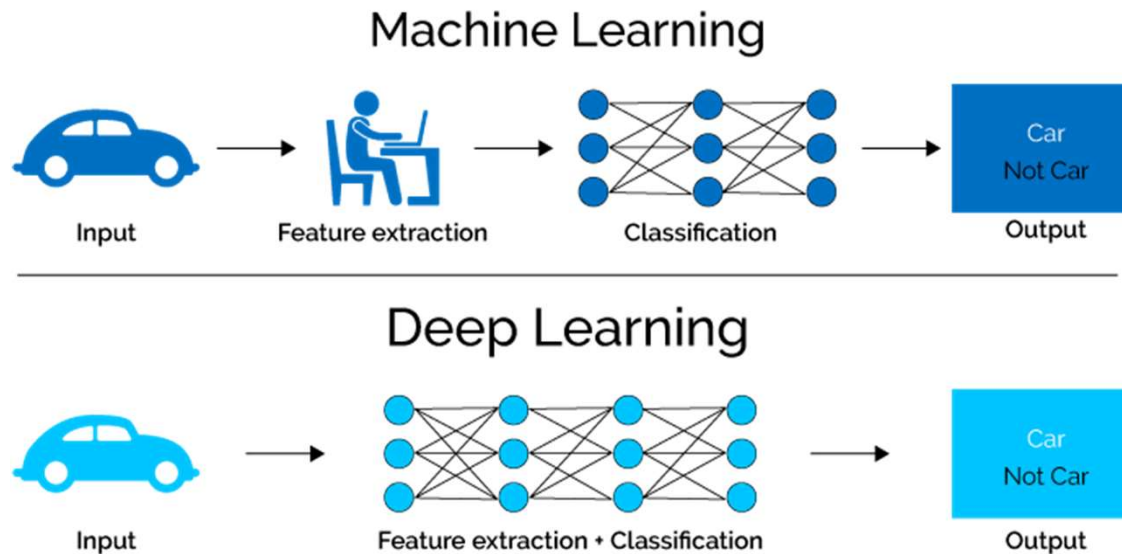
Thiết kế ANN

- Số lớp của mạng
- Số nơ-ron mỗi lớp ẩn
- Hàm kích hoạt tại mỗi nơ-ron
- Giải thuật huấn luyện mạng
- Hiệu đầu ra của mạng.

Mạng nơ-ron sâu

Deep Neural Network

Học máy và học sâu

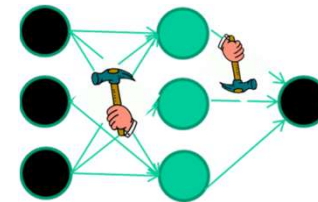


Nguồn: [CS224 NLP with DL course at Stanford](#)

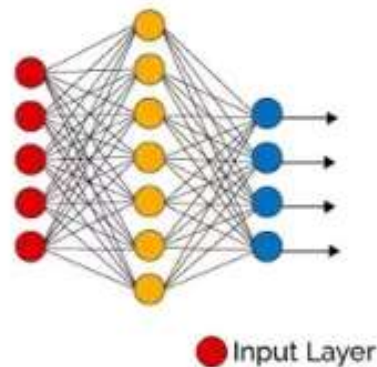
- Học máy cơ bản: Trích chọn đặc trưng (feature) từ ảnh → Học bộ phân loại từ các feature thu được → Hiệu chỉnh tham số.
- Học sâu: Feature được trích chọn tự động; Phân loại đối tượng, hiệu chỉnh tham số cũng có thể được tìm tự động

Mạng nơ-ron học sâu

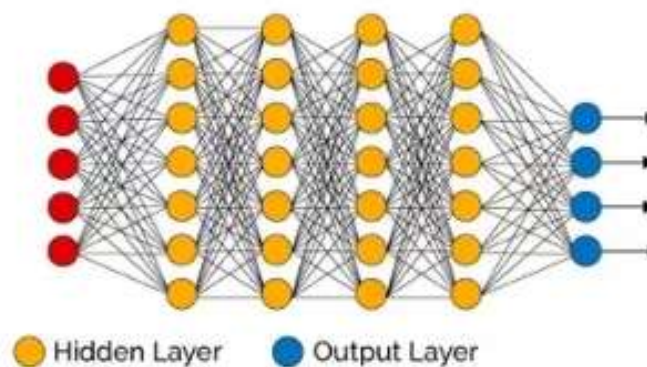
- Nhiều lớp ẩn
 - Số lượng lớn các trọng số phải học
 - Độ phức tạp tính toán lớn!!!
- Giảm số lượng các trọng số?



Simple Neural Network



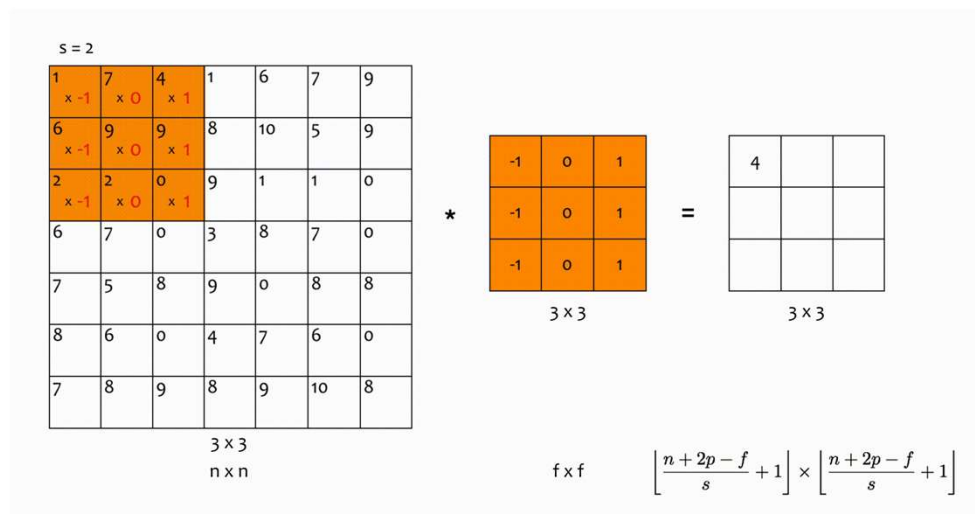
Deep Learning Neural Network



Nguồn: Internet

Mô hình mạng neural tích chập

- Convolutional Neural Network (CNN)
- Convolution (tích chập):



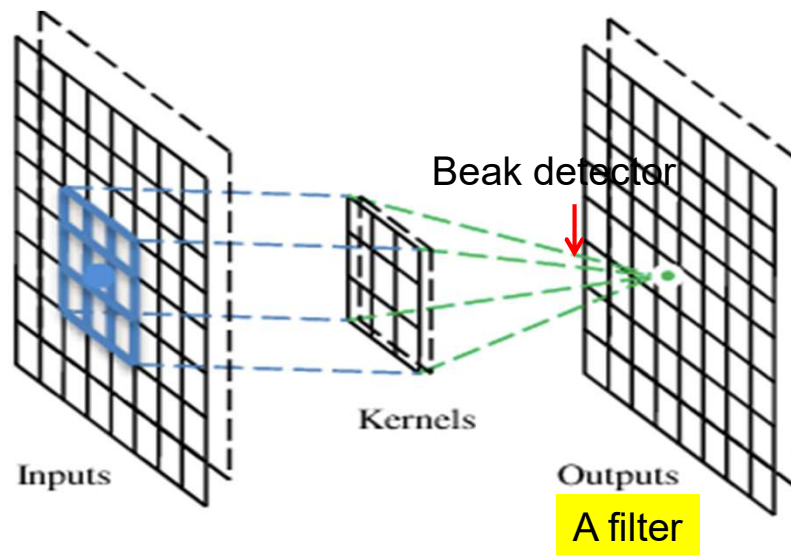
- Conv Layer: Các lớp tích chập nhằm trích chọn các đặc trưng của ảnh đầu vào

Một số khái niệm cơ bản

- **Filter** (còn gọi là Kernel hay Feature Detector): ma trận lọc. Thông thường, ở các lớp đầu tiên của Conv Layer sẽ có kích thước là $[5 \times 5 \times 3]$
- **Convolved Feature** (Activation Map hay Feature Map): đầu ra của ảnh khi cho bộ lọc quét qua hết các vị trí trong ảnh và thực hiện phép nhân vô hướng.
- **Receptive field**: vùng ảnh được chọn để tính tích chập, bằng đúng kích thước của bộ lọc.
- **Depth**: số lượng bộ lọc.
- **Stride**: khoảng cách dịch chuyển của bộ lọc sau mỗi lần tính. Ví dụ khi $\text{stride}=2$. Tức sau khi tính xong tại 1 vùng ảnh, nó sẽ dịch sang phải 2 pixel. Tương tự cho việc dịch xuống dưới.
- **Zero-Padding**: thêm các giá trị 0 ở xung quanh biên ảnh, để đảm bảo phép tích chập được thực hiện đủ trên toàn ảnh.

A convolutional layer

CNN là một neural network với một số lớp nhân chập (convolutional layers). Một lớp nhân chập thực hiện phép lọc với một số các bộ lọc trên ảnh.



[Các slides sau đây dùng từ nguồn: cs231n.stanford.edu](http://cs231n.stanford.edu)

Convolution

These are the network parameters to be learned.

1	0	0	0	0	1
0	1	0	0	1	0
0	0	1	1	0	0
1	0	0	0	1	0
0	1	0	0	1	0
0	0	1	0	1	0

6 x 6 image

1	-1	-1
-1	1	-1
-1	-1	1

Filter 1

-1	1	-1
-1	1	-1
-1	1	-1

Filter 2

⋮ ⋮

Each filter detects a small pattern (3 x 3).

Convolution

stride=1

1	0	0	0	0	1
0	1	0	0	1	0
0	0	1	1	0	0
1	0	0	0	1	0
0	1	0	0	1	0
0	0	1	0	1	0

6 x 6 image

1	-1	-1
-1	1	-1
-1	-1	1

Filter 1

Dot
product
→

3

-1

Convolution

If stride=2

1	0	0	0	0	1
0	1	0	0	1	0
0	0	1	1	0	0
1	0	0	0	1	0
0	1	0	0	1	0
0	0	1	0	1	0

6 x 6 image

1	-1	-1
-1	1	-1
-1	-1	1

Filter 1

3

-3

Convolution

stride=1

1	0	0	0	0	1
0	1	0	0	1	0
0	0	1	1	0	0
1	0	0	0	1	0
0	1	0	0	1	0
0	0	1	0	1	0

6 x 6 image

1	-1	-1
-1	1	-1
-1	-1	1

Filter 1

3	-1	-3	-1
-3	1	0	-3
-3	-3	0	1
3	-2	-2	-1

Convolution

-1	1	-1
-1	1	-1
-1	1	-1

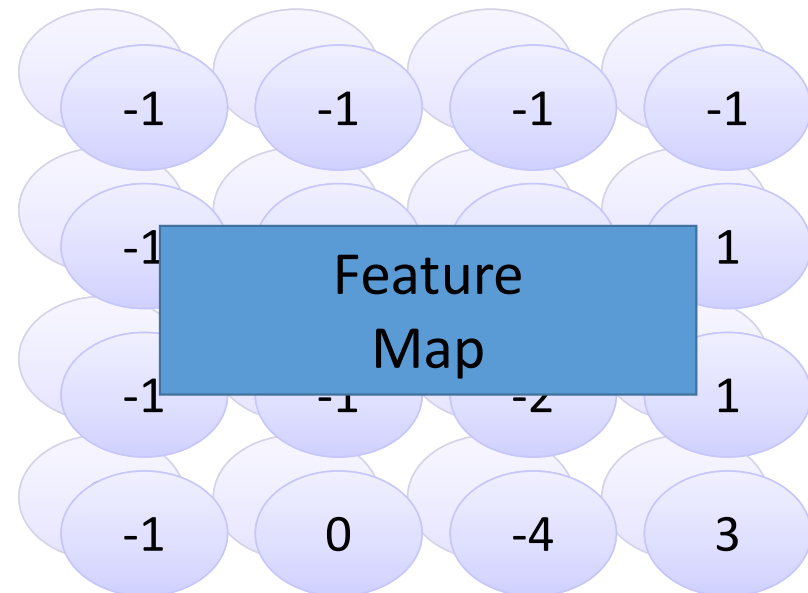
Filter 2

stride=1

1	0	0	0	0	1
0	1	0	0	1	0
0	0	1	1	0	0
1	0	0	0	1	0
0	1	0	0	1	0
0	0	1	0	1	0

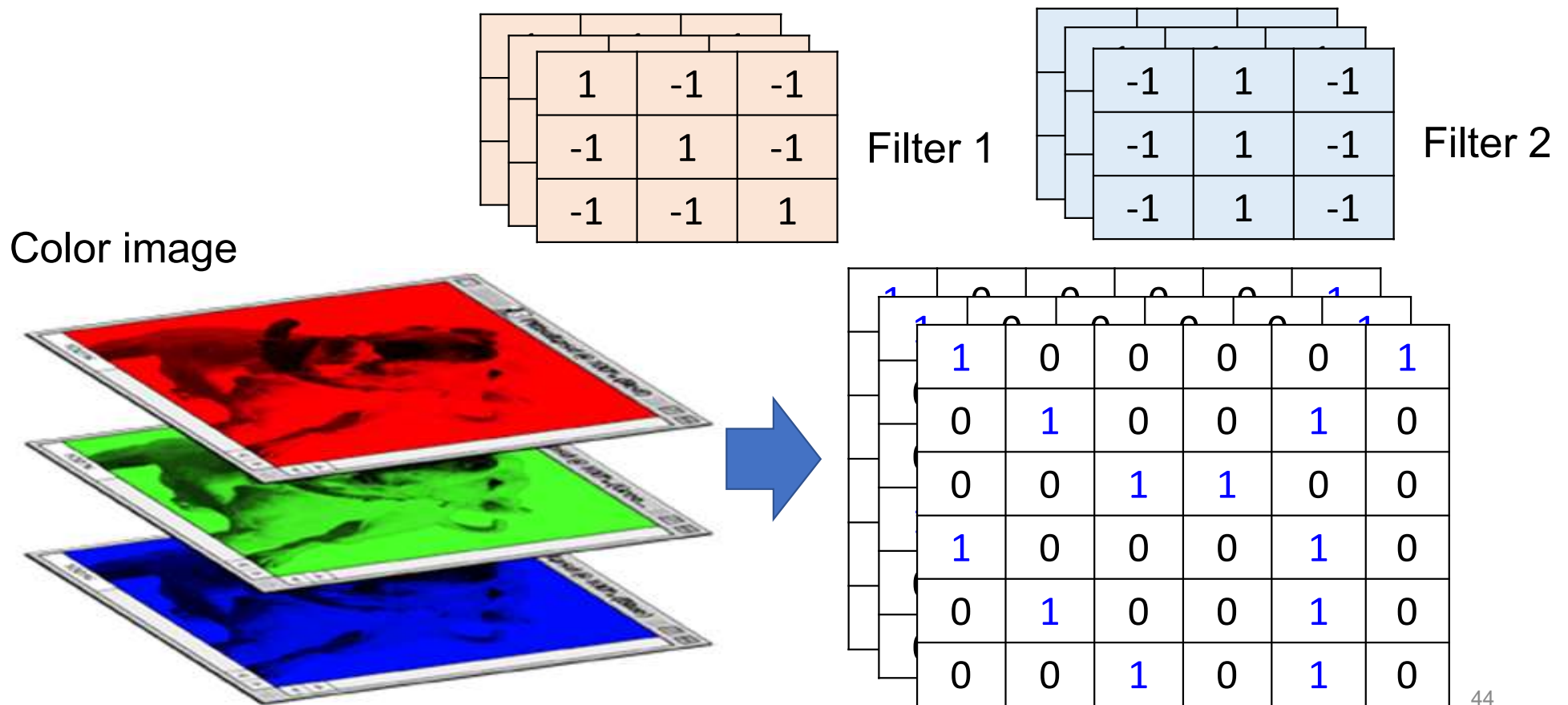
6 x 6 image

Repeat this for each filter



Two 4 x 4 images
Forming 2 x 4 x 4 matrix

Color image: RGB 3 channels



Convolution v.s. Fully Connected

1	0	0	0	0	1
0	1	0	0	1	0
0	0	1	1	0	0
1	0	0	0	1	0
0	1	0	0	1	0
0	0	1	0	1	0

image

1	-1	-1
-1	1	-1
-1	-1	1

-1	1	-1
-1	1	-1
-1	1	-1

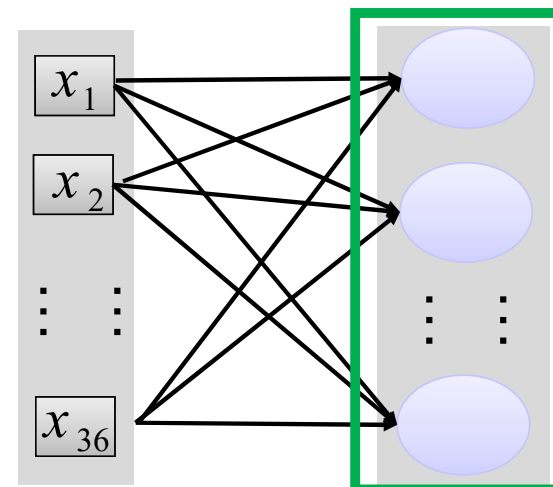


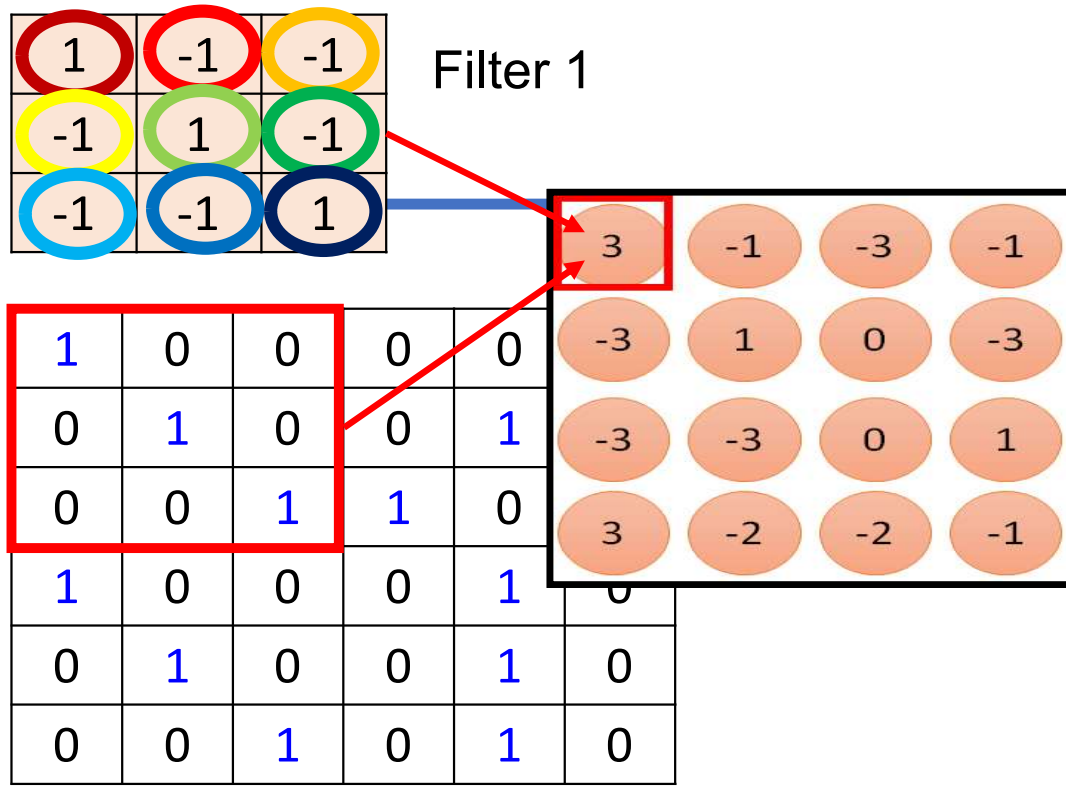
convolution

-1	-1	-1	-1
-1	-1	-2	1
-1	-1	-2	1
-1	0	-4	3

Fully-
connected

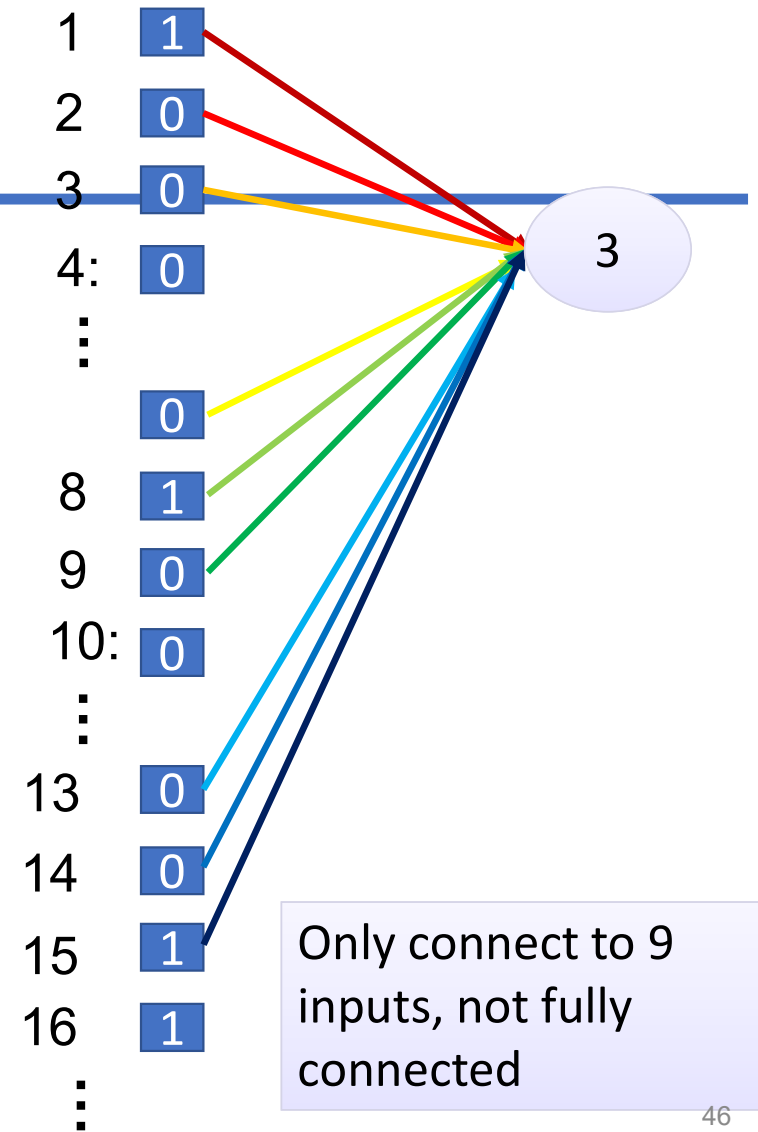
1	0	0	0	0	1
0	1	0	0	1	0
0	0	1	1	0	0
1	0	0	0	1	0
0	1	0	0	1	0
0	0	1	0	1	0

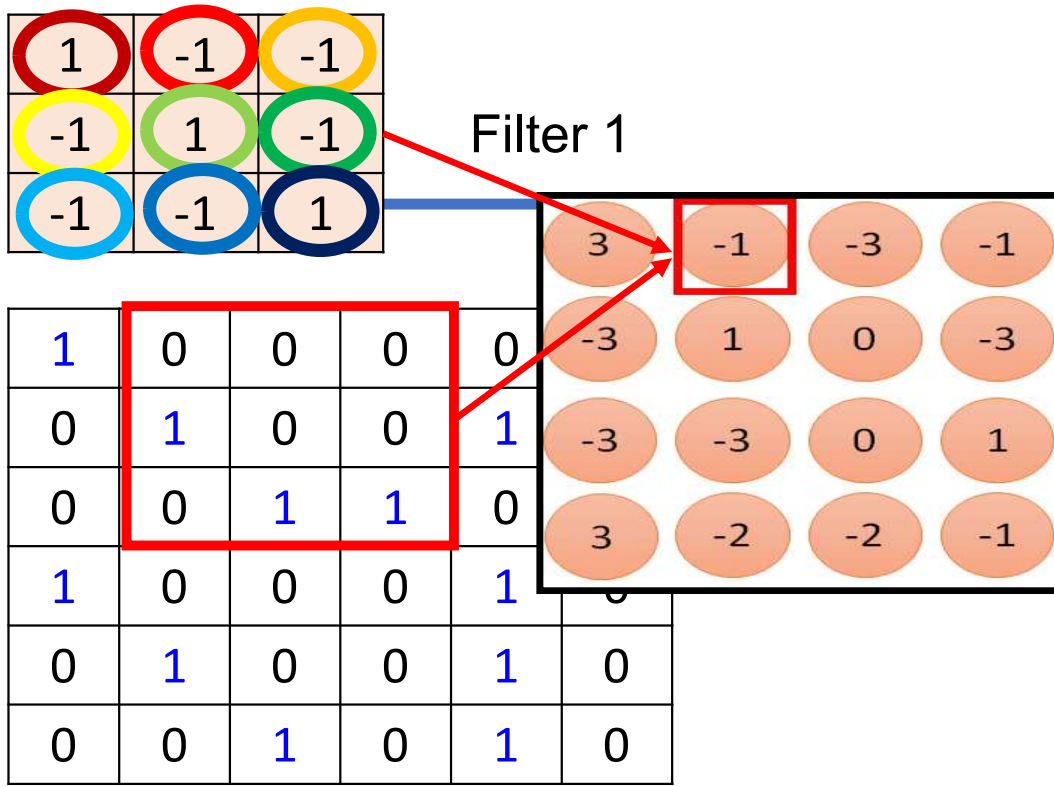




6 x 6 image

fewer parameters!

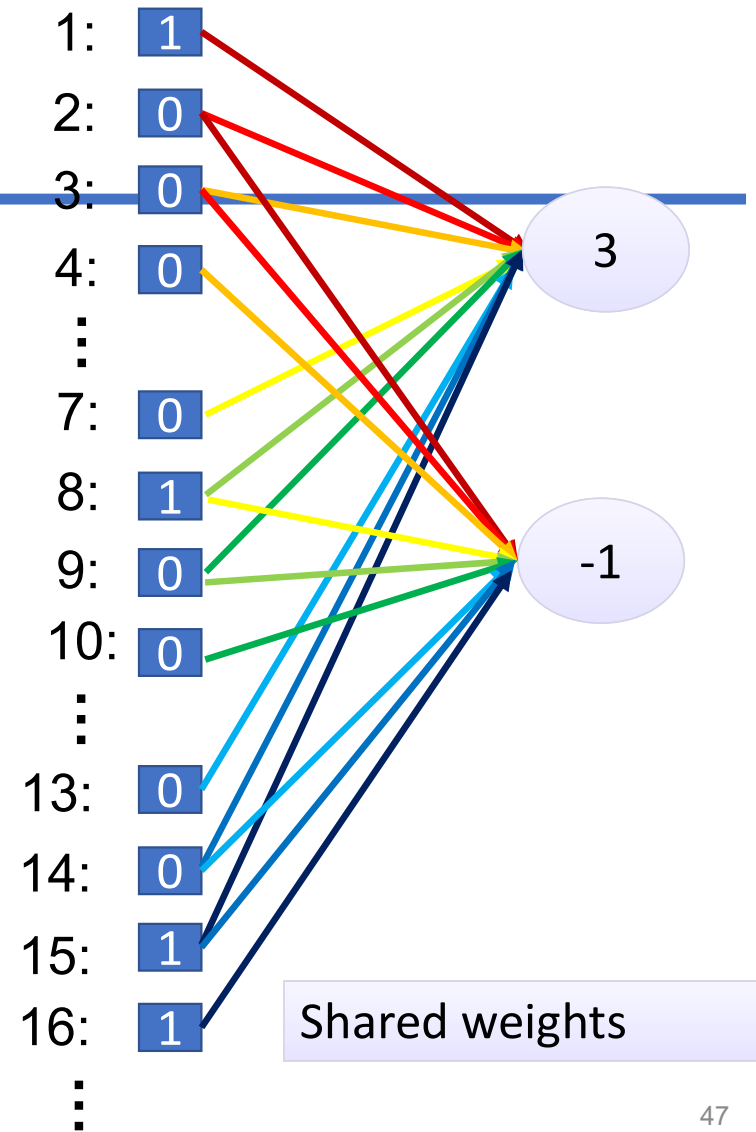




6 x 6 image

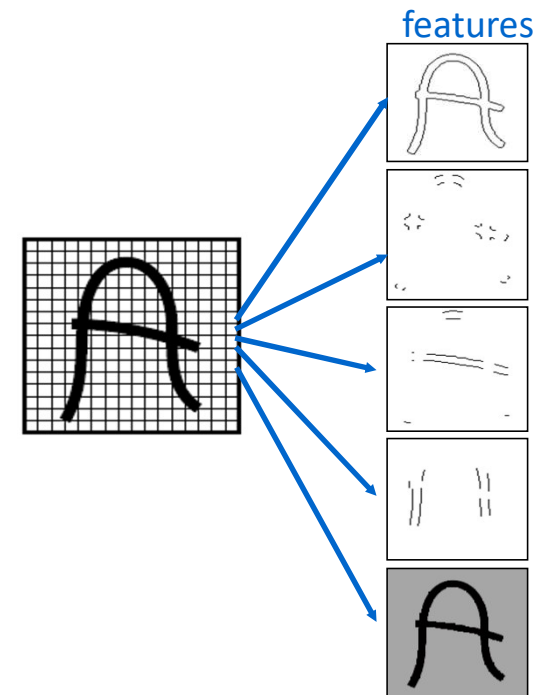
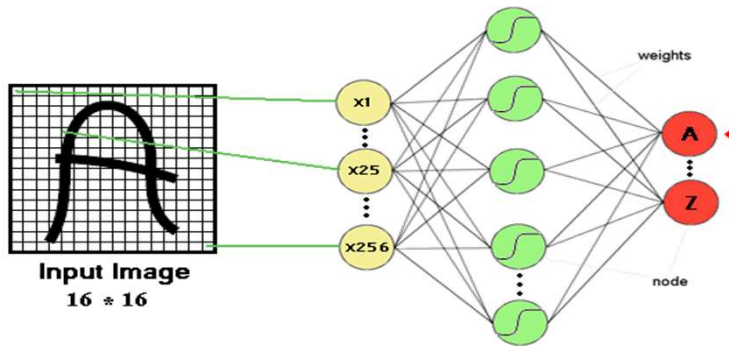
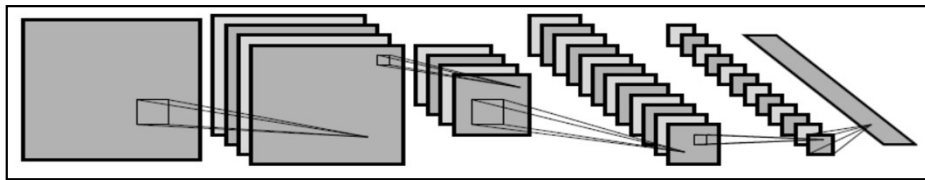
Fewer parameters

Even fewer parameters

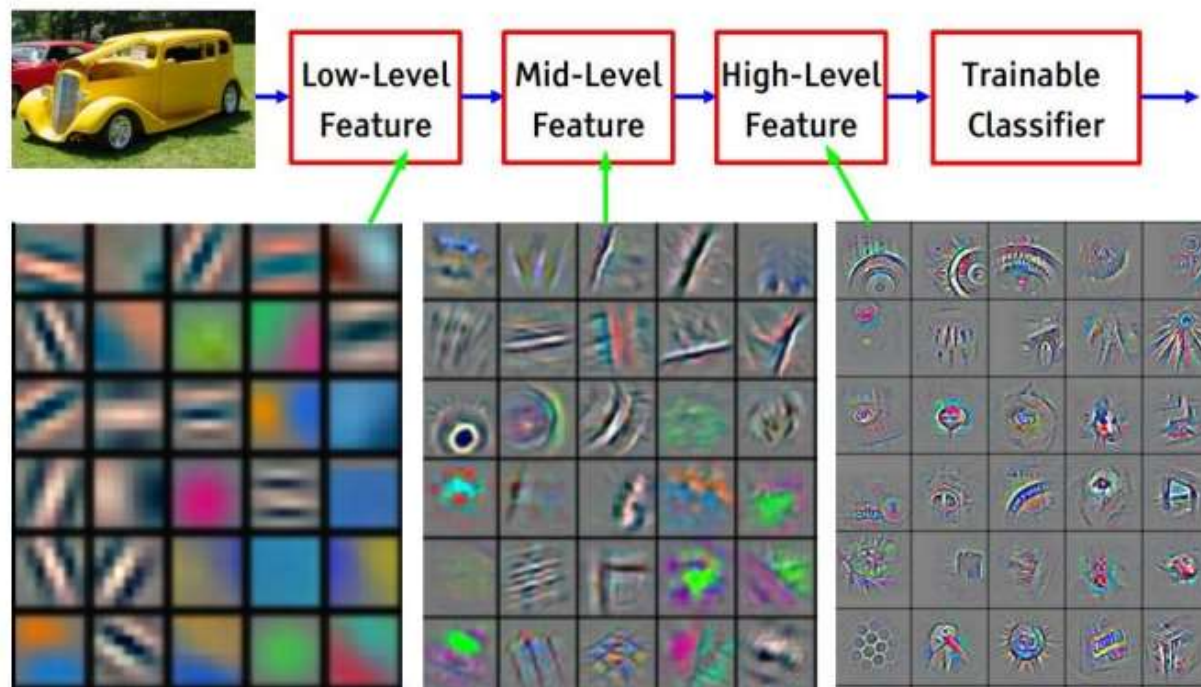


Feature extraction layer or Convolution layer

- Detect the same feature at different positions in the input image.

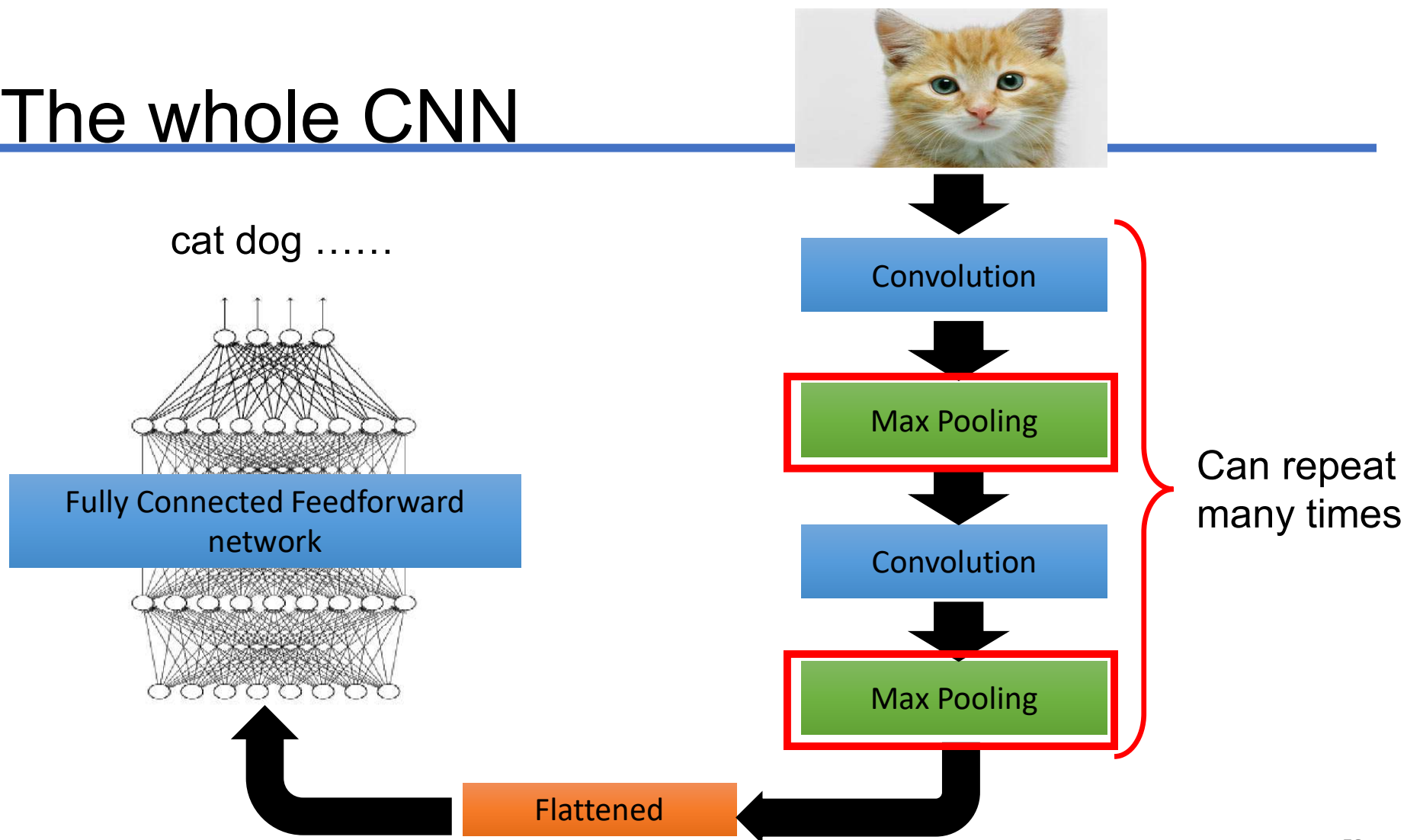


*[From recent Yann
LeCun slides]*



Feature visualization of convolutional net trained on ImageNet from [Zeiler & Fergus 2013]

The whole CNN



Max Pooling

1	-1	-1
-1	1	-1
-1	-1	1

Filter 1

-1	1	-1
-1	1	-1
-1	1	-1

Filter 2

3	-1	-3	-1
-3	1	0	-3
-3	-3	0	1
3	-2	-2	-1

-1	-1	-1	-1
-1	-1	-2	1
-1	-1	-2	1
-1	0	-4	3

Why Pooling

- Subsampling pixels will not change the object

bird



Subsampling

bird



We can subsample the pixels to make image smaller

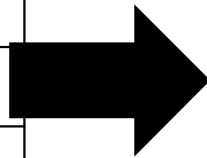


fewer parameters to characterize the image

Max Pooling

1	0	0	0	0	1
0	1	0	0	1	0
0	0	1	1	0	0
1	0	0	0	1	0
0	1	0	0	1	0
0	0	1	0	1	0

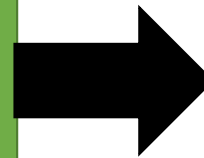
6 x 6 image



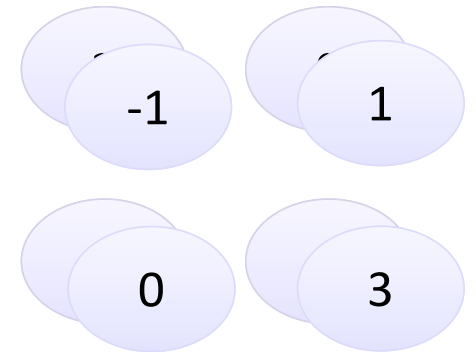
Conv



Max
Pooling



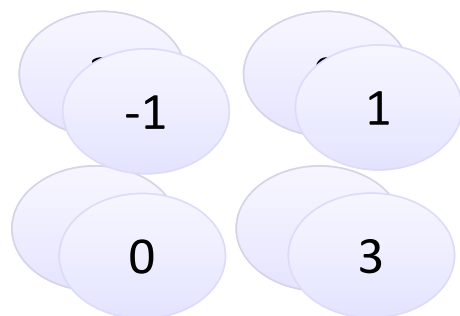
New image
but smaller



2 x 2 image

Each filter
is a channel

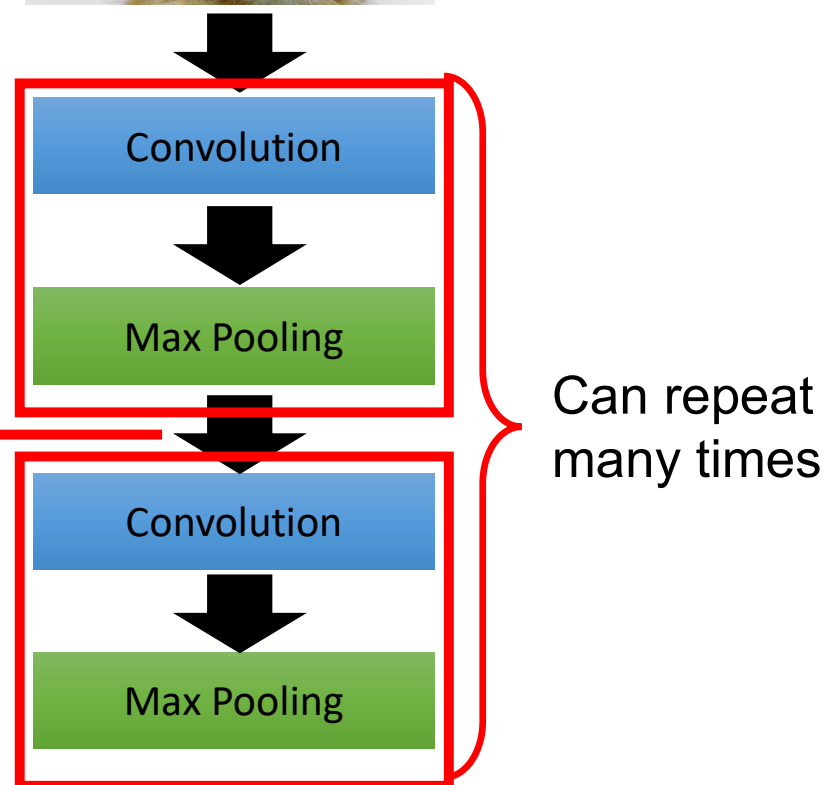
The whole CNN



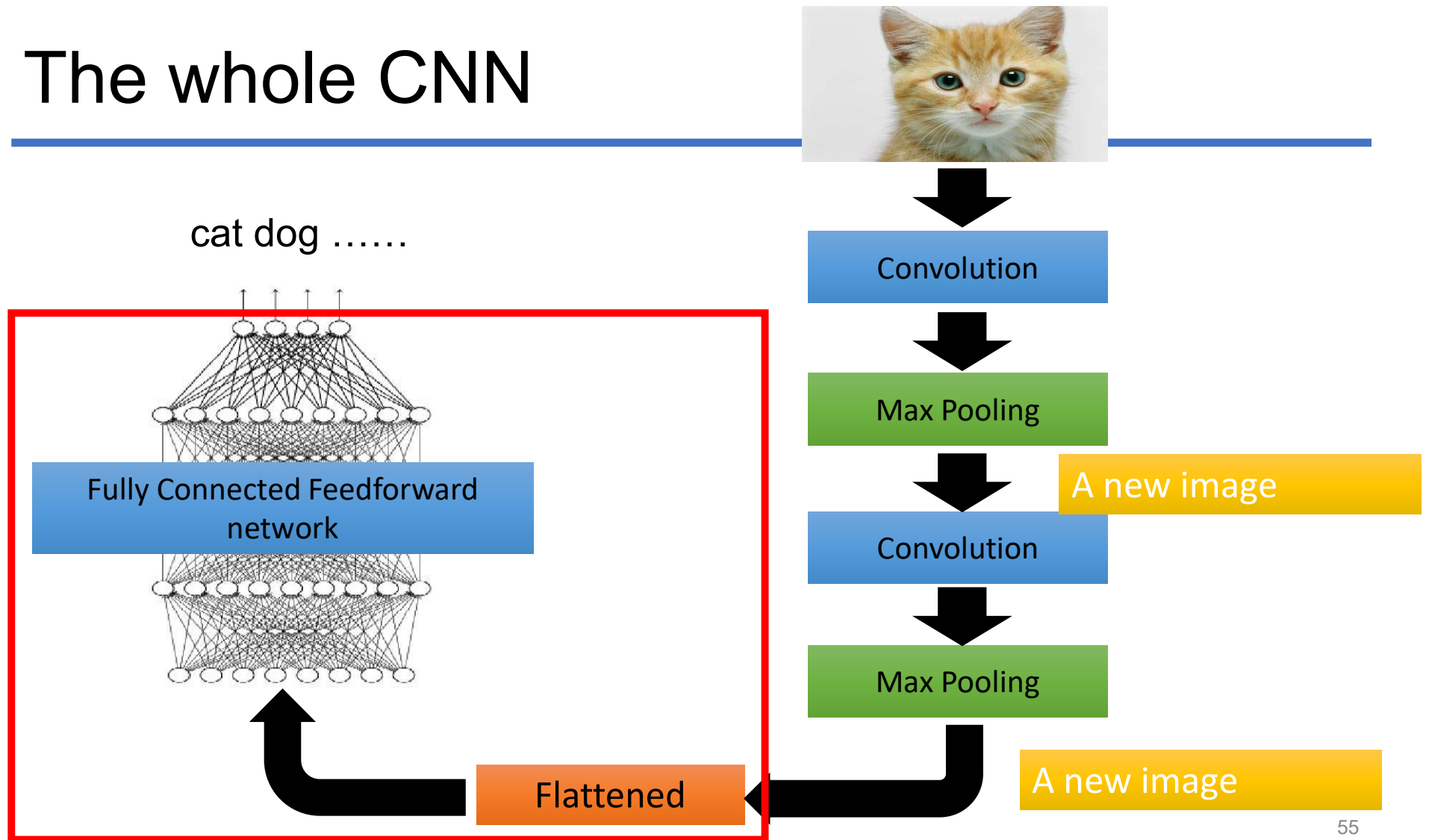
A new image

Smaller than the original image

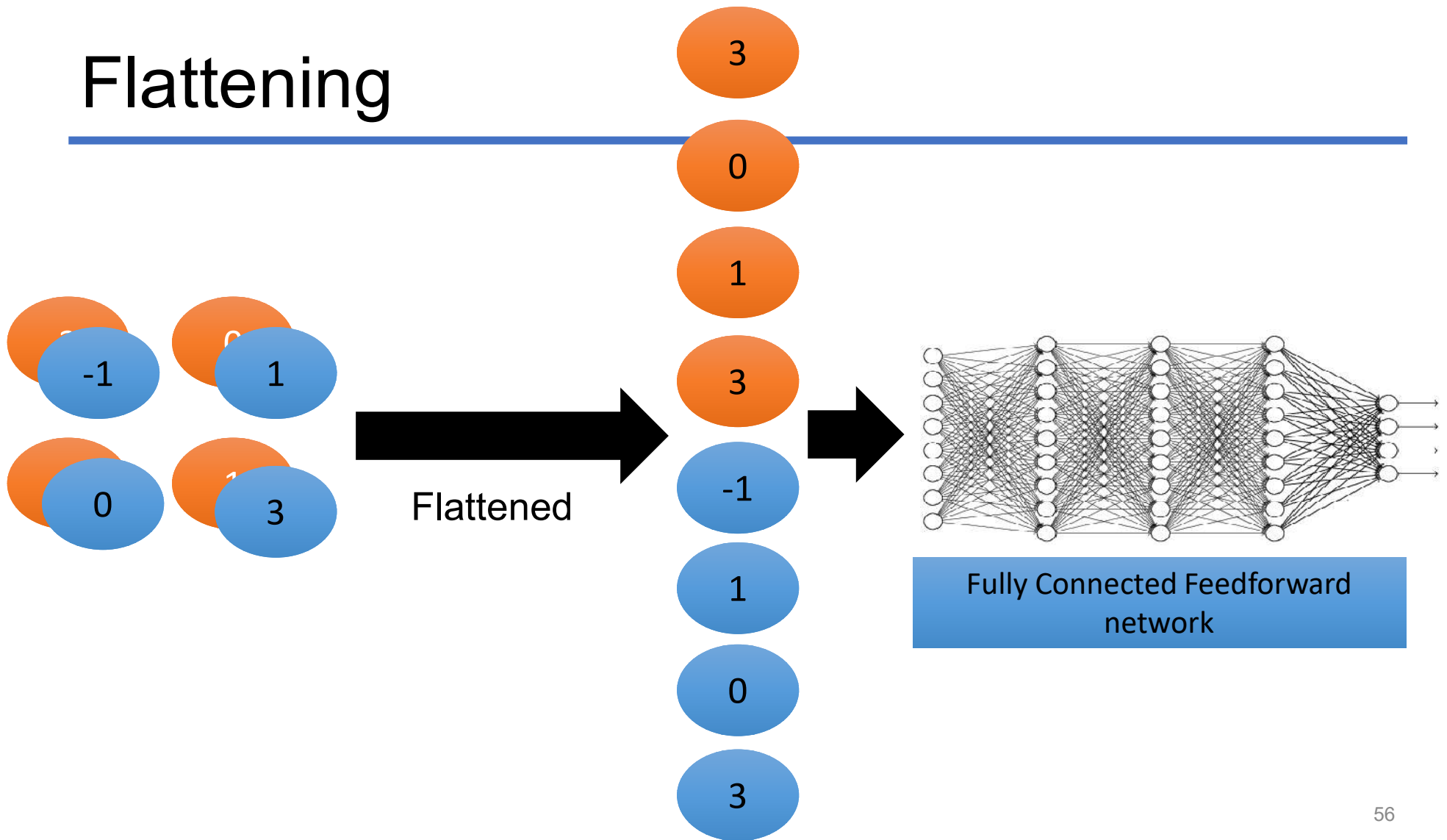
The number of channels is the number of filters



The whole CNN



Flattening



Thực hành ứng dụng ANN